



DesignWare® DW_apb_i2c

Databook

DW_apb_i2c **Product Code**

Copyright Notice and Proprietary Information

© 2023 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.

www.synopsys.com

Contents

Revision History	9
Preface	17
Product Code	17
Databook Organization	19
Related Documents	20
Web Resources	20
Reference Documentation	20
Synopsys Statement on Inclusivity and Diversity	21
Replacement of Noninclusive Terminology	21
Customer Support	21
Chapter 1	
Product Overview	23
1.1 DesignWare System Overview	24
1.2 General Product Description	26
1.2.1 DW_apb_i2c Block Diagram	26
1.3 Supported Features	27
1.3.1 I2C Features	27
1.3.2 DesignWare APB Completer Interface	28
1.4 Unsupported Features	28
1.5 Standards Compliance	29
1.6 Area and Power Numbers	29
1.7 Deliverables	29
1.8 Verification Environment Overview	29
1.9 Licenses	29
1.10 Where To Go From Here	30
Chapter 2	
Functional Description	31
2.1 Overview	32
2.2 I2C Terminology	35
2.2.1 I2C Bus Terms	35
2.2.2 Bus Transfer Terms	35
2.3 I2C Behavior	37
2.3.1 START and STOP Generation	37

2.3.2 Combined Formats	38
2.4 I2C Protocols	39
2.4.1 START and STOP Conditions	39
2.4.2 Addressing Target Protocol	39
2.4.3 Transmitting and Receiving Protocol	41
2.4.4 START BYTE Transfer Protocol	42
2.5 Tx FIFO Management and START, STOP and RESTART Generation	44
2.5.1 Tx FIFO Management When IC_EMPTYFIFO_HOLD_MASTER_EN = 0	44
2.5.2 Tx FIFO Management When IC_EMPTYFIFO_HOLD_MASTER_EN = 1	45
2.6 Multiple Controller Arbitration	49
2.7 Clock Synchronization	51
2.8 Operation Modes	52
2.8.1 Target Mode Operation	52
2.8.2 Controller Mode Operation	56
2.8.3 Disabling DW_apb_i2c	59
2.8.4 Terminating I ² C Transfers	59
2.9 Spike Suppression	61
2.10 Fast Mode Plus Operation	63
2.11 Bus Clear Feature	64
2.11.1 SDA Line Stuck at LOW Recovery	64
2.11.2 SCL Line is Stuck at LOW	65
2.12 Device ID	66
2.13 Ultra-Fast Speed Mode	67
2.14 IC_CLK Frequency Configuration	69
2.14.1 Minimum High and Low Counts in SS, FS, FM+ and high speed Modes With IC_CLK_FREQ_OPTIMIZATION = 0	70
2.14.2 Minimum High and Low Counts in SS, FS, FM+ and high speed Modes With IC_CLK_FREQ_OPTIMIZATION = 1	72
2.14.3 Minimum High and Low counts in Ultra-Fast mode (IC_ULTRA_FAST_MODE = 1)	72
2.14.4 Minimum IC_CLK Frequency	72
2.15 Clocks and Resets	81
2.15.1 Clocks	81
2.15.2 Reset Signals	81
2.16 SDA Hold Time	82
2.16.1 SDA Hold Timings in Receiver	82
2.16.2 SDA Hold Timings in Transmitter	84
2.17 DMA Controller Interface	86
2.17.1 Enabling the DMA Controller Interface	86
2.17.2 Overview of Operation	86
2.17.3 Transmit Watermark Level and Transmit FIFO Underflow	88
2.17.4 Choosing the Transmit Watermark Level	88
2.17.5 Selecting DEST_MSIZ and Transmit FIFO Overflow	89
2.17.6 Receive Watermark Level and Receive FIFO Overflow	90
2.17.7 Choosing the Receive Watermark level	90
2.17.8 Selecting SRC_MSIZ and Receive FIFO Underflow	90
2.17.9 Handshaking Interface Operation	91
2.18 APB Interface	95
2.18.1 APB 3.0 Support	95
2.18.2 APB 4.0 Support	96

2.19 I/O Connections	97
2.20 DW_apb_i2c Registers	98
2.20.1 Registers and Field Descriptions	98
2.20.2 Operation of Interrupt Registers	98
2.21 SMBus/PMBus	100
2.21.1 tTIMEOUT,MIN Parameter	100
2.21.2 Controller Device Clock Extension	100
2.21.3 Target Device Clock Extension	100
2.21.4 SMBDAT Low Timeout	101
2.21.5 Bus Protocols	101
2.21.6 SMBus Address Resolution Protocol	104
2.21.7 Notify ARP Controller Command Usage	108
2.21.8 SMBus Additional Target Address	109
2.21.9 SMBus Optional Signals	110
2.22 Multiple SAR Feature	112
2.22.1 Multiple Target Device Address based I ² C Write Transfer Handling	113
2.22.2 Multiple Target Device Address based I ² C Read Transfer Handling	114
2.22.3 Impact of Multiple Target Device Address Feature on SMBus ARP Sequence and UDID	115
2.22.4 SMBus Alert Support for IC_SARx Target Device Addresses	117
2.22.5 Advantages of Multiple SAR Feature	119
2.23 SMBus 3.1 Compliance	120
2.23.1 Configuring a Target Device Address as Default Target Address Device	120
2.23.2 Configuring a Target Address as Persistent Target Address Device	121
2.23.3 SMBus Device Categorization	121
2.24 UDID Feature	125
2.24.1 UDID of IC_SAR Target Address	125
2.24.2 UDID of IC_SAR2, IC_SAR3 and IC_SAR4 Target Device Addresses (IC_MULTI_SAR_EN=1) ..	125
2.25 Debug Interface	127
2.25.1 Controller FSM State Encoding	127
2.25.2 Target FSM State Encoding	128

Chapter 3

Parameter Descriptions	131
3.1 Top Level Parameters / Software Interface Configuration Parameters	132
3.2 Top Level Parameters / Device Configuration Parameters	134
3.3 Top Level Parameters / External Configuration Parameters	139
3.4 Top Level Parameters / Internal Configuration Parameters	140
3.5 Top Level Parameters / Bus Timing Parameters	143
3.6 Top Level Parameters / Additional Features Parameters	146
3.7 I2C Features Parameters	150
3.8 SMBus Features Parameters	153
3.9 SMBus Features / SMBus ARP Features Parameters	155
3.10 SMBus Features / IC_SAR SMBus ARP Features Parameters	156
3.11 SMBus Features / IC_SAR2 SMBus ARP Features Parameters	158
3.12 SMBus Features / IC_SAR3 SMBus ARP Features Parameters	160
3.13 SMBus Features / IC_SAR4 SMBus ARP Features Parameters	162
3.14 DesignWare Foundation Configuration Parameters	164

Chapter 4

Signal Descriptions	165
4.1 Interrupts Signals	167
4.2 I2C Interface (Controller/Target) Signals	183
4.3 APB Completer Interface Signals	186
4.4 DMA Interface Signals	189
4.5 SMBus Interface Signals	191
4.6 I2C Debug Signals	192

Chapter 5

Register Descriptions	197
5.1 DW_apb_i2c_addr_block1 Registers	204
5.1.1 IC_CON	205
5.1.2 IC_TAR	214
5.1.3 IC_SAR	217
5.1.4 IC_HS_MADDR	218
5.1.5 IC_DATA_CMD	219
5.1.6 IC_SS_SCL_HCNT	223
5.1.7 IC_UFM_SCL_HCNT	225
5.1.8 IC_SS_SCL_LCNT	227
5.1.9 IC_UFM_SCL_LCNT	229
5.1.10 IC_FS_SCL_HCNT	231
5.1.11 IC_UFM_TBUF_CNT	233
5.1.12 IC_FS_SCL_LCNT	234
5.1.13 IC_HS_SCL_HCNT	236
5.1.14 IC_HS_SCL_LCNT	238
5.1.15 IC_INTR_STAT	240
5.1.16 IC_INTR_MASK	245
5.1.17 IC_RAW_INTR_STAT	250
5.1.18 IC_RX_TL	259
5.1.19 IC_TX_TL	260
5.1.20 IC_CLR_INTR	261
5.1.21 IC_CLR_RX_UNDER	262
5.1.22 IC_CLR_RX_OVER	263
5.1.23 IC_CLR_TX_OVER	264
5.1.24 IC_CLR_RD_REQ	265
5.1.25 IC_CLR_TX_ABRT	266
5.1.26 IC_CLR_RX_DONE	267
5.1.27 IC_CLR_ACTIVITY	268
5.1.28 IC_CLR_STOP_DET	269
5.1.29 IC_CLR_START_DET	270
5.1.30 IC_CLR_GEN_CALL	271
5.1.31 IC_ENABLE	272
5.1.32 IC_STATUS	279
5.1.33 IC_TXFLR	287
5.1.34 IC_RXFLR	288
5.1.35 IC_SDA_HOLD	289
5.1.36 IC_TX_ABRT_SOURCE	291
5.1.37 IC_SLV_DATA_NACK_ONLY	299

5.1.38 IC_DMA_CR	301
5.1.39 IC_DMA_TDLR	303
5.1.40 IC_DMA_RDLR	304
5.1.41 IC_SDA_SETUP	305
5.1.42 IC_ACK_GENERAL_CALL	306
5.1.43 IC_ENABLE_STATUS	307
5.1.44 IC_FS_SPKLEN	310
5.1.45 IC_UFM_SPKLEN	311
5.1.46 IC_HS_SPKLEN	312
5.1.47 IC_CLR_RESTART_DET	313
5.1.48 IC_SCL_STUCK_AT_LOW_TIMEOUT	314
5.1.49 IC_SDA_STUCK_AT_LOW_TIMEOUT	315
5.1.50 IC_CLR_SCL_STUCK_DET	316
5.1.51 IC_DEVICE_ID	317
5.1.52 IC_SMBUS_CLK_LOW_SEXT	318
5.1.53 IC_SMBUS_CLK_LOW_MEXT	319
5.1.54 IC_SMBUS_THIGH_MAX_IDLE_COUNT	320
5.1.55 IC_SMBUS_INTR_STAT	322
5.1.56 IC_SMBUS_INTR_MASK	326
5.1.57 IC_SMBUS_RAW_INTR_STAT	330
5.1.58 IC_CLR_SMBUS_INTR	334
5.1.59 IC_OPTIONAL_SAR	336
5.1.60 IC_SMBUS_UDID_LSB	337
5.1.61 IC_SMBUS_UDID_WORD0	338
5.1.62 IC_SMBUS_UDID_WORD1	339
5.1.63 IC_SMBUS_UDID_WORD2	340
5.1.64 IC_SMBUS_UDID_WORD3	341
5.1.65 REG_TIMEOUT_RST	342
5.1.66 IC_COMP_PARAM_1	343
5.1.67 IC_COMP_VERSION	346
5.1.68 IC_COMP_TYPE	347
5.1.69 IC_SAR2	348
5.1.70 IC_SAR3	349
5.1.71 IC_SAR4	350
5.1.72 IC_CLR_WR_REQ	351
5.1.73 IC_CLR_SLV_ADDR_TAG	352
5.1.74 IC_SAR2_SMBUS_UDID_WORD0	353
5.1.75 IC_SAR2_SMBUS_UDID_WORD1	354
5.1.76 IC_SAR2_SMBUS_UDID_WORD2	355
5.1.77 IC_SAR2_SMBUS_UDID_WORD3	356
5.1.78 IC_SAR3_SMBUS_UDID_WORD0	357
5.1.79 IC_SAR3_SMBUS_UDID_WORD1	358
5.1.80 IC_SAR3_SMBUS_UDID_WORD2	359
5.1.81 IC_SAR3_SMBUS_UDID_WORD3	360
5.1.82 IC_SAR4_SMBUS_UDID_WORD0	361
5.1.83 IC_SAR4_SMBUS_UDID_WORD1	362
5.1.84 IC_SAR4_SMBUS_UDID_WORD2	363
5.1.85 IC_SAR4_SMBUS_UDID_WORD3	364
5.1.86 IC_SMBUS_ARP_CTRL	365

Chapter 6

Programming the DW_apb_i2c	369
6.1 Software Registers	370
6.2 Programming Example	371
6.3 Programming Flow for SCL and SDA Bus Recovery	379
6.4 Programming Flow for Reading the Device ID	380
6.5 Programming Flow of DW_apb_i2c in Ultra-Fast Mode	381
6.5.1 DW_apb_i2c Controller Mode	381
6.5.2 DW_apb_i2c Target Mode	382
6.6 SMBus Timeout Sequence in Controller Mode	383
6.7 SMBus Timeout Sequence in Target Mode	384
6.8 ARP Controller Sequence	385
6.9 ARP Target Sequence	386
6.10 SMBus SUSPEND Sequence in Host Mode	389
6.11 SMBus SUSPEND Sequence in Device Mode	390
6.12 SMBus ALERT Sequence in Host Mode	391
6.13 SMBus ALERT Sequence in Device Mode	392

Chapter 7

Verification	393
7.1 Verification Environment	394
7.1.1 VIP System Env – I ² C Controller/Target VIP	395
7.1.2 DUT System Environment	395
7.2 Testbench Directories and Files	396
7.3 Packaged Testcases	397

Chapter 8

Integration Considerations	399
8.1 Accessing Top-level Constraints	400
8.2 Performance	401
8.2.1 Power Consumption, Frequency, Area and DFT Coverage	401
8.3 Timing Exceptions	408

Appendix A

Synchronizer Methods	409
A.1 BCM Library Components	410
A.2 Synchronizer Methods	411
A.2.1 Synchronizers Used in DW_apb_i2c	411
A.2.2 Synchronizer 1: Simple Double Register Synchronizer	411
A.2.3 Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset	412

Appendix B

Internal Parameter Descriptions	413
--	------------

Appendix C

Glossary	417
-----------------	------------

Revision History

This table shows the revision history for the databook from release to release. This is being tracked from version 1.08a onward.

**Note**

- A change-bar version of this databook is available on the following IP web page under the “Documentation” section:
https://www.synopsys.com/dw/ipdir.php?c=DW_apb_i2c
 - Links and references to any chapter, section, table, figure, and page numbers in the Revision History table are assured to be valid only for the current version in the table.
-

Version	Date	Description
2.04a	April 2023	Note: Replaced noninclusive and insensitive terms with globally accepted inclusive terminology. See “Replacement of Noninclusive Terminology” on page 21. Added ■ “Synopsys Statement on Inclusivity and Diversity” on page 21 ■ “Reference Documentation” on page 20 ■ “Unsupported Features” on page 28 ■ “Area and Power Numbers” on page 29 ■ “Deliverables” on page 29 ■ “Where To Go From Here” on page 30 ■ “SMBus Device Categorization” on page 121 ■ “SMBus Alert Support for IC_SARx Target Device Addresses” on page 117 Updated ■ Updated the Revision History table for the version 2.02a ■ “Product Code” on page 17 ■ “Customer Support” on page 21 ■ “SMBus Alert Signal” on page 110 ■ Figure 6-1 on page 371, Figure 6-10 on page 383, Figure 6-14 on page 389 ■ “Verification” on page 393 ■ “Performance” on page 401 ■ Chapter 3, “Parameter Descriptions”, Chapter 4, “Signal Descriptions”, Chapter 5, “Register Descriptions” and Appendix Table B-1, “Internal Parameters” are auto extracted with change bars from the RTL.

Version	Date	Description
2.03a	December 2020	Added: ■ “Clocks and Resets” on page 81 ■ “Notify ARP Controller Command Usage” on page 108 ■ “Multiple SAR Feature” on page 112 ■ “SMBus 3.1 Compliance” on page 120 ■ “Debug Interface” on page 127 ■ “Timing Exceptions” on page 408 ■ “BCM Library Components” on page 410 Updated: ■ Version number changed for 2020.12a release ■ Table 2-10 on page 104 ■ “SMBus Address Resolution Protocol” on page 104 ■ “UDID Feature” on page 125 ■ Figure 6-9 on page 382 ■ Figure 6-11 on page 384 ■ Chapter 7, “Verification” ■ “Performance” on page 401 ■ Chapter 3, “Parameter Descriptions”, Chapter 4, “Signal Descriptions”, Chapter 5, “Register Descriptions” and Appendix Table B-1, “Internal Parameters” are auto extracted with change bars from the RTL. Renamed: ■ Appendix A, “Synchronizer Methods” ■ “SMBus Timeout Sequence in Controller Mode” on page 383 ■ “ARP Controller Sequence” on page 385 ■ “ARP Target Sequence” on page 386 ■ “SMBus SUSPEND Sequence in Host Mode” on page 389 ■ “SMBus SUSPEND Sequence in Device Mode” on page 390 ■ “SMBus ALERT Sequence in Host Mode” on page 391 ■ “SMBus ALERT Sequence in Device Mode” on page 392 Removed: ■ Software Driver section from Chapter 6, “Programming the DW_apb_i2c” ■ Index chapter

Version	Date	Description
2.02a	July 2018	Added: ■ “APB 3.0 Support” ■ “APB 4.0 Support” ■ “UDID Feature” Removed: ■ Chapter 2, “Building and Verifying a Component or Subsystem” and added the contents in the newly created user guide. ■ Section “Running Leda on Generated Code with coreConsultant” ■ Removed all instances of Leda Updated: ■ Version number changed for 2018.07a release ■ IC_RX_FULL_GEN_NACK parameter and feature was restored ■ “Parameter Descriptions”, “Signal Descriptions”, “Register Descriptions” and “Internal Parameter Descriptions” are auto extracted with change bars from the RTL.
2.01a	October 2016	Added: ■ “Running VCS XPROP Analyzer” ■ Entry for the xprop directory in Table 2-1 and Table 2-4. ■ Added note in “Overview” and “Tx FIFO Management When IC_EMPTYFI-FO_HOLD_MASTER_EN = 0” ■ Parameter Descriptions and Register Descriptions auto-extracted from the RTL Removed: ■ Removed the “Running Leda on Generated Code with coreConsultant” section, and reference to Leda directory in Table 2-1 ■ Removed the “Running Leda on Generated Code with coreAssembler” section, and reference to Leda directory in Table 2-4 Updated: ■ Version number changed for 2016.10a release ■ Modified Table 2-9 ■ Updated area and power numbers in sections “Area” and “Power Consumption” ■ Modified “APB Interface” ■ Updated description for SMBus ■ Updated the ic_smbalert_oe signal description ■ Moved Internal Parameter Descriptions to Appendix

Version	Date	Description
2.00a	June 2015	Added: ■ “Running SpyGlass® Lint and SpyGlass® CDC” ■ “Running SpyGlass on Generated Code with coreAssembler” ■ “Internal Parameter Descriptions” ■ New features: □ “Bus Clear Feature” □ “Device ID” □ “SMBus/PMBus” □ “Ultra-Fast Speed Mode” □ New parameter “IC_CLK_FREQ_OPTIMIZATION” □ Synchronizer Methods ■ Included a note regarding tBUF timing and setup/hold time. Updated: ■ “IC_CLK Frequency Configuration” updated for IC_CLK_FREQ_OPTIMIZATION and IC_ULTRA_FAST_MODE Configurations ■ “Signal Descriptions” auto-extracted from the RTL
1.22a	June 2014	Added: ■ New features: □ Blocking the Tx FIFO commands using IC_TX_CMD_BLOCK field in IC_ENABLE register □ Indication for first data byte received after the address in IC_DATA_CMD register □ Detection of STOP interrupt only if master is active ■ coreConsultant parameter (IC_AVOID_RX_FIFO_FLUSH_ON_TX_ABORT) introduced to avoid flushing of RX FIFO during TX Abort ■ New bits in IC_STATUS register for Indicating a reason for bus holding ■ Performance section in Integration considerations Updated: ■ Width of TX_FLUSH_CNT field in the IC_TX_ABORT_SOURCE register ■ Default Input/Output Delays in Signals chapter

Version	Date	Description
1.21a	May 2013	Added: ■ Section on Fast Mode Plus ■ Configuration Parameters: □ IC_RX_FULL_HLD_BUS_EN □ IC_SLV_RESTART_DET_EN ■ Signals: □ ic_restart_det_intr(_n) signal to enable restart detect in slave mode ■ Registers □ RESTART_DET bit of IC_INTR_STAT, IC_INTR_MASK and IC_RAW_INTR_STAT registers Bit detects a repeated start when the DW_apb_i2c is the addressed slave □ IC_CLR_RESTART_DET to clear the RESTART_DET interrupt □ MST_ON_HOLD bit to the IC_INTR_STAT, IC_INTR_MASK and IC_RAW_INTR_STAT registers. This bit indicates whether a master is holding the bus and the Tx FIFO is empty. Added the signal ic_mst_on_hold_intr(_n) ■ Programming flow for DW_apb_i2c master with TAR update Updated: ■ References to Fast Mode Plus ■ Registers: □ TX_FLUSH_CNT field of the IC_TX_ABRT_SOURCE register □ TX_ABRT field of the IC_RAW_INTR_STAT register □ IC_CON □ IC_RAW_INTR_STAT □ IC_SDA_HOLD Signals: ■ Active state of the ic_current_src_en signal ■ Programming flow for DW_apb_i2c as master in standard or fast mode ■ Method for deriving ic_clk values in high speed modes ■ Documentation template Removed: ■ Text stating that Fast Mode Plus is not supported Note in the IC_TX_ABRT_SOURCE register description stating DW_apb_i2c can be a master and slave at the same time
1.20a	October 2012	Added the product code on the cover and in Table 1-1.
1.20a	June 2012	Edited calculations for driving SDA in “high speed Modes” section; updated IC_ENABLE and IC_TX_ABRT_SOURCE registers.
1.17a	March 2012	Enhanced DW_ahb_dmac and DW_apb_i2c programming example; updated definition of IC_FS_SPKLEN and IC_HS_SPKLEN register descriptions; corrected programming values for dma_tx_req and dma_rx_req signals.
1.16b	December 2011	Enhanced description of IC_ADD_ENCODED_PARAMS parameter.
1.16b	November 2011	Version change for 2011.11a release.

Version	Date	Description
1.16a	October 2011	Version change for 2011.10a release.
1.15a	June 2011	Removed “Digital/Analog Domain Functional Partitioning” section (9.1) – irrelevant now with Spike Suppression functionality.
1.15a	June 2011	Updated system diagram in Figure 1-1; enhanced description of ic_rst_n signal; enhanced “Related Documents” section in Preface.
1.15a	April 2011	Clarified description of C_DEFAULT_SDA_HOLD parameter.
1.15a	April 2011	Corrected IC_DEFAULT_FS_SPKLEN and IC_DEFAULT_HS_SPKLEN default values.
1.15a	April 2011	Added spike suppression material; corrected R/W locations in timing diagrams in “Tx FIFO Management and START, STOP and RESTART Generation” section
1.14a	December 2010	Corrected subsection numbering in Registers chapter.
1.13a	October 2010	Added information on calculating maximum value for IC_DEFAULT_SDA_HOLD parameter and IC_SDA_HOLD register; “SDA Hold Time” section, description of IC_DEFAULT_SDA_HOLD parameter, and IC_SDA_HOLD register updated
1.12a	September 2010	Corrected DW_ahb_dmac response in “Receive Watermark Level and Receive FIFO Overflow” section
1.12a	September 2010	Corrected names of include files and vcs command used for simulation
1.11a	March 2010	Corrected information regarding how DW_apb_i2c communicates with slaves when operating in master mode; corrected default value for IC_DEFAULT_SDA_SETUP parameter; added SDA hold time information; added IC_SDA_HOLD register description; removed references to 300ns hold time in integration considerations; removed DW_apb_i2c Application Notes appendix.
1.10a	January 2010	Removed reference to I2C protocol created by Philips (NXP).
1.10a	December 2009	Corrected dependencies for IC_SS_SCL_HIGH_COUNT, IC_SS_SCL_LOW_COUNT, IC_FS_SCL_HIGH_COUNT, and IC_FS_SCL_LOW_COUNT parameters; corrected IC_RESTART_EN parameter description; modified description of IC_SDA_SETUP register; updated databook to new template for consistency with other IIP/VIP/PHY databooks.
1.10a	July 2009	Corrected equations for avoiding underflow when programming a source burst transaction.
1.10a	June 2009	Corrected name of IC_10BITADDR_SLAVE parameter in “Parameters” chapter.
1.10a	May 2009	Removed references to QuickStarts, as they are no longer supported.
1.10a	April 2009	Enhanced IC_CON description with table for IC_SLAVE_DISABLE and MASTER_MODE combinations that result in configuration errors.
1.10a	April 2009	Enhanced “Master Transmit and Master Receive” subsection to clarify reads for multiple bytes.

Version	Date	Description
1.10a	October 2008	IC_RX_FULL_GEN_NACK parameter removed; IC_INTR_MASK is active low; dependency changed for IC_HS_MASTER_CODE parameter; IC_SLAVE_DISABLE default changed to 1; values for high speed mode corrected in Table 8; debug_* signal default values corrected; version change for 2008.10a release.
1.09a	July 2008	Removed IC_RX_FULL_GEN_NACK configuration parameter and its conditional text. Changed reference to non-existent table for IC_*S_SCL_*CNT registers to link to “IC_CLK Frequency Configuration” section. Removed USE_FOUNDATION parameter.
1.09a	June 2008	Removed Synchronous value from IC_CLK_TYPE parameter; clarified that putting data into the FIFO generates a START and emptying the FIFO generates a STOP; clarified description of I2C_DYNAMIC_TAR_UPDATE parameter; clarification of IC_TAR description.
1.08b	February 2008	Modified note on restriction.

Preface

This databook provides information that you need to interface the DW_apb_i2c to the Advanced Peripheral Bus (APB). The DW_apb_i2c conforms to the [AMBA Specification, Revision 2.0](#) from Arm®.

The information in this databook includes an overview, pin and parameter descriptions, a memory map, and functional behavior of the component. An overview of the testbench, a description of the tests that are run to verify the coreKit, and synthesis information for the component are also provided.

Product Code

The following table lists all the components associated with the product license for DesignWare AMBA Fabric.

Table 1 Product Codes and Licenses for DesignWare AMBA Fabric

Component	Description	Product Licenses
DW_ahb	High Performance, Low Latency Interconnect Fabric for AMBA 2 AHB/AHB5	DesignWare AMBA Fabric Source License: DWC-AMBA-Fabric-Source (Product Code: 3768-0) Add-on - DesignWare AMBA Fabric Source AHB5 License: DWC-AMBA-AHB5-Fabric-Source (Product code: F944-0)
DW_ahb_eh2h	High Performance, High Bandwidth AMBA 2/AHB5 - AHB to AHB bridge	
DW_ahb_h2h	Area Efficient, Low Bandwidth AMBA 2/AHB5 - AHB to AHB Bridge	
DW_ahb_icm	Configurable Multi-Layer Interconnection Matrix AMBA 2 AHB/AHB5	
DW_ahb_ictl	Configurable Vectored Interrupt Controllers for AHB bus systems	

Component	Description	Product Licenses
DW_apb	High Performance, Low Latency Interconnect Fabric & Bridge for AMBA 2 APB(APB2)/APB3/APB4 for direct connect to AMBA 2 AHB Fabric	DesignWare AMBA Fabric Source License: DWC-AMBA-Fabric-Source (Product Code: 3768-0) Add-on - DesignWare AMBA Fabric Source AHB5 License: DWC-AMBA-AHB5-Fabric-Source (Product Code: F944-0) Add-on - DesignWare AXI Mission Critical License: DWC-AXI-SAFETY (Product Code: H615-0) Note: This license is available only with the Product Code H678-0. This is applicable only for DW_axi.
DW_apb_ictl	Configurable Vectored Interrupt Controllers for AMBA 2 APB bus systems	
DW_axi	High Performance, Hybrid Architecture, Low Latency Interconnect Fabric for AMBA 3 AXI/AMBA 4 AXI	
DW_axi_a2x	High Performance, Configurable Bridge Between AHB and AXI Components or AXI and AXI Components. With support for AMBA 3 AXI and AMBA 4 AXI	
DW_axi_gm	Simplify the connection of third party/custom Manager Controllers to any AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_gs	Simplify the connection of third party/custom Subordinate Controllers to any AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_hmx	Configurable, High Performance Interface from AHB Manager to an AXI Fabric or AXI Subordinate	
DW_axi_rs	Configurable Standalone Pipelining stage for AMBA 3 AXI/AMBA 4 AXI subsystems for Timing Management	
DW_axi_x2h	Bridge from AMBA 3 AXI/AMBA 4 AXI to AMBA 2 AHB/AHB5, enabling easy Integration of Legacy AHB Designs with newer AXI Systems	
DW_axi_x2p	High performance, Low Latency Interconnect Fabric and Bridge for AMBA 2 APB (APB2)/APB3/APB4 for direct connect to AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_x2x	Flexible Bridge between Multiple AMBA 3 AXI Components or Buses	

Component	Description	Product Licenses
DW_apb_gpio	General Purpose I/O Pad Control Peripheral for the AMBA APB Bus	DesignWare APB Peripheral Source License: DWC-APB-Periph-Source (Product Code: 3771-0) Add-on - DesignWare APB GPIO Mission Critical License: DWC-APB-GPIO-SAFETY (Product Code: H107-0) Note: This license is available only with the Product Code 3771-0. This is applicable only for DW_apb_gpio.
DW_apb_rap	Programmable controller for the Remap and Pause features of the DW_ahb AHB Interconnect	
DW_apb_rtc	Configurable High Range Counter with an AMBA APB Interface	
DW_apb_timers	Configurable System Counters Controlled through an AMBA APB interface	
DW_apb_wdt	A Programmable Watchdog Timer Peripheral for the AMBA APB bus	Add-on - DesignWare APB WDT Mission Critical License: DWC-APB-WDT-SAFETY (Product Code: H108-0) Note: This license is available only with the Product Code 3771-0. This is applicable only for DW_apb_wdt.
DW_apb_i2c	A Highly Configurable, Programmable Controller or Target I2C Device with an APB Interface	DesignWare APB Advanced Peripheral Source License: DWC-APB-Advanced-Source (Product Code: 3772-0)
DW_apb_i2s	A Configurable Controller or Target device for the three-wire Interface (I2S) for streaming stereo audio between devices	
DW_apb_ssi	A Configurable, Programmable, Full-duplex, Controller or Target Synchronous Serial Interface	
DW_apb_uart	A Configurable and Programmable Universal Asynchronous Receiver/Transmitter (UART) for the AMBA APB Bus	

Databook Organization

The chapters of this databook are organized as follows:

- [Chapter 1, “Product Overview”](#) “Product Overview” provides a system overview, a component block diagram, basic features, and an overview of the verification environment.
- [Chapter 2, “Functional Description”](#) describes the functional operation of the DW_apb_i2c.
- [Chapter 3, “Parameter Descriptions”](#) identifies the configurable parameters supported by the DW_apb_i2c.
- [Chapter 4, “Signal Descriptions”](#) provides a list and description of the DW_apb_i2c signals.
- [Chapter 5, “Register Descriptions”](#) describes the programmable registers of the DW_apb_i2c.

- [Chapter 6, “Programming the DW_apb_i2c”](#) provides information needed to program the configured DW_apb_i2c.
- [Chapter 7, “Verification”](#) provides information on verifying the configured DW_apb_i2c.
- [Chapter 8, “Integration Considerations”](#) includes information you need to integrate the configured DW_apb_i2c into your design.
- [Chapter A, “Synchronizer Methods”](#) documents the synchronizer methods (blocks of synchronizer functionality), and the list of BCM library components used in DW_apb_i2c.
- [Appendix B, “Internal Parameter Descriptions”](#) provides a list of internal parameter descriptions that might be indirectly referenced in expressions in the Signals chapter.
- [Appendix C, “Glossary”](#) provides a glossary of general terms.

Related Documents

Refer to the following documentation:

- *Using DesignWare Library IP in coreAssembler* – Contains information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools
- *coreAssembler User Guide* – Contains information on using coreAssembler
- *coreConsultant User Guide* – Contains information on using coreConsultant

To see a complete listing of documentation within the DesignWare Synthesizable Components for AMBA APB Protocol Specification v2.0, see the [Guide to Documentation for DesignWare Synthesizable Components for AMBA 2, AMBA 3 AXI, and AMBA 4 AXI](#) (Documentation Overview).

Web Resources

The following web links are various Synopsys online resources you may find useful:

- DesignWare IP product information:
<https://www.synopsys.com/designware-ip.html>
- Your custom DesignWare IP page:
<http://www.mydesignware.com>
- Documentation through SolvNetPlus:
<http://solvnetplus.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL):
<http://www.synopsys.com/keys>

Reference Documentation

The following references contain useful information concerning the protocols addressed by this DesignWare Library IP:

- AMBA Specification, Revision 2.0, Arm® for AHB and APB interface
- AMBA 3 APB Protocol Specification, v1.0, Arm® for APB3 interface
- AMBA 4 APB Protocol Specification v2.0, Arm® for APB4 interface

For additional references, see [“Standards Compliance”](#) on page 29.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Replacement of Noninclusive Terminology

The following table has the list and definition of the inclusive terminology used in this document:

Table 2 Replacement of Noninclusive Terminology

Noninclusive Terms	Replacement Terms	Definition
Master	Manager, Controller, Requester, Primary	Device or model that initiates and controls another device or peripheral.
Slave	Subordinate, Target, Completer, Secondary	Device or model that is controlled by and responds to a manager.



Note

To comply to the AMBA Specifications and to aid in seamless integration, Synopsys will not change parameter, signal, and register names.

Customer Support

To obtain support for your product, prepare the required files and contact the support center using one of the methods described:

- Prepare the following debug information, if applicable:
 - For environment set-up problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, select the following menu:
File > Build Debug Tar-file
 Check all the boxes in the dialog box that apply to your issue. This option gathers all the Synopsys product data needed to begin debugging an issue and writes it to the *<core tool startup directory>/debug.tar.gz* file.
 - For simulation issues outside of coreConsultant or coreAssembler:
 - Create a waveforms file (such as VPD, VCD or FSDB).
 - Identify the hierarchy path to the DesignWare instance.
 - Identify the timestamp of any signals or locations in the waveforms that are not understood.

- *For the fastest response, enter a case through SolvNetPlus:*
 - a. <https://solvnetplus.synopsys.com>

**Note**

SolvNetPlus does not support Internet Explorer.

- b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).
 - c. Complete the mandatory fields that are marked with an asterisk and click **Save**.
Make sure to include the following:
 - **Product L1:** DesignWare Library IP
 - **Product L2:** AMBA
 - d. After creating the case, attach any debug files you created.
For more information about general usage information, refer to the following article in SolvNetPlus:
<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>
- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
 - Include the Product L1 and Product L2 names, and Version number in your e-mail so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood
 - Attach any debug files you created.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

1

Product Overview

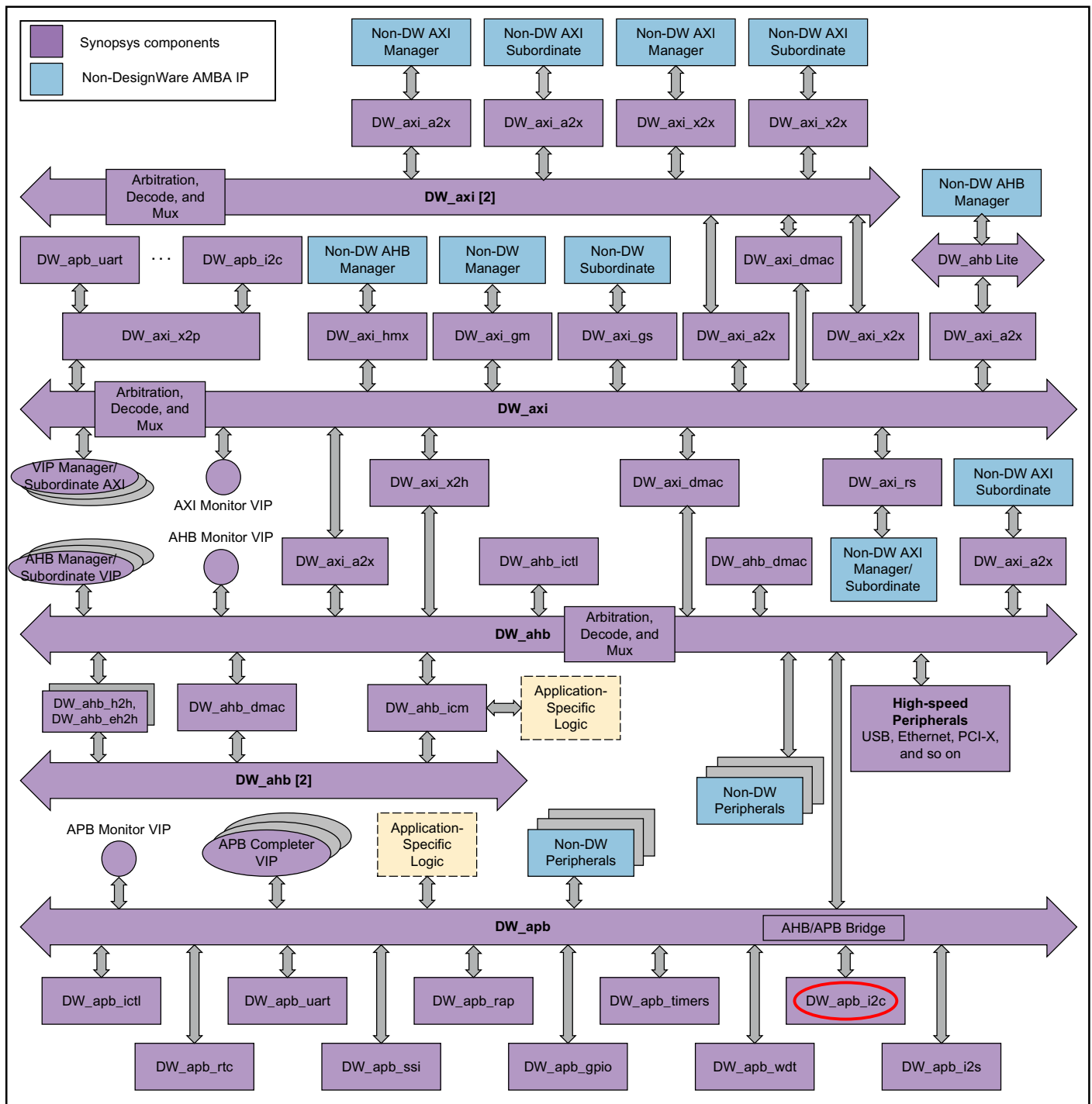
This chapter describes the DesignWare APB I²C Interface Peripheral, referred to as DW_apb_i2c. The DW_apb_i2c component is an AMBA 2.0-compliant Advanced Peripheral Bus (APB / APB3 / APB4) target device and is part of the family of DesignWare Synthesizable Components.

1.1 DesignWare System Overview

The Synopsys DesignWare Synthesizable Components environment is a parameterizable bus system containing AMBA version 2.0-compliant AHB (Advanced High-performance Bus) and APB (Advanced Peripheral Bus) components, and AMBA version 3.0-compliant AXI (Advanced eXtensible Interface) components.

[Figure 1-1](#) illustrates one example of this environment, including the AXI bus, the AHB bus, and the APB bus. Included in this subsystem are synthesizable IP for AXI/AHB/APB peripherals, bus bridges, and an AXI interconnect and AHB bus fabric. Also included are verification IP for AXI/AHB/APB manager/subordinate models and bus monitors. To access the product page and documentation for AMBA components, see the [DesignWare IP Solutions for AMBA Interconnect](#) page. (SolvNetPlus ID required)

Figure 1-1 Example of DW_apb_i2c in a Complete System



You can connect, configure, synthesize, and verify the DW_apb_i2c within a DesignWare subsystem using coreAssembler, documentation for which is available on the web in the *coreAssembler User Guide*.

If you want to configure, synthesize, and verify a single component such as the DW_apb_i2c component, you might prefer to use coreConsultant, documentation for which is available in the *coreConsultant User Guide*.

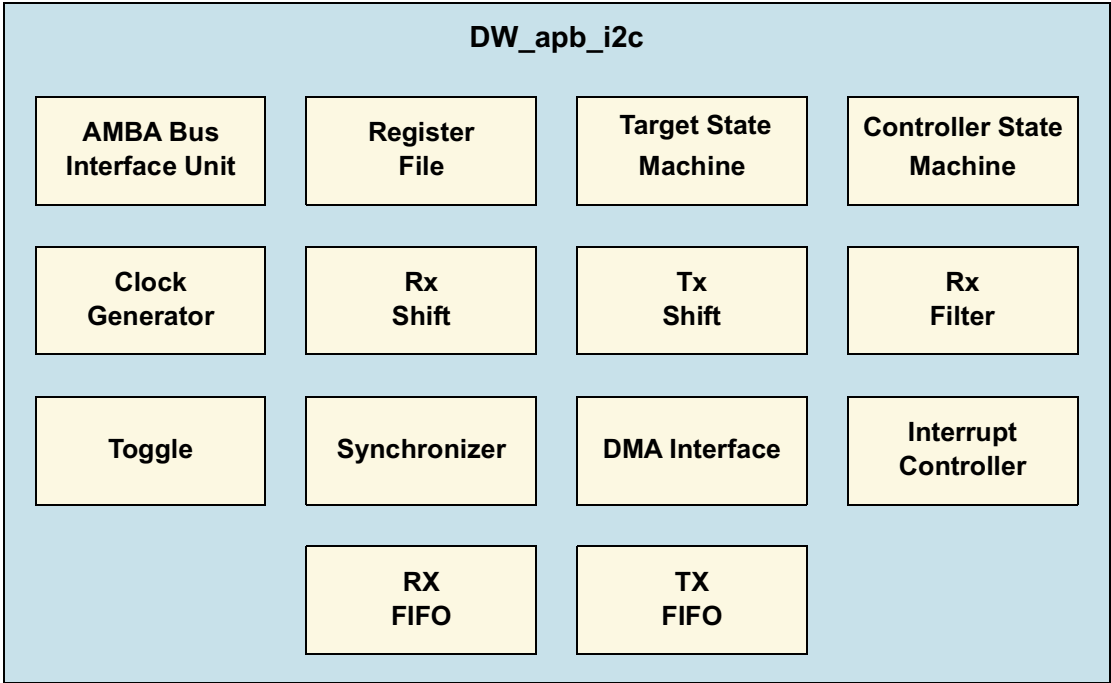
1.2 General Product Description

The DW_apb_i2c is a configurable, synthesizable, and programmable control bus that provides support for the communications link between integrated circuits in a system. It is a simple two-wire bus with a software-defined protocol for system control, which is used in temperature sensors and voltage level translators to EEPROMs, general-purpose I/O, A/D and D/A converters, CODECs, and many types of microprocessors.

1.2.1 DW_apb_i2c Block Diagram

Figure 1-2 illustrates a simple block diagram of DW_apb_i2c. For a more detailed block diagram and description of the component, see Chapter 2, “Functional Description”.

Figure 1-2 Block Diagram of DW_apb_i2c



1.3 Supported Features

DW_apb_i2c has the following features:

1.3.1 I2C Features

- Two-wire I²C serial interface – consists of a serial data line (SDA) and a serial clock (SCL)
- Three speeds:
 - Standard mode (0 to 100 Kb/s)
 - Fast mode (£ 400 Kb/s) or fast mode plus (£ 1000 Kb/s)¹
 - High-speed mode (£ 3.4 Mb/s)
- Clock synchronization
- Controller or target I²C operation
- 7- or 10-bit addressing
- 7- or 10-bit combined format transfers
- Bulk transmit mode
- Ignores CBUS addresses (an older ancestor of I²C that used to share the I²C bus)
- Transmit and receive buffers
- Interrupt or polled-mode operation
- Handles Bit and Byte waiting at all bus speeds
- Simple software interface consistent with DesignWare APB peripherals
- Component parameters for configurable software driver support
- DMA handshaking interface compatible with the DW_ahb_dmac handshaking interface
- Programmable SDA hold time (tHD;DAT)
- Bus clear feature
- Device ID feature
- SMBus/PMBus support
- SMBus Target detects and responds to ARP commands.
- Ultra-Fast mode support
- Multiple SAR Feature
- UDID feature support for all target device addresses (IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4)

The DW_apb_i2c requires external hardware components as support in order to be compliant in an I²C system. The descriptions are detailed later in this document.

It must also be noted that the DW_apb_i2c should only be operated either as (but not both):

- A controller in an I²C system and programmed only as a Controller; or
- A target device in an I²C system and programmed only as a Target.

1. In this document, references to fast mode also apply to fast mode plus, unless specifically stated otherwise.

1.3.2 DesignWare APB Completer Interface

- Support for APB data bus widths of 8, 16, and 32 bits
- Source code for this component is available on a per-project basis as a DesignWare Core; contact your local sales office for the details.

1.4 Unsupported Features

- Non-PTA - UDID Random number generation
- PMBus 1.3.1 Zone Protocols
- PMBus 1.3.1 Group Command Protocol
- PMBus 1.3.1 Extended Command Protocol
- Soft Reset through a programmable register
- SMBus non-ARP command Decoding Feature
- Advanced SMBus non-ARP Command Status Packing
- Advanced Block Write/Read command to enable the NVMe basic management commands
- Synopsys proprietary I²C/SMBus Host Controller Interface (HCI)
- Mission Critical Safety Features for automotive applications

1.5 Standards Compliance

The DW_apb_i2c component conforms to the AMBA Specification, AMBA 2 APB(APB2)/APB3/APB4. Readers are assumed to be familiar with this specification.

The DW_apb_i2c is designed for the following specifications:

- I²C Bus Specification, Version 6.0, dated April 2014
- SMBus specification Version 3.1, dated March 2018
- PMBus Specification Version 1.2, dated September 2010

1.6 Area and Power Numbers

For more information on power consumption, frequency, area and DFT coverage, see [“Performance”](#) on page 401.

1.7 Deliverables

DW_apb_i2c is packaged as a .run file. The license file required to use this Synopsys DesignWare IP is delivered separately. Use the Synopsys coreConsultant tool to configure, synthesize, and simulate the IP.

The <IP Name> image contains the following:

- System Verilog RTL source code
- System Verilog based test suite that integrates the DUT and AMBA SVT APB2/APB3/APB4 VIP
- Synthesis scripts for Synopsys Design Compiler and Synopsys Fusion Compiler; Synplify Pro Simulation regression scripts for Synopsys VCS tool
- Spyglass and VC SpyGlass Lint, CDC, RDC checker rules used for linting, Clock Domain Crossing checks and Reset Domain crossing checks respectively

1.8 Verification Environment Overview

The DW_apb_i2c includes an extensive verification environment, which sets up and uses Synopsys VCS tool to execute tests that verify the functionality of the configured component. You can then analyze the results of the simulation.

The *Verification* chapter discusses the testbench environment and provides an overview of the tests that are used to verify the DW_apb_i2c when you run component-level simulation.

1.9 Licenses

Before you begin using the DW_apb_i2c, you must have a valid license. For more information, see “Licenses” section in the *DesignWare Synthesizable Components for AMBA 2 Installation Guide*.

1.10 Where To Go From Here

At this point, you may want to get started working with the DW_apb_i2c component within a subsystem or by itself. Synopsys provides several tools within its coreTools suite of products for the purposes of configuration, synthesis, and verification of single or multiple synthesizable IP components – coreConsultant and coreAssembler. For information on the different coreTools, see *Guide to coreTools Documentation*.

For more information about configuring, synthesizing, and verifying just your DW_apb_i2c component, see “Overview of the coreConsultant Configuration and Integration Process” in *DesignWare Synthesizable Components for AMBA 2 User Guide*.

For more information about implementing your DW_apb_i2c component within a DesignWare subsystem using coreAssembler, see “Overview of the coreAssembler Configuration and Integration Process” section in *DesignWare Synthesizable Components for AMBA 2 User Guide*.

2

Functional Description

This chapter describes the functional behavior of DW_apb_i2c in more detail. Following topics are covered in this chapter:

- [“Overview” on page 32](#)
- [“I2C Terminology” on page 35](#)
- [“I2C Behavior” on page 37](#)
- [“I2C Protocols” on page 39](#)
- [“Tx FIFO Management and START, STOP and RESTART Generation” on page 44](#)
- [“Multiple Controller Arbitration” on page 49](#)
- [“Clock Synchronization” on page 51](#)
- [“Operation Modes” on page 52](#)
- [“Spike Suppression” on page 61](#)
- [“Fast Mode Plus Operation” on page 63](#)
- [“Bus Clear Feature” on page 64](#)
- [“Device ID” on page 66](#)
- [“Ultra-Fast Speed Mode” on page 67](#)
- [“IC_CLK Frequency Configuration” on page 69](#)
- [“Clocks and Resets” on page 81](#)
- [“SDA Hold Time” on page 82](#)
- [“DMA Controller Interface” on page 86](#)
- [“APB Interface” on page 95](#)
- [“I/O Connections” on page 97](#)
- [“DW_apb_i2c Registers” on page 98](#)
- [“SMBus/PMBus” on page 100](#)
- [“Multiple SAR Feature” on page 112](#)
- [“SMBus 3.1 Compliance” on page 120](#)
- [“UDID Feature” on page 125](#)
- [“Debug Interface” on page 127](#)

2.1 Overview

The I²C bus is a two-wire serial interface, consisting of a serial data line (SDA) and a serial clock (SCL). These wires carry information between the devices connected to the bus. Each device is recognized by a unique address and can operate as either a “transmitter” or “receiver,” depending on the function of the device. Devices can also be considered as controllers or targets when performing data transfers. A controller is a device that initiates a data transfer on the bus and generates the clock signals to permit that transfer. At that time, any device addressed is considered a target device.

**Note**

The DW_apb_i2c must only be programmed to operate in either controller or target mode only. Operating as a controller and target simultaneously is not supported.

The DW_apb_i2c module can operate in standard mode (with data rates 0 to 100 Kb/s), fast mode (with data rates less than or equal to 400 Kb/s), fast mode plus (with data rates less than or equal to 1000 Kb/s), high speed mode (with data rates less than or equal to 3.4 Mb/s), and Ultra-Fast Speed Mode (with data rates less than or equal to 5 Mb/s).

**Note**

In this document, references to fast mode also apply to fast mode plus, unless specifically stated otherwise.

The DW_apb_i2c can communicate with devices only of these modes as long as they are attached to the bus. Additionally, high speed mode and fast mode devices are downward compatible. For instance, high speed mode devices can communicate with fast mode and standard mode devices in a mixed-speed bus system; fast mode devices can communicate with standard mode devices in 0 to 100 Kb/s I²C bus system. However:

1. Standard mode devices are not upward compatible and should not be incorporated in a fast-mode I²C bus system as they cannot follow the higher transfer rate and unpredictable states would occur.
2. Ultra-Fast mode devices are not downward compatible and should not be incorporated in traditional I²C speeds (High Speed, Fast/Fast Mode Plus speed, Standard mode speed) as Ultra-Fast mode follows the higher transfer rate (up to 5Mb/s) with only write transfers and there is no acknowledgment from the target device.

An example of high speed mode devices are LCD displays, high-bit count ADCs, and high capacity EEPROMs. These devices typically need to transfer large amounts of data. Most maintenance and control applications, the common use for the I²C bus, typically operate at 100 kHz (in standard and fast modes).

An example of Ultra-Fast speed mode devices are LED controllers and other devices that do not need feedback. These devices typically need to transfer large amounts of data greater than 1Mhz.

Any DW_apb_i2c device can be attached to an I²C-bus and every device can talk with any controller device, passing information back and forth. There needs to be at least one controller (such as a microcontroller or DSP) on the bus but there can be multiple controller, which require them to arbitrate for ownership. Multiple controllers and arbitration are explained later in this chapter.

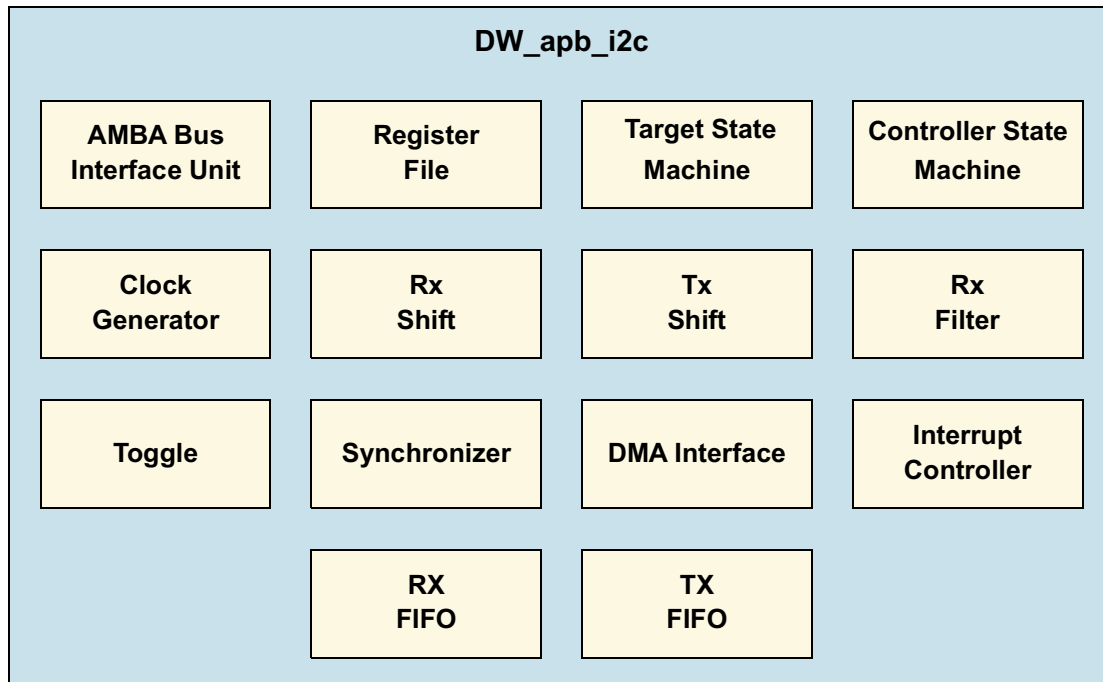
The DW_apb_i2c also supports SMBus and PMBus protocols for System Management and Power management.

**Note**

In this databook, any reference to SMBus implicitly refers to PMBus also and vice versa.

The DW_apb_i2c is made up of an AMBA APB completer interface, an I²C interface, and FIFO logic to maintain coherency between the two interfaces. A simplified block diagram of the component is illustrated in [Figure 2-1](#).

Figure 2-1 DW_apb_i2c Block Diagram



The following define the file names and functions of the blocks in [Figure 2-1](#):

- AMBA Bus Interface Unit—DW_apb_i2c_biu.v—Takes the APB interface signals and translates them into a common generic interface that allows the register file to be bus protocol-agnostic.
- Register File—DW_apb_i2c_regfile—Contains configuration registers and is the interface with software.
- Target State Machine—DW_apb_i2c_slv fsm—Follows the protocol for a target and monitors bus for address match.
- Controller State Machine—DW_apb_i2c_mst fsm—Generates the I²C protocol for the controller transfers.
- Clock Generator—DW_apb_i2c_clk_gen.v—Calculates the required timing to do the following:
 - Generate the SCL clock when configured as a controller
 - Check for bus idle
 - Generate a START and a STOP
 - Setup the data and hold the data
- Rx Shift—DW_apb_i2c_rx_shift—Takes data into the design and extracts it in byte format.

- Tx Shift – DW_apb_i2c_tx_shift – Presents data supplied by CPU for transfer on the I²C bus.
- Rx Filter – DW_apb_i2c_rx_filter – Detects the events in the bus; for example, start, stop and arbitration lost.
- Toggle – DW_apb_i2c_toggle – Generates pulses on both sides and toggles to transfer signals across clock domains.
- Synchronizer – DW_apb_i2c_sync – Transfers signals from one clock domain to another.
- DMA Interface – DW_apb_i2c_dma – Generates the handshaking signals to the central DMA controller in order to automate the data transfer without CPU intervention.
- Interrupt Controller – DW_apb_i2c_intctl – Generates the raw interrupt and interrupt flags, allowing them to be set and cleared.
- RX FIFO/TX FIFO – DW_apb_i2c_fifo – Holds the RX FIFO and TX FIFO register banks and controllers, along with their status levels.

2.2 I2C Terminology

The following terms are used throughout this manual and are defined as follows:

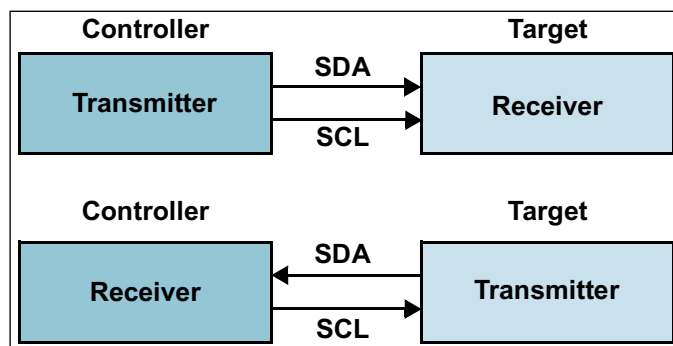
2.2.1 I2C Bus Terms

The following terms relate to how the role of the I²C device and how it interacts with other I²C devices on the bus.

- **Transmitter** – The device that sends data to the bus. A transmitter can either be a device that initiates the data transmission to the bus (a *controller-transmitter*) or responds to a request from the controller to send data to the bus (a *target-transmitter*).
- **Receiver** – The device that receives data from the bus. A receiver can either be a device that receives data on its own request (a *controller-receiver*) or in response to a request from the controller (a *target-receiver*).
- **Controller** – The component that initializes a transfer (START command), generates the clock (SCL) signal and terminates the transfer (STOP command). A controller can be either a transmitter or a receiver.
- **Target** – The device addressed by the controller. A target can be either receiver or transmitter.

These concepts are illustrated in [Figure 2-2](#).

Figure 2-2 Controller/Target and Transmitter/Receiver Relationships



- **Multi-Controller** – the ability for more than one controller to co-exist on the bus at the same time without collision or data loss.
- **Arbitration** – the predefined procedure that authorizes only one controller at a time to take control of the bus. For more information about this behavior, see [“Multiple Controller Arbitration”](#) on page 49.
- **Synchronization** – the predefined procedure that synchronizes the clock signals provided by two or more controllers. For more information about this feature, see [“Clock Synchronization”](#) on page 51.
- **SDA** – data signal line (Serial Data)
- **SCL** – clock signal line (Serial Clock)

2.2.2 Bus Transfer Terms

The following terms are specific to data transfers that occur to/from the I²C bus.

- **START (RESTART)** – data transfer begins with a START or RESTART condition. The level of the SDA data line changes from high to low, while the SCL clock line remains high. When this occurs, the bus becomes busy.



START and RESTART conditions are functionally identical.

- **STOP** – data transfer is terminated by a STOP condition. This occurs when the level on the SDA data line passes from the low state to the high state, while the SCL clock line remains high. When the data transfer has been terminated, the bus is free or idle once again. The bus stays busy if a RESTART is generated instead of a STOP condition.

2.3 I2C Behavior

The DW_apb_i2c can be controlled through software to be either:

- An I²C controller only, communicating with other I²C targets; OR
- An I²C target only, communicating with one more I²C controllers.

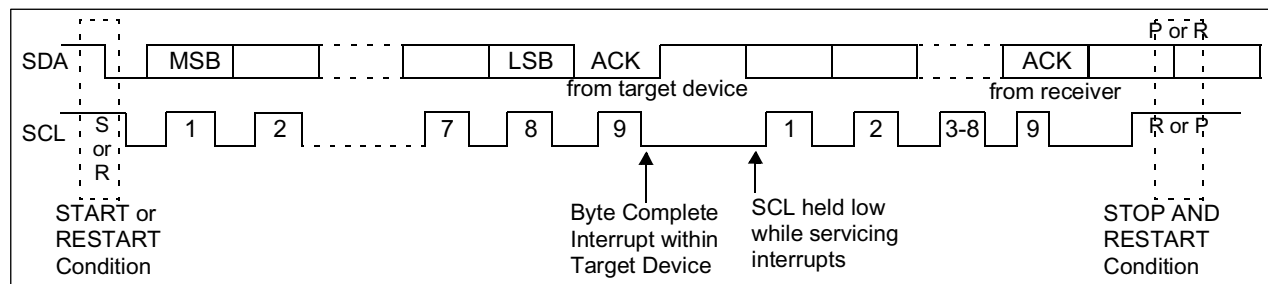
The controller is responsible for generating the clock and controlling the transfer of data. The target is responsible for either transmitting or receiving data to/from the controller. The acknowledgment of data is sent by the device that is receiving data, which can be either a controller or a target device. As mentioned previously, the I²C protocol also allows multiple controllers to reside on the I²C bus and uses an arbitration procedure to determine bus ownership.

Each target has a unique address that is determined by the system designer. When a controller wants to communicate with a target, the controller transmits a START/RESTART condition that is then followed by the target's address and a control bit (R/W) to determine if the controller wants to transmit data or receive data from the target. The target then sends an acknowledge (ACK) pulse after the address.

If the controller (controller-transmitter) is writing to the target (target-receiver), the receiver gets one byte of data. This transaction continues until the controller terminates the transmission with a STOP condition. If the controller is reading from a target (controller-receiver), the target transmits (target-transmitter) a byte of data to the controller, and the controller then acknowledges the transaction with the ACK pulse. This transaction continues until the controller terminates the transmission by not acknowledging (NACK) the transaction after the last byte is received, and then the controller issues a STOP condition or addresses another target after issuing a RESTART condition. This behavior is illustrated in [Figure 2-3](#).

In Ultra-Fast Speed Mode, the controller can issue only the write transfers to the targets with always not acknowledging (NACK) from the targets. Read transfers are not allowed in this mode.

Figure 2-3 Data Transfer on the I²C Bus



The DW_apb_i2c is a synchronous serial interface. The SDA line is a bidirectional signal and changes only while the SCL line is low, except for STOP, START, and RESTART conditions. The output drivers are open-drain or open-collector to perform wire-AND functions on the bus. The maximum number of devices on the bus is limited by only the maximum capacitance specification of 400 pF. Data is transmitted in byte packages.

The I²C protocols implemented in DW_apb_i2c are described in more details in [“I2C Protocols”](#) on page 39.

2.3.1 START and STOP Generation

When operating as an I²C controller, putting data into the transmit FIFO causes the DW_apb_i2c to generate a START condition on the I²C bus. If the IC_EMPTYFIFO_HOLD_MASTER_EN parameter is set to 0, allowing the transmit FIFO to empty causes the DW_apb_i2c to generate a STOP condition on the I²C bus. If IC_EMPTYFIFO_HOLD_MASTER_EN is set to 1, then writing a 1 to IC_DATA_CMD[9] causes the

DW_apb_i2c to generate a STOP condition on the I²C bus; a STOP condition is not issued if this bit is not set, even if the transmit FIFO is empty.

When operating as a target, the DW_apb_i2c does not generate START and STOP conditions, as per the protocol. However, if a read request is made to the DW_apb_i2c, it holds the SCL line low until read data has been supplied to it. This stalls the I²C bus until read data is provided to the target DW_apb_i2c, or the DW_apb_i2c target is disabled by writing a 0 to bit 0 of the IC_ENABLE register.

2.3.2 Combined Formats

The DW_apb_i2c supports mixed read and write combined format transactions in both 7-bit and 10-bit addressing modes.

To initiate combined format transfers, IC_CON.IC_RESTART_EN should be set to 1. With this value set and operating as a controller, when the DW_apb_i2c completes an I²C transfer, it checks the transmit FIFO and executes the next transfer. If the direction of this transfer differs from the previous transfer, the combined format is used to issue the transfer. If the transmit FIFO is empty when the current I²C transfer completes – depending on the value of IC_EMPTYFIFO_HOLD_MASTER_EN:

- Either a STOP is issued or,
- IC_DATA_CMD[9] is checked and:
 - If set to 1, a STOP bit is issued.
 - If set to 0, the SCL is held low until the next command is written to the transmit FIFO.

For more details, see [“Tx FIFO Management and START, STOP and RESTART Generation”](#) on page 44.



Note

Mixed write and read transactions in both 7-bit and 10-bit addressing modes are not applicable for Ultra-Fast Mode (IC_ULTRA_FAST_MODE=1) as read transfers are not supported in Ultra-Fast Mode.

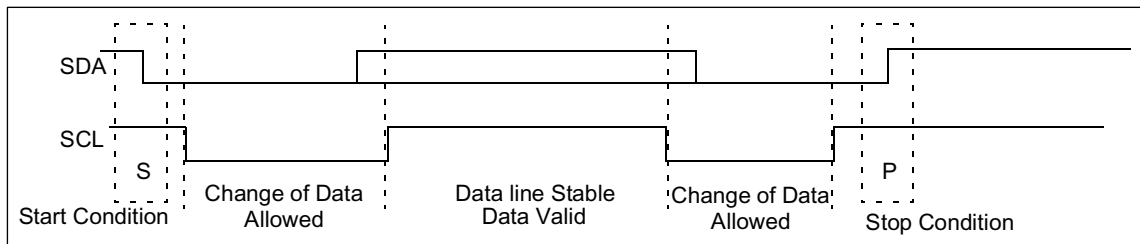
2.4 I2C Protocols

The DW_apb_i2c has the protocols discussed in this section.

2.4.1 START and STOP Conditions

When the bus is idle, both the SCL and SDA signals are pulled high through external pull-up resistors on the bus. When the controller wants to start a transmission on the bus, the controller issues a START condition. This is defined to be a high-to-low transition of the SDA signal while SCL is 1. When the controller wants to terminate the transmission, the controller issues a STOP condition. This is defined to be a low-to-high transition of the SDA line while SCL is 1. [Figure 2-4](#) shows the timing of the START and STOP conditions. When data is being transmitted on the bus, the SDA line must be stable when SCL is 1.

Figure 2-4 START and STOP Condition



Note

The signal transitions for the START/STOP conditions, as depicted in [Figure 2-4](#), reflect those observed at the output signals of the Controller driving the I²C bus. Care should be taken when observing the SDA/SCL signals at the input signals of the Target(s), because unequal line delays may result in an incorrect SDA/SCL timing relationship.

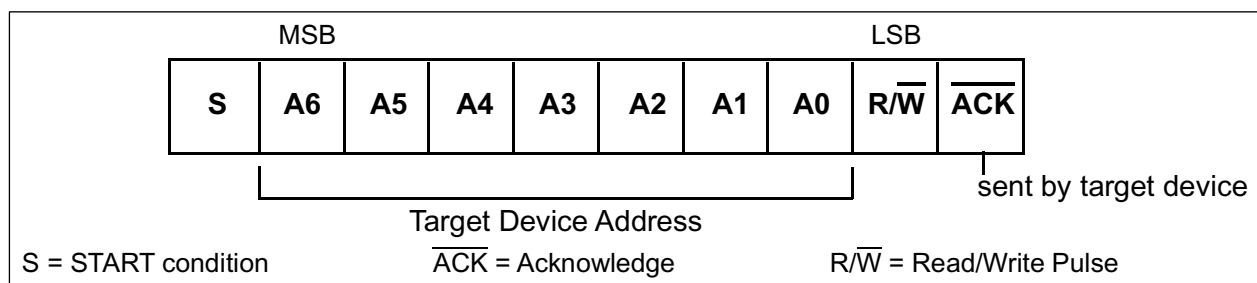
2.4.2 Addressing Target Protocol

There are two address formats: the 7-bit address format and the 10-bit address format.

2.4.2.1 7-bit Address Format

During the 7-bit address format, the first seven bits (bits 7:1) of the first byte set the target device address and the LSB bit (bit 0) is the R/W bit as shown in [Figure 2-5](#). When bit 0 (R/W) is set to 0, the controller writes to the target. When bit 0 (R/W) is set to 1, the controller reads from the target.

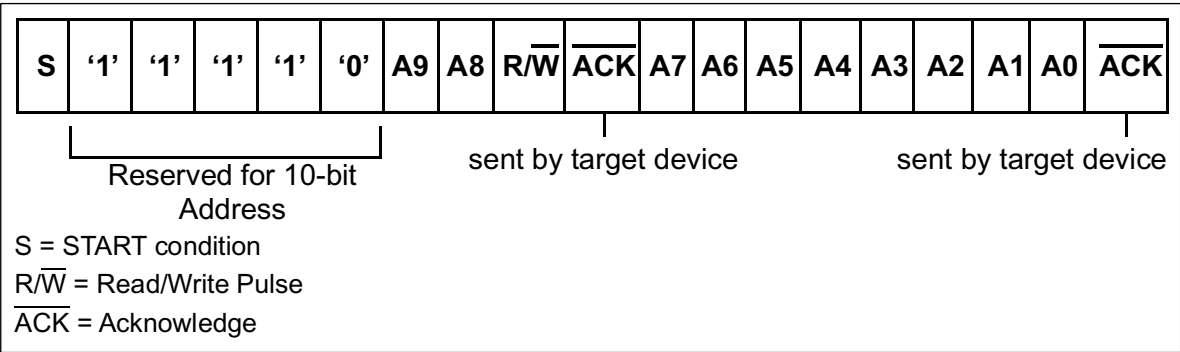
Figure 2-5 7-bit Address Format



2.4.2.2 10-bit Address Format

During 10-bit addressing, two bytes are transferred to set the 10-bit address. The transfer of the first byte contains the following bit definition. The first five bits (bits 7:3) notify the targets that this is a 10-bit transfer followed by the next two bits (bits 2:1), which set the targets address bits 9:8, and the LSB bit (bit 0) is the R/W bit. The second byte transferred sets bits 7:0 of the target address. [Figure 2-6](#) shows the 10-bit address format.

Figure 2-6 10-bit Address Format



[Table 2-1](#) defines the special purpose and reserved first byte addresses.

Table 2-1 I²C/SMBus Definition of Bits in First Byte

Target Address	R/W Bit	Description
0000 000	0	General Call Address. DW_apb_i2c places the data in the receive buffer and issues a General Call interrupt.
0000 000	1	START byte. For more details, see “START BYTE Transfer Protocol” on page 42.
0000 001	X	CBUS address. DW_apb_i2c ignores these accesses.
0000 010	X	Reserved.
0000 011	X	Reserved.
0000 1XX	X	High-speed controller code (for more information, see “Multiple Controller Arbitration” on page 49)
1111 1XX	1	Device ID
1111 0XX	X	10-bit target addressing
0001 000	X	SMBus Host
0001 100	X	SMBus Alert Response Address
1100 001	X	SMBus Device Default Address

DW_apb_i2c does not restrict you from using these reserved addresses. However, if you use these reserved addresses, you may run into incompatibilities with other I²C components.

2.4.3 Transmitting and Receiving Protocol

The controller can initiate data transmission and reception to/from the bus, acting as either a controller-transmitter or controller-receiver. A target responds to requests from the controller to either transmit data or receive data to/from the bus, acting as either a target-transmitter or target-receiver, respectively.

2.4.3.1 Controller-Transmitter and Target-Receiver

All data is transmitted in byte format, with no limit on the number of bytes transferred per data transfer. After the controller sends the address and R/W bit or the controller transmits a byte of data to the target, the target-receiver must respond with the acknowledge signal (ACK). When a target-receiver does not respond with an ACK pulse, the controller terminates the transfer by issuing a STOP condition. The target must leave the SDA line high so that the controller can terminate the transfer.

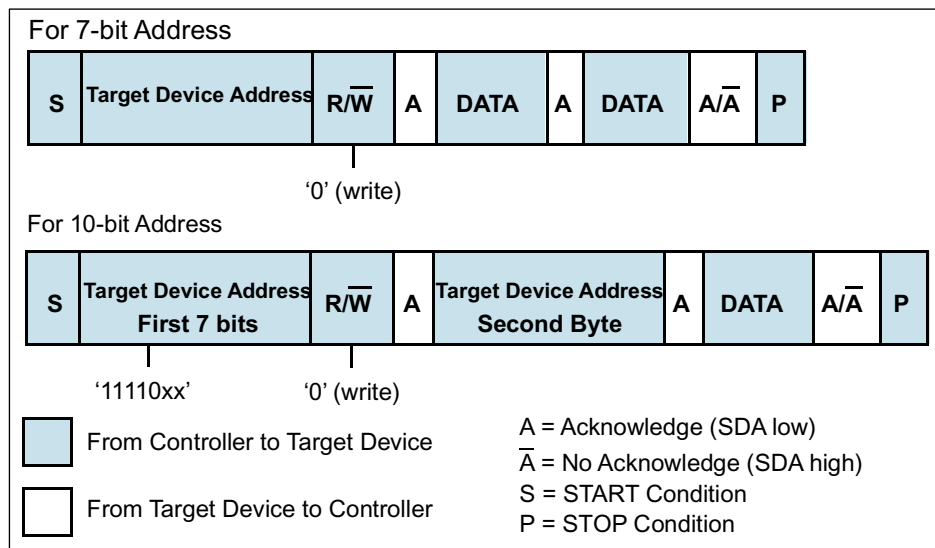
If the controller-transmitter is transmitting data as shown in [Figure 2-7](#), then the target-receiver responds to the controller-transmitter with an acknowledge pulse after every byte of data is received.



Note

In Ultra-Fast Mode, the target-receiver always responds with the No Acknowledge signal (NACK) for the Address and the write data from the controller.

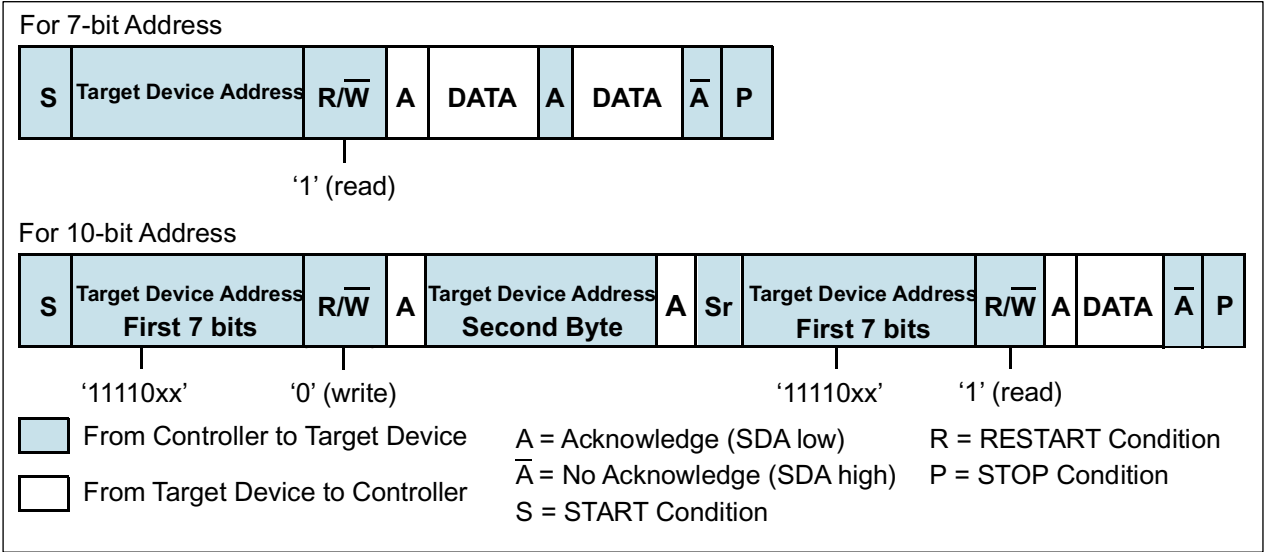
Figure 2-7 Controller-Transmitter Protocol



2.4.3.2 Controller-Receiver and Target-Transmitter

If the controller is receiving data as shown in [Figure 2-8](#), then the controller responds to the target-transmitter with an acknowledge pulse after a byte of data has been received, except for the last byte. This is the way the controller-receiver notifies the target-transmitter that this is the last byte. The target-transmitter relinquishes the SDA line after detecting the No Acknowledge (NACK) so that the controller can issue a STOP condition.

Figure 2-8 Controller-Receiver Protocol



When a controller does not want to relinquish the bus with a STOP condition, the controller can issue a RESTART condition. This is identical to a START condition except it occurs after the ACK pulse. Operating in controller mode, the DW_apb_i2c can then communicate with the same target using a transfer of a different direction. For a description of the combined format transactions that the DW_apb_i2c supports, see [“Combined Formats”](#) on page 38.

 **Note**

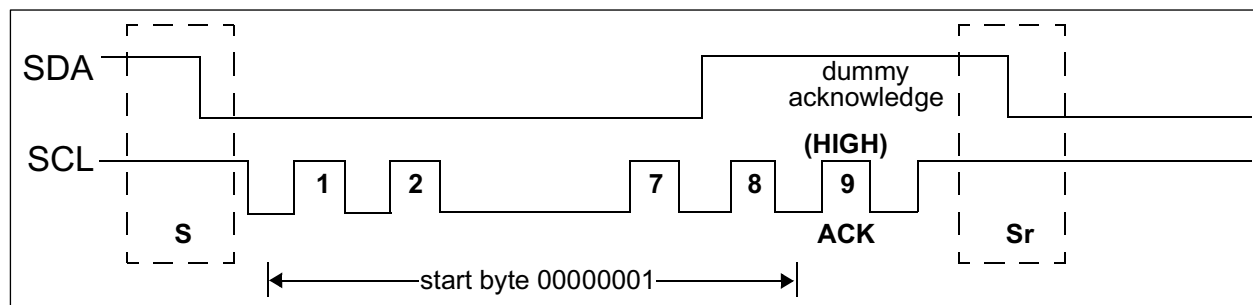
Before the target address register (IC_TAR) can be reprogrammed,

- DW_apb_i2c must be completely disabled if I2C_DYNAMIC_TAR_UPDATE = 0
- DW_apb_i2c must be inactive on the serial port if I2C_DYNAMIC_TAR_UPDATE = 1

2.4.4 START BYTE Transfer Protocol

The START BYTE transfer protocol is set up for systems that do not have an on-board dedicated I²C hardware module. When the DW_apb_i2c is addressed as a target, it always samples the I²C bus at the highest speed supported so that it never requires a START BYTE transfer. However, when DW_apb_i2c is a controller, it supports the generation of START BYTE transfers at the beginning of every transfer in case a target device requires it.

This protocol consists of seven zeros being transmitted followed by a 1, as illustrated in [Figure 2-9](#). This allows the processor that is polling the bus to under-sample the address phase until 0 is detected. Once the microcontroller detects a 0, it switches from the under sampling rate to the correct rate of the controller.

Figure 2-9 START BYTE Transfer

The START BYTE procedure is as follows:

1. Controller generates a START condition.
2. Controller transmits the START byte (0000 0001).
3. Controller transmits the ACK clock pulse. (Present only to conform with the byte handling format used on the bus)
4. No target sets the ACK signal to 0.
5. Controller generates a RESTART (R) condition.

A hardware receiver does not respond to the START BYTE because it is a reserved address and resets after the RESTART condition is generated.

2.5 Tx FIFO Management and START, STOP and RESTART Generation

When operating as a controller, the DW_apb_i2c component supports two modes of Tx FIFO management. You use the IC_EMPTYFIFO_HOLD_MASTER_EN parameter to select between these two modes:

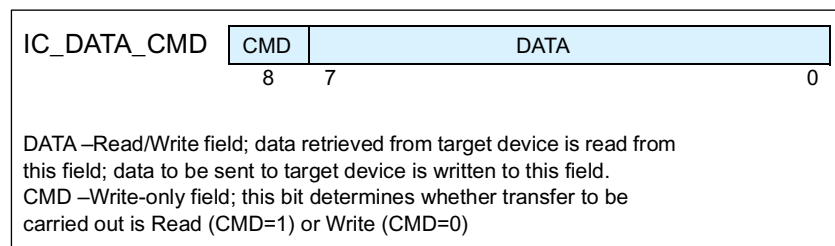
- IC_EMPTYFIFO_HOLD_MASTER_EN equals 0, illustrated in [Figure 2-10](#)
- IC_EMPTYFIFO_HOLD_MASTER_EN equals 1, illustrated in [Figure 2-13](#)

2.5.1 Tx FIFO Management When IC_EMPTYFIFO_HOLD_MASTER_EN = 0

When the value of IC_EMPTYFIFO_HOLD_MASTER_EN is 0, the component generates a STOP on the bus whenever the Tx FIFO becomes empty. If RESTART generation capability is enabled, the component generates a RESTART when the direction of the transfer in the Tx FIFO commands changes from Read to Write or vice-versa; if RESTART is not enabled, a STOP followed by a START is generated in this situation.

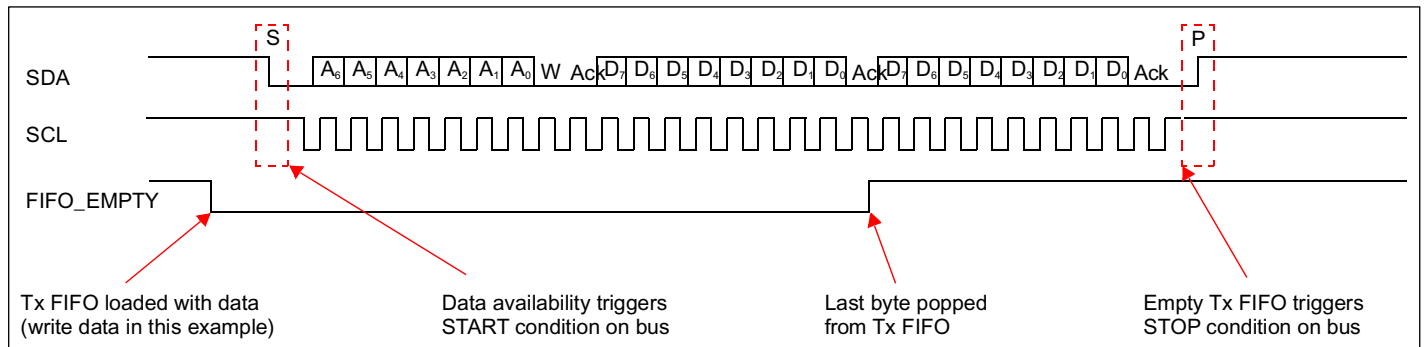
[Figure 2-10](#) shows the bits in the IC_DATA_CMD register if IC_EMPTYFIFO_HOLD_MASTER_EN = 0.

Figure 2-10 IC_DATA_CMD Register if IC_EMPTYFIFO_HOLD_MASTER_EN = 0

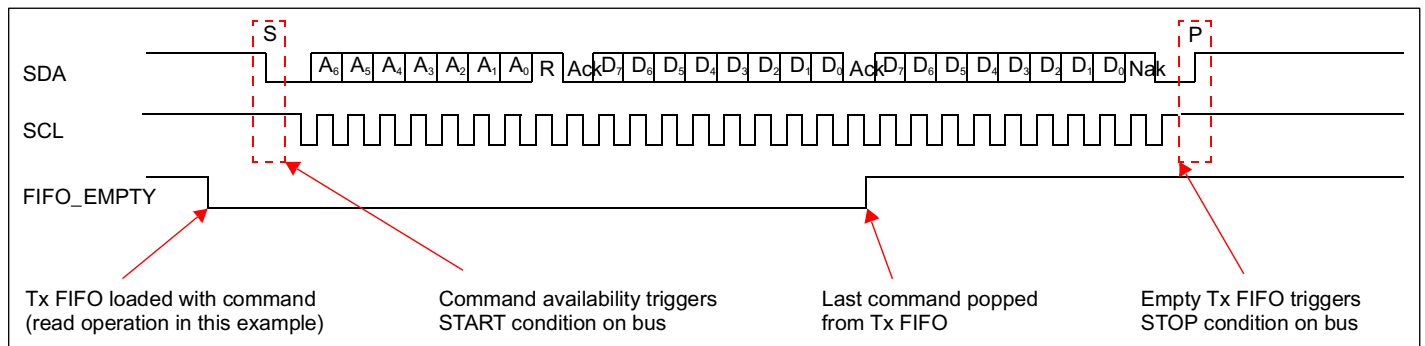


[Figure 2-11](#) shows a timing diagram that illustrates the behavior of the DW_apb_i2c when Tx FIFO becomes empty while operating as a controller transmitter when IC_EMPTYFIFO_HOLD_MASTER_EN=0.

Figure 2-11 Controller Transmitter — Tx FIFO Becomes Empty If IC_EMPTYFIFO_HOLD_MASTER_EN = 0



[Figure 2-12](#) shows a timing diagram that illustrates the behavior of the DW_apb_i2c when Tx FIFO becomes empty while operating as a controller receiver when IC_EMPTYFIFO_HOLD_MASTER_EN=0.

Figure 2-12 Controller Receiver — Tx FIFO Becomes Empty If IC_EMPTYFIFO_HOLD_MASTER_EN = 0**Note**

When IC_EMPTYFIFO_HOLD_MASTER_EN = 0, if the TX_EMPTY_CTRL bit of the IC_CON register is set to 1 and the transmit threshold (TX_THLD) is set to 0, then one I²C frame (consisting of multiple commands) always breaks into multiple frames based on the number of commands in each frame. This is because DW_apb_i2c issues a TX_EMPTY interrupt after the end of each data (TX_EMPTY_CTRL=1) is sent on the line and the data (TX_THLD=0) is pushed based on TX_EMPTY interrupt.

2.5.2 Tx FIFO Management When IC_EMPTYFIFO_HOLD_MASTER_EN = 1

When the value of IC_EMPTYFIFO_HOLD_MASTER_EN is 1, the component does not generate a STOP if the Tx FIFO becomes empty; in this situation the component holds the SCL line low, stalling the bus until a new entry is available in the Tx FIFO. A STOP condition is generated only when you specifically request it by setting bit 9 (Stop bit) of the command written to IC_DATA_CMD register.

Figure 2-13 shows the bits in the IC_DATA_CMD register if IC_EMPTYFIFO_HOLD_MASTER_EN = 1.

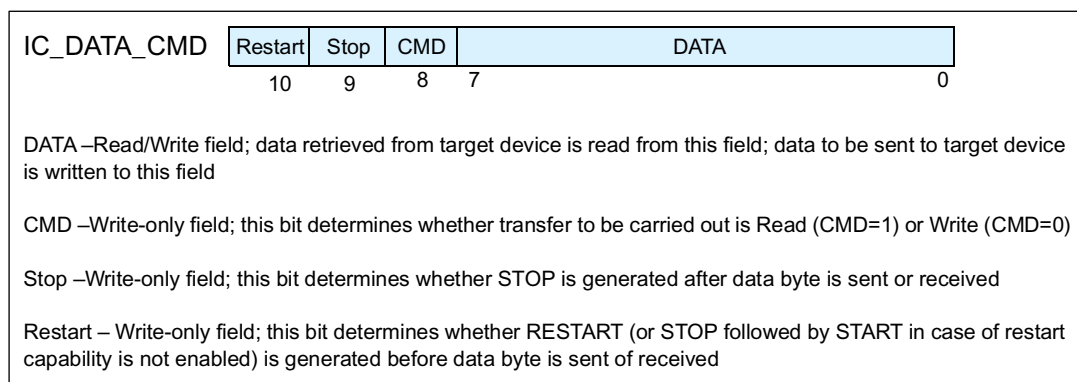
Figure 2-13 IC_DATA_CMD Register if IC_EMPTYFIFO_HOLD_MASTER_EN = 1

Figure 2-14 illustrates the behavior of the DW_apb_i2c when the Tx FIFO becomes empty while operating as a controller transmitter, as well as showing the generation of a STOP condition when IC_EMPTYFIFO_HOLD_MASTER_EN=1.

Figure 2-14 Controller Transmitter — Tx FIFO Empties/STOP Generation If IC_EMPTYFIFO_HOLD_MASTER_EN = 1

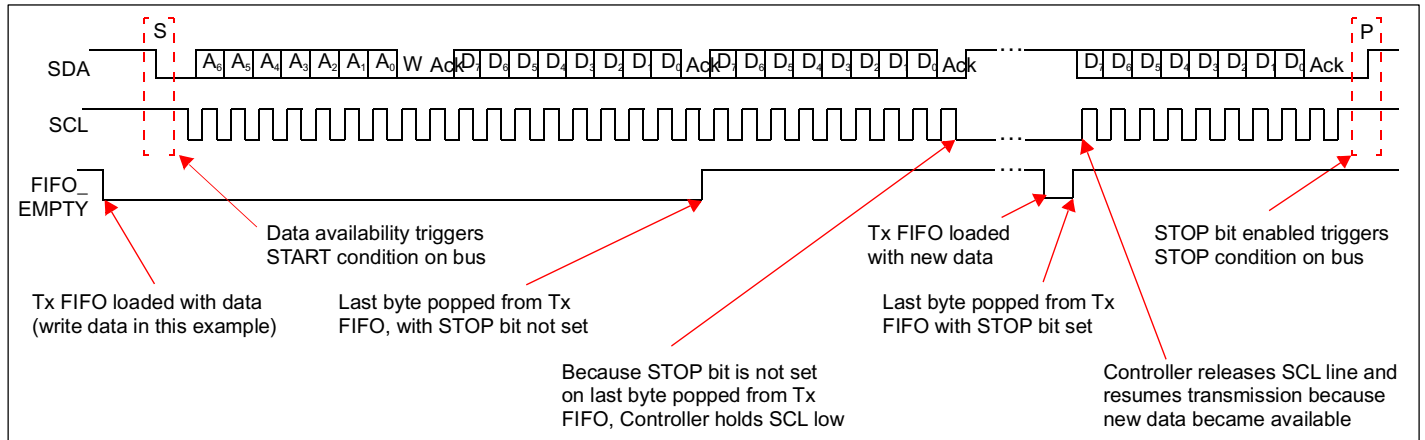


Figure 2-15 illustrates the behavior of the DW_apb_i2c when the Tx FIFO becomes empty while operating as a controller receiver, as well as showing the generation of a STOP condition when IC_EMPTYFIFO_HOLD_MASTER_EN=1.

Figure 2-15 Controller Receiver — Tx FIFO Empties/STOP Generation If IC_EMPTYFIFO_HOLD_MASTER_EN = 1

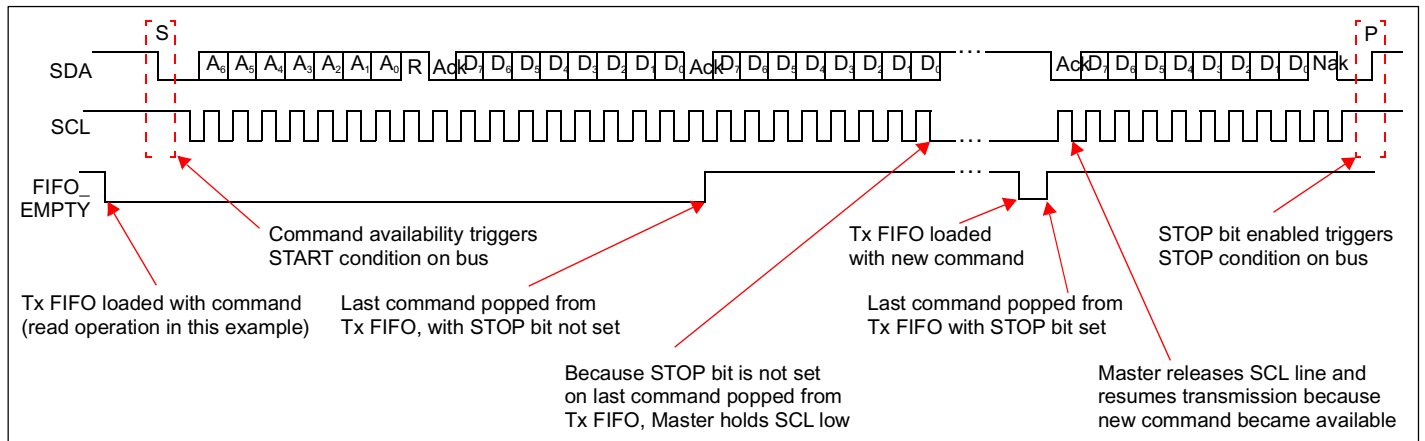


Figure 2-16 and Figure 2-17 illustrate configurations where you can control the generation of RESTART conditions on the I2C bus. If bit 10 (Restart) of the IC_DATA_CMD register is set and the restart capability is enabled (IC_RESTART_EN=1), a RESTART is generated before the data byte is written to or read from the target. If the restart capability is not enabled a STOP followed by a START is generated in place of the RESTART. Figure 2-16 illustrates this situation during operation as a controller transmitter.

Figure 2-16 Controller Transmitter — Restart Bit of IC_DATA_CMD Is Set (IC_EMPTYFIFO_HOLD_MASTER_EN = 1)

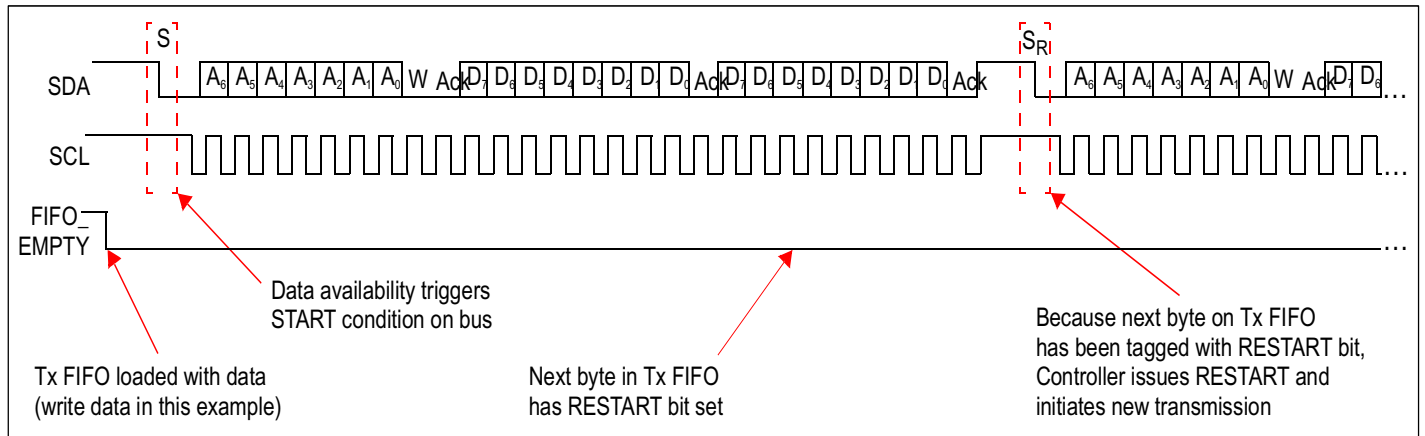


Figure 2-17 illustrates the same situation, but during operation as a controller receiver.

Figure 2-17 Controller Receiver — Restart Bit of IC_DATA_CMD Is Set (IC_EMPTYFIFO_HOLD_MASTER_EN = 1)

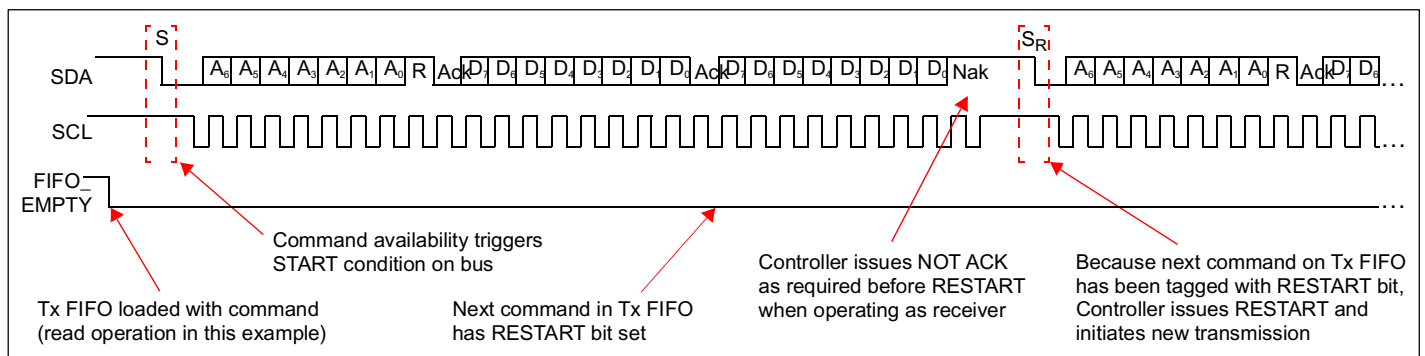


Figure 2-18 illustrates operation as a controller transmitter where the Stop bit of the IC_DATA_CMD register is set and the Tx FIFO is not empty (IC_EMPTYFIFO_HOLD_MASTER_EN=1).

Figure 2-18 Controller Transmitter — Stop Bit of IC_DATA_CMD Set/Tx FIFO Not Empty (IC_EMPTYFIFO_HOLD_MASTER_EN=1)

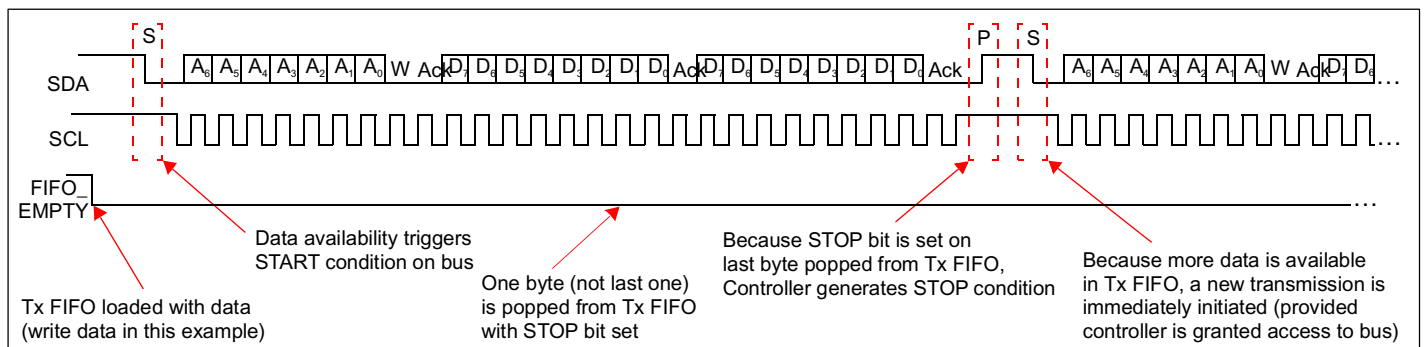


Figure 2-19 illustrates operation as a controller transmitter where the first byte loaded into the Tx FIFO is allowed to go empty with the Restart bit set (IC_EMPTYFIFO_HOLD_MASTER_EN=1).

Figure 2-19 Controller Transmitter — First Byte Loaded Into Tx FIFO Allowed to Empty, Restart Bit Set (IC_EMPTYFIFO_HOLD_MASTER_EN=1)

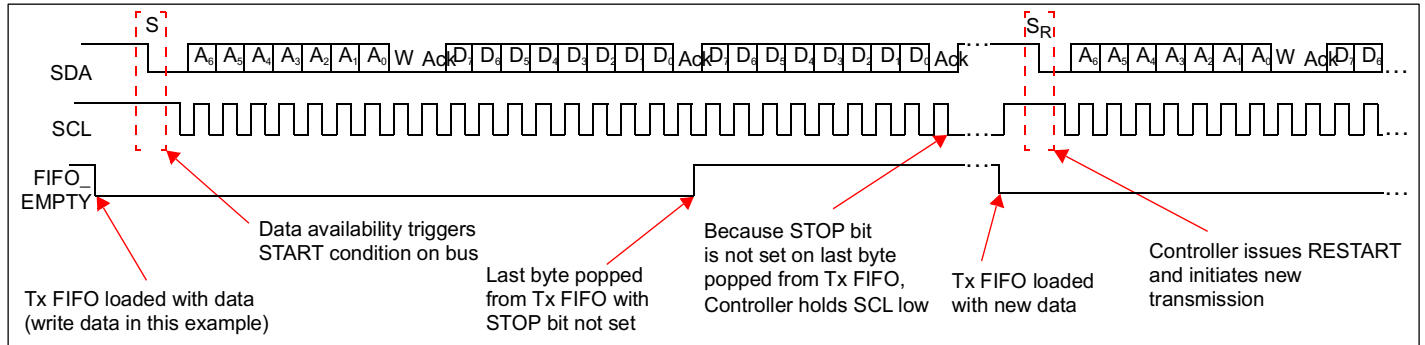


Figure 2-20 illustrates operation as a controller receiver where the Stop bit of the IC_DATA_CMD register is set and the Tx FIFO is not empty (IC_EMPTYFIFO_HOLD_MASTER_EN=1).

Figure 2-20 Controller Receiver — Stop Bit of IC_DATA_CMD Set/Tx FIFO Not Empty (IC_EMPTYFIFO_HOLD_MASTER_EN=1 and IC_ULTRA_FAST_MODE=0)

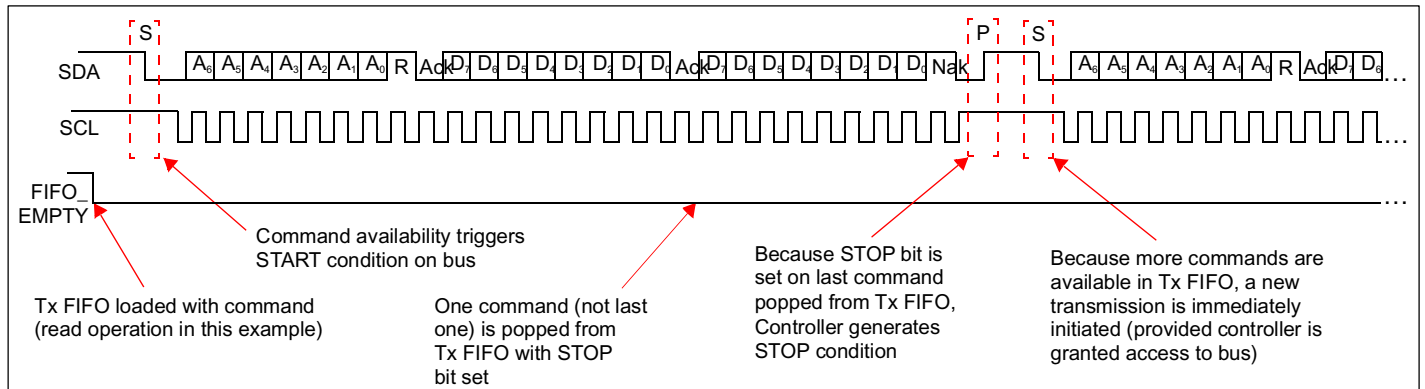
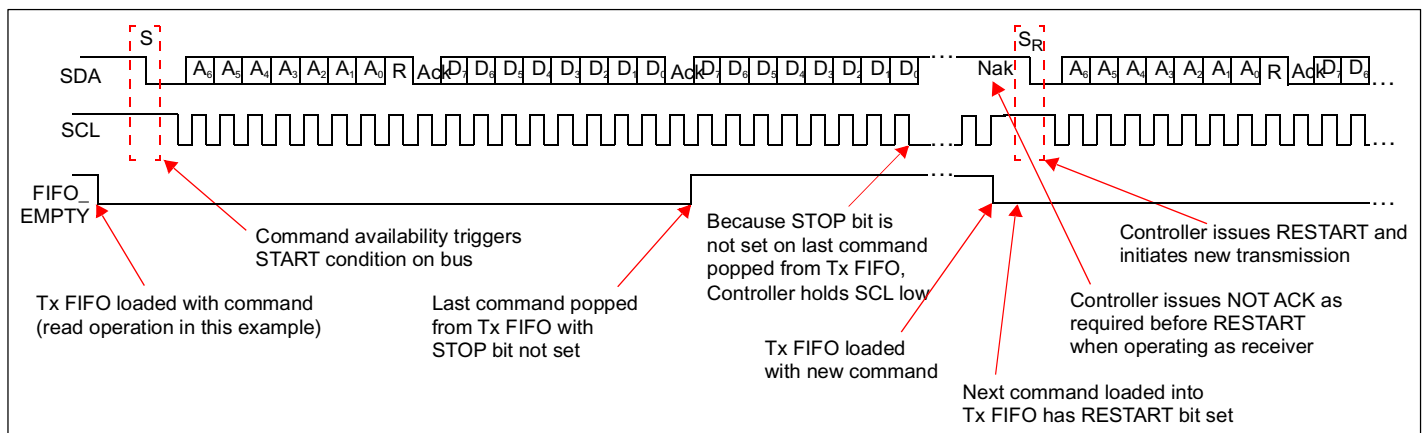


Figure 2-21 illustrates operation as a controller receiver where the first command loaded after the Tx FIFO is allowed to empty and the Restart bit is set (IC_EMPTYFIFO_HOLD_MASTER_EN=1).

Figure 2-21 Controller Receiver — First Command Loaded After Tx FIFO Allowed to Empty/Restart Bit Set (IC_EMPTYFIFO_HOLD_MASTER_EN=1 and IC_ULTRA_FAST_MODE=0)



2.6 Multiple Controller Arbitration

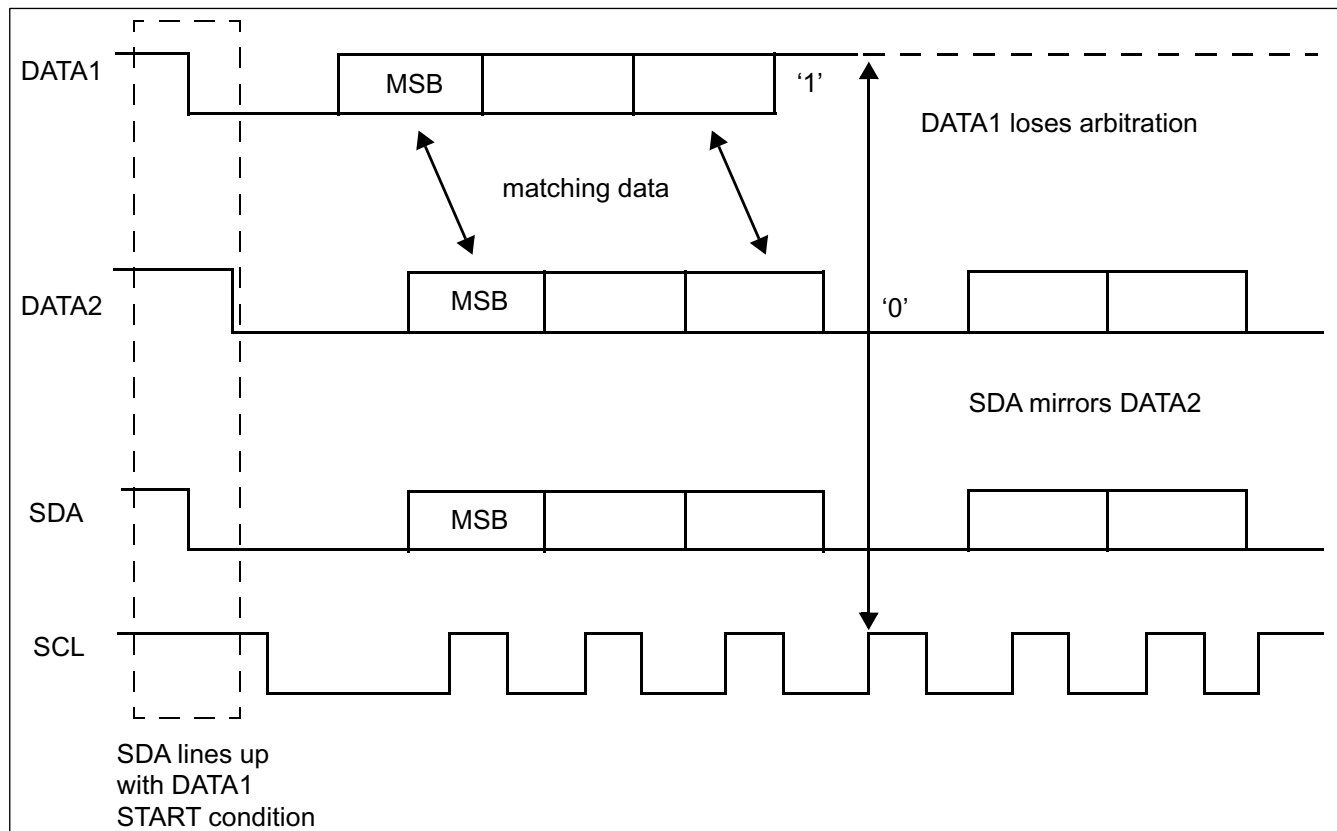
The DW_apb_i2c bus protocol allows multiple controllers to reside on the same bus. If there are two controllers on the same I²C-bus, there is an arbitration procedure if both try to take control of the bus at the same time by generating a START condition at the same time. Once a controller (for example, a microcontroller) has control of the bus, no other controller can take control until the first controller sends a STOP condition and places the bus in an idle state.

Arbitration takes place on the SDA line, while the SCL line is 1. The controller, which transmits a 1 while the other controller transmits 0, loses arbitration and turns off its data output stage. The controller that lost arbitration can continue to generate clocks until the end of the byte transfer. If both controllers are addressing the same target device, the arbitration could go into the data phase.

Upon detecting that it has lost arbitration to another controller, the DW_apb_i2c stops generating SCL (ic_clk_oe).

Figure 2-22 illustrates the timing of when two controllers are arbitrating on the bus.

Figure 2-22 Multiple Controller Arbitration



For high-speed mode, the arbitration cannot go into the data phase because each controller is programmed with a unique high-speed controller code. This 8-bit code is defined by the system designer and is set by writing to the high speed Controller Mode Code Address Register, IC_HS_MADDR. Because the codes are unique, only one controller can win arbitration, which occurs by the end of the transmission of the high-speed controller code.

Control of the bus is determined by address or controller code and data sent by competing controllers, so there is no central controller nor any order of priority on the bus.

Arbitration is not allowed between the following conditions:

- A RESTART condition and a data bit
- A STOP condition and a data bit
- A RESTART condition and a STOP condition

Target devices are not involved in the arbitration process.

**Note**

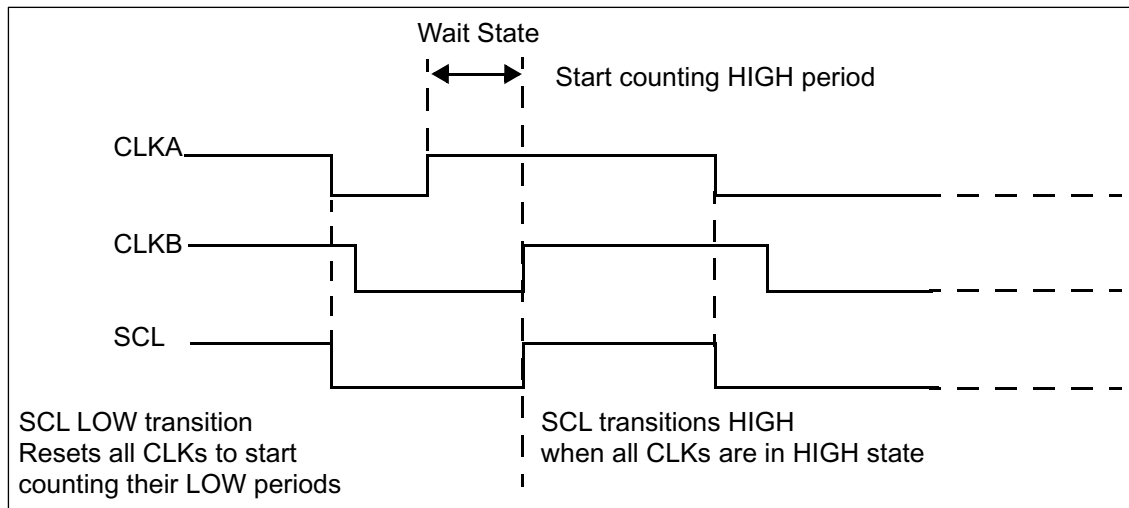
Multi-controller arbitration is not applicable in Ultra-Fast mode (IC_ULTRA_FAST_MODE=1) as single controller is present.

2.7 Clock Synchronization

When two or more controllers try to transfer information on the bus at the same time, they must arbitrate and synchronize the SCL clock. All controllers generate their own clock to transfer messages. Data is valid only during the high period of SCL clock. Clock synchronization is performed using the wired-AND connection to the SCL signal. When the controller transitions the SCL clock to 0, the controller starts counting the low time of the SCL clock and transitions the SCL clock signal to 1 at the beginning of the next clock period. However, if another controller is holding the SCL line to 0, then the controller goes into a HIGH wait state until the SCL clock line transitions to 1.

All controllers then count off their high time, and the controller with the shortest high time transitions the SCL line to 0. The controllers then counts out their low time and the one with the longest low time forces the other controller into a HIGH wait state. Therefore, a synchronized SCL clock is generated, which is illustrated in [Figure 2-23](#). Optionally, targets may hold the SCL line low to slow down the timing on the I²C bus.

Figure 2-23 Multi-Controller Clock Synchronization



Note

Clock Synchronization is not supported in Ultra-Fast Mode (IC_ULTRA_FAST_MODE=1) as single controller is present in the Ultra-Fast Mode system.

2.8 Operation Modes

This section provides information on operation modes.

**Note**

It is important to note that the DW_apb_i2c should only be set to operate as an I²C Controller, or I²C Target, but not both simultaneously. This is achieved by ensuring that bit 6 (IC_SLAVE_DISABLE) and 0 (MASTER_MODE) of the IC_CON register are never set to 0 and 1, respectively.

2.8.1 Target Mode Operation

This section discusses target mode procedures.

2.8.1.1 Initial Configuration

To use the DW_apb_i2c as a target, perform the following steps:

1. Disable the DW_apb_i2c by writing a '0' to bit 0 of the IC_ENABLE register.
2. Write to the IC_SAR register (bits 9:0) to set the target address. This is the address to which the DW_apb_i2c responds.
3. Write to the IC_CON register to specify which type of addressing is supported (7- or 10-bit by setting bit 3). Enable the DW_apb_i2c in target-only mode by writing a '0' into bit 6 (IC_SLAVE_DISABLE) and a '0' to bit 0 (MASTER_MODE).

**Note**

Targets and controllers do not have to be programmed with the same type of addressing 7- or 10-bit address. For instance, a target can be programmed with 7-bit addressing and a controller with 10-bit addressing, and vice versa.

4. Enable the DW_apb_i2c by writing a '1' in bit 0 of the IC_ENABLE register.

**Note**

Depending on the reset values chosen, steps 2 and 3 may not be necessary because the reset values can be configured. For instance, if the device is only going to be a controller, there would be no need to set the target address because you can configure DW_apb_i2c to have the target disabled after reset and to enable the controller after reset. The values stored are static and do not need to be reprogrammed if the DW_apb_i2c is disabled.

**Attention**

It is recommended that the DW_apb_i2c Target be brought out of reset only when the I²C bus is IDLE. De-asserting the reset when a transfer is ongoing on the bus causes internal synchronization flip-flops used to synchronize SDA and SCL to toggle from a reset value of 1 to the actual value on the bus. This can result in SDA toggling from 1 to 0 while SCL is 1, thereby causing a false START condition to be detected by the DW_apb_i2c Target. This scenario can also be avoided by configuring the DW_apb_i2c with IC_SLAVE_DISABLE = 1 and IC_MASTER_MODE = 1 so that the Target interface is disabled after reset. It can then be enabled by programming IC_CON[0] = 0 and IC_CON[6] = 0 after the internal SDA and SCL have synchronized to the value on the bus; this takes approximately 6 ic_clk cycles after reset de-assertion.

2.8.1.2 Target-Transmitter Operation for a Single Byte

When another I²C controller device on the bus addresses the DW_apb_i2c and requests data, the DW_apb_i2c acts as a target-transmitter and the following steps occur:

1. The other I²C controller device initiates an I²C transfer with an address that matches the target address in the IC_SAR register of the DW_apb_i2c.
2. The DW_apb_i2c acknowledges the sent address and recognizes the direction of the transfer to indicate that it is acting as a target-transmitter.
3. The DW_apb_i2c asserts the RD_REQ interrupt (bit 5 of the IC_RAW_INTR_STAT register) and holds the SCL line low. It is in a wait state until software responds.

If the RD_REQ interrupt has been masked, due to IC_INTR_MASK[5] register (M_RD_REQ bit field) being set to 0, then it is recommended that a hardware and/or software timing routine be used to instruct the CPU to perform periodic reads of the IC_RAW_INTR_STAT register.

- a. Reads that indicate IC_RAW_INTR_STAT[5] (R_RD_REQ bit field) being set to 1 must be treated as the equivalent of the RD_REQ interrupt being asserted.
- b. Software must then act to satisfy the I²C transfer.
- c. The timing interval used should be in the order of 10 times the fastest SCL clock period the DW_apb_i2c can handle. For example, for 400 kb/s, the timing interval is 25µs.

**Note**

The value of 10 is recommended here because this is approximately the amount of time required for a single byte of data transferred on the I²C bus.

4. If there is any data remaining in the Tx FIFO before receiving the read request, then the DW_apb_i2c asserts a TX_ABORT interrupt (bit 6 of the IC_RAW_INTR_STAT register) to flush the old data from the TX FIFO.

**Note**

Because the DW_apb_i2c's Tx FIFO is forced into a flushed/reset state whenever a TX_ABRT event occurs, it is necessary for software to release the DW_apb_i2c from this state by reading the IC_CLR_TX_ABRT register before attempting to write into the Tx FIFO. See register IC_RAW_INTR_STAT for more details.

If the TX_ABRT interrupt has been masked, due to of IC_INTR_MASK[6] register (M_TX_ABRT bit field) being set to 0, then it is recommended that re-using the timing routine (described in the previous step), or a similar one, be used to read the IC_RAW_INTR_STAT register.

- a. Reads that indicate bit 6 (R_TX_ABRT) being set to 1 must be treated as the equivalent of the TX_ABRT interrupt being asserted.
 - b. There is no further action required from software.
 - c. The timing interval used should be similar to that described in the previous step for the IC_RAW_INTR_STAT[5] register.
5. Software writes to the IC_DATA_CMD register with the data to be written (by writing a '0' in bit 8).
 6. Software must clear the RD_REQ and TX_ABRT interrupts (bits 5 and 6, respectively) of the IC_RAW_INTR_STAT register before proceeding.

If the RD_REQ and/or TX_ABRT interrupts have been masked, then clearing of the IC_RAW_INTR_STAT register is already been performed when either the R_RD_REQ or R_TX_ABRT bit has been read as 1.

7. The DW_apb_i2c releases the SCL and transmits the byte.
8. The controller may hold the I²C bus by issuing a RESTART condition or release the bus by issuing a STOP condition.

**Note**

Target-Transmitter Operation for a Single Byte is not applicable in Ultra-Fast Mode as Read transfers are not supported.

2.8.1.3 Target-Receiver Operation for a Single Byte

When another I²C controller device on the bus addresses the DW_apb_i2c and is sending data, the DW_apb_i2c acts as a target-receiver and the following steps occur:

1. The other I²C controller device initiates an I²C transfer with an address that matches the DW_apb_i2c's target address in the IC_SAR register.
2. The DW_apb_i2c acknowledges the sent address and recognizes the direction of the transfer to indicate that the DW_apb_i2c is acting as a target-receiver.
3. DW_apb_i2c receives the transmitted byte and places it in the receive buffer.

**Note**

If the Rx FIFO is completely filled with data when a byte is pushed, and `IC_RX_FULL_HLD_BUS_EN = 0`, then an overflow occurs and the DW_apb_i2c continues with subsequent I²C transfers. Because a NACK is not generated, software must recognize the overflow when indicated by the DW_apb_i2c (by the `R_RX_OVER` bit in the `IC_INTR_STAT` register) and take appropriate actions to recover from lost data. Hence, there is a real time constraint on software to service the Rx FIFO before the latter overflows, as there is no way to re-apply pressure to the remote transmitting controller. You must select a deep enough Rx FIFO depth to satisfy the interrupt service interval of the system.

If the Rx FIFO is completely filled with data when a byte is pushed, and `IC_RX_FULL_HLD_BUS_EN = 1`, and bit 9 of the `IC_CON` register (`RX_FIFO_FULL_HLD_CTRL`) is programmed to HIGH, then the DW_apb_i2c target holds the I²C SCL line low until the Rx FIFO has some space, and then continues with the next read request.

4. DW_apb_i2c asserts the `RX_FULL` interrupt (`IC_RAW_INTR_STAT[2]` register).

If the `RX_FULL` interrupt has been masked, due to setting `IC_INTR_MASK[2]` register to 0 or setting `IC_TX_TL` to a value larger than 0, then it is recommended that a timing routine (described in “[Target-Transmitter Operation for a Single Byte](#)” on page 53) be implemented for periodic reads of the `IC_STATUS` register. Reads of the `IC_STATUS` register, with bit 3 (`RFNE`) set at 1, must then be treated by software as the equivalent of the `RX_FULL` interrupt being asserted.

5. Software may read the byte from the `IC_DATA_CMD` register (bits 7:0).
6. The other controller device may hold the I²C bus by issuing a RESTART condition, or release the bus by issuing a STOP condition.

2.8.1.4 Target-Transfer Operation For Bulk Transfers

In the standard I²C protocol, all transactions are single byte transactions and the programmer responds to a remote controller read request by writing one byte into the target’s TX FIFO. When a target (target-transmitter) is issued with a read request (`RD_REQ`) from the remote controller (controller-receiver), at a minimum there should be at least one entry placed into the target-transmitter’s TX FIFO. DW_apb_i2c is designed to handle more data in the TX FIFO so that subsequent read requests can take that data without raising an interrupt to get more data. Ultimately, this eliminates the possibility of significant latencies being incurred between raising the interrupt for data each time had there been a restriction of having only one entry placed in the TX FIFO.

This mode only occurs when DW_apb_i2c is acting as a target-transmitter. If the remote controller acknowledges the data sent by the target-transmitter and there is no data in the target’s TX FIFO, the DW_apb_i2c holds the I²C SCL line low while it raises the read request interrupt (`RD_REQ`) and waits for data to be written into the TX FIFO before it can be sent to the remote controller.

If the `RD_REQ` interrupt is masked, due to bit 5 (`M_RD_REQ`) of the `IC_INTR_STAT` register being set to 0, then it is recommended that a timing routine be used to activate periodic reads of the `IC_RAW_INTR_STAT` register. Reads of `IC_RAW_INTR_STAT` that return bit 5 (`R_RD_REQ`) set to 1 must be treated as the equivalent of the `RD_REQ` interrupt referred to in this section. This timing routine is similar to that described in “[Target-Transmitter Operation for a Single Byte](#)” on page 53.

The `RD_REQ` interrupt is raised upon a read request, and like interrupts, must be cleared when exiting the interrupt service handling routine (ISR). The ISR allows you to either write 1 byte or more than 1 byte into

the Tx FIFO. During the transmission of these bytes to the controller, if the controller acknowledges the last byte, then the target must raise the RD_REQ again because the controller is requesting for more data.

If the programmer knows in advance that the remote controller is requesting a packet of n bytes, then when another controller addresses DW_apb_i2c and requests data, the Tx FIFO could be written with n number bytes and the remote controller receives it as a continuous stream of data. For example, the DW_apb_i2c target continues to send data to the remote controller as long as the remote controller is acknowledging the data sent and there is data available in the Tx FIFO. There is no need to hold the SCL line low or to issue RD_REQ again.

If the remote controller is to receive n bytes from the DW_apb_i2c but the programmer wrote a number of bytes larger than n to the Tx FIFO, then when the target finishes sending the requested n bytes, it clears the Tx FIFO and ignores any excess bytes.

The DW_apb_i2c generates a transmit terminate (TX_ABRT) event to indicate the clearing of the Tx FIFO in this example. At the time an ACK/NACK is expected, if a NACK is received, then the remote controller has all the data it wants. At this time, a flag is raised within the target's state machine to clear the leftover data in the Tx FIFO. This flag is transferred to the processor bus clock domain where the FIFO exists and the contents of the Tx FIFO is cleared at that time.

**Note**

Target Transmitter Operation for Bulk Transfers is not applicable in Ultra-Fast Mode (IC_ULTRA_FAST_MODE=1) as Controller Read Transfers are not supported.

2.8.2 Controller Mode Operation

This section discusses controller mode procedures.

2.8.2.1 Initial Configuration

The initial configuration procedure for Controller Mode Operation depends on the configuration parameter I2C_DYNAMIC_TAR_UPDATE. When set to “Yes” (1), the target address and address format can be changed dynamically without having to disable DW_apb_i2c. This parameter only applies to when DW_apb_i2c is acting as a controller because the target requires the component to be disabled before any changes can be made to the address. For more information about this parameter, see [Chapter 3, “Parameter Descriptions”](#). For more information about how this parameter affects the IC_TAR register, see [Chapter 5, “Register Descriptions”](#).

The procedures are very similar and are only different with regard to where the IC_10BITADDR_MASTER bit is set (either bit 4 of IC_CON register or bit 12 of IC_TAR register).

2.8.2.1.1 I2C_DYNAMIC_TAR_UPDATE = 0

To use the DW_apb_i2c as a controller when the I2C_DYNAMIC_TAR_UPDATE configuration parameter is set to “No” (0), perform the following steps:

1. Disable the DW_apb_i2c by writing 0 to bit 0 of the IC_ENABLE register.
2. Write to the IC_CON register to set the maximum speed mode supported (bits 2:1) and the desired speed of the DW_apb_i2c controller-initiated transfers, either 7-bit or 10-bit addressing (bit 4). Ensure that bit 6 (IC_SLAVE_DISABLE) is written with a ‘1’ and bit 0 (MASTER_MODE) is written with a ‘1’.

**Note**

Target and controllers do not have to be programmed with the same type of addressing 7- or 10-bit address. For instance, a target can be programmed with 7-bit addressing and a controller with 10-bit addressing, and vice versa.

3. Write to the IC_TAR register the address of the I²C device to be addressed (bits 9:0). This register also indicates whether a General Call or a START BYTE command is going to be performed by I²C.
4. *Only applicable for high-speed mode transfers.* Write to the IC_HS_MADDR register the desired controller code for the DW_apb_i2c. The controller code is programmer-defined.
5. Enable the DW_apb_i2c by writing a 1 to bit 0 of the IC_ENABLE register.
6. Now write transfer direction and data to be sent to the IC_DATA_CMD register. If the IC_DATA_CMD register is written before the DW_apb_i2c is enabled, the data and commands are lost as the buffers are kept cleared when DW_apb_i2c is disabled.

This step generates the START condition and the address byte on the DW_apb_i2c. Once DW_apb_i2c is enabled and there is data in the TX FIFO, DW_apb_i2c starts reading the data.

**Note**

Depending on the reset values chosen, steps 2, 3, 4, and 5 may not be necessary because the reset values can be configured. The values stored are static and do not need to be reprogrammed if the DW_apb_i2c is disabled, with the exception of the transfer direction and data.

2.8.2.1.2 I2C_DYNAMIC_TAR_UPDATE = 1

To use the DW_apb_i2c as a controller when the I2C_DYNAMIC_TAR_UPDATE configuration parameter is set to “Yes” (1), perform the following steps:

1. Disable the DW_apb_i2c by writing 0 to bit 0 of the IC_ENABLE register.
2. Write to the IC_CON register to set the maximum speed mode supported for target operation (bits 2:1) and to specify whether the DW_apb_i2c starts its transfers in 7/10 bit addressing mode when the device is a target (bit 3).
3. Write to the IC_TAR register the address of the I²C device to be addressed. It also indicates whether a General Call or a START BYTE command is going to be performed by I²C. The desired speed of the DW_apb_i2c controller-initiated transfers, either 7-bit or 10-bit addressing, is controlled by the IC_10BITADDR_MASTER bit field (bit 12).
4. *Only applicable for high-speed mode transfers.* Write to the IC_HS_MADDR register the desired controller code for the DW_apb_i2c. The controller code is programmer-defined.
5. Enable the DW_apb_i2c by writing a 1 to bit 0 of the IC_ENABLE register.
6. Now write the transfer direction and data to be sent to the IC_DATA_CMD register. If the IC_DATA_CMD register is written before the DW_apb_i2c is enabled, the data and commands are lost as the buffers are kept cleared when DW_apb_i2c is not enabled.

**Note**

When a DW_apb_i2c Controller is configured with IC_EMPTYFIFO_HOLD_MASTER_EN = 0, then for multiple I²C transfers, perform additional writes to the Tx FIFO such that the Tx FIFO does not become empty during the I²C transaction. If the Tx FIFO is completely emptied at any stage, then further writes to the Tx FIFO results in an independent I²C transaction.

2.8.2.2 Dynamic IC_TAR or IC_10BITADDR_MASTER Update

The DW_apb_i2c supports dynamic updating of the IC_TAR (bits 9:0) and IC_10BITADDR_MASTER (bit 12) bit fields of the IC_TAR register. In order to perform a dynamic update of the IC_TAR register, the I2C_DYNAMIC_TAR_UPDATE configuration parameter must be set to Yes (1). You can dynamically write to the IC_TAR register provided the software ensures that there are no other commands in the Tx FIFO that use the existing TAR address. If the software does not ensure this, then IC_TAR should be re-programmed only if the following conditions are met:

- DW_apb_i2c is not enabled (IC_ENABLE[0]=0);
OR
DW_apb_i2c is enabled (IC_ENABLE[0]=1); AND DW_apb_i2c is NOT engaged in any Controller (tx, rx) operation (IC_STATUS[5]=0); AND DW_apb_i2c is enabled to operate in Controller mode (IC_CON[0]=1); AND there are NO entries in the Tx FIFO (IC_STATUS[2]=1);¹

You can change the TAR address dynamically without losing the bus, only if the following conditions are met.

- DW_apb_i2c is enabled (IC_ENABLE[0]=1); AND IC_EMPTYFIFO_HOLD_MASTER_EN configuration parameter is set to 1; AND DW_apb_i2c is enabled to operate in Controller mode (IC_CON[0]=1); AND there are NO entries in the Tx FIFO and the controller is in HOLD state (IC_INTR_STAT[13]=1);



DW_apb_i2c uses the TAR address if either of the following conditions is true:

- The command has either RESTART or STOP bit set.
 - The direction is changed in commands with a read command following a write command or vice versa
- The updated TAR address comes into effect only when the next START or RESTART occurs on the bus.

2.8.2.3 Controller Transmit and Controller Receive

The DW_apb_i2c supports switching back and forth between reading and writing dynamically. To transmit data, write the data to be written to the lower byte of the I²C Rx/Tx Data Buffer and Command Register (IC_DATA_CMD). The CMD bit [8] should be written to 0 for I²C write operations. Subsequently, a read command may be issued by writing “don’t cares” to the lower byte of the IC_DATA_CMD register, and a 1 should be written to the CMD bit. The DW_apb_i2c controller continues to initiate transfers as long as there are commands present in the transmit FIFO. If the transmit FIFO becomes empty – depending on the value of IC_EMPTYFIFO_HOLD_MASTER_EN, the controller either inserts a STOP condition after completing the current transfers, or it checks to see if IC_DATA_CMD[9] is set to 1.

- If set to 1, it issues a STOP condition after completing the current transfer.
- If set to 0, it holds SCL low until next command is written to the transmit FIFO.

For more details, see [“Tx FIFO Management and START, STOP and RESTART Generation”](#) on page 44.

1. If the software or application is aware the DW_apb_i2c is not using the TAR address for the pending commands in the Tx FIFO, then it is possible to update the TAR address even while the Tx FIFO has entries (IC_STATUS[2]= 0).

**Note**

Controller Receiver Mode is not supported in Ultra-Fast Mode (IC_ULTRA_FAST_MODE=1) as Controller Read transfers are not supported.

2.8.3 Disabling DW_apb_i2c

The register IC_ENABLE_STATUS is added to allow software to unambiguously determine when the hardware has completely shutdown in response to bit 0 of the IC_ENABLE register being set from 1 to 0. Only one register is required to be monitored, as opposed to monitoring two registers (IC_STATUS and IC_RAW_INTR_STAT) which is a requirement for DW_apb_i2c versions 1.05a or earlier.

**Note**

When IC_EMPTYFIFO_HOLD_MASTER_EN = 1, the DW_apb_i2c Controller can be disabled only if the current command being processed—when the ic_enable de-assertion occurs—has the STOP bit set to 1.

When an attempt is made to disable the DW_apb_i2c controller while processing a command without the STOP bit set, the DW_apb_i2c controller continues to remain active, holding the SCL line low until a new command is received in the Tx FIFO.

When IC_EMPTYFIFO_HOLD_MASTER_EN = 1 and the DW_apb_i2c controller is processing a command without the STOP bit set, you can issue the ABORT (IC_ENABLE[1]) to relinquish the I²C bus and then disable DW_apb_i2c.

2.8.3.1 Procedure

1. Define a timer interval (t_{i2c_poll}) equal to the 10 times the signaling period for the highest I²C transfer speed used in the system and supported by DW_apb_i2c. For example, if the highest I²C transfer mode is 400 kb/s, then this t_{i2c_poll} is 25 μ s.
2. Define a maximum time-out parameter, MAX_T_POLL_COUNT, such that if any repeated polling operation exceeds this maximum value, an error is reported.
3. Execute a blocking thread/process/function that prevents any further I²C controller transactions to be started by software, but allows any pending transfers to be completed.

**Note**

This step can be ignored if DW_apb_i2c is programmed to operate as an I²C target only.

4. The variable POLL_COUNT is initialized to zero.
5. Set bit 0 of the IC_ENABLE register to 0.
6. Read the IC_ENABLE_STATUS register and test the IC_EN bit (bit 0). Increment POLL_COUNT by one. If POLL_COUNT $\geq$ MAX_T_POLL_COUNT, exit with the relevant error code.
7. If IC_ENABLE_STATUS[0] is 1, then sleep for t_{i2c_poll} and proceed to the previous step. Otherwise, exit with a relevant success code.

2.8.4 Terminating I²C Transfers

The ABORT control bit of the IC_ENABLE register allows the software to relinquish the I²C bus before completing the issued transfer commands from the Tx FIFO. In response to an ABORT request, the

controller issues the STOP condition over the I²C bus, followed by Tx FIFO flush. Terminating the transfer is allowed only in controller mode of operation.

2.8.4.1 Procedure

1. Stop filling the Tx FIFO (IC_DATA_CMD) with new commands.
2. When operating in DMA mode, disable the transmit DMA by setting TDMAE to 0.
3. Set bit 1 of the IC_ENABLE register (ABORT) to 1.
4. Wait for the M_TX_ABRT interrupt.
5. Read the IC_TX_ABRT_SOURCE register to identify the source as ABRT_USER_ABRT.

2.9 Spike Suppression

The DW_apb_i2c contains programmable spike suppression logic that match requirements imposed by the I²C Bus Specification for Standard, Fast, High or U^Fm Speed modes (tSP).

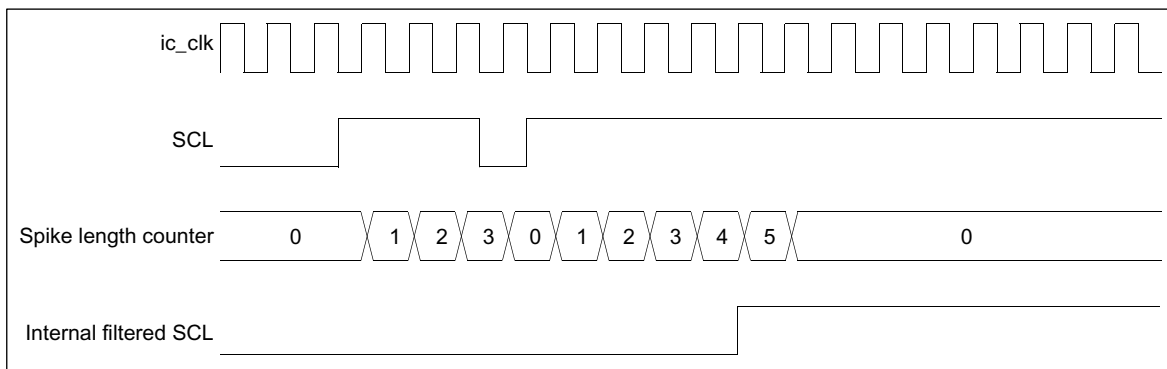
This logic is based on counters that monitor the input signals (SCL and SDA), checking if they remain stable for a predetermined amount of ic_clk cycles before they are sampled internally. There is one separate counter for each signal (SCL and SDA). The number of ic_clk cycles can be programmed and should be calculated taking into account the frequency of ic_clk and the relevant spike length specification.

Each counter is started whenever its input signal changes its value. Depending on the behavior of the input signal, one of the following scenarios occurs:

- The input signal remains unchanged until the counter reaches its count limit value. When this happens, the internal version of the signal is updated with the input value, and the counter is reset and stopped. The counter is not restarted until a new change on the input signal is detected.
- The input signal changes again before the counter reaches its count limit value. When this happens, the counter is reset and stopped, but the internal version of the signal is not updated. The counter remains stopped until a new change on the input signal is detected.

The timing diagram in [Figure 2-24](#) illustrates the behavior described earlier.

Figure 2-24 Spike Suppression Example



The count limit value used in this example is 5 and is calculated for a 10 ns ic_clk period and for SS/FS operation (50 ns spike suppression).



Note

There is a 2-stage synchronizer on the SCL input, but for the sake of simplicity this synchronization delay is not included in the timing diagram in [Figure 2-24](#).

The I²C Bus Specification calls for different maximum spike lengths according to the operating mode — 50 ns for SS and FS; 10 ns for high speed, 10 ns for U^Fm, so three registers are required to store the values needed for each case:

- Register IC_FS_SPKLEN holds the maximum spike length for SS and FS modes
- Register IC_HS_SPKLEN holds the maximum spike value for high speed mode.
- Register IC_UFM_SPKLEN holds the maximum spike value for U^Fm.

**Note**

- IC_HS_SPKLEN is implemented only if the component is configured for high speed operation; that is, (IC_MAX_SPEED = High).
- IC_UFM_SPKLEN is implemented only if the component is configured for Ultra-Fast mode; that is, (IC_ULTRA_FAST_MODE=1).
- IC_FS_SPKLEN and IC_HS_SPKLEN are not implemented when configured for Ultra-Fast mode; that is, (IC_ULTRA_FAST_MODE=1).

These registers are 8 bits wide and accessible through the APB interface for read and write purposes; however, they can be written to only when the DW_apb_i2c is disabled. The minimum value that can be programmed into these registers is 1; attempting to program a value smaller than 1 results in the value 1 being written.

The default value for these registers is automatically calculated in coreConsultant based on the value of ic_clk period, but this value can be overridden when configuring the component.

**Note**

- Because the minimum value that can be programmed into the IC_FS_SPKLEN, IC_HS_SPKLEN, and IC_UFM_SPKLEN registers is 1, the spike length specification can be exceeded for low frequencies of ic_clk. Consider the simple example of a 10 MHz (100 ns period) ic_clk; in this case, the minimum spike length that can be programmed is 100 ns, which means that spikes up to this length are suppressed.
- Standard synchronization logic (two flip-flops in series) is implemented upstream of the spike suppression logic and is not affected in any way by the contents of the spike length registers or the operation of the spike suppression logic; the two operations (synchronization and spike suppression) are completely independent.
Because the SCL and SDA inputs are asynchronous to ic_clk, there is one ic_clk cycle uncertainty in the sampling of these signals; that is, depending on when they occur relative to the rising edge of ic_clk, spikes of the same original length might show a difference of one ic_clk cycle after being sampled.
- Spike suppression is symmetrical; that is, the behavior is exactly the same for transitions from 0 to 1 and from 1 to 0.

2.10 Fast Mode Plus Operation

In fast mode plus, the DW_apb_i2c allows the fast mode operation to be extended to support speeds up to 1000 Kb/s. To enable the DW_apb_i2c for fast mode plus operation, perform the following steps before initiating any data transfer:

1. Configure the Maximum Speed mode of DW_apb_i2c Controller or Target to Fast Mode or High Speed mode (`IC_MAX_SPEED_MODE` >= 2).
2. Set `ic_clk` frequency greater than or equal to 32 MHz (see “[Standard Mode \(SM\), Fast Mode \(FM\), and Fast Mode Plus \(FM+\) with IC_CLK_FREQ_OPTIMIZATION = 0](#)” on page 72).
3. Program the `IC_CON` register [2:1] = 2'b10 for fast mode or fast mode plus.
4. Program `IC_FS_SCL_LCNT` and `IC_FS_SCL_HCNT` registers to meet the fast mode plus SCL (see “[IC_CLK Frequency Configuration](#)” on page 69).
5. Program the `IC_FS_SPKLEN` register to suppress the maximum spike of 50ns.
6. Program the `IC_SDA_SETUP` register to meet the minimum data setup time (`tSU; DAT`).

2.11 Bus Clear Feature

DW_apb_i2c supports the bus clear feature that provides graceful recovery of data (SDA) and clock (SCL) lines during unlikely events in which either the clock or data line is stuck at LOW.



Note Bus clear feature is not supported in High-speed mode.

The following sections describes the SDA and SCL lines stuck at LOW recovery mechanisms:

- [“SDA Line Stuck at LOW Recovery”](#) on page 64
- [“SCL Line is Stuck at LOW”](#) on page 65

2.11.1 SDA Line Stuck at LOW Recovery

In case of SDA line stuck at LOW, the controller performs the following actions to recover as shown in [Figure 2-25](#) and [Figure 2-26](#):

1. Controller sends a maximum of 9 clock pulses to recover the bus LOW within those 9 clocks.
 - ❑ The number of clock pulses varies with the number of bits that remain to be sent by the target. As the maximum number of bits is 9, controller sends up to 9 clock pulses and allows the target to recover it.
 - ❑ The controller attempts to assert a Logic 1 on the SDA line and check whether SDA is recovered. If the SDA is not recovered, it continues to send a maximum of 9 SCL clocks.
2. If SDA line is recovered within 9 clock pulses then the controller sends the STOP to release the bus.
3. If SDA line is not recovered even after the 9th clock pulse then system needs a hardware reset.

The detailed flow to recover the SDA stuck at LOW is explained in the section [“Programming Flow for SCL and SDA Bus Recovery”](#) on page 379.

Figure 2-25 SDA Recovery with 9 SCL Clocks

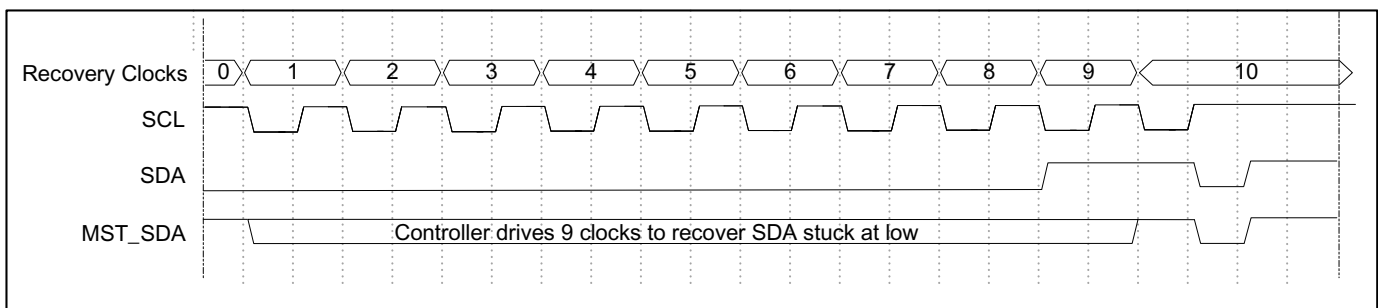
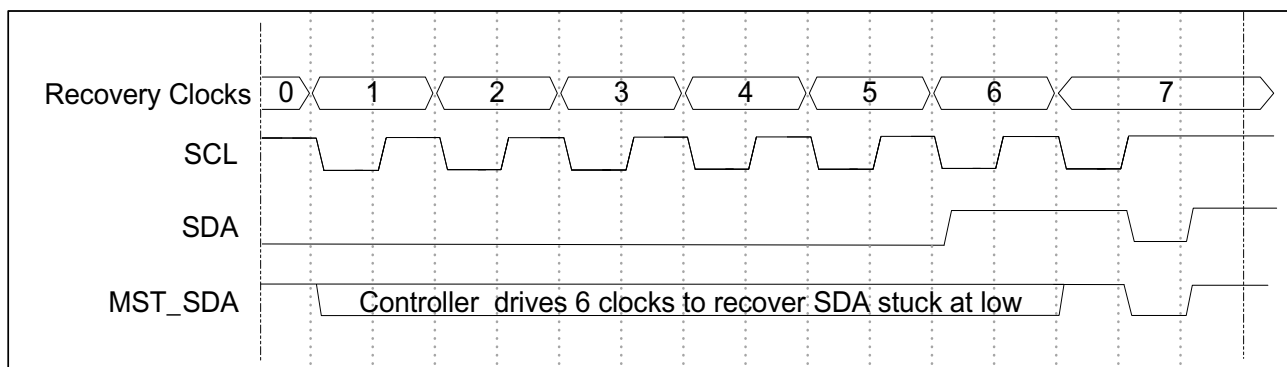


Figure 2-26 SDA Recovery with 6 SCL Clocks

2.11.2 SCL Line is Stuck at LOW

In the unlikely event (due to an electric failure of a circuit) where the clock (SCL) is stuck to LOW, there is no effective method to overcome this problem but to reset the bus using the hardware reset signal. The detailed flow to recover the SCL stuck at LOW is explained in [“Programming Flow for SCL and SDA Bus Recovery”](#) on page 379.

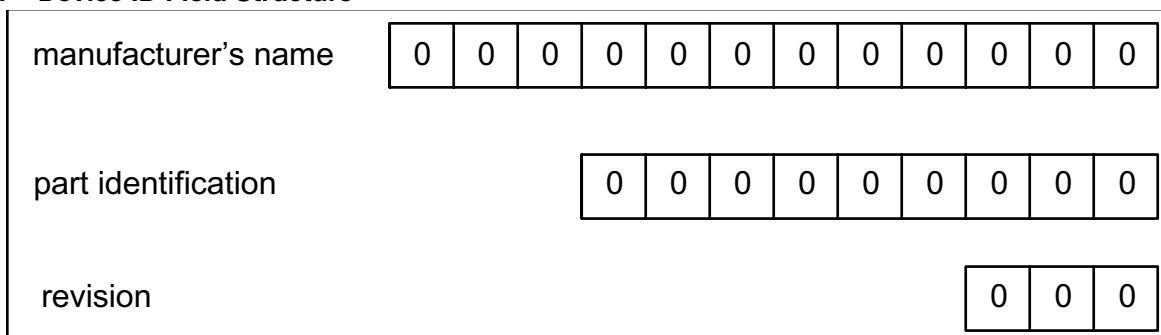
2.12 Device ID

A Device ID field is an optional 3-byte read-only (24 bits) word, which provides the following information:

- Twelve bits with the manufacturer's name, which is unique for every manufacturer.
- Nine bits with the part identification, which is assigned by the manufacturer.
- Three bits with the die revision, which is assigned by the manufacturer.

Figure 2-27 shows the Device ID field structure.

Figure 2-27 Device ID Field Structure



For reading the Device ID of a particular target, the controller can follow the procedure in [“Programming Flow for Reading the Device ID”](#) on page 380. The Device ID that is read is available in RX FIFO, which can be read using IC_DATA_CMD register.

In case of a target, you have to configure the Device ID using the IC_DEVICE_ID_VALUE coreConsultant parameter and you can read the Device ID of the target using IC_DEVICE_ID register.



Note

Device ID is not supported for 10-bit addressing and High Speed transfers (high speed mode).

2.13 Ultra-Fast Speed Mode

The Ultra-Fast Speed mode is a variant of I²C Bus Speed mode that operates from DC (0) to 5 MHz transmitting data in one direction. It is useful for speeds greater than 1 MHz to drive LED controllers and other gaming systems that do not need feedback.

Ultra-Fast speed mode is based on the standard I²C Protocol, which consists of START, target address, command bit, ninth clock (ACK cycle) and a STOP bit. The command bit should be always 'write' (0) only since it is a unidirectional bus (except for the START byte). The data bit on the ninth (ACK) cycle is driven high by the controller, ignoring the ACK cycle due to unidirectional nature of bus. The driver used for Ultra-Fast Mode is push-pull driver.

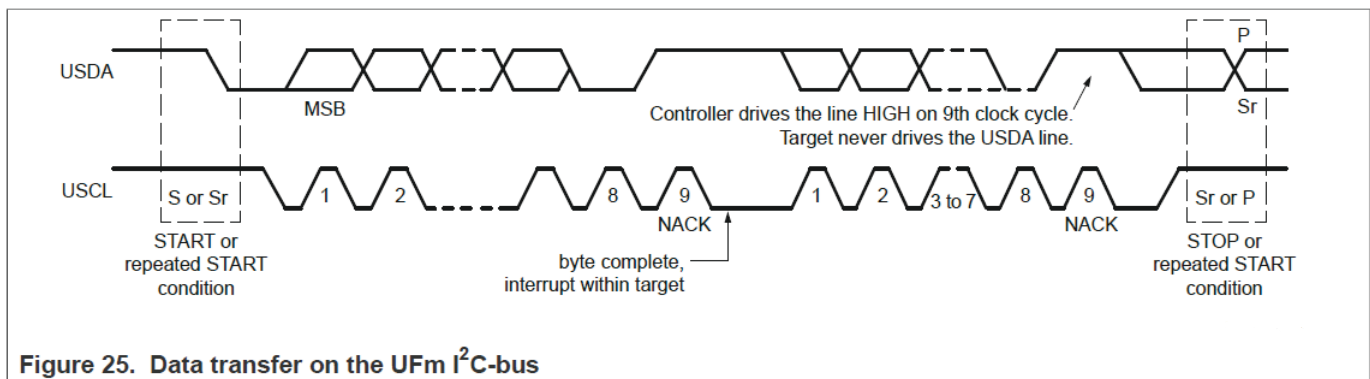
The Controller consists of serial clock (ic_clk_oe, USCL) and a serial data (ic_data_oe, USDA) output signals. The Output signals are Active-Low in nature.

The target consists of serial clock (ic_clk_in_a, USCL) and serial data (ic_data_in_a, USDA) input signals. The input signals are Active-High in nature.

The U^{Fm} I²C-bus does not have the multi-controller capability and hence, it does not consist of wired-AND open-drain driver. In the U^{Fm} I²C bus, the controller is the only device that initiates a data transfer (write transfer) on the bus and provides the clock signals to support that transfer. All other devices are considered as targets. Because of single controller support, the arbitration, synchronization, clock stretching mechanisms are not applicable.

The Byte format, START and STOP generation are same as in other modes of the I²C Protocol except for the ignorance of ACK cycle. The target never drives anything on the bus hence, the controller always drives NACK during the ninth cycle of the transfer as shown in [Figure 2-28](#) (source: I2C Bus Specification Version 7.0).

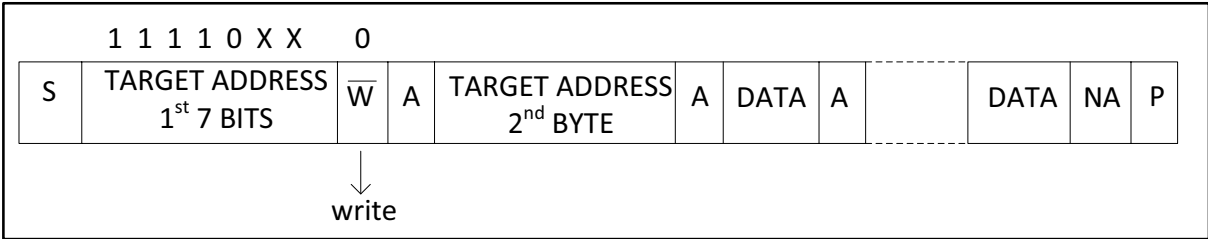
Figure 2-28 U^{Fm}-I2C Byte Transfer



In U^{Fm}-I²C mode, the target is not allowed to hold the clock LOW if it cannot receive another complete byte of data or while it is performing some other function, for example, servicing an internal interrupt. The ninth clock cycle that represents ACK/NACK of the byte is not applicable because target does not respond and it is preserved in U^{Fm} to be compatible with the I²C Protocol. The 8th bit of the address that represents Read or write transfer should be always set to write (0), since Read is not supported in U^{Fm} (except for the START Byte).

The Combined format of I²C Protocol is not supported in U^{Fm}-I²C mode. The 10-bit addressing that expands the number of possible devices is supported in U^{Fm}-I²C mode and it behaves similar to other modes as shown in [Figure 2-29](#) (Only write transfer is supported).

Figure 2-29 10-bit Addressing Write Transfer



The UfM-I²C mode supports START byte and general call features similar to other I²C modes. If the target is not responsive (determined through external feedback and not through UfM I²C-bus), then the target can reset through software reset or external hardware reset.

2.14 IC_CLK Frequency Configuration

When the DW_apb_i2c is configured as a Standard (SS), Fast (FS)/Fast-Mode Plus (FM+), or High Speed (HS) controller, the *CNT registers must be set before any I²C bus transaction can take place in order to ensure proper I/O timing. The *CNT registers are:

- IC_SS_SCL_HCNT
- IC_SS_SCL_LCNT
- IC_FS_SCL_HCNT
- IC_FS_SCL_LCNT
- IC_HS_SCL_HCNT
- IC_HS_SCL_LCNT

When the DW_apb_i2c is configured as a Ultra-Fast Mode controller, the *CNT registers must be set before any I²C bus transaction can take place in order to ensure proper I/O timing. The *CNT registers for this mode are:

- IC_UFM_SCL_HCNT
- IC_UFM_SCL_LCNT



Note

- The skew must be smaller than $T(\text{SCL Low period}) - T(\text{SU;DAT})$.
- The tBUF timing and setup/hold time of START, STOP and RESTART registers uses *HCNT/ *LCNT register settings for the corresponding speed mode.
- It is not necessary to program any of the *CNT registers if the DW_apb_i2c is enabled to operate only as an I²C target, since these registers are used only to determine the SCL timing requirements for operation as an I²C controller.

Table 2-2 lists the derivation of I²C timing parameters from the *CNT programming registers.

Table 2-2 Derivation of I²C Timing Parameters from *CNT Registers

Timing Parameter	Symbol	Standard Speed	Fast Speed / Fast Speed Plus	High Speed (100 pf)	High Speed (400 pf)
LOW period of the SCL clock	tLOW	IC_SS_SCL_LCNT	IC_FS_SCL_LCNT	IC_HS_SCL_LCNT	IC_HS_SCL_LCNT
HIGH period of the SCL clock	tHIGH	IC_SS_SCL_HCNT	IC_FS_SCL_HCNT	IC_HS_SCL_HCNT	IC_HS_SCL_HCNT
Setup time for a repeated START condition	tSU;STA	IC_SS_SCL_LCNT	IC_FS_SCL_HCNT	IC_HS_SCL_LCNT	$(\text{IC_HS_SCL_LCNT})/2$
Hold time (repeated) START condition*	tHD;STA	IC_SS_SCL_HCNT	IC_FS_SCL_HCNT	IC_HS_SCL_LCNT	$(\text{IC_HS_SCL_LCNT})/2$
Setup time for STOP condition	tSU;STO	IC_SS_SCL_HCNT	IC_FS_SCL_HCNT	IC_HS_SCL_LCNT	$(\text{IC_HS_SCL_LCNT})/2$

Timing Parameter	Symbol	Standard Speed	Fast Speed / Fast Speed Plus	High Speed (100 pf)	High Speed (400 pf)
Bus free time between a STOP and a START condition	tBUF	IC_SS_SCL_LCNT	IC_FS_SCL_LCNT	NA	NA
Spike length	tSP	IC_FS_SPKLEN	IC_FS_SPKLEN	IC_HS_SPKLEN	IC_HS_SPKLEN
Data hold time	tHD;DAT	IC_SDA_HOLD	IC_SDA_HOLD	IC_SDA_HOLD	IC_SDA_HOLD
Data setup time	tSU;DAT	IC_SDA_SETUP	IC_SDA_SETUP	IC_SDA_SETUP	IC_SDA_SETUP

**Note**

- When IC_CLK_FREQ_OPTIMIZATION=0:
 - SCL_HCNT is considered as:

$$(IC_*_SCL_HCNT + IC_*_SPKLEN + 7)$$
 - SCL_LCNT is to be considered as:

$$(IC_*_SCL_LCNT + 1)$$
- When IC_CLK_FREQ_OPTIMIZATION=1:
 - SCL_HCNT is to be considered as:

$$(IC_*_SCL_HCNT + IC_*_SPKLEN + 3)$$
 - SCL_LCNT is to be considered as:

$$IC_*_SCL_LCNT$$

2.14.1 Minimum High and Low Counts in SS, FS, FM+ and high speed Modes With IC_CLK_FREQ_OPTIMIZATION = 0.

When the DW_apb_i2c operates as an I²C controller, in both transmit and receive transfers:

- IC_SS_SCL_LCNT and IC_FS_SCL_LCNT register values must be larger than IC_FS_SPKLEN + 7.
- IC_SS_SCL_HCNT and IC_FS_SCL_HCNT register values must be larger than IC_FS_SPKLEN + 5.
- If the component is programmed to support high speed, IC_HS_SCL_LCNT register value must be larger than IC_HS_SPKLEN + 7.
- If the component is programmed to support high speed, IC_HS_SCL_HCNT register value must be larger than IC_HS_SPKLEN + 5.

Details regarding the DW_apb_i2c high and low counts are as follows:

- The minimum value of IC_*_SPKLEN + 7 for the *_LCNT registers is due to the time required for the DW_apb_i2c to drive SDA after a negative edge of SCL.
- The minimum value of IC_*_SPKLEN + 5 for the *_HCNT registers is due to the time required for the DW_apb_i2c to sample SDA during the high period of SCL.
- The DW_apb_i2c adds one cycle to the programmed *_LCNT value in order to generate the low period of the SCL clock; this is due to the counting logic for SCL low counting to (*_LCNT + 1).

- The DW_apb_i2c adds $IC_*_SPKLEN + 7$ cycles to the programmed $*_HCNT$ value in order to generate the high period of the SCL clock; this is due to the following factors:
 - The counting logic for SCL high counts to $(*_HCNT+1)$.
 - The high count logic has an internal latency of 1 clock cycle. Hence, this adds one more cycle to the calculations.
 - The digital filtering applied to the SCL line incurs a delay of $SPKLEN + 2\ ic_clk$ cycles, where $SPKLEN$ is:
 - IC_FS_SPKLEN if the component is operating in SS or FS
 - IC_HS_SPKLEN if the component is operating in high speed.
 This filtering includes metastability removal and the programmable spike suppression on SDA and SCL edges.
- Whenever SCL is driven 1 to 0 by the DW_apb_i2c—that is, completing the SCL high time—an internal logic latency of three ic_clk cycles is incurred. Consequently, the minimum SCL low time of which the DW_apb_i2c is capable is nine (9) ic_clk periods ($7 + 1 + 1$), while the minimum SCL high time is thirteen (13) ic_clk periods ($6 + 1 + 3 + 3$).

**Note**

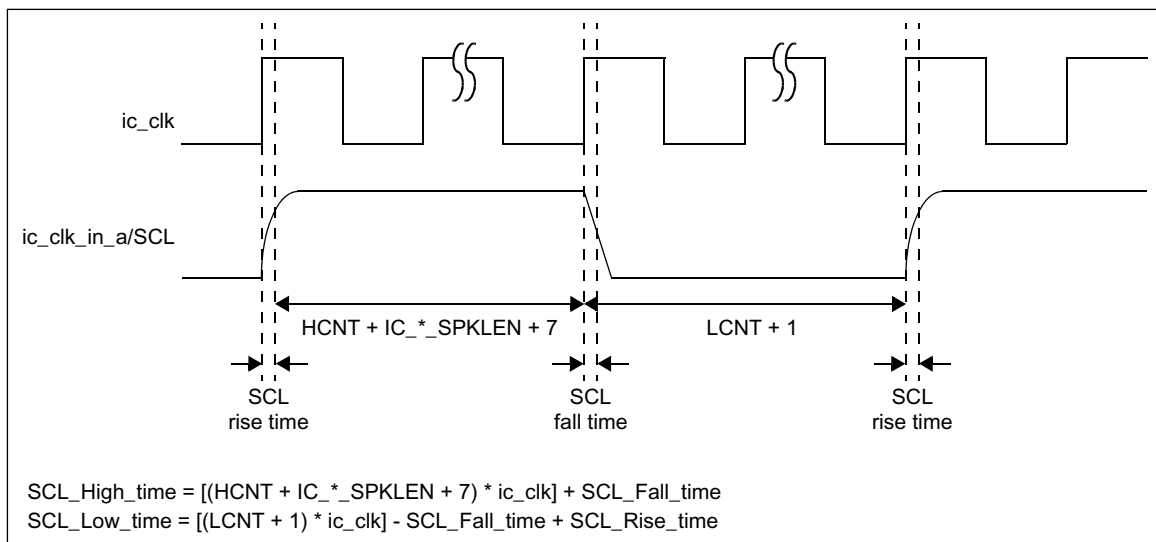
The total high time and low time of SCL generated by the DW_apb_i2c controller is also influenced by the rise time and fall time of the SCL line, as shown in the illustration and equations in

Figure 2-30. It should be noted that the SCL rise and fall time parameters vary, depending on external factors such as:

- Characteristics of IO driver
- Pull-up resistor value
- Total capacitance on SCL line, and so on

These characteristics are beyond the control of the DW_apb_i2c.

Figure 2-30 Impact of SCL Rise Time and Fall Time on Generated SCL



2.14.2 Minimum High and Low Counts in SS, FS, FM+ and high speed Modes With IC_CLK_FREQ_OPTIMIZATION = 1

The minimum high and low counts in SS, FS, FM+ and high speed Modes with the IC_CLK_FREQ_OPTIMIZATION parameter set to one is such that:

- The total SCL LOW period is driven by DW_apb_i2c is IC_*_LCNT register value. The hardware does not support a value less than 6 to be written to the IC_*_LCNT register. Additionally, the minimum SCL low time of which the DW_apb_i2c is capable is 6 ic_clk periods.
- The total SCL HIGH period driven by DW_apb_i2c is IC_*_HCNT register value + SPKLEN + 3. Additionally, the minimum SCL high time of which the DW_apb_i2c is capable is 5 ic_clk periods [1+1+3].

The total high time and low time of SCL generated by the DW_apb_i2c controller is also influenced by the rise time and fall time of the SCL line. The SCL rise and fall time parameters vary depending on external factors such as:

- Characteristics of IO driver
- Pull-up resistor value
- Total capacitance on SCL line, and so on

These characteristics are beyond the control of the DW_apb_i2c.

2.14.3 Minimum High and Low counts in Ultra-Fast mode (IC_ULTRA_FAST_MODE = 1)

When the DW_apb_i2c operates as an I²C controller:

- The IC_UFM_SCL_HCNT register value must be equal or larger than 3.
- The IC_UFM_SCL_LCNT register Value must be equal or larger than 5.

2.14.4 Minimum IC_CLK Frequency

This section describes the minimum ic_clk frequencies that the DW_apb_i2c supports for each speed mode, and the associated high and low count values. In target mode, IC_SDA_HOLD (Thd;dat) and IC_SDA_SETUP (Tsu;dat) need to be programmed to satisfy the I²C protocol timing requirements.

The following examples are for the case where IC_FS_SPKLEN and IC_HS_SPKLEN are programmed to 2.

2.14.4.1 Standard Mode (SM), Fast Mode (FM), and Fast Mode Plus (FM+) with IC_CLK_FREQ_OPTIMIZATION = 0

This section details how to derive a minimum ic_clk value for standard and fast modes of the DW_apb_i2c. Although the following method shows how to do fast mode calculations, you can also use the same method in order to do calculations for standard mode and fast mode plus.



Note

The following computations do not consider the SCL_Rise_time and SCL_Fall_time.

Given conditions and calculations for the minimum DW_apb_i2c ic_clk value in fast mode:

- Fast mode has data rate of 400kb/s; implies SCL period of $1/400\text{khz} = 2.5\mu\text{s}$
- Minimum hcnt value of 14 as a seed value; IC_HCNT_FS = 14

■ Protocol minimum SCL high and low times:

- MIN_SCL_LOWtime_FS = 1300ns
- MIN_SCL_HIGHtime_FS = 600ns

Derived equations:

$$\frac{\text{SCL_PERIOD_FS}}{\text{IC_HCNT_FS} + \text{IC_LCNT_FS}} = \text{IC_CLOCK_PERIOD}$$

$$\text{IC_LCNT_FS} \times \text{IC_CLOCK_PERIOD} = \text{MIN_SCL_LOWtime_FS}$$

Combined, the previous equations produce the following:

$$\text{IC_LCNT_FS} \frac{\text{SCL_PERIOD_FS}}{\text{IC_LCNT_FS} + \text{IC_HCNT_FS}} = \text{MIN_SCL_LOWtime_FS}$$

Solving for IC_LCNT_FS:

$$\text{IC_LCNT_FS} \frac{2.5 \text{ s}}{\text{IC_LCNT_FS} + 14} = 1.3 \text{ s}$$

The previous equation gives:

$$\text{IC_LCNT_FS} = \text{roundup}(15.166) = 16$$

These calculations produce IC_LCNT_FS = 16 and IC_HCNT_FS = 14, giving an ic_clk value of:

$$\frac{2.5 \text{ s}}{16 + 14} = 83.3\text{ns} = 12\text{Mhz}$$

Testing these results shows that protocol requirements are satisfied.

2.14.4.2 High-Speed (HS) Mode With IC_CLK_FREQ_OPTIMIZATION = 0

The method used for standard and fast modes can also be used to derive ic_clk values for high speed modes. For example, given a high speed mode with a 100pf bus loading, using the standard and fast modes method produces the following:

- IC_LCNT_HS = 17
- IC_HCNT_HS = 14
- ic_clk = 105.4 MHz

Table 2-3 lists the minimum ic_clk values for all modes with high and low count values.

Table 2-3 ic_clk in Relation to High and Low Counts When IC_CLK_FREQ_OPTIMIZATION = 0

Speed Mode	ic_clk _{freq} (MHz)	Minimum Value of IC_*_SPKL EN	SCL Low Time in ic_clks	SCL Low Program Value	SCL Low Time	SCL High Time in ic_clks	SCL High Program Value	SCL High Time
SS	2.7	1	13	12	4.7 ms	14	6	5.2 ms
FS	12.0	1	16	15	1.33 ms	14	6	1.16 ms

Speed Mode	ic_clk _{freq} (MHz)	Minimum Value of IC_*_SPKL EN	SCL Low Time in ic_clks	SCL Low Program Value	SCL Low Time	SCL High Time in ic_clks	SCL High Program Value	SCL High Time
FM+	32	2	16	15	500 ns	16	7	500 ns
HS (400pf)	51	1	17	16	333 ns	14	6	274 ns
HS (100pf)	105.4	1	17	16	161 ns	14	6	132 ns

**Note**

- The IC_*_SCL_LCNT and IC_*_SCL_HCNT registers are programmed using the SCL low and high program values in [Table 2-3](#), which are calculated using SCL low count minus 1, and SCL high counts minus 8, respectively.

The values in [Table 2-3](#) are based on IC_SDA_RX_HOLD = 0. The maximum IC_SDA_RX_HOLD value depends on the IC_*CNT registers in Controller mode, as described in “[SDA Hold Timings in Receiver](#)” on page 82.

- In order to compute the HCNT and LCNT considering RC timings, use the following equations:

$$\text{SCL_High_time} = [(\text{IC_*_SCL_HCNT} + \text{IC_*_SPKLEN} + 7) * \text{ic_clk}] + \text{SCL_Fall_time}$$

$$\text{SCL_Low_time} = [(\text{IC_*_SCL_LCNT} + 1) * \text{ic_clk}] - \text{SCL_Fall_time} + \text{SCL_Rise_time}$$

2.14.4.3 SM, FM, FM+ and high speed Modes With IC_CLK_FREQ_OPTIMIZATION = 1

2.14.4.3.1 Controller Mode

This section describes the minimum ic_clk frequencies that the DW_apb_i2c supports for each speed mode and the associated high and low count values. The following examples are for the case where IC_FS_SPKLEN = 1, IC_HS_SPKLEN = 1 and IC_CLK_FREQ_OPTIMIZATION = 1.

Following calculations show how to derive a minimum ic_clk value for fast mode of the DW_apb_i2c. Although the following method shows how to do fast mode calculations, you can also use the same method in order to do calculations for any speed mode.

**Note**

The computation in this section does not consider SCL_Rise_time and SCL_Fall_time.

Following are the conditions and calculations for the minimum DW_apb_i2c ic_clk value in fast mode:

- Fast mode has data rate of 400kb/s; implies SCL period of $1/400\text{KHz} = 2.5 \mu\text{s}$
- Minimum hcnt value of 5 as a seed value; IC_HCNT_FS = 5
- Protocol minimum SCL high and low times:
 - MIN_SCL_LOWtime_FS = 1300 ns
 - MIN_SCL_HIGHtime_FS = 600 ns

Following are the derived equations:

$$\text{SCL_PERIOD_FS} / (\text{IC_HCNT_FS} + \text{IC_LCNT_FS}) = \text{IC_CLOCK_PERIOD}$$

$$\text{IC_LCNT_FS} \times \text{IC_CLOCK_PERIOD} = \text{MIN_SCL_LOWtime_FS}$$

Following is the result of combining previous equations:

$$IC_LCNT_FS \times SCL_PERIOD_FS / (IC_LCNT_FS + IC_HCNT_FS) = MIN_SCL_LOWtime_FS$$

By solving for IC_LCNT_FS:

$$IC_LCNT_FS \times 2.5 \mu s / (IC_LCNT_FS + 5) = 1.3 \mu s$$

The previous equation provides:

$$IC_LCNT_FS = \text{roundup}(5.417) = 6$$



Note

Minimum IC_*_LCNT value should be equal 6. If derived value is less than 6, consider IC_LCNT_FS as 6 only.

These calculations produce IC_LCNT_FS = 6 and IC_HCNT_FS = 5, providing an ic_clk value of:

$$2.5 \mu s / (6 + 5) = 227.27 \text{ ns} = 4.4 \text{ MHz}$$

Testing these results shows that the protocol requirements are satisfied.

[Table 2-4](#) lists the minimum ic_clk values for all modes with high and low count values.

Table 2-4 ic_clk in Relation to High and Low Counts When IC_CLK_FREQ_OPTIMIZATION = 1

Speed Mode	ic_clk Frequency (MHz)	Minimum Value of IC_*_SPK LEN	SCL Low Time in ic_clks	SCL Low Program Value	SCL Low Time in ns	SCL High Time in ic_clks	SCL High Program Value	SCL High Time in ns
SS	1.1	1	6	6	5454.545	5	1	4545.455
FS	4.4	1	6	6	1363.636	5	1	1136.364
FM+	11	1	6	6	545.4545	5	1	454.5455
HS (400pf)	18.7	1	6	6	320.8527	5	1	267.3773
HS (100pf)	37.4	1	6	6	160.4236	5	1	133.6864



Note

- The IC_*_SCL_LCNT and IC_*_SCL_HCNT registers are programmed using the SCL low and high program values in [Table 2-4](#), which are calculated as SCL low count, and SCL high count minus 4, respectively. The values in [Table 2-4](#) are based on IC_SDA_RX_HOLD = 0. The maximum IC_SDA_RX_HOLD value depends on the IC_*CNT registers in controller mode, as described in [“SDA Hold Timings in Receiver”](#) on page [82](#).
- To compute the HCNT and LCNT considering RC timings, use the following equations:

$$SCL_High_time = [(IC_*_SCL_HCNT + IC_*_SPKLEN + 3) * ic_clk] + SCL_Fall_time$$

$$SCL_Low_time = [IC_*_SCL_LCNT * ic_clk] - SCL_Fall_time + SCL_Rise_time$$

2.14.4.3.2 Target Mode

DW_apb_i2c in target mode requires minimum 5 ic_clk cycles [SPKLEN + 3 (Metastability removal, worst case) + 1] to drive SDA after a falling edge of SCL. Therefore, the ic_clk frequency must be selected such that the maximum data hold time (thd;dat)/data valid time (tVD;DAT) is not violated.

For example, in high speed mode with a 100pf bus loading (SCLH clock frequency upto 3.4 MHz), the maximum data hold time is 70 ns. Therefore, the minimum frequency in which DW_apb_i2c can operate in target mode without violating thd;dat is $70\text{ns}/5 = 14\text{ns} = 71.42\text{ MHz}$.

Table 2-5 lists the minimum IC_CLK frequency in target mode when IC_CLK_FREQ_OPTIMIZATION is set to 1.

Table 2-5 Minimum IC_CLK Frequency in Target Mode When IC_CLK_FREQ_OPTIMIZATION=1

Speed Mode	ic_clk Frequency (MHz)	Minimum Value of IC_*_SPKLEN	Minimum data hold time in ic_clks	Maximum data hold time
SS	1.45	1	5	3.45 μs
FS	5.56	1	5	0.9 μs
FM+	11.11	1	5	0.45 μs
HS (400pf)	35.71	1	5	140 ns
HS (100pf)	71.42	1	5	70 ns

2.14.4.4 ULTRA-FAST Mode

2.14.4.4.1 Controller Mode

This section describes the minimum ic_clk frequency that the DW_apb_i2c supports for Ultra-Fast speed mode and the associated high and low count values.

The following calculations show how to derive a minimum ic_clk value.



Note The following computations do not consider the SCL_Rise_time and SCL_Fall_time.

Given conditions and calculations for the minimum DW_apb_i2c ic_clk value in Ultra-Fast mode:

- Fast mode has data rate of 5000kb/s; implies SCL period of $1/5000\text{khz} = 200\text{ns}$
- Minimum hcnt value of 3 as a seed value; IC_UFM_SCL_HCNT = 3
- Protocol minimum SCL high and low times:
 - MIN_SCL_LOWtime_UFm = 50 ns
 - MIN_SCL_HIGHtime_UFm = 50 ns

Derived equations:

- $\text{SCL_PERIOD_UFm} / (\text{IC_HCNT_UFm} + \text{IC_LCNT_UFm}) = \text{IC_CLOCK_PERIOD}$
- $\text{IC_LCNT_UFm} \times \text{IC_CLOCK_PERIOD} = \text{MIN_SCL_LOWtime_UFm}$

Combined, the previous equations produce the following:

$$\text{IC_LCNT_UFm} \times \text{SCL_PERIOD_UFm} / (\text{IC_LCNT_UFm} + \text{IC_HCNT_UFm}) = \text{MIN_SCL_LOWtime_UFm}$$

Solving for IC_LCNT_UFm:

$$IC_LCNT_UFm \times 200ns / (IC_LCNT_UFm + 3) = 50 ns$$

The previous equation gives:

$$IC_LCNT_UFm = 1$$



Note

Minimum IC_SCL_UFM_LCNT value should be equal 5. If derived value is less than 5, consider IC_LCNT_UFm as 5 only.

These calculations produce IC_LCNT_UFm = 5 and IC_HCNT_UFm = 3, giving an ic_clk value of:

$$200 ns / (5 + 3) = 25ns = 40MHz$$

Testing these results shows that protocol requirements are satisfied.

[Table 2-6](#) describes the relation between the High and Low counts with ic_clk frequency

Table 2-6 ic_clk in relation to High and Low Counts when IC_ULTRA_FAST_MODE=1

Speed	ic_clk (freq) (Mhz)	SCL Low Program Value	SCL Low Time in ic_clks	SCL Low Time	SCL High Program Value	SCL HighTime in ic_clks	SCL HighTime
UltraFast Mode	40	5	5	125 ns	3	3	75 ns



Note

- The IC_UFM_SCL_LCNT and IC_UFM_SCL_HCNT registers are programmed using the SCL low and high program values in [Table 2-6](#), which are calculated as SCL low count, and SCL high count, respectively. The values in [Table 2-6](#) are based on IC_SDA_RX_HOLD = 0. The maximum IC_SDA_RX_HOLD value depends on the IC_UFM_SCL_LCNT registers in Controller mode, as described in [“SDA Hold Timings in Receiver”](#) on page 82.
- In order to compute the HCNT and LCNT considering RC timings, use the following equations:

$$SCL_High_time = [IC_UFM_SCL_HCNT * ic_clk] + SCL_Fall_time$$

$$SCL_Low_time = [IC_UFM_SCL_LCNT * ic_clk] - SCL_Fall_time + SCL_Rise_time$$

2.14.4.4.2 Target Mode

DW_apb_i2c in target mode requires minimum of 2 ic_clk cycles for SCL High period and SCL Low Period. Therefore, the minimum ic_clk frequency for the target mode is 40 MHz.

2.14.4.5 Calculating High and Low Counts with IC_CLK_FREQ_OPTIMIZATION = 0

The following calculations show how to calculate SCL high and low counts for each speed mode in the DW_apb_i2c. For the calculations to work, the ic_clk frequencies used must not be less than the minimum ic_clk frequencies specified in [Table 2-3](#) on page 73.

The DW_apb_i2c coreConsultant GUI can automatically calculate SCL high and low count values. By specifying an integer ic_clk period value in nanoseconds for the IC_CLOCK_PERIOD parameter, SCL high and low count values are automatically calculated for each speed mode. The ic_clk period must not specify a clock of a lower frequency than required for all supported speed modes. It is possible that the

automatically calculated values may result in a baud rate higher than the maximum rate specified by the protocol. If this happens, either the low or high count values can be scaled up to reduce the baud rate.

The equation to calculate the proper number of ic_clk signals required for setting the proper SCL clocks high and low times is as follows:

$$IC_xCNT = (ROUNDUP(MIN_SCL_xxxtime * OSCFREQ, 0))$$

ROUNDUP is an explicit Excel function call that is used to convert a real number to its equivalent integer number.

MIN_SCL_HIGHTime = Minimum High Period

MIN_SCL_HIGHTime = 4000 ns for 100 kbps
 600 ns for 400 kbps
 260 ns for 1000 kbps
 60 ns for 3.4 Mbs, bus loading = 100pF
 120 ns for 3.4 Mbs, bus loading = 400pF

MIN_SCL_LOWtime = Minimum Low Period

MIN_SCL_LOWtime = 4700 ns for 100 kbps
 1300 ns for 400 kbps
 500 ns for 1000 kbps
 160 ns for 3.4Mbs, bus loading = 100pF
 320 ns for 3.4Mbs, bus loading = 400pF

OSCFREQ = ic_clk Clock Frequency (Hz).

For example:

OSCFREQ = 100 MHz
 I2Cmode = fast, 400 kbit/s
 MIN_SCL_HIGHTime = 600 ns.
 MIN_SCL_LOWtime = 1300 ns.

$$IC_xCNT = (ROUNDUP(MIN_SCL_HIGH_LOWtime * OSCFREQ, 0))$$

$$IC_HCNT = (ROUNDUP(600 \text{ ns} * 100 \text{ MHz}, 0))$$

IC_HCNTSCL PERIOD = 60

$$IC_LCNT = (ROUNDUP(1300 \text{ ns} * 100 \text{ MHz}, 0))$$

IC_LCNTSCL PERIOD = 130

Actual MIN_SCL_HIGHTime = $60 * (1/100 \text{ MHz}) = 600 \text{ ns}$

Actual MIN_SCL_LOWtime = $130 * (1/100 \text{ MHz}) = 1300 \text{ ns}$

**Note**

Once the default values for SCL HighCount and LowCount are computed by the coreConsultant GUI, check that the values are consistent with the required baud rate. In case the computed values do not match with the required values, you can manually scale the values, as described in the section “[High-Speed \(HS\) Mode With IC_CLK_FREQ_OPTIMIZATION = 0](#)” on page 73.

2.14.4.6 Calculating High and Low counts with IC_CLK_FREQ_OPTIMIZATION = 1

The following calculations show how to calculate SCL high and low counts for each speed mode in the DW_apb_i2c. For the calculations to work, the ic_clk frequencies used must not be less than the minimum ic_clk frequencies specified in [Table 2-4](#) on page 75.

The DW_apb_i2c coreConsultant GUI can automatically calculate SCL high and low count values. By specifying an integer ic_clk period value in nanoseconds for the IC_CLOCK_PERIOD parameter, SCL high and low count values are automatically calculated for each speed mode. The ic_clk period must not specify a clock of a lower frequency than required for all supported speed modes. It is possible that the automatically calculated values may result in a baud rate higher than the maximum rate specified by the protocol. If this happens, either the low or high count values can be scaled up to reduce the baud rate. For more information, see “[Controller Mode](#)” on page 74.

The equation to calculate the proper number of ic_clk signals required for setting the proper SCL clocks high and low times is as follows:

$$IC_xCNT = (ROUNDUP(MIN_SCL_xxxtime * OSCFREQ, 0))$$

ROUNDUP is an explicit Excel function call that is used to convert a real number to its equivalent integer number.

MIN_SCL_HIGHTime = Minimum High Period

MIN_SCL_HIGHTime = 4000 ns for 100 kbps

600 ns for 400 kbps

260 ns for 1000 kbps

60 ns for 3.4 Mbps, bus loading = 100pF

160 ns for 3.4 Mbps, bus loading = 400pF

MIN_SCL_LOWtime = Minimum Low Period

MIN_SCL_LOWtime = 4700 ns for 100 kbps

1300 ns for 400 kbps

500 ns for 1000 kbps

120 ns for 3.4Mbps, bus loading = 100pF

320 ns for 3.4Mbps, bus loading = 400pF

OSCFREQ = ic_clk Clock Frequency (Hz).

For example:

OSCFREQ = 100 MHz

I2Cmode = fast, 400 kbit/s

$$\text{MIN_SCL_HIGHtime} = 600 \text{ ns.}$$
$$\text{MIN_SCL_LOWtime} = 1300 \text{ ns.}$$
$$\text{IC_xCNT} = (\text{ROUNDUP}(\text{MIN_SCL_HIGH_LOWtime} * \text{OSCFREQ}, 0))$$
$$\text{IC_HCNT} = (\text{ROUNDUP}(600 \text{ ns} * 100 \text{ MHz}, 0))$$
$$\text{IC_HCNTSCL PERIOD} = 60$$
$$\text{IC_LCNT} = (\text{ROUNDUP}(1300 \text{ ns} * 100 \text{ MHz}, 0))$$
$$\text{IC_LCNTSCL PERIOD} = 130$$
$$\text{Actual MIN_SCL_HIGHtime} = 60 * (1/100 \text{ MHz}) = 600 \text{ ns}$$
$$\text{Actual MIN_SCL_LOWtime} = 130 * (1/100 \text{ MHz}) = 1300 \text{ ns}$$
**Note**

When the default values for SCL HighCount and LowCount are computed by the coreConsultant GUI, check that the values are consistent with the required baud rate. In case the computed values do not match with the required values, you can manually scale the values, as described in [“Controller Mode”](#) on page 74.

**Note**

Contact Synopsys if you require support for IC_*_SCL_HCNT and IC_*_SCL_LCNT calculations.

2.15 Clocks and Resets

This section describes the clocks and resets present in the DW_apb_i2c.

2.15.1 Clocks

Clocks present in the DW_apb_i2c:

- pclk - APB clock for bus interface unit
- ic_clk - Peripheral clock on which DW_apb_i2c runs. It is used to clock transfers in standard, fast and high-speed mode

When the configuration parameter IC_CLK_TYPE is set to 0, pclk and ic_clk are identical and when IC_CLK_TYPE is 1, they are asynchronous.



Note

If PCLK and IC_CLK are asynchronous (IC_CLK_TYPE=ASYNC) then the following condition must be met for DW_apb_i2c to function properly:

- When IC_HAS_ASYNC_FIFO = 0, $(SCL_LOW_COUNT * ic_clock_period) > (3 * pclk_period + 3 * ic_clock_period)$ Where, SCL_LOW_COUNT specifies the low count value in terms of ic_clk for the respective speed mode.
- When IC_HAS_ASYNC_FIFO = 1, $pclk_period < (9 * scl_period) / 2$ Where, pclk_period specifies the clock period of the application clock. and scl_period specifies the SCL period.

2.15.2 Reset Signals

Resets present in the DW_apb_i2c:

- presetn - The APB interface domain active-low reset. This signal is asserted asynchronously, but de-assertion must be synchronous to pclk. DW_apb_i2c does not have the logic to perform this synchronization, so it must be taken care externally.
- ic_rst_n - DW_apb_i2c peripheral clock active-low reset. Used to reset flip-flops that are clocked by the ic_clk clock. This signal does not reset DW_apb_i2c control, configuration, and status registers. The signal is asserted asynchronously, but is de-asserted synchronously after the rising edge of ic_clk. The synchronization must be provided external to this component.

2.16 SDA Hold Time

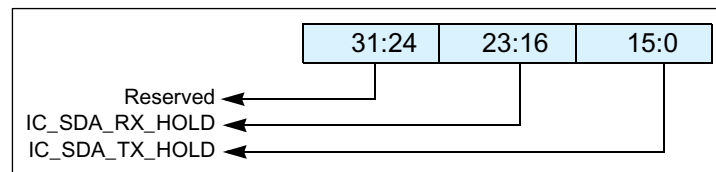
The I²C protocol specification requires 300ns of hold time on the SDA signal (t_{HD;DAT}) in standard mode and fast mode, and a hold time long enough to bridge the undefined part between logic 1 and logic 0 of the falling edge of SCL in high speed mode and fast mode plus.

Board delays on the SCL and SDA signals can mean that the hold-time requirement is met at the I²C controller, but not at the I²C target (or vice-versa). As each application encounters differing board delays, the DW_apb_i2c contains a software programmable register (IC_SDA_HOLD) to enable dynamic adjustment of the SDA hold-time.

The bits [15:0] are used to control the hold time of SDA during transmit in both target and controller mode (after SCL goes from HIGH to LOW).

The bits [23:16] are used to extend the SDA transition (if any) whenever SCL is HIGH in the receiver (in either controller or target mode).

Figure 2-31 IC_SDA_HOLD Register



If different SDA hold times are required for different speed modes, the IC_SDA_HOLD register must be reprogrammed when the speed mode is being changed. The IC_SDA_HOLD register can be programmed only when the DW_apb_i2c is disabled (IC_ENABLE[0] = 0).

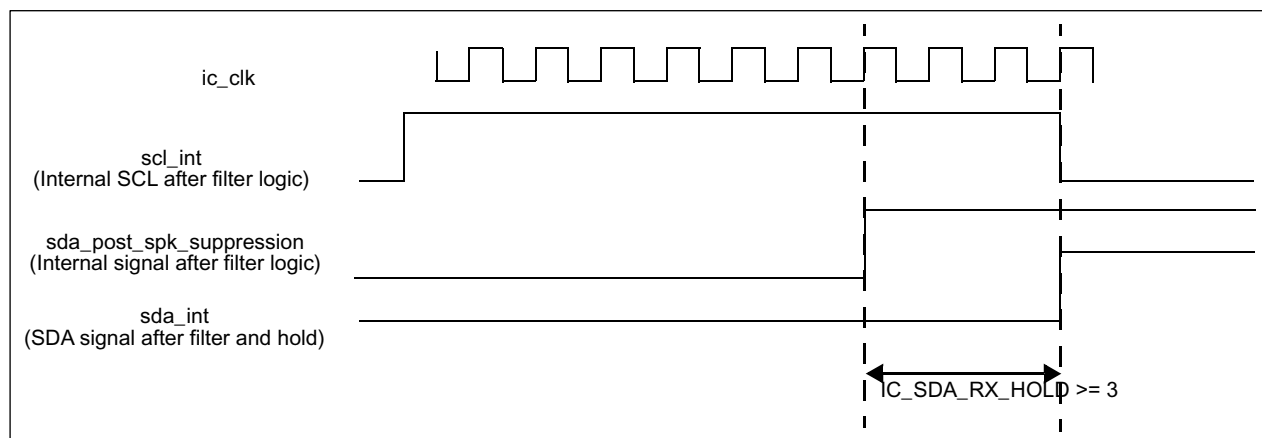
The reset value of the IC_SDA_HOLD register can be set through the coreConsultant parameter IC_DEFAULT_SDA_HOLD

2.16.1 SDA Hold Timings in Receiver

When DW_apb_i2c acts as a receiver, according to the I²C protocol, the device should internally hold the SDA line to bridge undefined gap between logic 1 and logic 0 of SCL.

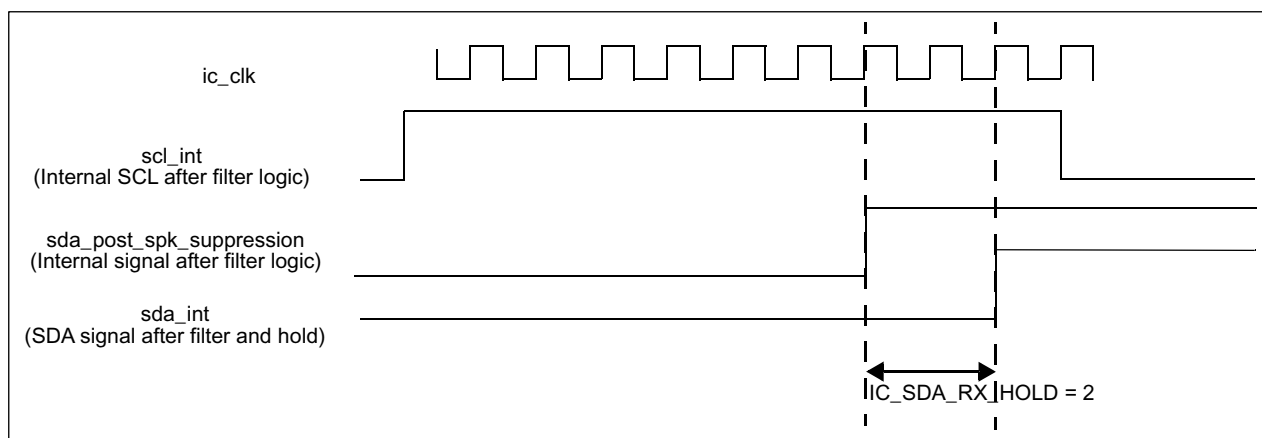
IC_SDA_RX_HOLD can be used to alter the internal hold time which DW_apb_i2c applies to the incoming SDA line. Each value in the IC_SDA_RX_HOLD register represents a unit of one ic_clk period. The minimum value of IC_SDA_RX_HOLD is 0. This hold time is applicable only when SCL is HIGH. The receiver does not extend the SDA after SCL goes LOW internally.

Figure 2-32 shows the DW_apb_i2c as receiver with IC_SDA_RX_HOLD programmed to greater than or equal to 3.

Figure 2-32 DW_apb_i2c as Receiver With IC_SDA_RX_HOLD Programmed to Greater Than or Equal to 3

If **IC_SDA_RX_HOLD** is greater than 3, **DW_apb_i2c** does not hold SDA beyond 3 **ic_clk** cycles, because SCL goes LOW internally.

Figure 2-33 shows the **DW_apb_i2c** as receiver with **IC_SDA_RX_HOLD** programmed to 2.

Figure 2-33 DW_apb_i2c as Receiver With IC_SDA_RX_HOLD Programmed to 2

The maximum values of **IC_SDA_RX_HOLD** that can be programmed in the register for the respective speed modes are derived from the equations show in Table 2-7.

Table 2-7 Maximum Values for IC_SDA_RX_HOLD

Speed Mode	Maximum IC_SDA_RX_HOLD Value
Standard Mode	$IC_SS_SCL_HCNT - IC_FS_SPKLEN - 3$
Fast Mode or Fast Mode Plus	$IC_FS_SCL_HCNT - IC_FS_SPKLEN - 3$
High Speed (IC_CAP_LOADING =100)	$\text{Min} \{ IC_FS_SCL_HCNT - IC_FS_SPKLEN - 3, IC_HS_SCL_LCNT - IC_HS_SPKLEN - 3 \}$
High Speed (IC_CAP_LOADING =400)	$\text{Min} \{ IC_FS_SCL_HCNT - IC_FS_SPKLEN - 3, (IC_HS_SCL_LCNT/2) - IC_HS_SPKLEN - 3 \}$

**Note**

The maximum values in [Table 2-7](#) is applicable in Controller mode. In Target mode, make sure the IC_SDA_RX_HOLD does not exceed the maximum SCL fall time (t_f in SS and FS mode or t_{fcl} in HS Mode).

2.16.2 SDA Hold Timings in Transmitter

The IC_SDA_TX_HOLD register can be used to alter the timing of the generated SDA (`ic_data_oe`) signal by the DW_apb_i2c. Each value in the IC_SDA_TX_HOLD register represents a unit of one `ic_clk` period.

When the DW_apb_i2c is operating in Controller Mode, the minimum $t_{HD}:DAT$ timing is one `ic_clk` period. Therefore even when IC_SDA_TX_HOLD has a value of zero, the DW_apb_i2c drives SDA (`ic_data_oe`) one `ic_clk` cycle after driving SCL (`ic_clk_oe`) to logic 0. For all other values of IC_SDA_TX_HOLD, the following is true:

- Drive on SDA (`ic_data_oe`) occurs IC_SDA_TX_HOLD `ic_clk` cycles after driving SCL (`ic_clk_oe`) to logic 0

When the DW_apb_i2c is operating in Target Mode, the minimum $t_{HD}:DAT$ timing is $SPKLEN + 7$ `ic_clk` periods, where SPKLEN is:

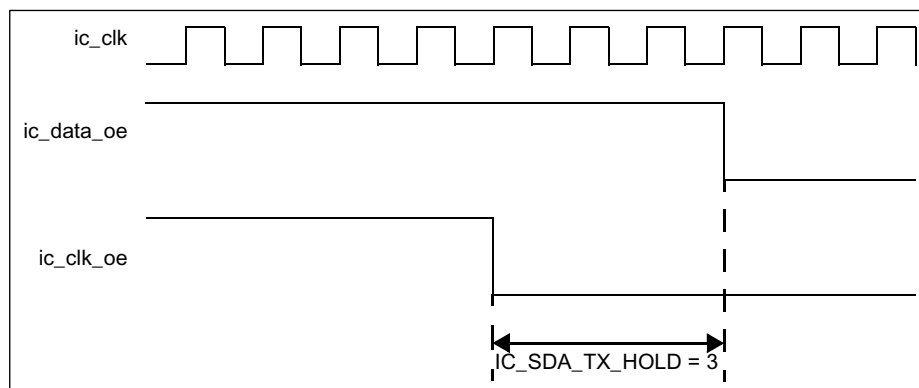
- IC_FS_SPKLEN if the component is operating in standard mode, fast mode, or fast mode plus
- IC_HS_SPKLEN if the component is operating in high speed mode

This delay allows for synchronization and spike suppression on the SCL (`ic_clk_in_a`) sample. Therefore, even when IC_SDA_TX_HOLD has a value less than $SPKLEN + 7$, the DW_apb_i2c drives SDA (`ic_data_oe`) $SPKLEN + 7$ `ic_clk` cycles after SCL (`ic_clk_in`) has transitioned to logic 0. For all other values of IC_SDA_TX_HOLD, the following is true:

- Drive on SDA (`ic_data_oe`) occurs IC_SDA_TX_HOLD `ic_clk` cycles after SCL (`ic_clk_in_a`) has transitioned to logic 0.

[Figure 2-34](#) shows the $t_{HD}:DAT$ timing generated by the DW_apb_i2c operating in Controller Mode when IC_SDA_TX_HOLD = 3.

Figure 2-34 DW_apb_i2c Controller Implementing $t_{HD}:DAT$ with IC_SDA_HOLD = 3



**Note**

The programmed SDA hold time cannot exceed at any time the duration of the low part of scl. Therefore the programmed value cannot be larger than N_SCL_LOW-2 , where N_SCL_LOW is the duration of the low part of the scl period measured in ic_clk cycles.

2.17 DMA Controller Interface

The DW_apb_i2c has an optional built-in DMA capability that can be selected at configuration time; it has a handshaking interface to a DMA Controller to request and control transfers. The APB bus is used to perform the data transfer to or from the DMA. While the DW_apb_i2c DMA operation is designed in a generic way to fit any DMA controller as easily as possible, it is designed to work seamlessly, and best used, with the DesignWare DMA Controller, the DW_ahb_dmac. The settings of the DW_ahb_dmac that are relevant to the operation of the DW_apb_i2c are discussed here, mainly bit fields in the DW_ahb_dmac channel control register, CTLx, where *x* is the channel number.

**Note**

When the DW_apb_i2c interfaces to the DW_ahb_dmac, the DW_ahb_dmac is always a flow controller; that is, it controls the block size. This must be programmed by software in the DW_ahb_dmac. The DW_ahb_dmac always transfers data using DMA burst transactions if possible, for efficiency. For more information, see the *DesignWare DW_ahb_dmac Databook*. Other DMA controllers act in a similar manner.

The relevant DMA settings are discussed in the following sections.

**Note**

The DMA output dma_finish is a status signal to indicate that the DMA block transfer is complete. DW_apb_i2c does not use this status signal, and therefore does not appear in the I/O port list.

2.17.1 Enabling the DMA Controller Interface

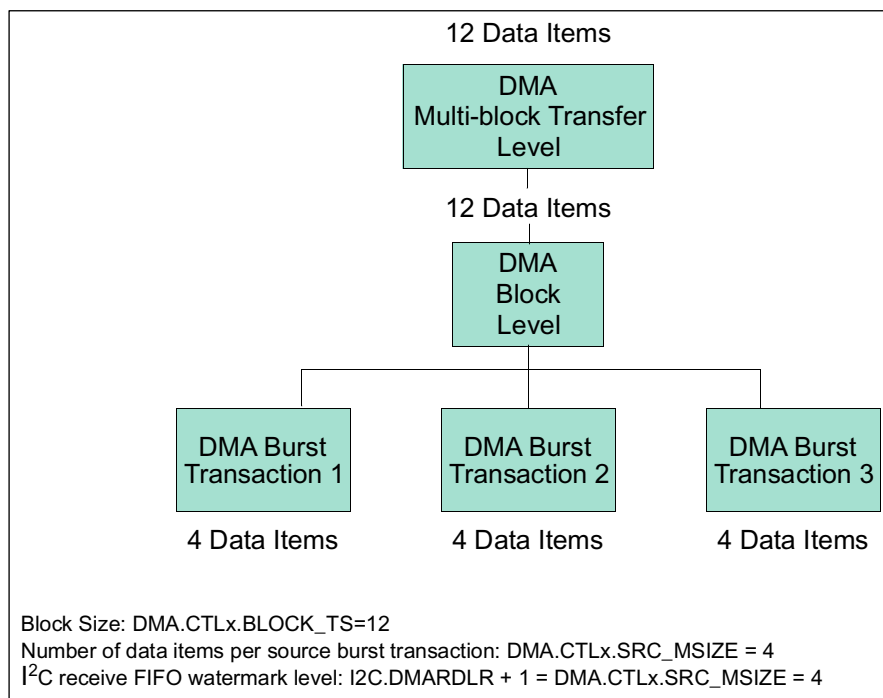
To enable the DMA Controller interface on the DW_apb_i2c, you must write the DMA Control Register (IC_DMA_CR). Writing a 1 into the TDMAE bit field of IC_DMA_CR register enables the DW_apb_i2c transmit handshaking interface. Writing a 1 into the RDMAE bit field of the IC_DMA_CR register enables the DW_apb_i2c receive handshaking interface.

2.17.2 Overview of Operation

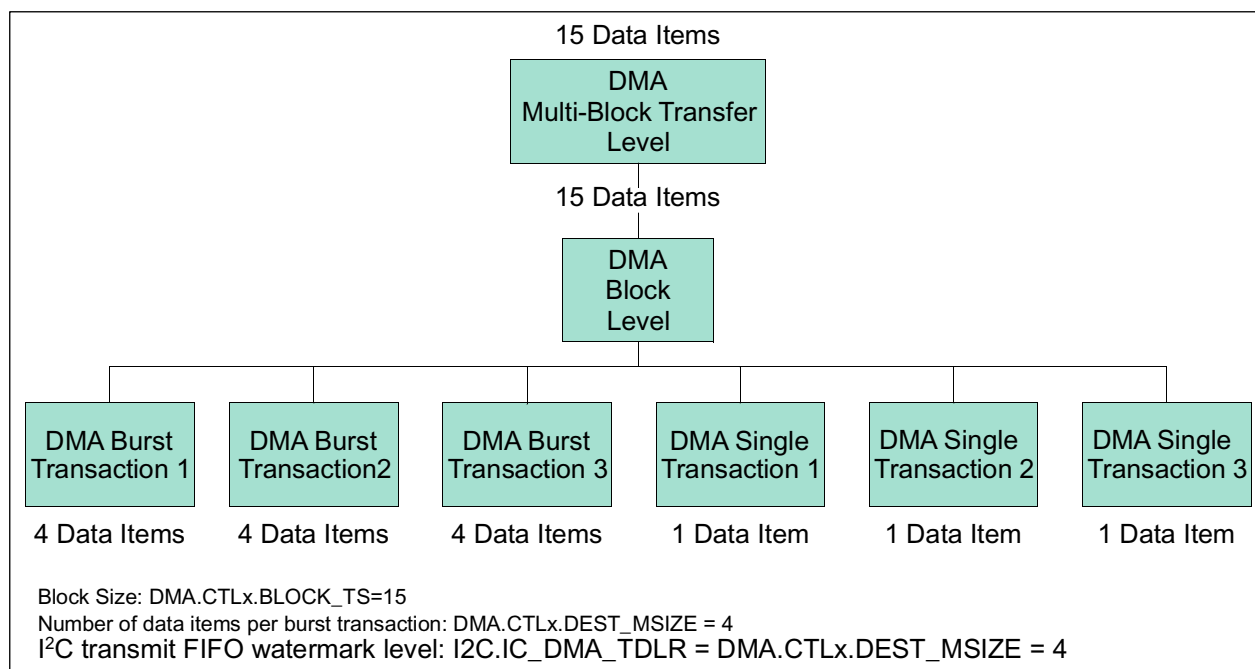
As a block flow control device, the DMA Controller is programmed by the processor with the number of data items (block size) that are to be transmitted or received by DW_apb_i2c; this is programmed into the BLOCK_TS field of the DW_ahb_dmac CTLx register.

The block is broken into a number of transactions, each initiated by a request from the DW_apb_i2c. The DMA Controller must also be programmed with the number of data items (in this case, DW_apb_i2c FIFO entries) to be transferred for each DMA request. This is also known as the burst transaction length and is programmed into the SRC_MSIZE/DEST_MSIZE fields of the DW_ahb_dmac CTLx register for source and destination, respectively.

Figure 2-35 shows a single block transfer, where the block size programmed into the DMA Controller is 12 and the burst transaction length is set to 4. In this case, the block size is a multiple of the burst transaction length. Therefore, the DMA block transfer consists of a series of burst transactions. If the DW_apb_i2c makes a transmit request to this channel, four data items are written to the DW_apb_i2c TX FIFO. Similarly, if the DW_apb_i2c makes a receive request to this channel, four data items are read from the DW_apb_i2c RX FIFO. Three separate requests must be made to this DMA channel before all 12 data items are written or read.

Figure 2-35 Breakdown of DMA Transfer into Burst Transactions

When the block size programmed into the DMA Controller is not a multiple of the burst transaction length, as shown in [Figure 2-36](#), a series of burst transactions followed by single transactions are needed to complete the block transfer.

Figure 2-36 Breakdown of DMA Transfer into Single and Burst Transactions

2.17.3 Transmit Watermark Level and Transmit FIFO Underflow

During DW_apb_i2c serial transfers, transmit FIFO requests are made to the DW_ahb_dmac whenever the number of entries in the transmit FIFO is less than or equal to the DMA Transmit Data Level Register (IC_DMA_TDLR) value; this is known as the watermark level. The DW_ahb_dmac responds by writing a burst of data to the transmit FIFO buffer, of length CTLx.DEST_MSIZEx.

If IC_EMPTYFIFO_HOLD_MASTER_EN parameter is set to 0, data should be fetched from the DMA often enough for the transmit FIFO to perform serial transfers continuously; that is, when the FIFO begins to empty another DMA request should be triggered. Otherwise, the FIFO runs out of data causing a STOP to be inserted on the I²C bus. To prevent this condition, you must set the watermark level correctly.

2.17.4 Choosing the Transmit Watermark Level

Consider the example where the assumption is made:

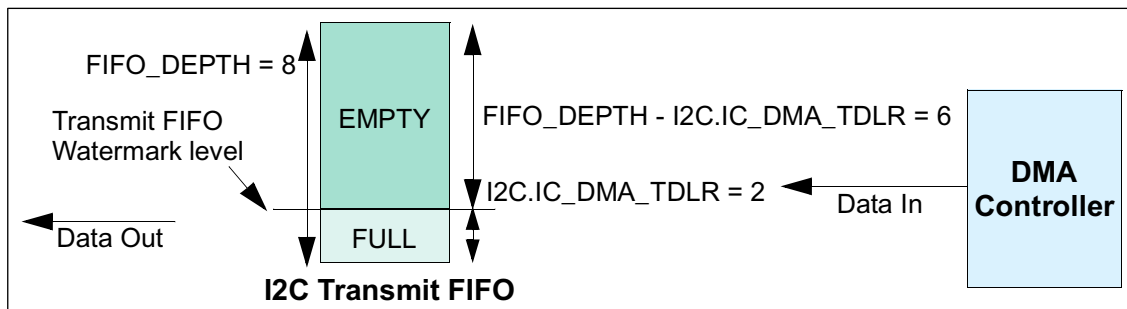
$$\text{DMA.CTLx.DEST_MSIZE} = \text{FIFO_DEPTH} - \text{I2C.IC_DMA_TDLR}$$

Here the number of data items to be transferred in a DMA burst is equal to the empty space in the Transmit FIFO. Consider two different watermark level settings.

2.17.4.1 Case 1: IC_DMA_TDLR = 2

- Transmit FIFO watermark level = I2C.IC_DMA_TDLR = 2
- DMA.CTLx.DEST_MSIZEx = FIFO_DEPTH - I2C.IC_DMA_TDLR = 6
- I²C transmit FIFO_DEPTH = 8
- DMA.CTLx.BLOCK_TS = 30

Figure 2-37 Case 1 Watermark Levels



Therefore, the number of burst transactions needed equals the block size divided by the number of data items per burst:

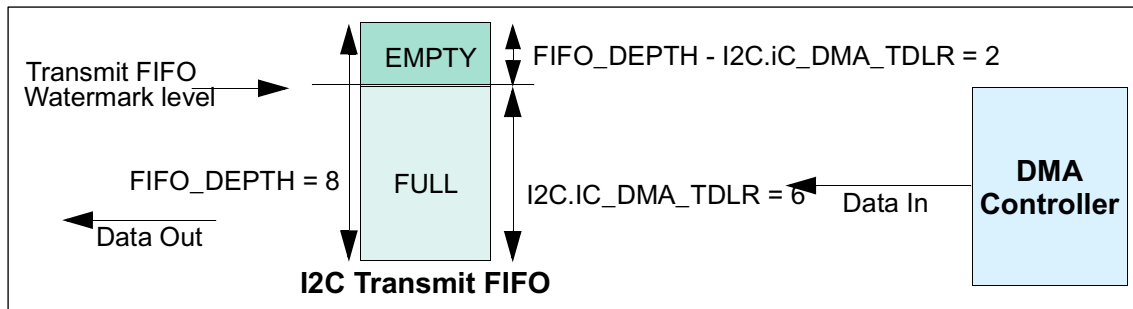
$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 30 / 6 = 5$$

The number of burst transactions in the DMA block transfer is 5. But the watermark level, I2C.IC_DMA_TDLR, is quite low. Therefore, the probability of an I²C underflow is high where the I²C serial transmit line needs to transmit data, but where there is no data left in the transmit FIFO. This occurs because the DMA has not had time to service the DMA request before the transmit FIFO becomes empty.

2.17.4.2 Case 2: IC_DMA_TDLR = 6

- Transmit FIFO watermark level = I2C.IC_DMA_TDLR = 6
- DMA.CTLx.DEST_MSIZ = FIFO_DEPTH - I2C.IC_DMA_TDLR = 2
- I²C transmit FIFO_DEPTH = 8
- DMA.CTLx.BLOCK_TS = 30

Figure 2-38 Case 2 Watermark Levels



Number of burst transactions in Block:

$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 30 / 2 = 15$$

In this block transfer, there are 15 destination burst transactions in a DMA block transfer. But the watermark level, I2C.IC_DMA_TDLR, is high. Therefore, the probability of an I²C underflow is low because the DMA controller has plenty of time to service the destination burst transaction request before the I²C transmit FIFO becomes empty.

Thus, the second case has a lower probability of underflow at the expense of more burst transactions per block. This provides a potentially greater amount of AMBA bursts per block and worse bus utilization than the former case.

Therefore, the goal in choosing a watermark level is to minimize the number of transactions per block, while at the same time keeping the probability of an underflow condition to an acceptable level. In practice, this is a function of the ratio of the rate at which the I²C transmits data to the rate at which the DMA can respond to destination burst requests.

For example, promoting the channel to the highest priority channel in the DMA, and promoting the DMA controller interface to the highest priority controller in the AMBA layer, increases the rate at which the DMA controller can respond to burst transaction requests. This in turn allows you to decrease the watermark level, which improves bus utilization without compromising the probability of an underflow occurring.

2.17.5 Selecting DEST_MSIZ and Transmit FIFO Overflow

As can be seen from [Figure 2-38](#), programming DMA.CTLx.DEST_MSIZ to a value greater than the watermark level that triggers the DMA request may cause overflow when there is not enough space in the I²C transmit FIFO to service the destination burst request. Therefore, the following equation must be adhered to in order to avoid overflow:

$$\text{DMA.CTLx.DEST_MSIZ} \leq \text{I2C.FIFO_DEPTH} - \text{I2C.IC_DMA_TDLR} \quad (1)$$

In [Case 2: IC_DMA_TDLR = 6](#), the amount of space in the transmit FIFO at the time the burst request is made is equal to the destination burst length, DMA.CTLx.DEST_MSIZEx. Thus, the transmit FIFO may be full, but not overflowed, at the completion of the burst transaction.

Therefore, for optimal operation, DMA.CTLx.DEST_MSIZEx should be set at the FIFO level that triggers a transmit DMA request; that is:

$$\text{DMA.CTLx.DEST_MSIZE} = \text{I2C.FIFO_DEPTH} - \text{I2C.IC_DMA_TDLR} \quad (2)$$

This is the setting used in [Figure 2-36](#).

Adhering to equation (2) reduces the number of DMA bursts needed for a block transfer, and this in turn improves AMBA bus utilization.



The transmit FIFO is not full at the end of a DMA burst transfer if the I²C has successfully transmitted one data item or more on the I²C serial transmit line during the transfer.

2.17.6 Receive Watermark Level and Receive FIFO Overflow

During DW_apb_i2c serial transfers, receive FIFO requests are made to the DW_ahb_dmac whenever the number of entries in the receive FIFO is at or above the DMA Receive Data Level Register; that is, IC_DMA_RDLR+1. This is known as the watermark level. The DW_ahb_dmac responds by fetching a burst of data from the receive FIFO buffer of length CTLx.SRC_MSIZEx.

Data should be fetched by the DMA often enough for the receive FIFO to accept serial transfers continuously; that is, when the FIFO begins to fill, another DMA transfer is requested. Otherwise, the FIFO fills with data (overflow). To prevent this condition, you must correctly set the watermark level.

2.17.7 Choosing the Receive Watermark level

Similar to choosing the transmit watermark level described earlier, the receive watermark level, IC_DMA_RDLR+1, should be set to minimize the probability of overflow, as shown in [Figure 2-39](#). It is a trade-off between the number of DMA burst transactions required per block versus the probability of an overflow occurring.

2.17.8 Selecting SRC_MSIZEx and Receive FIFO Underflow

As can be seen in [Figure 2-39](#), programming a source burst transaction length greater than the watermark level may cause underflow when there is not enough data to service the source burst request. Therefore, equation 3 following must be adhered to avoid underflow.

If the number of data items in the receive FIFO is equal to the source burst length at the time the burst request is made – DMA.CTLx.SRC_MSIZEx – the receive FIFO may be emptied, but not underflowed, at the completion of the burst transaction. For optimal operation, DMA.CTLx.SRC_MSIZEx should be set at the watermark level; that is:

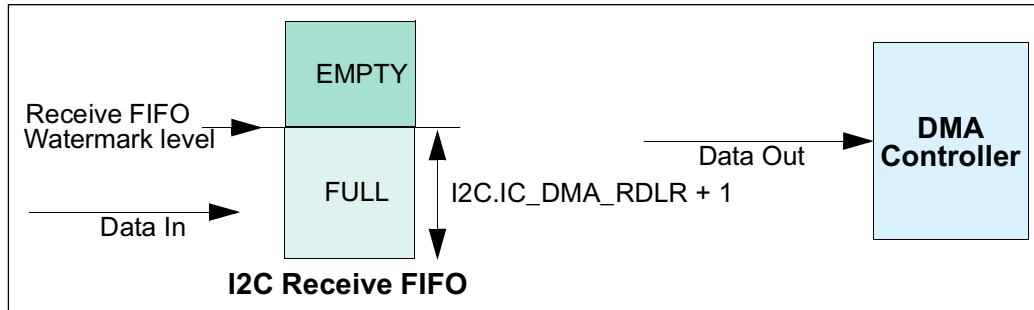
$$\text{DMA.CTLx.SRC_MSIZE} = \text{I2C.IC_DMA_RDLR} + 1 \quad (3)$$

Adhering to equation (3) reduces the number of DMA bursts in a block transfer, which in turn can avoid underflow and improve AMBA bus utilization.



Note The receive FIFO is not empty at the end of the source burst transaction if the I²C has successfully received one data item or more on the I²C serial receive line during the burst.

Figure 2-39 I²C Receive FIFO



2.17.9 Handshaking Interface Operation

The following sections discuss the handshaking interface.

2.17.9.1 dma_tx_req, dma_rx_req

The request signals for source and destination, `dma_tx_req` and `dma_rx_req`, are activated when their corresponding FIFOs reach the watermark levels as discussed earlier.

The DW_ahb_dmac uses rising-edge detection of the `dma_tx_req` signal/`dma_rx_req` to identify a request on the channel. Upon reception of the `dma_tx_ack`/`dma_rx_ack` signal from the DW_ahb_dmac to indicate the burst transaction is complete, the DW_apb_i2c de-asserts the burst request signals, `dma_tx_req`/`dma_rx_req`, until `dma_tx_ack`/`dma_rx_ack` is de-asserted by the DW_ahb_dmac.

When the DW_apb_i2c samples that `dma_tx_ack`/`dma_rx_ack` is de-asserted, it can re-assert the `dma_tx_req`/`dma_rx_req` of the request line if their corresponding FIFOs exceed their watermark levels (back-to-back burst transaction). If this is not the case, the DMA request lines remain de-asserted.

Figure 2-40 shows a timing diagram of a burst transaction where `pclk` = `hclk`.

Figure 2-40 Burst Transaction – `pclk` = `hclk`

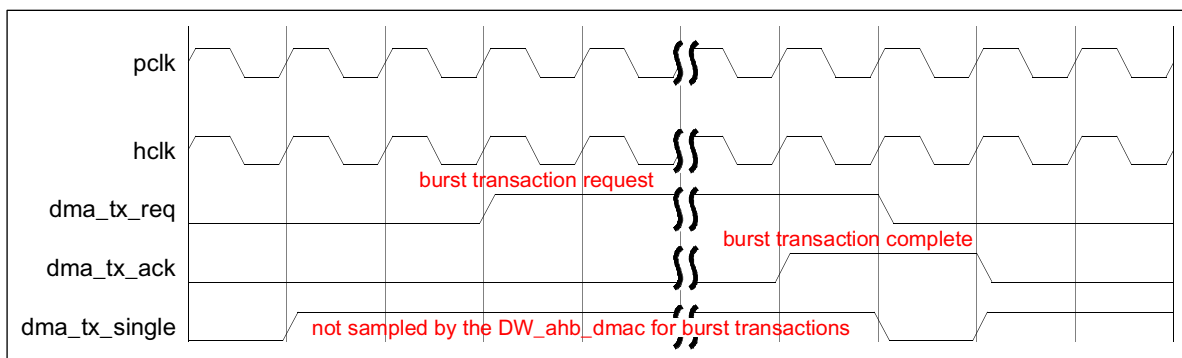
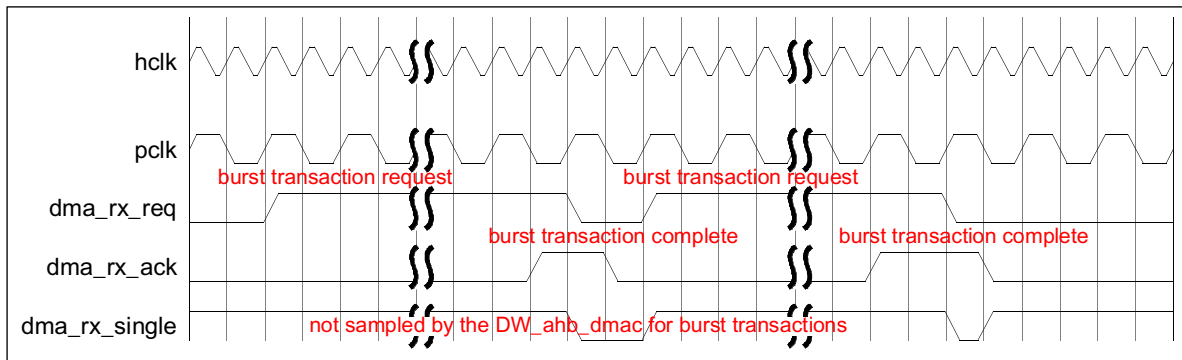


Figure 2-41 shows two back-to-back burst transactions where the `hclk` frequency is twice the `pclk` frequency.

Figure 2-41 Back-to-Back Burst Transactions – $hclk = 2 \cdot pclk$ 

The handshaking loop is as follows:

- dma_tx_req/dma_rx_req asserted by DW_apb_i2c
- dma_tx_ack/dma_rx_ack asserted by DW_ahb_dmac
- dma_tx_req/dma_rx_req de-asserted by DW_apb_i2c
- dma_tx_ack/dma_rx_ack de-asserted by DW_ahb_dmac
- dma_tx_req/dma_rx_req reasserted by DW_apb_i2c, if back-to-back transaction is required

**Note**

The burst transaction request signals, dma_tx_req and dma_rx_req, are generated in the DW_apb_i2c off pclk and sampled in the DW_ahb_dmac by hclk. The acknowledge signals, dma_tx_ack and dma_rx_ack, are generated in the DW_ahb_dmac off hclk and sampled in the DW_apb_i2c of pclk. The handshaking mechanism between the DW_ahb_dmac and the DW_apb_i2c supports quasi-synchronous clocks; that is, hclk and pclk must be phase-aligned, and the hclk frequency must be a multiple of the pclk frequency.

Two things to note here:

1. The burst request lines, dma_tx_req signal/dma_rx_req, once asserted remain asserted until their corresponding dma_tx_ack/dma_rx_ack signal is received even if the respective FIFO's drop below their watermark levels during the burst transaction.
2. The dma_tx_req/dma_rx_req signals are de-asserted when their corresponding dma_tx_ack/dma_rx_ack signals are asserted, even if the respective FIFOs exceed their watermark levels.

2.17.9.2 dma_tx_single, dma_rx_single

The dma_tx_single signal is a status signal. It is asserted when there is at least one free entry in the transmit FIFO and cleared when the transmit FIFO is full. The dma_rx_single signal is a status signal. It is asserted when there is at least one valid data entry in the receive FIFO and cleared when the receive FIFO is empty.

These signals are needed by only the DW_ahb_dmac for the case where the block size, CTLx.BLOCK_TS, that is programmed into the DW_ahb_dmac is not a multiple of the burst transaction length, CTLx.SRC_MSIZE, CTLx.DEST_MSIZE, as shown in Figure 2-36. In this case, the DMA single outputs inform the DW_ahb_dmac that it is still possible to perform single data item transfers, so it can access all data items in the transmit/receive FIFO and complete the DMA block transfer. The DMA single outputs

from the DW_apb_i2c are not sampled by the DW_ahb_dmac otherwise. This is illustrated in the following example.

Consider first an example where the receive FIFO channel of the DW_apb_i2c is as follows:

$\text{DMA.CTLx.SRC_MSIZE} = \text{I2C.iC_DMA_RDLR} + 1 = 4$

$\text{DMA.CTLx.BLOCK_TS} = 12$

For the example in [Figure 2-35](#), with the block size set to 12, the `dma_rx_req` signal is asserted when four data items are present in the receive FIFO. The `dma_rx_req` signal is asserted three times during the DW_apb_i2c serial transfer, ensuring that all 12 data items are read by the DW_ahb_dmac. All DMA requests read a block of data items and no single DMA transactions are required. This block transfer is made up of three burst transactions.

Now, for the following block transfer:

$\text{DMA.CTLx.SRC_MSIZE} = \text{I2C.IC_DMA_RDLR} + 1 = 4$

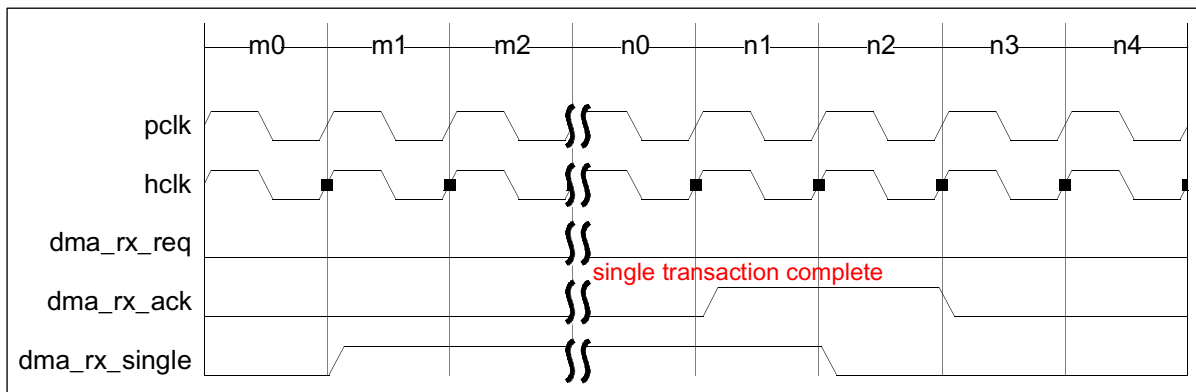
$\text{DMA.CTLx.BLOCK_TS} = 15$

The first 12 data items are transferred as already described using three burst transactions. But when the last three data frames enter the receive FIFO, the `dma_rx_req` signal is not activated because the FIFO level is below the watermark level. The DW_ahb_dmac samples `dma_rx_single` and completes the DMA block transfer using three single transactions. The block transfer is made up of three burst transactions followed by three single transactions.

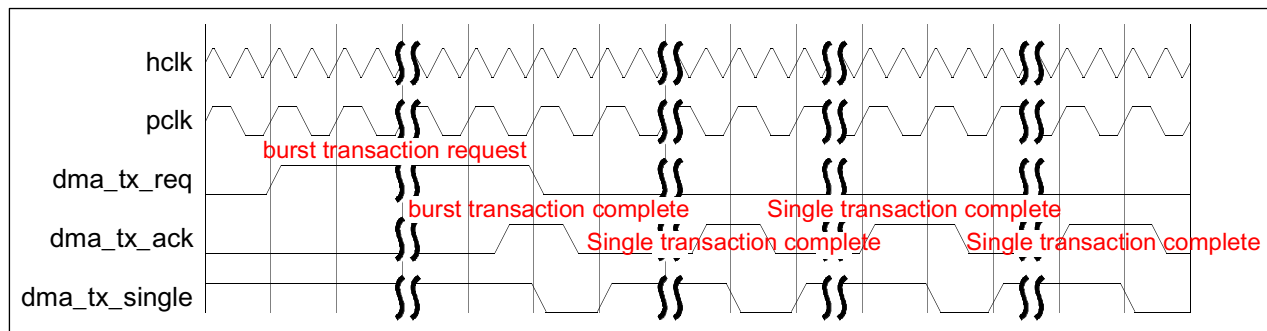
[Figure 2-42](#) shows a single transaction. The handshaking loop is as follows:

- `dma_tx_single/dma_rx_single` asserted by DW_apb_i2c
- `dma_tx_ack/dma_rx_ack` asserted by DW_ahb_dmac
- `dma_tx_single/dma_rx_single` de-asserted by DW_apb_i2c
- `dma_tx_ack/dma_rx_ack` de-asserted by DW_ahb_dmac.

Figure 2-42 Single Transaction



[Figure 2-43](#) shows a burst transaction, followed by three back-to-back single transactions, where the `hclk` frequency is twice the `pclk` frequency.

Figure 2-43 Burst Transaction + 3 Back-to-Back Singles – $hclk = 2 \cdot pclk$ **Note**

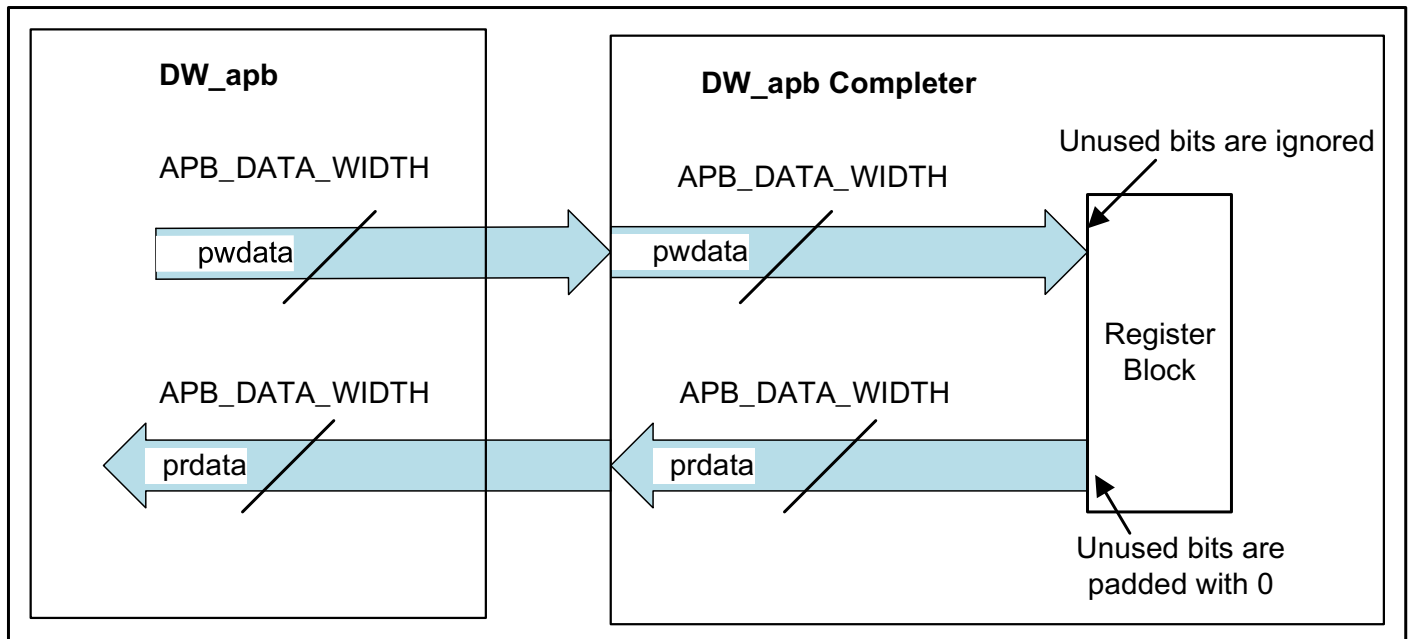
The single transaction request signals, dma_tx_single and dma_rx_single, are generated in the DW_apb_i2c on the pclk edge and sampled in DW_ahb_dmac on hclk. The acknowledge signals, dma_tx_ack and dma_rx_ack, are generated in the DW_ahb_dmac on the hclk edge and sampled in the DW_apb_i2c on pclk. The handshaking mechanism between the DW_ahb_dmac and the DW_apb_i2c supports quasi-synchronous clocks; that is, hclk and pclk must be phase aligned and the hclk frequency must be a multiple of pclk frequency.

2.18 APB Interface

The host processor accesses data, control, and status information on the DW_apb_i2c peripheral through the AMBA APB 2.0 interface. This peripheral supports APB data bus widths of 8, 16, or 32 bits, which is set with the APB_DATA_WIDTH parameter.

Figure 2-44 shows the read/write buses between the DW_apb and the APB completer.

Figure 2-44 Read/Write Buses Between the DW_apb and an APB Completer



The register interface of DW_apb_i2c is compliant to APB 2.0, APB 3.0, and APB 4.0 specifications. The SLAVE_INTERFACE_TYPE parameter is used to select the APB interface type of the register interface.

2.18.1 APB 3.0 Support

The following signals are included as part of APB 3.0 interface.

The PREADY signal should always be kept to its default value that is high for all APB accesses except for the IC_DATA_CMD register access. During the IC_DATA_CMD access, the PREADY signal is de-asserted to stall the APB transaction under the following conditions:

- Tx FIFO is empty: The APB transaction completes and PREADY is asserted if the data is received in the Rx FIFO before the register read/write timeout happens.
- Tx FIFO is full: The APB transaction completes and PREADY is asserted if the data is read out of the Tx FIFO before the register read/write timeout happens.

If timeout happens, then the APB transaction is unsuccessful that is the RX FIFO is not read or the TX FIFO is not written. The APB transaction must be re-initiated by application for successful completion.

The PSLVERR signal functionality is enabled when the SLVERR_RESP_EN parameter is set to 1, so that DW_apb_i2c register interface can provide APB completer error response (if required). The DW_apb_i2c generates the error response under the following conditions:

- Enabling both controller mode and target mode in IC_CON register, which is an invalid programming.

- Writing high into CMD bit of the IC_DATA_CMD register, when IC_SLV_DISABLE bit of IC_CON register is low.
- The DW_apb_i2c stalls the APB transaction by pulling PREADY signal low, because the Rx FIFO is empty or the Tx FIFO is full. To avoid locking of the bus for the large number of clock cycles, a timeout option is provided through configuration parameter REG_TIMEOUT_VALUE. The timeout is triggered under the following conditions:
 - Rx FIFO remains empty or
 - TX FIFO remains full

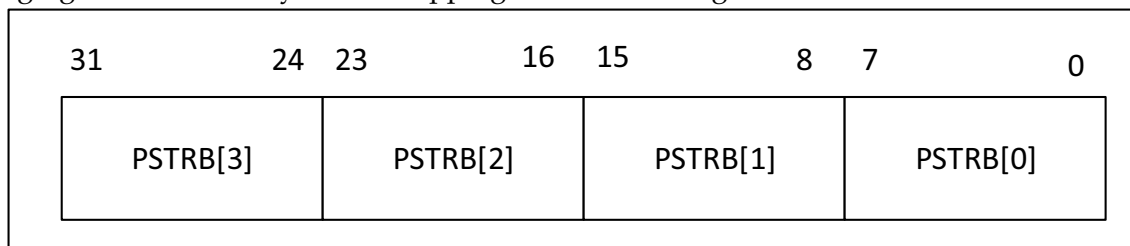
If the duration is equal to the timeout period that is REG_TIMEOUT_VALUE, then APB interface asserts PSLVERR signal to indicate the register read/write timeout.

2.18.2 APB 4.0 Support

The DW_apb_i2c register interface is compliant to the APB 4.0 specification and to adhere to this compliance, the PSTRB and PPROT signals are added to the APB interface.

In the write transaction to this interface, the PSTRB signal indicates validity of PWDATA bytes. DW_apb_i2c component selectively writes to the bytes of the addressed register whose corresponding bit in the PSTRB signal is high. Bytes strobed low by the corresponding PSTRB bits are not modified. The incoming strobe bits for a read transaction is always zero as per the protocol.

Following figure shows the byte lane mapping of the PSTRB signal.



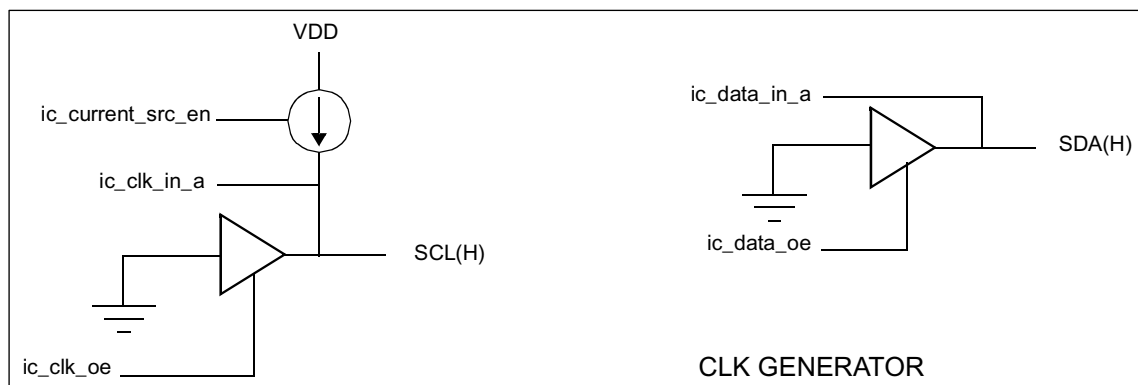
Note

- The PSTRB is not supported for write transaction on the IC_DATA_CMD register.
- The PPROT signal is added for interface consistency and PPROT signal is not used internally.
- APB3/APB4 configurations (SLAVE_INTERFACE_TYPE >= 1), rx_underflow and tx_overflow interrupts are not generated.

2.19 I/O Connections

As illustrated in Figure 2-45, the I²C interface consists of two wires, a clock (SCL) and data (SDA). For high speed systems, the names are SCLH and SDAH. For high speed mode, a current source pull-up may be used on the SCLH line. It is enabled during some active controller transactions. The SDA and SDAH connections are the same at any speed. There are no special connections required for the DesignWare APB completer interface side of the DW_apb_i2c.

Figure 2-45 I/O Connection to I²C Interface



2.20 DW_apb_i2c Registers

Chapter 5, “Register Descriptions” lists all the registers available in DW_apb_i2c. Note that there are references to both hardware parameters and software registers throughout the chapter Chapter 5, “Register Descriptions”. Parameters and many of the register bits are prefixed with an IC_*. However, the software register bits are distinguished in Chapter 5, “Register Descriptions” by italics. For instance, IC_MAX_SPEED_MODE is a hardware parameter and configured once using Synopsys coreConsultant, whereas the IC_SLAVE_DISABLE bit in the IC_CON register controls whether I²C has its target disabled.



Note

A read operation to an address location that contains unused bits results in a 0 value being returned on each of the unused bits.

2.20.1 Registers and Field Descriptions

Registers in DW_apb_i2c are on the pclk domain, but status bits reflect actions that occur in the ic_clk domain. Therefore, there is delay when the pclk register reflects the activity that occurred on the ic_clk side.

Some registers may be written only when the DW_apb_i2c is disabled, programmed by the IC_ENABLE register. Software should not disable the DW_apb_i2c while it is active. If the DW_apb_i2c is in the process of transmitting when it is disabled, it stops as well as deletes the contents of the transmit buffer after the current transfer is complete. The target continues receiving until the remote controller terminates the transfer, in which case the DW_apb_i2c could be disabled. Registers that cannot be written to when the DW_apb_i2c is enabled are indicated in their descriptions.

Unless the clocks pclk and ic_clk are identical (IC_CLK_TYPE = 0), there is a two-register delay for synchronous and asynchronous modes.

2.20.2 Operation of Interrupt Registers

Table 2-8 lists the operation of the DW_apb_i2c interrupt registers and how they are set and cleared. Some bits are set by hardware and cleared by software, whereas other bits are set and cleared by hardware.

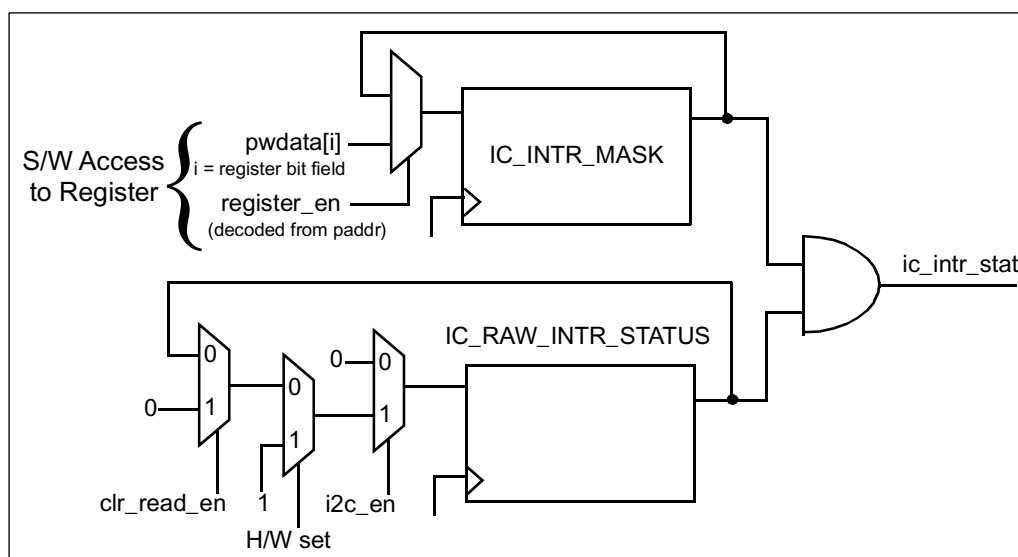
Table 2-8 Clearing and Setting of Interrupt Registers

Interrupt Bit Fields	Set by Hardware/ Cleared by Software	Set and Cleared by Hardware
MST_ON_HOLD	No	Yes
RESTART_DET	Yes	No
GEN_CALL	Yes	No
START_DET	Yes	No
STOP_DET	Yes	No
ACTIVITY	Yes	No
RX_DONE	Yes	No
TX_ABRT	Yes	No
RD_REQ	Yes	No

Interrupt Bit Fields	Set by Hardware/ Cleared by Software	Set and Cleared by Hardware
TX_EMPTY	No	Yes
TX_OVER	Yes	No
RX_FULL	No	Yes
RX_OVER	Yes	No
RX_UNDER	Yes	No

Figure 2-46 shows the operation of the interrupt registers where the bits are set by hardware and cleared by software.

Figure 2-46 Interrupt Scheme



2.21 SMBus/PMBus

The SMBus is designed to provide a predictable communication line between a system and its devices. It describes the Device timeout definitions and their conditions.

**Note**

As per SMBus specification maximum operation frequency, High Speed mode is not supported when SMBus feature is enabled in DW_apb_i2c.

2.21.1 tTIMEOUT,MIN Parameter

This Parameter allows a controller or target to conclude that a defective device is holding the clock low indefinitely or a controller is intentionally trying to drive devices off the bus. It is highly recommended that a target device release the bus (stop driving the bus and let SMBCLK and SMBDAT float high) when it detects any single clock held low longer than tTIMEOUT,MIN. Devices that have detected this condition must reset their communication interface and be able to receive a new START condition in no later than tTIMEOUT,MAX.

The DW_apb_i2c enables the bus clear feature in SMBus mode and the you can use the IC_SCL_STUCK_AT_LOW_TIMEOUT Register to program the tTIMEOUT,MIN value to detect the SMBCLK low timeout.

The DW_apb_i2c target device resets its communication interface and release both SCL and SDA lines after detecting the SCL_STUCK_TIMEOUT interrupt.

The DW_apb_i2c controller has a provision to generate the Termination which completes the current transfer and generate STOP condition on the bus through programming the IC_ENABLE[1] register bit.

2.21.2 Controller Device Clock Extension

The interval tLOW:MEXT is defined as the cumulative time a controller device is allowed to extend its clock cycles within one byte in a message as measured from:

- START to ACK
- ACK to ACK
- ACK to STOP.

The DW_apb_i2c Controller uses the IC_SMBUS_CLOCK_LOW_MEXT register to detect the Controller device clock extension timeout and generates SMBUS_CLK_LOW_MEXT interrupt.

2.21.3 Target Device Clock Extension

The interval tLOW:SEXT is the cumulative time a given target device is allowed to extend the clock cycles in one message from the initial START to the STOP.

The DW_apb_i2c controller uses the IC_SMBUS_CLOCK_LOW_SEXT register to detect the Target device clock extension timeout and generates SMBUS_CLK_LOW_SEXT interrupt.

A controller is allowed to terminate the transaction in progress to any target that violates the tLOW:SEXT or tTIMEOUT,MIN specifications through the enabling the user termination (IC_ENABLE[1]).

2.21.4 SMBDAT Low Timeout

A malfunctioning device holds the SMBDAT line low indefinitely. This would prevent the controller from issuing a STOP condition and ending a transaction. If SMBDAT is still low $t_{\text{TIMEOUT},\text{MAX}}$ after SMBCLK has gone high at the end of a transaction the controller should hold SMBCLK low for at least $t_{\text{TIMEOUT},\text{MAX}}$ in an attempt to reset the SMBus interface of all of the devices on the bus.

The DW_apb_i2c enables the Bus clear feature in SMBus mode and you can use the IC_SDA_STUCK_AT_LOW_TIMEOUT Register to program the SMBDAT timeout value to detect the SMBDAT low timeout. If SMBDAT line is stuck at low, the SDA_STUCK_TIMEOUT termination is generated and software can enable the SMBUS_CLK_RESET register bit of IC_ENABLE register to hold the SCL low for IC_SCL_STUCK_AT_LOW_TIMEOUT which in turn resets the SMBus interface of all devices on the bus.

2.21.5 Bus Protocols

A typical SMBus device has a set of commands by which data can be read and written. All commands are one byte long while their arguments and return values can vary in length. In accordance with the SMBus specification, the most significant bit (MSB) is transferred first. There are sixteen possible command protocols for any given device. These commands are Quick Command, Send Byte, Receive Byte, Write Byte, Write Word, Read Byte, Read Word, Process Call, Block Read, Block Write, and Block Write-Block Read Process Call, SMBus Host Notify protocol, Write 32 protocol, Read 32 protocol, Write 64 protocol, Read 64 protocol.

SMBus protocols for message transactions are generally different from I²C data transfer commands. It is still possible to program an SMBus controller to deliver I²C data transfer commands. Table 2-9 describes the derivation of SMBus Bus Protocols through TX-FIFO commands in DW_apb_i2c.

In the SMBus Controller mode, all the receive data bytes are available in RX-FIFO. In the SMBus Target mode, all the bus protocol command codes and data bytes are received in the RX-FIFO and read request data bytes must be sent using TX-FIFO, similar to the I²C mode.

Table 2-9 SMBus Bus Protocols Usage in DW_apb_i2c

Protocol	Required TX-FIFO Commands	Command/Data (IC_DATA_CMD[7:0])	CMD bit (IC_DATA_CMD[8])	STOP bit (IC_DATA_CMD[9])	Remarks
Quick Command	1	Not Applicable	Set the command [R/W]	Set to 1	Set IC_TAR[11] and IC_TAR[16] to 1
Send Byte	1	Data Byte	Set to 0	Set to 1	
Receive Byte	1	Not Applicable	Set to 1	Set to 1	
Write Byte	2	Command Code	Set to 0	Set to 0	
		Data Byte	Set to 0	Set to 1	
Write Word	3	Command Code	Set to 0	Set to 0	
		Data Byte Low	Set to 0	Set to 0	
		Data Byte High	Set to 0	Set to 1	

Protocol	Required TX-FIFO Commands	Command/Data (IC_DATA_CMD[7:0])	CMD bit (IC_DATA_CMD[8])	STOP bit (IC_DATA_CMD[9])	Remarks
Read Byte	2	Command Code	Set to 0	Set to 0	
		Not Applicable	Set to 1	Set to 1	
Read Word	3	Command Code	Set to 0	Set to 0	
		Not Applicable	Set to 1	Set to 0	
		Not Applicable	Set to 1	Set to 1	
Process Call	5	Command Code	Set to 0	Set to 0	
		Data Byte Low	Set to 0	Set to 0	
		Data Byte High	Set to 0	Set to 0	
		Not Applicable	Set to 1	Set to 0	
		Not Applicable	Set to 1	Set to 1	
Block Write	N+2	Command Code	Set to 0	Set to 0	
		Data Byte	Set to 0	Set to 0	
		(N+1) Data Byte N	Set to 0	Set to 1	
Block Read	N+2	Command Code	Set to 0	Set to 0	
		Not Applicable	Set to 1	Set to 0	
		(N+1) Not Applicable	Set to 1	Set to 1	
Block Write-Block Read Process Call	M+N+3	Command Code	Set to 0	Set to 0	
		Data Byte 1	Set to 0	Set to 0	
		(M+1) Data Byte M	Set to 0	Set to 0	
		(M+1+1) Not Applicable	Set to 1	Set to 0	
		(M+1+2) Not Applicable	Set to 1	Set to 0	
		(M+1+N+1) Not Applicable	Set to 1	Set to 1	
SMBUS Host Notify Protocol	3	Device Address	Set to 0	Set to 0	Set IC_TAR[6:0] to SMB Host Address (0001 000)
		Data Byte Low	Set to 0	Set to 0	
		Data Byte High	Set to 0	Set to 1	

Protocol	Required TX-FIFO Commands	Command/Data (IC_DATA_CMD[7:0])	CMD bit (IC_DATA_CMD[8])	STOP bit (IC_DATA_CMD[9])	Remarks
Write 32	5	Command Code	Set to 0	Set to 0	
		Data Byte 1	Set to 0	Set to 0	
		Data Byte 2	Set to 0	Set to 0	
		Data Byte 3	Set to 0	Set to 0	
		Data Byte 4	Set to 0	Set to 1	
Read 32	5	Command code	Set to 0	Set to 0	
		Not Applicable (1)	Set to 1	Set to 0	
		Not Applicable (2)	Set to 1	Set to 0	
		Not Applicable (3)	Set to 1	Set to 0	
		Not Applicable (4)	Set to 1	Set to 1	
Write 64	9	Command code	Set to 0	Set to 0	
		Data Byte 1	Set to 0	Set to 0	
		Data Byte 2	Set to 0	Set to 0	
		Data Byte 3	Set to 0	Set to 0	
		Data Byte 4	Set to 0	Set to 0	
		Data Byte 5	Set to 0	Set to 0	
		Data Byte 6	Set to 0	Set to 0	
		Data Byte 7	Set to 0	Set to 0	
		Data Byte 8	Set to 0	Set to 1	
Read 64	9	Command code	Set to 0	Set to 0	
		Not Applicable (1)	Set to 1	Set to 0	
		Not Applicable (2)	Set to 1	Set to 0	
		Not Applicable (3)	Set to 1	Set to 0	
		Not Applicable (4)	Set to 1	Set to 0	
		Not Applicable (5)	Set to 1	Set to 0	
		Not Applicable (6)	Set to 1	Set to 0	
		Not Applicable (7)	Set to 1	Set to 0	
		Not Applicable (8)	Set to 1	Set to 1	

DW_apb_i2c Target can be enabled to receive only Quick command through enabling the SMBUS_SLAVE_QUICK_EN bit in the IC_CON Register. Whenever this bit is selected the target only receives quick commands and are not accept other Bus Protocols. The DW_apb_i2c target issues the SMBUS_QUICK_DET interrupt upon receiving the QUICK command.

SMBus introduces a Packet Error checking Mechanism through appending PEC Byte at the end of the Bus Protocol. This can be achieved through adding an extra command (PEC byte) while transferring and decoding it while receiving by the software.

2.21.6 SMBus Address Resolution Protocol

SMBus target device address conflicts can be resolved by dynamically assigning a new unique address to each target device by the Host. This feature allows the devices to be 'hot-plugged' in to the system.

SMBus introduces a 128-bit Unique Device ID (UDID) for each device in the system to isolate each device for the purpose of address assignment. DW_apb_i2c uses the IC_SMBUS_UDID_MSB parameter for upper constant 96 bits and 'IC_SMBUS_ARP_UDID_LSB' register for lower variable 32 bits of the UDID.

DW_apb_i2c uses the PERSISTANT_SLV_ADDR_EN register bit in IC_CON register to indicate whether the DW_apb_i2c supports persistent target address.

DW_apb_i2c controller can issue general and directed Address Resolution Protocol (ARP) commands to assign the dynamic address for the targets in the SMBus system.

Table 2-10 describes the derivation of SMBus ARP commands through Tx-FIFO commands in DW_apb_i2c.

Table 2-10 Derivation of SMBus ARP Command Through Tx-FIFO Commands in DW_apb_i2c

ARP Command	Required TX-FIFO Commands	Command/Data (IC_DATA_CMD[7:0])	CMD bit (IC_DATA_CMD[8])	STOP bit (IC_DATA_CMD[9])	Remarks
Prepare for ARP	2	Command = '0000 0001'	Set to 0	Set to 0	Set IC_TAR[6:0] to SMB Default Address (1100 001)
		PEC Byte	Set to 0	Set to 1	
Reset Device (General)	2	Command = '0000 0010'	Set to 0	Set to 0	Set IC_TAR[6:0] to SMB Default Address (1100 001)
		PEC Byte	Set to 0	Set to 1	
Get UDID (General)	20	Command = '0000 0011'	Set to 0	Set to 0	1. Set IC_TAR[6:0] to SMB Default Address (1100 001). 2. 16 Reads to be performed for the 128 UDID bytes. 3. Last read command for the target device address.
		Not Applicable	Set to 1	Set to 0	
		Not Applicable (16 times)	Set to 1	Set to 0	
		Not Applicable	Set to 1	Set to 0	
		PEC Byte	Set to 1	Set to 1	

ARP Command	Required TX-FIFO Commands	Command/Data (IC_DATA_CMD[7:0])	CMD bit (IC_DATA_CMD[8])	STOP bit (IC_DATA_CMD[9])	Remarks
Assign Address	20	Command = '0000 0100'	Set to 0	Set to 0	- Set IC_TAR[6:0] to SMB Default Address (1100 001). 16 Writes to be performed for the 128 UDID byte. - Last Write command for the Assigned target device address.
		Byte Count = 17	Set to 0	Set to 0	
		UDID Byte 15 to 0	Set to 0	Set to 0	
		Assigned Address	Set to 0	Set to 0	
		PEC Byte	Set to 0	Set to 1	
Get UDID (Directed)	20	{target device address[6:0], 1'b1}	Set to 0	Set to 0	- Set IC_TAR[6:0] to SMB Default Address (1100 001). - One read to be performed to get the byte count. 16 Reads to be performed for the 128 UDID byte. - Last Read command for the target device address.
		Not Applicable	Set to 1	Set to 0	
		Not Applicable (16 times)	Set to 1	Set to 0	
		Not Applicable	Set to 1	Set to 0	
		PEC Byte	Set to 1	Set to 1	
Reset Device (Directed)	2	Command = {target device Address[6:0], 0}	Set to 0	Set to 0	Set IC_TAR[6:0] to SMB Default Address (1100 001)
		PEC byte	Set to 0	Set to 1	
Notify ARP Controller	3	Device Address = '1100 001' {Device Address, 1'b0}	Set to 0	Set to 0	Set IC_TAR[6:0] to SMB Host Address (0001 000)
		Data Byte Low = '0000 0000'	Set to 0	Set to 0	
		Data Byte High = '0000 0000'	Set to 0	Set to 1	

**Note**

- DW_apb_i2c target hardware:
 - Handles the generation, detection, and NACKing of the wrong PEC (CRC8 $C(X)=X8+X2+X1+1$) for the ARP Commands.
 - Does not handle the PEC for Non-ARP commands.
- DW_apb_i2c controller hardware does not handle PEC for both ARP and non-ARP commands.

2.21.6.1 Procedure to Perform ARP in Controller Mode

To use the DW_apb_i2c as a SMBus Controller/Host for assigning the unique address to each target device to resolve the target device address conflicts, perform the following steps:

1. After a reset or a cold power up, the SMBus host or controller issues a “Prepare to ARP” command to indicate that the controller is carrying an ARP to assign dynamic addresses to all devices. Target must flush any pending host notify commands.
2. An acknowledgment received for the “Prepare to ARP” command indicates that ARP-capable devices exist in the system and the “Get UDID” command must be issued. A NACK indicates that ARP-capable devices do not exist or currently all targets have their addresses resolved. In this case, the controller must complete steps outlined from Step 8 onwards. The DW_apb_i2c controller indicates NACK reception through 'ABRT_7B_ADDR_NOACK' and 'ABRT_TXDATA_NOACK' bits of IC_TX_ABRT_SOURCE register.
3. DW_apb_i2c controller issues 'Get UDID' to receive the UDID information of the target for assigning the dynamic address.
4. If the first three bytes of the “Get UDID” command are ACK'ed and the receive byte count is 0x11, then the controller issues the “Assign Address” command. Else, the controller must complete steps outlined in step 8 onwards to indicate that the ARP is complete. DW_apb_i2c controller indicates NACK reception through ABRT_7B_ADDR_NOACK and ABRT_TXDATA_NOACK bits of the IC_TX_ABRT_SOURCE register.
5. The controller issues the “Assign Address” command to assign the Dynamic address to the target whose UDID is received through “Get UDID command”.
6. If the assigned address packet is ACK'ed, then controller removes the assigned address from the address pool and moves to Step 3 to get UDID of another target. If the packet is not ACK'ed, then controller does not remove the address from the address pool and moves to Step 3 to get UDID of same target or another target.
7. If the Assign Address is ACK'ed, then controller stores the assigned address in the used address pool with the UDID characteristics of the device.
8. The controller moves to Step 3 to issue a 'Get UDID' command again to receive the UDID of another target. If it receives NACK for 'Get UDID', the controller moves to Step 9.
9. The DW_apb_i2c can be switched to Target mode to detect device requests for Host Notify Protocol.
10. If the DW_apb_i2c switched to target mode and DW_apb_i2c detects the Host Notify Protocol, then this indicates that a target is requesting for the dynamic address and the controller has to undergo the ARP as outlined in Step 11.
11. If the DW_apb_i2c is in Controller mode, then move to Step 3 for performing ARP procedure, otherwise move to Step 12.
12. The DW_apb_i2c is switched to Controller Mode and moves to Step 3 to perform ARP procedure.

The detailed flow diagram is explained in [Figure 6-12](#) on page 385.

2.21.6.2 Procedure to Perform ARP in Target Mode

The DW_apb_i2c as a SMBus target performs the following tasks:

- Decodes the ARP commands and responds based on internal state flags SMBUS_SLAVE_ADDR_VALID and 'SMBUS_SLAVE_ADDR_RESOLVED' of the IC_STATUS register.
- Generates and Validates the PEC byte of ARP commands

- Generates ACK for the PEC byte only if it matches the CRC value calculated on data it received. If not, NACK the PEC byte.

When another SMBus Controller/Host device on the bus generates the ARP commands and requests to participate in the ARP, the DW_apb_i2c acts as a SMBus target and performs the following steps:

1. After a reset or a cold power up, the DW_apb_i2c target device checks whether it supports a persistent target address or default target address.
2. If DW_apb_i2c has a persistent target address (PSA) or Default Target Address (DSA), which is indicated by the Address Valid flag being set, then PSA/DSA is set in the Target Device Address Register (IC_SAR/IC_SARx; for x <= IC_NUM_SARS) register. If the flag is not set, then proceed to Step 4.
3. DW_apb_i2c persistent target stores the persistent address in IC_SAR and sets Address Valid flag to 1 and Address Resolved Flag to 0.
4. DW_apb_i2c Non Persistent target (non-PSA) clears both Address Valid and Address Resolved Flags.
5. DW_apb_i2c Checks whether any Packet received has ARP Default address in the target device address field of the packet to decide on ARP command or normal command. If there is a match then DW_apb_i2c target proceeds to Step 6, otherwise to Step 25.
6. If DW_apb_i2c detects a packet addressed to the SMBus Device Default Address, it checks the command field to determine if this is the "Prepare to ARP" command. If so, then it proceeds to Step 7, otherwise it proceeds to Step 8.
7. Upon receipt of the "Prepare to ARP" command, the DW_apb_i2c acknowledges the packet and clears the Address Resolved flag in order to participate in the ARP Process. DW_apb_i2c proceeds to Step 5 and waits for another SMBus Packet.
8. The DW_apb_i2c checks the command field to verify if the "Reset Device" command is issued. If yes, the DW_apb_i2c proceeds to Step 9, otherwise it proceeds to Step 10.
9. Upon receipt of the "Reset Device" command, the DW_apb_i2c acknowledges the packet and clears the Address Resolved and Address Valid (If non-PSA and ic_con[19]=0) flags. The Address Valid flag of the DSA device remains set. DW_apb_i2c proceeds to Step 5 and waits for another SMBus Packet.
10. The device checks the command to verify if the "Assign Address" command is issued. If yes, then it proceeds to Step 11, otherwise proceeds to Step 13.
11. Upon receipt of the "Assign Address" command, the DW_apb_i2c compares its UDID with one its received bytes. If any byte does not match, then DW_apb_i2c does not acknowledge that byte and subsequent bytes also. If all bytes in the UDID matches, then the DEVICE proceeds to Step 12, otherwise it proceeds to Step 5 and waits for another SMBus packet.
12. After the UDID is matched in Step 11, the DW_apb_i2c receives the target device address and sets the IC_SAR register with this target device address. The DW_apb_i2c sets its Address Valid and Address Resolved flags, which means it has received the dynamic address and is no longer respond to the "Get UDID" command unless it receives the "Prepare to ARP" or "Reset Device" commands. DW_apb_i2c now proceeds to Step 5 and waits for another SMBus packet.
13. The DW_apb_i2c checks the command field to verify if the "Get UDID" command is issued. If yes, then it proceeds to Step 14, otherwise to Step 19.
14. Upon receipt of the "Get UDID" command, the DW_apb_i2c checks its Address Resolved flag to determine whether it must participate in an ARP process. If set, then its address has already been resolved by the ARP Controller, so the device proceeds to Step 5 and waits for another SMBus packet. If the ARP Flag is cleared, then it proceeds to Step 15.
15. The DW_apb_i2c returns its UDID and monitors the SMBus data line for collisions. If a collision is detected at any time, DW_apb_i2c generates the ARB_LOST and ABRT_SLV_ARBLOST bits and stops

transmitting. Further, it proceeds to Step 5 and waits for another SMBus packet. If collisions are not detected, then DW_apb_i2c proceeds to Step 16.

16. The DW_apb_i2c check its Address Valid (AV) flag to determine the value to return for the Device Target Address field. If the AV flag is set, then it proceeds to Step 17, otherwise it proceeds to Step 18.
17. When the AV flag is set, the current IC_SAR is valid, therefore the device returns this for the Device target Address field (with bit 0 set) and monitors the SMBus data line for collisions. DW_apb_i2c proceeds to Step 5 and waits for another SMBus Packet.
18. When the AV flag is not set, the current target device address (IC_SAR) is invalid. Therefore, the DW_apb_i2c returns a value of FFh and monitors the SMBus data line for collisions. The device requires an address assignment if the ARP controller receives the FFh value. DW_apb_i2c proceeds to Step 5 and waits for another SMBus packet.
19. The DW_apb_i2c may be receiving a directed command. If the Address Valid flag is set and address is the same as in IC_SAR, then proceed to Step 20 otherwise, proceed to Step 5 to wait for another SMBus packet.
20. If the Address Valid flag is set, check if the command is a directed “Reset Device” command. If yes, then proceed to Step 21, otherwise proceed to Step 22.
21. Upon receipt of the “Reset Device” command, the DW_apb_i2c acknowledges the packet and clears the Address Resolved and Address Valid (If non-PSA and ic_con[19]=0) flags. The Address Valid flag of the DSA device remains set. DW_apb_i2c proceeds to Step 5 and waits for another SMBus Packet.
22. DW_apb_i2c checks whether the received command is a “Directed Get UDID” command. If yes, then proceed to Step 23 and return the UDID information. If not, then proceed to Step 24.
23. If the received command is a “Directed Get UDID” command, then return the UDID information and current target address, proceed to Step 5 and wait for another SMBus Packet.
24. If the received command is a “Directed Get UDID” command, the DW_apb_i2c has not received a valid ARP command and hence DW_apb_i2c NACKs the command and proceeds to Step 5 and wait for another SMBus Packet.
25. If the Address Valid bit is set then it proceeds to Step 26, otherwise it proceeds to Step 5 and waits for another SMBus Packet. The received address is not the SMBus Device Default Address and the packet may be addresses to the DW_apb_i2c's core function. The device checks its Address Valid bit to determine whether to respond.
26. When the address valid bit is set, DW_apb_i2c has a valid target address. It compares the received target address to its target address, and if there is a match, DW_apb_i2c proceeds to Step 27, otherwise it proceeds to Step 5 and waits for another SMBus Packet.
27. The DW_apb_i2c receives a packet addresses to its core function and hence it acknowledges the packet and processes it accordingly. DW_apb_i2c proceeds to step 5 and waits for another SMBus Packet.

The detailed flow diagram is explained in [Figure 6-13](#) on page 387.

2.21.7 Notify ARP Controller Command Usage

This section explains the SMBus Notify ARP Controller command usage. Notify ARP Controller command notifies the ARP Controller that the device requires Address resolution. This informs the ARP Controller that a new device is present in the system.

2.21.7.1 SMBus Specification Requirement for Notify ARP Controller Command

As per the SMBus specification the Notify ARP Controller command can be generated under the following circumstances:

- When the device is powered on.
- When the device senses a bus collision during a read operation directed to its Assigned Target Device Address.

2.21.7.2 DW_apb_i2c Usage Model for Notify ARP Controller Command

System level prerequisites:

- The system must account for the change from Controller to Target or vice-versa.
- During operating mode change, if DW_apb_i2c receives an incoming ARP command when it is not enabled, then DW_apb_i2c NACKs the ARP command. The host is expected to know that a Notify ARP Controller Command is requested, and the Host can re-initiate the ARP sequence.

To support Notify ARP Controller command in DW_apb_i2c:

1. Disable DW_apb_i2c as specified in [“Disabling DW_apb_i2c”](#) on page 59.
2. Write the registers IC_CON.IC_SLAVE_DISABLE = 1, and IC_CON.MASTER_MODE = 1 to change the DW_apb_i2c mode of operation to the Controller mode from the Target mode.
3. Write the register IC_TAR = SMBus Host Address (0001 000).
4. Enable DW_apb_i2c by writing the register IC_ENABLE.ENABLE = 1.
5. Write the IC_DATA_CMD register as per [Table 2-10](#) to generate the Notify ARP Controller command.
6. DW_apb_i2c sends the Notify ARP Controller command on the I²C bus.
7. Poll for the IC_STATUS.ACTIVITY register until it is read as 0.
8. Disable DW_apb_i2c as specified in [“Disabling DW_apb_i2c”](#) on page 59.
9. Write the registers IC_CON.IC_SLAVE_DISABLE = 0, and IC_CON.MASTER_MODE = 0 to change the DW_apb_i2c mode of operation to the Target mode from the Controller mode.

After SMBus Host receives the Notify ARP Controller Command, the Host initiates full ARP sequence.

DW_apb_i2c SMBus host when acting as target does not differentiate between the SMBus Host Notify Protocol and Notify ARP Controller commands and uses a single interrupt for indication of the same. The firmware can read the contents on receive FIFO and decide accordingly.

- If the firmware reads all zeros, it is a Notify ARP Controller command.
- Else it is SMBus Host Notify Protocol.

2.21.8 SMBus Additional Target Address

DW_apb_i2c supports second optional target address decode capability. It can be configured to contain an extra target address register IC_OPTIONAL_SAR. If configured with this additional register, you can write any valid target address to this register which is matched against an incoming target address on SMBus. A match of incoming address with either IC_SAR register or IC_OPTIONAL_SAR register causes DW_apb_i2c to acknowledge the transaction and respond to it accordingly. Use of this additional target address register is controlled by OPTIONAL_SAR_CTRL (IC_CON[16]) bit. If OPTIONAL_SAR_CTRL bit

is programmed to be 1, then IC_OPTIONAL_SAR register is used to match the incoming address. All restrictions of IC_SAR register applies to IC_OPTIONAL_SAR register as well.

The default value that IC_OPTIONAL_SAR register obtains after reset can be configured by the IC_OPTIONAL_SAR_DEFAULT parameter.

2.21.9 SMBus Optional Signals

The SMBus standard supports these optional signals:

- SMBus Suspend Signal
- SMBus Alert Signal

As these signals are optional, DW_apb_i2c can be configured to include these signals through IC_SMBUS_SUSPEND_ALERT parameter.

2.21.9.1 SMBus Suspend Signal

The SMBus suspend signal (SMBSUS#) is an optional signal which is asserted by the system controller (mostly the Host) to indicate that the system should enter in low power suspend mode. It is output from the system controller and input to all other devices. This signal is an active low signal. DW_apb_i2c implements this functionality using following signals:

- ic_smbsus_in_n
- ic_smbsus_out_n

Output signal ic_smbsus_out_n is controlled directly by the SMBUS_SUS_CTRL bit (IC_ENABLE[17]). If this bit is programmed to 1, ic_smbsus_out_n signal goes to 0 as soon as controller finishes any ongoing transfer. For coming out of the suspend mode, you need to clear this bit, which de-asserts the ic_smbsus_out_n signal.

Input signal ic_smbsus_in_n generates interrupt ic_smbsus_det_intr (or ic_smbsus_det_intr_n) on the falling edge. This interrupt can be used by the software to enter the Low Power Mode. Current status of this ic_smbsus_in_n can be read from SMBUS_SUSPEND_STATUS bit of IC_STATUS (19) register.

2.21.9.2 SMBus Alert Signal

The SMBus alter signal (SMBALERT#) is other optional signal specified by the SMBus standard. It can be used by simple devices to request the attention of the host. Devices can use the SMBALERT# signal to request the attention of the host with controller functionality. This signal is input to host device and output from all other devices. Since multiple devices may implement SMBALERT#, it is required to be a wired-AND signal. Upon detecting a SMBALERT# signal, a host must send an alert response address which is acknowledge by alerting the device and it sends the address to the host and de-asserts the alert signal. If host still detects an asserted alert signal, it repeats sending alert response address. DW_apb_i2c implements this functionality using following signals:

- ic_smbalert_in_n
- ic_smbalert_oe

Output signal ic_smbalert_oe is open drain/open collector pull down driver and should be used similar to ic_clk_oe and ic_data_oe on a system implementation. Assertion of ic_smbalert_oe is controlled by SMBUS_ALERT_CTRL bit (IC_ENABLE[18]). Once asserted, DW_apb_i2c waits for alert response address

to be sent by controller. Upon receiving it, contents of IC_SAR[7:0] register are sent to the controller. When successful, DW_apb_i2c clears the SMBUS_ALERT_CTRL bit and de-asserts the ic_smbalert_oe signal.

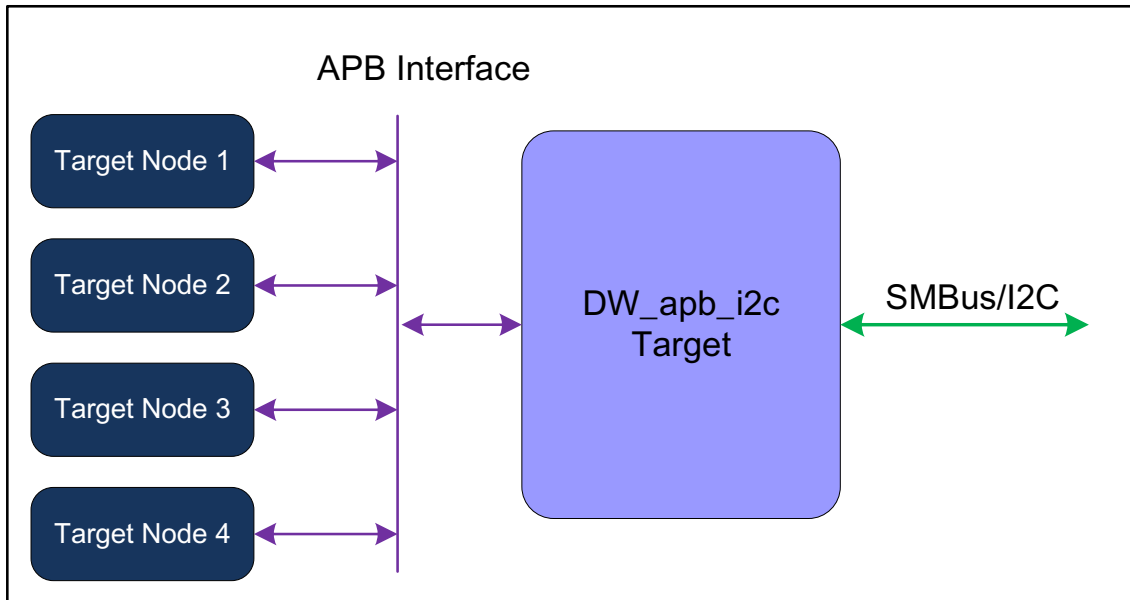
When Multiple SAR feature is enabled (IC_MULTI_SAR_EN = 1), DW_apb_i2c has four target device addresses IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4. For more information on SMBus Alert support when Multiple SAR feature is enabled, refer to [“SMBus Alert Support for IC_SARx Target Device Addresses”](#) on page 117.

Input signal ic_smbalert_in_n generates interrupt ic_smbalert_det_intr (or ic_smbalert_det_intr_n) on falling edge. If working as host, you need to service this interrupt by sending read byte command with Alert Response Address. Current status of ic_smbalert_in_n can be read from SMBUS_ALERT_STATUS bit (IC_STATUS[20])

2.22 Multiple SAR Feature

NVMe management interface uses the Management Component Transport Protocol based on the SMBus specification. This type of applications may have single physical I²C target port with the multiple target nodes. These applications require multiple target device addresses on DW_apb_i2c to support multiple target nodes. This feature can be used in any application where multiple target nodes are required on the single SMBus DW_apb_i2c target device. A typical SMBus Multiple Target address-based subsystem is as shown in [Figure 2-47](#).

Figure 2-47 SMBus Multiple Target Address Based System



With the help of the Multiple Target Devices Address feature, SMBus DW_apb_i2c instance is connected to the multiple target nodes through the APB interface. The SMBus/I²C transfers received on the I²C bus can be sent to different target nodes. Each target node has distinct Target Device Address supported in the DW_apb_i2c instance.



Note

When DW_apb_i2c is configured with Multiple SAR feature:

- If DW_apb_i2c is acting as SMBus target, at least one of the target devices selected in the IC_NUM_SARS parameter must be enabled.
- If DW_apb_i2c is acting as SMBus host, then IC_SAR Target Address must be enabled.

**Note**

- The Multiple Target Device Address feature is enabled only when the SMBus feature is enabled.
- The DW_apb_i2c parameters and registers corresponds to IC_SAR target device address unless stated otherwise.
- DW_apb_i2c always responds with IC_SAR target device address as Device Address in response to SMBus Alert Response Address (ARA) access.
- Optional SAR address is not supported when Multiple SAR feature is enabled.
- Only IC_SAR target device address accepts SMBus Host Address as part of the SMBus Host Notify protocol.

2.22.1 Multiple Target Device Address based I²C Write Transfer Handling

Whenever an I²C Write Transfer with the specific (programmed) Target Device Address is received,

1. The I²C write transfer related address is acknowledged. Subsequently received data is stored in the RX FIFO.
2. On acceptance of I²C write transfer the following interrupts are asserted:
 - a. IC_RAW_INTR_STAT.WR_REQ indicates the reception of new write request.
 - b. IC_RAW_INTR_STAT.SLV_ADDRx_TAG indicates for which target device address the I²C transfer has been received (for x = 1; x <= IC_NUM_SARS).
3. DW_apb_i2c continues to receive the data till the I²C controller sends the data.
4. The following interrupts are asserted based on the programmed threshold values through the IC_RX_TL register.
 - a. IC_RAW_INTR_STAT.RX_FULL interrupt reads the data from RX FIFO (corresponding to the data reception for the Target Device Address - SLV_ADDRx_TAG).

**Note**

Other interrupts are asserted based on the respective condition such as RX_UNDER, RX_OVER.

5. The data can be read from the RX FIFO with the IC_DATA_CMD register using the interrupts or DMA interface through the APB Interface.
6. If the RX FIFO is full, then the Clock is stretched until the RX FIFO has some space to store the received data from the I²C line.
7. The following two conditions are considered at the end of I²C transfer related to Target Device Address:
 - a. STOP condition: Irrespective of the next transfer address corresponds to the current transfer address.
 - b. RESTART condition: Unless new address is the same as the last address.

8. The DW_apb_i2c Target stretches the clock until all the data in the RX FIFO is read in the following scenario:
 - a. After receiving the last data byte from the I²C bus based on the conditions.
 - b. An I²C transfer is directed to the DW_apb_i2c target instance.

The SLV_ISO_SAR_DATA_CLK_STRETCH bit of the IC_STATUS register records clock stretching status.
9. After all the data is read from the RX FIFO, the DW_apb_i2c target releases the clock to accept the subsequent I²C transfers.
10. After receiving the last byte from the I²C bus, the clock is not stretched in the following scenarios:
 - a. An I²C transfer is not received until all the data in the RX FIFO is read completely.
 - b. An I²C transfer is received, which is not directed to the DW_apb_i2c target instance until all the data in the RX FIFO is read.
 - c. An I²C transfer is received after restart, which is directed to the DW_apb_i2c target instance and the address is same as the last I²C transfer address.
11. Subsequently, the DW_apb_i2c target can receive the next I²C transfer.

2.22.2 Multiple Target Device Address based I²C Read Transfer Handling

Whenever an I²C Read Transfer with the specific (programmed) Target Device Address is received,

1. The I²C read transfer related address is acknowledged.
2. On acceptance of I²C read transfer, the following interrupts are asserted, and the clock is stretched until the data corresponding the target device address is written into the TX FIFO through the IC_DATA_CMD register.
 - a. IC_RAW_INTR_STAT.RD_REQ indicates the reception of new read request.
 - b. IC_RAW_INTR_STAT.SLV_ADDRx_TAG indicates the target device address for which the I²C transfer is received.
 - c. IC_RAW_INTR_STAT.TX_EMPTY reads the data (corresponding to the Target Device Address - SLV_ADDRx_TAG) from the application based on the programmed threshold values through the IC_TX_TL register.



Note Other interrupts are asserted based on the respective condition such as TX_OVER.

3. After the data corresponding to the Target Device Address is written into the TX FIFO, DW_apb_i2c releases the clock line and sends the data corresponding to the I²C target read transfer on the I²C line.
4. During this transfer, if the TX FIFO becomes empty, then clock is stretched until the data is available in the TX FIFO.
5. This process continues until the NACK is received. Then controller issues a STOP condition.
6. After NACK and STOP is received, the DW_apb_i2c flushes the TX FIFO data.
7. Subsequently, the DW_apb_i2c target can receive the next I²C transfer.

2.22.3 Impact of Multiple Target Device Address Feature on SMBus ARP Sequence and UDID

The SMBus address resolution must happen to all the targets in the system. With the multiple SAR feature, the SMBus host must resolve IC_NUM_SARS number of SAR addresses by repeating the ARP protocol for IC_NUM_SARS times. This is to understand the bus even though we have single I²C Target instance. DW_apb_i2c supports the SMBus ARP enable/disable for each of the IC_SARx target device addresses. Only the ARP enabled target device addresses participate in the ARP sequence.



Note

The DW_apb_i2c resolves IC_NUM_SARS number of SAR addresses in a defined ARP sequence - starting with IC_SAR, followed by IC_SAR2, IC_SAR3 and IC_SAR4 in sequence.

DW_apb_i2c supports the following two ways using which each IC_SARx target device address can identify itself with its unique UDID.

- If the configuration parameter IC_MULTI_SAR_UDID_CTRL is set to 0, each target device address is identified and differentiated using UDID[LOG2_IC_NUM_SARS-1:0] bits. For more information, refer to [“Use of UDID\[LOG2_IC_NUM_SARS-1:0\] bits \(IC_MULTI_SAR_UDID_CTRL = 0\)”](#) on page 115.
- Otherwise (IC_MULTI_SAR_UDID_CTRL = 1), the DW_apb_i2c adds separated 128-bit UDID registers for each of the IC_SARx target device address. These UDID register values are used to identify and differentiate between them. For more information, refer to [“Use of Separate 128-bit UDID Registers \(IC_MULTI_SAR_UDID_CTRL = 1\)”](#) on page 117.

2.22.3.1 Use of UDID[LOG2_IC_NUM_SARS-1:0] bits (IC_MULTI_SAR_UDID_CTRL = 0)

As per SMBus specification, each target (with unique SAR) must identify itself with Unique UDID. To address this UDID requirement, DW_apb_i2c reuses the lower LOG2_IC_NUM_SARS-1:0 bits of Vendor-specific ID field of the UDID. Other bits are taken from the following UDID_REG*[127:LOG2_IC_NUM_SARS] configured/programmed in DW_apb_i2c.

Where, * UDID_REG is defined as following:

- IC_SMBUS_UDID_HC = 1, UDID_REG = {IC_SMBUS_UDID_LSB_DEFAULT, IC_SMBUS_UDID_LSB register}
- IC_SMBUS_UDID_HC = 0, UDID_REG = {IC_SMBUS_UDID_WORD3 register, IC_SMBUS_UDID_WORD2 register, IC_SMBUS_UDID_WORD1 register, IC_SMBUS_UDID_WORD0 register}

Example-1:

Where IC_NUM_SARS = 4 (LOG2_IC_NUM_SARS = 2), with all target device addresses are ARP enabled. In this case, lower 2-bits of Vendor-specific ID field of UDID are mapped to respective SAR as follows:

Table 2-11 SMBus Vendor ID to SAR Mapping with IC_NUM_SARS=4 and all targets are ARP Enabled

Target Device Address	Vendor-Specific ID[1:0]/UDID[1:0]	Is ARP Enabled (IC_CON.IC_SARx_SMBUS_ARP_EN==1)?
IC_SAR	2'b00	Yes
IC_SAR2	2'b01	Yes

Target Device Address	Vendor-Specific ID[1:0]/UDID[1:0]	Is ARP Enabled (IC_CON.IC_SARx_SMBUS_ARP_EN==1)?
IC_SAR3	2'b10	Yes
IC_SAR4	2'b11	Yes

Example-2:

Where IC_NUM_SARS = 4 (LOG2_IC_NUM_SARS = 2), with two-target device addresses being ARP enabled. In this case, lower 2-bits of Vendor-specific ID field of UDID mapped to respective SAR as follows:

Table 2-12 SMBus Vendor ID to SAR Mapping with IC_NUM_SARS=4 and two Targets are ARP Enabled

Target Device Address	Vendor-Specific ID[1:0]/UDID[1:0]	Is ARP Enabled (IC_CON.IC_SARx_SMBUS_ARP_EN==1)?
IC_SAR	2'b00	Yes
IC_SAR2	2'b01	Yes
IC_SAR3	Not Applicable	No
IC_SAR4	Not Applicable	No

The system ensures that UDID generation is done considering the 'Vendor-specific ID' and SAR. Where, UDID[1:0] = 2'b00 is required for IC_SAR0/IC_SAR, UDID[1:0] = 2'b01 is required for IC_SAR1, and so on.

Example-3:

Where IC_NUM_SARS = 2 (LOG2_IC_NUM_SARS = 1), with all target device addresses are ARP enabled. In this case, lower 1-bit of Vendor-specific ID field of UDID mapped to respective SAR as follows:

Table 2-13 SMBus Vendor ID to SAR Mapping with IC_NUM_SARS=2 and all targets are ARP Enabled

Target Device Address	Vendor-specific ID[0]/UDID[0]	Is ARP Enabled (IC_CON.IC_SARx_SMBUS_ARP_EN==1)?
IC_SAR	1'b0	Yes
IC_SAR2	1'b1	Yes

Example-4:

Where IC_NUM_SARS = 2 (LOG2_IC_NUM_SARS = 1), with only one target device address is ARP enabled. In this case, lower 1-bit of Vendor-specific ID field of UDID mapped to respective SAR as follows:

Table 2-14 SMBus Vendor ID to SAR Mapping with IC_NUM_SARS=2 and only one Target is ARP Enabled

Target Device Address	Vendor-specific ID[0]/UDID[0]	Is ARP Enabled (IC_CON.IC_SARx_SMBUS_ARP_EN==1)?
IC_SAR	1'b0	Yes
IC_SAR2	Not Applicable	No

The system ensures that UDID generation is done considering the Vendor-specific ID and SAR. Where, UDID[0] = 1'b0 is required for IC_SAR0/IC_SAR and UDID[0] = 1'b1 is required for IC_SAR2.

2.22.3.2 Use of Separate 128-bit UDID Registers (IC_MULTI_SAR_UDID_CTRL = 1)

As per the SMBus specification, each Target (with unique SAR) must identify itself with Unique UDID. To address this UDID requirement, DW_apb_i2c adds separate 128-bit UDID registers for each IC_SARx target device address. These UDID registers are defined as following:

- UDID of IC_SAR target device address:
 - If IC_SMBUS_UDID_HC = 1
UDID = {IC_SMBUS_UDID_MSB, IC_SMBUS_UDID_LSB register}
 - If IC_SMBUS_UDID_HC = 0
UDID = {IC_SMBUS_UDID_WORD3 register, IC_SMBUS_UDID_WORD2 register, IC_SMBUS_UDID_WORD1 register, IC_SMBUS_UDID_WORD0 register}
- UDID of IC_SAR2 target device address:
 - If IC_SAR2_SMBUS_UDID_HC = 1
UDID = IC_SAR2_SMBUS_UDID_DEFAULT
 - If IC_SAR2_SMBUS_UDID_HC = 0
UDID = {IC_SAR2_SMBUS_UDID_WORD3 register, IC_SAR2_SMBUS_UDID_WORD2 register, IC_SAR2_SMBUS_UDID_WORD1 register, IC_SAR2_SMBUS_UDID_WORD0 register}
- UDID of IC_SAR3 target device address:
 - If IC_SAR3_SMBUS_UDID_HC = 1
UDID = IC_SAR3_SMBUS_UDID_DEFAULT
 - If IC_SAR3_SMBUS_UDID_HC = 0
UDID = {IC_SAR3_SMBUS_UDID_WORD3 register, IC_SAR3_SMBUS_UDID_WORD2 register, IC_SAR3_SMBUS_UDID_WORD1 register, IC_SAR3_SMBUS_UDID_WORD0 register}
- UDID of IC_SAR4 target device address:
 - If IC_SAR4_SMBUS_UDID_HC = 1
UDID = IC_SAR4_SMBUS_UDID_DEFAULT
 - If IC_SAR4_SMBUS_UDID_HC = 0
UDID = {IC_SAR4_SMBUS_UDID_WORD3 register, IC_SAR4_SMBUS_UDID_WORD2 register, IC_SAR4_SMBUS_UDID_WORD1 register, IC_SAR4_SMBUS_UDID_WORD0 register}

DW_apb_i2c uses the mentioned 128-bits UDID values to identify IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4 target device addresses during the SMBus ARP sequence.

2.22.4 SMBus Alert Support for IC_SARx Target Device Addresses

DW_apb_i2c supports SMBus ARA capability for each target device address (IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4). The DW_apb_i2c implements this functionality using following signals:

- ic_smbalert_in_n

■ ic_smbalert_oe[IC_NUM_SARS-1:0]

SMBus ARA capability on individual target device address is enabled through IC_ENABLE register bits as shown in [Table 2-10](#). Assertion of ic_smbalert_oe[IC_NUM_SARS-1:0] is controlled by SMBus Alert enable register bits.



Note

The target address must be enabled (IC_SARx_En == 1) to have SMBus Alert capability.

Table 2-15 SMBus ARA Enable Register Bit for IC_SARx Target Device Address

Target Address	SMBus Alert Enable Register Bit	SMBus Alert Output Port
IC_SAR	IC_ENABLE.SMBUS_ALERT_EN (IC_ENABLE[18])	ic_smbalert_oe[0]
IC_SAR2	IC_ENABLE.IC_SAR2_SMBUS_ALERT_EN (IC_ENABLE[23])	ic_smbalert_oe[1]
IC_SAR3	IC_ENABLE.IC_SAR3_SMBUS_ALERT_EN (IC_ENABLE[24])	ic_smbalert_oe[2]
IC_SAR4	IC_ENABLE.IC_SAR4_SMBUS_ALERT_EN (IC_ENABLE[25])	ic_smbalert_oe[3]

Once SMBus ARA is enabled on any target device address, DW_apb_i2c waits for Alert Response Address to be sent from host. Upon receiving it, contents of the IC_SARx[7:0] register are sent to the controller. If SMBus Alert is enabled on multiple target addresses, the first target address (sent in response to ARA from host) is derived based on priority - the target address which wins the arbitration. The DW_apb_i2c target computes this arbitration after receiving Alert response Address from host.

When ARA is successful (target device address is sent), DW_apb_i2c clears the SMBus ARA enable register bit (IC_ENABLE.IC_SARx_SMBUS_ALERT_EN) and de-asserts the respective ic_smbalert_oe signal. If SMBus Alert is enabled on multiple target device addresses, after processing of the first ARA from host, the DW_apb_i2c waits for the Alert Response Address from host for the next target device address.

DW_apb_i2c supports both SMBus Alert formats - with PEC or without PEC byte. If host ACKs the target device address sent in response to ARA, DW_apb_i2c target goes into PEC transmit state and sends the computed PEC byte. If the host NACKs the target device address sent in response to ARA, the SMBus ARA completes with STOP bit followed by NACK.

Input signal ic_smbalert_in_n generates interrupt ic_smbalert_det_intr (or ic_smbalert_det_intr_n) on falling edge. If working as host, you need to service this interrupt by sending read byte command with Alert Response Address. Current status of ic_smbalert_in_n can be read from SMBUS_ALERT_STATUS bit (IC_STATUS[20]).

The following example illustrates the SMBus Alert Response Address process with two target devices, IC_NUM_SARS = 2.

Configuration:

- IC_MULTI_SAR_EN = 1
- IC_NUM_SARS = 2
- IC_SMBUS_SUSPEND_ALERT = 1

Register programming:

- IC_SAR = 0x66
- IC_SAR2 = 0x55
- IC_ENABLE.IC_SMBUS_ALERT_EN = 1
- IC_ENABLE.IC_SAR2_SMBUS_ALERT_EN = 1

In this case, DW_apb_i2c asserts both bits `ic_smbalert_oe[0]` and `ic_smbalert_oe[1]` as SMBus Alert is enabled on both target device addresses and DW_apb_i2c waits for Alert Response Address to be sent from host. Upon receiving it, contents of IC_SAR2 is sent to host (as between IC_SAR (0x66) and IC_SAR2 (0x55), IC_SAR2 will win the arbitration). When ARA is successful, DW_apb_i2c clears the SMBus Alert enable register bit for IC_SAR2 (IC_ENABLE.IC_SAR2_SMBUS_ALERT_EN) and de-asserts the `ic_smbalert_oe[1]` signal.

The SMBus Alert for IC_SAR target device address is still asserted and indicated through the `ic_smbalert_oe[0]` signal. The DW_apb_i2c waits for one more Alert Response Address to be sent from host. Upon receiving it, contents of IC_SAR is sent to host (as now SMBus Alert is enabled for IC_SAR target TBD address only). When ARA is successful, DW_apb_i2c clears the SMBus Alert enable register bit for IC_SAR (IC_ENABLE.SMBUS_ALERT_EN) and de-asserts the `ic_smbalert_oe[0]` signal.

2.22.4.1 Handling Multi-bit SMBus Alert Output Port

SMBALERT is a wired-AND signal just as the SMBCLK and SMBDAT signals are. With multiple SAR and SMBus Alert feature, DW_apb_i2c has up to 4 SMBus Alert output ports `ic_smbalert_oe[0]`, `ic_smbalert_oe[1]`, `ic_smbalert_oe[2]` and `ic_smbalert_oe[3]`. These SMBus Alert outputs are implemented by IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4 target devices. These SMBus Alert outputs are required to be a wired-AND signal. In the SMBus system it can be handled in two ways.

1. Perform an AND of all four SMBus Alert output ports external to DW_apb_i2c and implement a wired-AND functionality on the AND gate output of 4 SMBus Alert ports.
2. Take individual 4 SMBus Alert outputs and implement a wired-AND functionality for each of them.

2.22.5 Advantages of Multiple SAR Feature

The following are the advantages of using the Multiple SAR feature.

- DW_apb_i2c target supports Multiple Target Device Addresses in the single instance.
 - This helps in saving the area and power.
 - This also helps in reducing the software/firmware handling overhead for multiple instances.
 - Having multiple DW_apb_i2c instances results in additional capacitive load on the I²C bus, which affects the overall performance of the I²C bus. But in this approach, it reduces the capacitive load on the I²C bus significantly because we have only once instance compared to multiple instances.
- You can modify the I²C Target device address by reprogramming the IC_SAR register. You must disable the DW_apb_i2c, and then change the IC_SAR register. During this flow the I²C target does not accept any I²C transfers, this may result loss of data. The new proposed architecture ensures that there is no data-loss because the I²C target can receive up to 4 target addresses without any program-change.
- This proposed architecture ensures that there is no mix-up of data corresponding to the different I²C transfers with different target device addresses. This is done by enabling clock stretching at the end of I²C write transfer until all data is read out of the RX FIFO.

2.23 SMBus 3.1 Compliance

A device type Default Target Device Address (DSA) TBD is added as per the SMBus 3.1 specification. The Default Target Address is an ARP-capable device. With Multiple SAR enhancement, DW_apb_i2c adds up to four logical targets in a single instance of the DW_apb_i2c. Each target can be configured as PSA/Non-PSA/DSA device type with flexible configuration and programming options listed in [Table 2-16](#).

Table 2-16 Configuration and Programming Options for Target Addresses

Target Device Address	Power-on State of AV Flag (Configuration Parameter)	Register Field for Software Programmable AV Flag	ARP Capability (IC_SMBUS_ARP = 1)
IC_SAR	IC_PERSISTANT_SLV_ADDR_DEFAULT	IC_CON.SMBUS_PERSISTENT_SLV_ADDR_EN	IC_CON.SMBUS_ARP_EN
IC_SAR2	IC_PERSISTANT_SLV_ADDR2_DEFAULT	IC_CON.SMBUS_PERSISTENT_SLV_ADDR_EN	IC_CON.IC_SAR2_SMBUS_ARP_EN
IC_SAR3	IC_PERSISTANT_SLV_ADDR3_DEFAULT	IC_CON.SMBUS_PERSISTENT_SLV_ADDR_EN	IC_CON.IC_SAR3_SMBUS_ARP_EN
IC_SAR4	IC_PERSISTANT_SLV_ADDR4_DEFAULT	IC_CON.SMBUS_PERSISTENT_SLV_ADDR_EN	IC_CON.IC_SAR4_SMBUS_ARP_EN

The difference between a DSA device type and a PSA device type is because of the Address Valid (AV) flag and SMBus Address after power-on reset.

DSA Device: Default target Address (DSA) is a device that has a manufacturer-assigned, read-only target address that is loaded from ROM after the device is powered on. A device with the default address can change its address during the ARP process by writing the new address to a volatile memory location. After a power cycle or power-on reset, the assigned address stored in volatile memory is lost, then the device responds to its default address.

PSA Device: Persistent Target Address (PSA) is an assigned target address that is retained through the loss of device power. So, after a device address is assigned in the ARP process, it is stored in the non-volatile memory. After a power cycle, the device wakes up and responds to the previously assigned and stored address.

2.23.1 Configuring a Target Device Address as Default Target Address Device

DW_apb_i2c target device address is considered as DSA device with the following configuration/programming (for $x = 1, x < \text{IC_NUM_SARS}$):

1. Configure IC_SMBUS_ARP = 1
2. Configure IC_PERSISTANT_SLV_ADDRx_DEFAULT = 1
3. Configure IC_DEFAULT_SLAVE_ADDRx = Default target Address
4. Program IC_CON.IC_SARx_SMBUS_ARP_EN = 1

This configuration and programming ensures that the target behaves as Default Target Address (DSA) device:

- The AV flag is set after the power-on reset.
- The target address is set to default address, which can be updated by the ARP process.

2.23.2 Configuring a Target Address as Persistent Target Address Device

DW_apb_i2c target address is considered as PSA device with the following configuration/programming (for $x = 1, x < IC_NUM_SARS$):

1. Configure `IC_SMBUS_ARP = 1`
2. Program `IC_CON.SMBUS_PERSISTENT_SLV_ADDRx_EN = 1`
3. Configure `IC_SARx = Persistent Target Address`
4. Program `IC_CON.IC_SARx_SMBUS_ARP_EN = 1`

This configuration and programming ensures that the target behaves as Persistent Target Address (PSA) device:

- The AV flag is set using the `IC_CON` register.
- The target device address is set to PSA address, which can be updated by the ARP process.



Note

The AV flag and address is read from non-volatile memory. Ensure that the values of the register fields `IC_CON.SMBUS_PERSISTENT_SLV_ADDRx_EN` and `IC_SARx` are retained even after the loss of device power or hard reset.

2.23.3 SMBus Device Categorization

The SMBus specification (v3.1) categorizes devices as shown in [Table 2-17](#).

Table 2-17 SMBus Device Categorization

Device Category	IC_SMBUS_ARP_CTRL. IC_SARx_SMBUS_DEVICE_TYPE
Non-ARP capable	2'b00
ARP-capable	2'b01
Fixed and Discoverable	2'b10
Fixed and not Discoverable	2'b11

DW_apb_i2c supports the following four device categories by programming the `IC_SMBUS_ARP_CTRL` register:

- **Non-ARP-capable device**
 - a. Target address is fixed.
 - b. DW_apb_i2c target shall NACK all the ARP commands.
 - c. Address Valid (AV) and Address Resolved (AR) flags have no significance.
- **ARP-capable device**
 - a. Target address is assigned using SMBus host-based ARP sequence.
 - b. DW_apb_i2c target shall accept and ACK all the ARP commands.
 - c. Address Valid (AV) and Address Resolved (AR) flags shall be cleared after reset commands.

- **Fixed and Discoverable device**

- a. Target address is fixed.
- b. DW_apb_i2c target shall accept and ACK all the ARP commands.
- c. Address Valid (AV) flag shall not be cleared after Reset Device command.
- d. Address Resolved (AR) flag shall be set after Assign Address command.
Address Resolved (AR) flag shall be cleared after reset command.
- e. DW_apb_i2c target shall NACK the “Data 17 (Assign Address)” byte as the target address is fixed.
- f. The Data byte (17) of the Assign Address command is NACK’ed without validating the PEC byte.
- g. Address Valid (AV) flag shall be always set to 1.

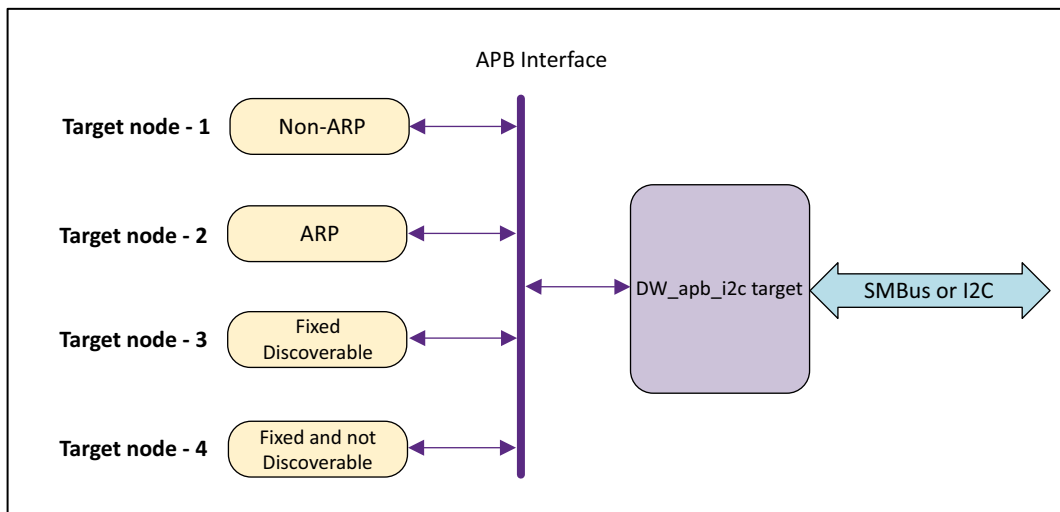
- **Fixed and not Discoverable device**

- a. Target address is fixed.
- b. DW_apb_i2c target shall accept and ACK the directed the Get UDID command.
- c. DW_apb_i2c target shall NACK any other ARP commands.
- d. Address Valid (AV) and Address Resolved (AR) flags shall be always set to 1.
- e. Address Valid (AV) and Address Resolved (AR) flags shall not be cleared after Reset Device command.

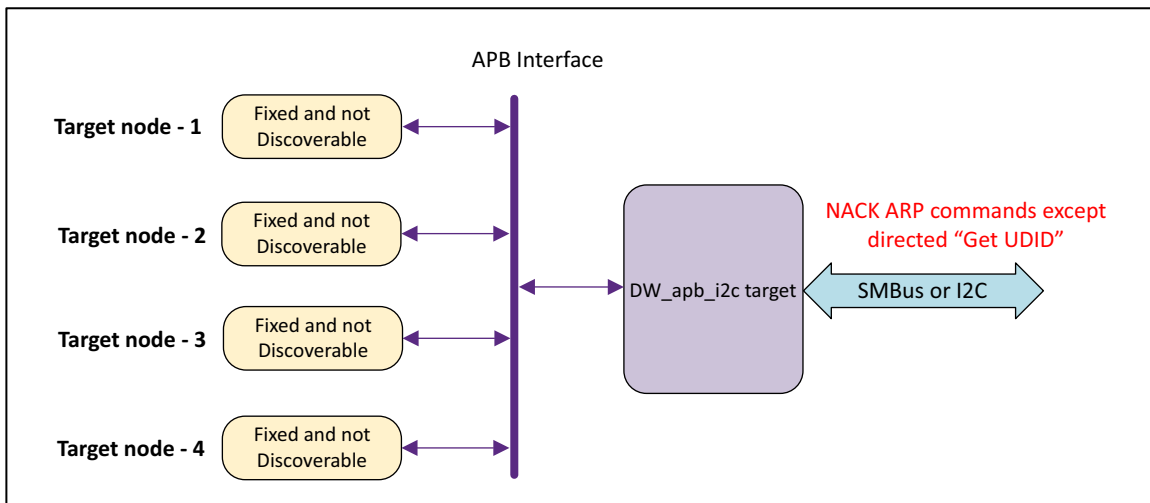
The configuration parameter IC_SMBUS_FD_FND_SUPPORT_EN can be enabled to support the above mentioned SMBus device categories.

2.23.3.1 SMBus Multiple Target Device Address System with different Device Types

DW_apb_i2c target extends the support to program different device types for different target addresses when multiple target device address feature is enabled. The default device types of different nodes can be configured using the parameter IC_SARx_DFLT_SMBUS_DEVICE_TYPE. A typical multi node SMBus target with different device type is shown in [Figure 2-48](#). DW_apb_i2c target will acknowledge (ACK) the ARP commands when the multiple device types are configured as shown in [Figure 2-48](#) due to the presence of ARP and Fixed Discoverable Target nodes.

Figure 2-48 SMBus Multi Address System with Different Device Types

DW_apb_i2c will not Acknowledge (NACK) the ARP commands (except directed Get UDID) only when all the target nodes are configured or programmed to fixed and not-discoverable device type as shown in [Figure 2-49](#).

Figure 2-49 Fixed and not Discoverable Device Type for Multiple Target Nodes

2.23.3.2 Configuring DW_apb_i2c to Support Different SMBus Device Types

Different SMBus device types can be selected with the following configuration/programming (for $x = 1, x < \text{IC_NUM_SARS}$):

1. Configure `IC_SMBUS_ARP = 1`
2. Configure `IC_SMBUS_FD_FND_SUPPORT_EN = 1`
3. Configure the default Device type for different IC_SAR target address - `IC_SARx_DFLT_SMBUS_DEVICE_TYPE`

4. Program IC_SMBUS_ARP_CTRL.IC_SARx_SMBUS_DEVICE_TYPE to update the default device type value.
5. Program the IC_SARx Target device addresses for ARP device type.

The SMBUS_ARP_EN (bit-18), IC_SAR2_SMBUS_ARP_EN (bit-23), IC_SAR3_SMBUS_ARP_EN (bit-24) and IC_SAR4_SMBUS_ARP_EN (bit-25) fields in IC_CON register cannot be used to enable the ARP capability when the IC_SMBUS_FD_FND_SUPPORT_EN parameter is enabled. Instead, the user can program the IC_SARx_SMBUS_DEVICE_TYPE bit-field of the IC_SMBUS_ARP_CTRL register to enable the ARP capability and select the desired device type.



- DW_apb_i2c Target considers Target device address to be Fixed for FD and FnD device types and it is highly recommended to configure the default target device address values for these devices.
- DW_apb_i2c Target considers UDID address to be Fixed for FnD device types and it is highly recommended to configure the default UDID address values for these devices.

2.24 UDID Feature

SMBus target device address conflicts can be resolved by dynamically assigning a new unique address to each target device through the SMBus Host Controller. The SMBus Host controller uses the SMBus Address Resolution Protocol to assign unique dynamic address for the targets in the SMBus system. This SMBus Address Resolution protocol uses the unique 128-bit Unique Device ID (UDID), which helps to isolate each device in the SMBus system for address assignment.

The DW_apb_i2c provides configuration parameters and registers to configure and program the required UDID values for each target device address. If DW_apb_i2c is configured with multiple target device address support (IC_MULTI_SAR_EN=1), it supports up to four target addresses (IC_SAR, IC_SAR2, IC_SAR3 and IC_SAR4). Otherwise (IC_MULTI_SAR_EN=0), DW_apb_i2c has only one target address IC_SAR. The UDID setting is explained in the subsequent sections.

2.24.1 UDID of IC_SAR Target Address

The DW_apb_i2c supports the 128-bit SMBus UDID through the IC_SMBUS_UDID_HC configuration parameter.

When IC_SMBUS_UDID_HC=1, the SMBus UDID is partially configurable and programmable that is Upper 96-bits of the UDID is configurable through the IC_SMBUS_UDID_MSB parameter and lower 32-bits are programmable through the register IC_SMBUS_UDID_LSB. The register's default value is configurable through the IC_SMBUS_UDID_LSB_DEFAULT parameter.

When IC_SMBUS_UDID_HC=0, the SMBus UDID is completely software programmable through the IC_SMBUS_UDID_WORD0/1/2/3 registers. This feature is useful when you want to update the SMBus UDID through the software. The register IC_SMBUS_UDID_WORD0 default value is configurable through the parameter IC_SMBUS_UDID_LSB_DEFAULT. The IC_SMBUS_UDID_WORD1/2/3 register, default value is derived based on the configurable parameter IC_SMBUS_UDID_MSB.

Software programmable UDID feature helps you to update the UDID quickly using the application software.

2.24.2 UDID of IC_SAR2, IC_SAR3 and IC_SAR4 Target Device Addresses (IC_MULTI_SAR_EN=1)

DW_apb_i2c supports the 128-bit SMBus UDID for IC_SARx (for x = 2, x <= IC_NUM_SARS) target device addresses through the IC_SARx_SMBUS_UDID_HC configuration parameter.

When IC_SARx_SMBUS_UDID_HC=1, the SMBus UDID is configurable only. The 128-bits of the UDID is configured through the IC_SARx_SMBUS_UDID_DEFAULT parameters.

When IC_SARx_SMBUS_UDID_HC=0, the SMBus UDID is configurable and completely software programmable through the registers IC_SARx_SMBUS_UDID_WORD0/1/2/3. The default value of these registers is shown in [Table 2-18](#).

Table 2-18 Default Value of SMBus UDID Registers

Register Name (for x = 2, x <= IC_NUM_SARS)	Default Value (for x = 2, x <= IC_NUM_SARS)
IC_SARx_SMBUS_UDID_WORD0	IC_SARx_SMBUS_UDID_DEFAULT[31:0]
IC_SARx_SMBUS_UDID_WORD1	IC_SARx_SMBUS_UDID_DEFAULT[63:32]

Register Name (for x = 2, x <= IC_NUM_SARS)	Default Value (for x = 2, x <= IC_NUM_SARS)
IC_SARx_SMBUS_UDID_WORD2	IC_SARx_SMBUS_UDID_DEFAULT[95:64]
IC_SARx_SMBUS_UDID_WORD3	IC_SARx_SMBUS_UDID_DEFAULT[127:96]

2.25 Debug Interface

This section explains the important DW_apb_i2c controller and target debug signals' encoding and respective description.

2.25.1 Controller FSM State Encoding

DW_apb_i2c has a controller FSM state encoded debug signal output - debug_mst_cstate[4:0]. Its encoding is explained in [Table 2-19](#).

Table 2-19 Controller FSM State Encoding

debug_mst_cstate [4:0]	State Name	State Description
5'b00000	IDLE	Idle state and no activity in controller FSM. The controller FSM is waiting for commands from application.
5'b00001	GEN_START	This state generates the Start condition on the I ² C bus.
5'b00010	POP_TX_DATA	In this state controller FSM pops the data from TX FIFO and decodes the command as I ² C Write or I ² C Read.
5'b00011	TX_HS_MCODE	Transmits the high-speed controller code on the I ² C bus. This state is present only when configuration parameter IC_MAX_SPEED_MODE = 3.
5'b00100	TX7_1ST_ADDR	Address phase of the I ² C command. This state is used only in the 7-bit addressing mode.
5'b00101	GEN_STOP	This state is used for the generation of the Stop condition on the I ² C line.
5'b00110	CHECK_IC_TAR	This controller FSM state sends general call or start byte on to the I ² C bus.
5'b00111	RX_BYTE	In this state RX data is received from Target I ² C. This state is present in the configuration where IC_ULTRA_FAST_MODE = 0.
5'b01000	GEN_RSTRT_HS	This state generates the Restart condition in high-speed mode.
5'b01001	GEN_RSTRT_7BIT	This state generates the Restart condition in non-high-speed mode for 7-bit addressing mode.
5'b01010	GEN_RSTRT_10BIT	This state generates the Restart in non-high-speed mode for 10-bit addressing mode.
5'b01011	TX_BYTE	In this controller FSM state Tx data is sent to the I ² C targets.
5'b01100	TX10_1ST_ADDR	Address phase of the I ² C command. This state is used only in the 10-bit addressing mode. In this state first 7-bit address is sent.
5'b01101	TX10_2ND_ADDR	Address phase of the I ² C command. This state is used only in the 10-bit addressing mode. In this state subsequent 3-bit of the address is sent.
5'b01110	GEN_RSTRT_SBYTE	This state generates the Restart condition on the I ² C bus for Start Byte transfer. This state is present only when configuration parameter IC_MAX_SPEED_MODE = 3.

debug_mst_cstate [4:0]	State Name	State Description
5'b01111	GEN_SPLIT_STOP	This state is used to split a combined transaction (Read followed write I ² C command or Write followed by read I ² C command) when restart is disabled. This state generates the Stop for current transfer before starting the next command.
5'b10001	DEVICE_SLV_ADDR	This state sends the Target device address from which Device ID is required to be read. This state is present in the configuration where IC_DEVICE_ID = 1.
5'b10010	DEVICE_ID_RSTRT	This controller FSM state generates Restart condition for the Device ID Feature. This state is present in the configuration where IC_DEVICE_ID = 1.
5'b10011	SMBUS_CLK_RESET	This controller FMS state resets the system by holding the clock low for protocol specified number of clock cycles. This state is present in the configuration where IC_SMBUS = 1.
5'b10100	SDA_RECOVERY	This controller FSM state recovers the SDA through generating SCL for required number clock cycles. This state is present in the configuration where IC_BUS_CLEAR_FEATURE = 1.
5'b10101	GEN_SPLIT_START	This state is used to split a combined transaction (Read followed write I ² C command or Write followed by read I ² C command) when restart is disabled. This state generates the Start before initiating the next command.
5'b10111	HOLD_RX_BYTE	If Rx FIFO is full and this is not the last command, then DW_apb_i2c stretches the SCL clock until there is a space in Rx FIFO. This state is present in the configuration where IC_ULTRA_FAST_MODE = 0 and IC_RX_FULL_HLD_BUS_EN = 1, or IC_EMPTYFIFO_HOLD_MASTER_EN = 1.
5'b11011	HOLD_TX_BYTE	If the Tx FIFO is empty and is not the last command, then DW_apb_i2c stretches the SCL clock until there is a command in the Tx FIFO. This state is present in the configuration where IC_EMPTYFIFO_HOLD_MASTER_EN = 1.

2.25.2 Target FSM State Encoding

DW_apb_i2c has a target FSM state encoded debug signal output - debug_slv_cstate[3:0]. Its encoding is explained in [Table 2-20](#).

Table 2-20 Target FSM State Encoding

debug_slv_cstate [3:0]	State Name	Description
4'b0000	IDLE	Idle state and no activity in target FSM. Target FSM is waiting for commands on the I ² C bus.
4'b0001	RX_1BYTE	This target FSM state is used to receive the 1st byte that is Address of the I ² C command.
4'b0010	RX10_2ND_ADDR	This target FSM state is used to receive the 2nd byte of 10-bit addressing.

debug_slv_cs state [3:0]	State Name	Description
4'b0011	WAIT_RX_NEMPTY	In this state target FSM waits for the RX FIFO to become empty and it stretches the SCL clock. This state is present when configuration parameter IC_MULTI_SAR_EN = 1.
4'b0100	ABORT_RX_ADDR	This target FSM state is used to terminate the address reception due to a start detection.
4'b0101	RX_LOOP	The receive data loop state, where the target FSM receives the data from the I ² C controller.
4'b0110	WAIT_TX_DATA	In this state target FSM waits for processor/system software to supply the data for target transmission. This state is present in the configuration where IC_ULTRA_FAST_MODE = 0.
4'b0111	TX_LOOP	The Transmit data loop state, where the target FSM transmits the data to the I ² C controller. This state is present in the configuration where IC_ULTRA_FAST_MODE = 0.
4'b1000	INSPECT_DATA_CMD	In this state target FSM checks the 8th bit of the IC_DATA_CMD register. The transition to this state happens from WAIT_TX_DATA state. This state is present in the configuration where IC_ULTRA_FAST_MODE = 0.
4'b1001	WAIT_RX_FULL	In this state target FSM waits for Rx FIFO to have a space. During this process clock is stretched until FIFO has a space. This state is present in the configuration where IC_RX_FULL_HLD_BUS_EN = 1.
4'b1010	RX_DEVICE_ADDR	In this state target FSM receives the Device Target Address. This state is present in the configuration where IC_DEVICE_ID = 1.
4'b1011	TX_DEVICE_LOOP	In this state Target FSM transmits the Device Target Address. This state is present in the configuration where IC_DEVICE_ID = 1.
4'b1100	RX_PEC	This target FSM state is used to receive the PEC bytes of the SMBus ARP commands such as prepare ARP, reset Device, assign address and so on. This state is present only when DW_apb_i2c is configured with SMBus ARP capability, IC_SMBUS_ARP = 1.
4'b1101	TX_PEC	This target FSM state is used to transmit the PEC bytes of the SMBus ARP commands such as get UDID (general), get UDID (directed) and so on. This state is present only when DW_apb_i2c is configured with SMBus ARP capability, IC_SMBUS_ARP = 1.
4'b1110	TX_UDID_LOOP	This target FSM state is used to transmit the 128-bit UDID of target for the SMBus ARP commands such as get UDID (general), get UDID (directed) and so on. This state is present only when DW_apb_i2c is configured with SMBus ARP capability, IC_SMBUS_ARP = 1.
4'b1111	RX_UDID_LOOP	This target FSM state is used to receive the 128-bit UDID of Target for the SMBus ARP commands such as prepare ARP, reset device, assign address and so on. This state is present only when DW_apb_i2c is configured with SMBus ARP capability, IC_SMBUS_ARP = 1.

Parameter Descriptions

This chapter details all the configuration parameters. **You can use the coreConsultant GUI configuration reports to determine the complete configuration state of the controller.** Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

The parameter descriptions in this chapter include the **Enabled:** attribute which indicates the values required to be set on other parameters before you can change the value of this parameter.

These tables define all of the configuration options for this component.

- [“Top Level Parameters / Software Interface Configuration Parameters” on page 132](#)
- [“Top Level Parameters / Device Configuration Parameters” on page 134](#)
- [“Top Level Parameters / External Configuration Parameters” on page 139](#)
- [“Top Level Parameters / Internal Configuration Parameters” on page 140](#)
- [“Top Level Parameters / Bus Timing Parameters” on page 143](#)
- [“Top Level Parameters / Additional Features Parameters” on page 146](#)
- [“I2C Features Parameters” on page 150](#)
- [“SMBus Features Parameters” on page 153](#)
- [“SMBus Features / SMBus ARP Features Parameters” on page 155](#)
- [“SMBus Features / IC_SAR SMBus ARP Features Parameters” on page 156](#)
- [“SMBus Features / IC_SAR2 SMBus ARP Features Parameters” on page 158](#)
- [“SMBus Features / IC_SAR3 SMBus ARP Features Parameters” on page 160](#)
- [“SMBus Features / IC_SAR4 SMBus ARP Features Parameters” on page 162](#)
- [“DesignWare Foundation Configuration Parameters” on page 164](#)

3.1 Top Level Parameters / Software Interface Configuration Parameters

Table 3-1 Top Level Parameters / Software Interface Configuration Parameters

Label	Description
System Configuration	
Register Interface Type	Select Register Interface type as APB2, APB3 or APB4. By default, DW_apb_i2c supports APB2 interface. Values: ■ APB2 (0) ■ APB3 (1) ■ APB4 (2) Default Value: APB2 Enabled: DWC-APB-Advanced-Source source license exists. Parameter Name: SLAVE_INTERFACE_TYPE
APB Completer Error Response Enable	Enable Completer Error response signaling. The component will refrain From signaling an error response if this parameter is disabled. Values: ■ false (0) ■ true (1) Default Value: false Enabled: SLAVE_INTERFACE_TYPE>0 Parameter Name: SLVERR_RESP_EN
APB data bus width	Width of the APB data bus. Values: 8, 16, 32 Default Value: 8 Enabled: Always Parameter Name: APB_DATA_WIDTH
Width of Register timeout counter	Defines the width of Register timeout counter. If set to zero, the timeout counter register is disabled, and timeout is triggered as soon as the transaction tries to read an empty RX_FIFO or write to a full TX_FIFO. As these are the only cases where PREADY signal goes low, it ensures that PREADY is tied high throughout. Setting values from 4 through 32 for this parameter configures the timeout period from 2⁴ to 2⁸ pclk cycles. Values: 0, 4, 5, 6, 7, 8 Default Value: 4 Enabled: SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1 Parameter Name: REG_TIMEOUT_WIDTH

Label	Description
Hardcode Register timeout counter value	Checking this parameter makes Register timeout counter a read-only register. The register can be programmed by user if the hardcode option is turned off. Values: ■ false (0) ■ true (1) Default Value: false Enabled: SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0 Parameter Name: HC_REG_TIMEOUT_VALUE
Register Timeout counter default value	Defines the reset value of Register timeout counter register. This value can be over - ridden by programming the timeout counter register before enabling the component, if the HC_REG_TIMEOUT_VALUE is left un-checked Values: 4, ..., POW_2_REG_TIMEOUT_WIDTH Default Value: 8 Enabled: SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0 Parameter Name: REG_TIMEOUT_VALUE

3.2 Top Level Parameters / Device Configuration Parameters

Table 3-2 Top Level Parameters / Device Configuration Parameters

Label	Description
Device Configuration	
Highest speed I2C mode supported	Maximum I2C mode supported. Controls the reset value of the SPEED bit field [2:1] of the I2C Control Register (IC_CON). Count registers are used to generate the outgoing clock SCL on the I2C interface. For speed modes faster than the configured maximum speed mode, the corresponding registers are not present in the top-level RTL. For unsupported speed modes those registers are not present as described below. ■ If this parameter is set to "Standard Mode" then the IC_FS_SCL_*, IC_HS_MADDR, and IC_HS_SCL_* registers are not present. ■ If this parameter is set to "Fast Mode" then the IC_HS_MADDR, and IC_HS_SCL_* registers are not present. Values: ■ Standard Mode (0x1) ■ Fast Mode or Fast Mode Plus (0x2) ■ High Speed Mode (0x3) Default Value: (IC_ULTRA_FAST_MODE == 1)? 1 : (IC_SMBUS == 1 ? 2 : 3) Enabled: IC_ULTRA_FAST_MODE == 0 Parameter Name: IC_MAX_SPEED_MODE
Has I2C default target device address of?	Reset Value of DW_apb_i2c Target Device Address. Controls the reset value of Register (IC_SAR). The default values cannot be any of the reserved address locations: 0x00 to 0x07 or 0x78 to 0x7f. Values: 0x000, ..., 0x3ff Default Value: 0x055 Enabled: Always Parameter Name: IC_DEFAULT_SLAVE_ADDR
Has I2C default target device address 2 of?	Reset Value of DW_apb_i2c Target Device Address 2. Controls the reset value of Register (IC_SAR2). The default values cannot be any of the reserved address locations: 0x00 to 0x07 or 0x78 to 0x7f. Values: 0x000, ..., 0x3ff Default Value: 0x056 Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) Parameter Name: IC_DEFAULT_SLAVE_ADDR2
Has I2C default target device address 3 of?	Reset Value of DW_apb_i2c Target Device Address 3. Controls the reset value of Register (IC_SAR3). The default values cannot be any of the reserved address locations: 0x00 to 0x07 or 0x78 to 0x7f. Values: 0x000, ..., 0x3ff Default Value: 0x057 Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) Parameter Name: IC_DEFAULT_SLAVE_ADDR3

Label	Description
Has I2C default target device address 4 of?	Reset Value of DW_apb_i2c Target Device Address 4. Controls the reset value of Register (IC_SAR4). The default values cannot be any of the reserved address locations: 0x00 to 0x07 or 0x78 to 0x7f. Values: 0x000, ..., 0x3ff Default Value: 0x058 Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) Parameter Name: IC_DEFAULT_SLAVE_ADDR4
Has I2C default target address of?	Reset value of DW_apb_i2c target address. Controls the reset value of the IC_TAR bit field (9:0) of the I2C Target Address Register (IC_TAR). The default values cannot be any of the reserved address locations: 0x00 to 0x07 or 0x78 to 0x7f. Values: 0x000, ..., 0x3ff Default Value: 0x055 Enabled: Always Parameter Name: IC_DEFAULT_TAR_SLAVE_ADDR
Has High Speed mode controller code of?	High Speed mode controller code of the DW_apb_i2c block. Controls the reset value of I2C HS Controller Mode Code Address Register (IC_HS_MADDR). This is a unique code that alerts other controllers on the I2C bus that a high-speed mode transfer is going to begin. For more information about this code, refer to "Multiple Controller Arbitration" section in data book. Values: 0x0, ..., 0x7 Default Value: 0x1 Enabled: (IC_MAX_SPEED_MODE == 3) && (IC_ULTRA_FAST_MODE == 0) Parameter Name: IC_HS_MASTER_CODE
Is an I2C Controller?	Controls whether DW_apb_i2c has its controller enabled to be a controller after reset. This parameter controls the reset value of bit 0 of the I2C Control Register (IC_CON). To enable the component to be a controller, you must write a 1 in bit 0 of the IC_CON register. Note: If this parameter is checked (1), then you must ensure that the parameter IC_SLAVE_DISABLE is checked (1) as well. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: Always Parameter Name: IC_MASTER_MODE

Label	Description
Disable Target after reset?	Controls whether DW_apb_i2c has its target enabled or disabled after reset. If checked, the DW_apb_i2c target interface is disabled after reset. The target also can be disabled by programming a 1 into IC_CON[6]. By default the target is enabled. Note: If this parameter is unchecked (0), then you must ensure that the parameter IC_MASTER_MODE is unchecked (0) as well. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: Always Parameter Name: IC_SLAVE_DISABLE
Supports 10-bit addressing in target mode?	Controls whether DW_apb_i2c target supports 7 or 10 bit addressing on the I2C interface after reset when acting as a target. Controls reset value of part of Register IC_CON. The DW_apb_i2c module will respond to this number of address bits when acting as a target; it can be reprogrammed by software. Values: ■ false (0x0) ■ true (0x1) Default Value: IC_SMBUS == 1 ? 0 : 1 Enabled: IC_SMBUS==0 Parameter Name: IC_10BITADDR_SLAVE
Supports 10-bit addressing in controller mode?	Controls whether DW_apb_i2c supports 7 or 10 bit addressing on the I2C interface after reset when acting as a controller. Controls reset value of part of Register IC_CON. Controller generated transfers will use this number of address bits. Additionally, it can be reprogrammed by software by writing to the IC_CON register. Values: ■ false (0x0) ■ true (0x1) Default Value: IC_SMBUS == 1 ? 0 : 1 Enabled: IC_SMBUS==0 Parameter Name: IC_10BITADDR_MASTER
Depth of transmit buffer is?	Depth of transmit buffer. The buffer is 9 bits wide; 8 bits for the data, and 1 bit for the read or write command. Values: 2, ..., 256 Default Value: 8 Enabled: Always Parameter Name: IC_TX_BUFFER_DEPTH
Depth of receive buffer is?	Depth of receive buffer, the buffer is 8 bits wide. Values: 2, ..., 256 Default Value: 8 Enabled: Always Parameter Name: IC_RX_BUFFER_DEPTH

Label	Description
Transmit buffer threshold value is?	Reset value for threshold level of transmit buffer. This parameter controls the reset value of the I2C Transmit FIFO Threshold Level Register (IC_TX_TL). Values: 0x0, ..., IC_TX_BUFFER_DEPTH-1 Default Value: 0x0 Enabled: Always Parameter Name: IC_TX_TL
Receive buffer threshold value is?	Reset value for threshold level of receive buffer. This parameter controls the reset value of the I2C Receive FIFO Threshold Level Register (IC_RX_TL). Values: 0x0, ..., IC_RX_BUFFER_DEPTH-1 Default Value: 0x0 Enabled: Always Parameter Name: IC_RX_TL
Allow re-start conditions to be sent when acting as a controller?	Controls the reset value of bit 5 (IC_RESTART_EN) in the IC_CON register. By default, this parameter is checked, which allows RESTART conditions to be sent when DW_apb_i2c is acting as a controller. Some older targets do not support handling RESTART conditions; however, RESTART conditions are used in several I2C operations. When the RESTART is disabled, the DW_apb_i2c controller is incapable of performing the following functions: ■ Sending a START BYTE ■ Performing any high-speed mode operation ■ Performing direction changes in combined format mode ■ Performing a read operation with a 10-bit address Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: Always Parameter Name: IC_RESTART_EN
Hardware reset value for IC_SDA_SETUP register	Determines the reset value for the register IC_SDA_SETUP, which in turn controls the time delay - in terms of number of ic_clk clock periods - introduced in the rising edge of SCL, relative to SDA changing when a read-request is serviced. The relevant I2C requirement is t[su:DAT] as detailed in the I2C Bus Specifications. Values: 0x02, ..., 0xff Default Value: 0x64 Enabled: IC_ULTRA_FAST_MODE ==0 Parameter Name: IC_DEFAULT_SDA_SETUP

Label	Description
Hardware reset value for IC_SDA_HOLD register	Determines the reset value for the register IC_SDA_HOLD, which in turn controls the SDA hold time implemented by DW_apb_i2c (when transmitting or receiving, as either controller or target) as a controller/target transmitter or receiver). The relevant I2C requirement is t[HD:DAT] as detailed in the I2C Bus Specifications. The programmed SDA hold time as transmitter cannot exceed at any time the duration of the low part of scl. Therefore it is recommended that the configured default value should not be larger than N_SCL_LOW-2, where N_SCL_LOW is the duration of the low part of the scl period measured in ic_clk cycles, for the maximum speed mode the component is configured for. Values: 0x000001, ..., 0xfffff Default Value: [<functionof> IC_USE_COUNTS IC_CLOCK_PERIOD IC_ULTRA_FAST_MODE] Enabled: Always Parameter Name: IC_DEFAULT_SDA_HOLD
Acknowledge I2C general calls on reset?	This parameter determines the reset value for the register IC_ACK_GENERAL_CALL, which in turn controls whether I2C general call addresses are to be responded or not. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: IC_ULTRA_FAST_MODE == 0 Parameter Name: IC_DEFAULT_ACK_GENERAL_CALL

3.3 Top Level Parameters / External Configuration Parameters

Table 3-3 Top Level Parameters / External Configuration Parameters

Label	Description
External Configuration	
Include DMA handshaking interface signals?	Configures the inclusion of DMA handshaking interface signals. When checked, includes the DMA handshaking interface signals at the top-level I/O. For more information about these signals, see "Signal Descriptions" in data book. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_HAS_DMA
Single Interrupt output port present?	If unchecked, each interrupt source has its own output. If checked, all interrupt sources are combined into a single output. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_INTR_IO
Polarity of Interrupts is active high?	Configures the active level of the output interrupt lines. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: Always Parameter Name: IC_INTR_POL

3.4 Top Level Parameters / Internal Configuration Parameters

Table 3-4 Top Level Parameters / Internal Configuration Parameters

Label	Description
Internal Configuration	
Add Encoded Parameters	Adding the encoded parameters gives firmware an easy and quick way of identifying the DesignWare component within an I/O memory map. Some critical design-time options determine how a driver should interact with the peripheral. There is a minimal area overhead by including these parameters. Allows a single driver to be developed for each component which will be self-configurable. When bit 7 of the IC_COMP_PARAM_1 is read and contains a '1' the encoded parameters can be read via software. If this bit is a '0' then the entire register is '0' regardless of the setting of any of the other parameters that are encoded in the register's bits. For details about this register, see the IC_COMP_PARAM_1 register. Note: Unique drivers must be developed for each configuration of the DW_apb_i2c. Based on the configuration, the registers in the IP can differ; thus the same driver cannot be used with different configurations of the IP. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: Always Parameter Name: IC_ADD_ENCODED_PARAMS
Specify clock counts directly instead of supplying clock frequency?	Determines whether *CNT values are provided directly or by specifying the ic_clk clock frequency and letting coreConsultant (or coreAssembler) calculate the count values. When this parameter is checked, the reset values of the *CNT registers are specified by the corresponding *COUNT configuration parameters which may be user-defined or derived (see standard, fast, fast mode plus, and high speed mode parameters later in this table). When unchecked (default setting), the reset values of the *CNT registers are calculated from the configuration parameter IC_CLOCK_PERIOD. Note: For fast mode plus, reprogram the IC_FS_SCL_*CNT register to achieve the required data rate when unchecked. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_USE_COUNTS

Label	Description
Hard code the count values for each mode?	By checking this parameter, the *CNT registers are set to read only. Unchecking this parameter (default setting) allows the *CNT registers to be writable. Regardless of the setting, the *CNT registers are always readable and have reset values from the corresponding *COUNT configuration parameters, which may be user defined or derived (see standard, fast, fast mode plus, or high speed mode parameters later in this table). Note: Since the DW_apb_i2c uses the same high and low count registers for fast mode and fast mode plus operation, if this parameter is checked (1) the IC_FS_SCL_*CNT registers are hard coded to either one of the fast mode and fast mode plus. Consequently, DW_apb_i2c can operate in either fast mode or fast mode plus, but not in both modes simultaneously. For fast mode plus, it is recommended that this parameter be Unchecked (0). Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_HC_COUNT_VALUES
ic_clk has a period of? (ns integers only)	Specifies the period of incoming ic_clk, used to generate outgoing I2C interface SCL clock. (ns integers only) When the count values are used to generate the IC_CLOCK_PERIOD then the IC_MAX_SPEED_MODE setting determines the actual period IC_MAX_SPEED_MODE = Standard => 500ns IC_MAX_SPEED_MODE = Fast => 100ns IC_MAX_SPEED_MODE = High => 10ns IC_ULTRA_FAST_MODE = 1 => 25ns Note: For fast mode plus, user has to reprogram the IC_FS_SCL_*CNT register to achieve required data rate. Values: 2, ..., 2147483647 Default Value: [<functionof> IC_MAX_SPEED_MODE IC_ULTRA_FAST_MODE] Enabled: IC_USE_COUNTS == 0 Parameter Name: IC_CLOCK_PERIOD
Relationship between pclk and ic_clk is?	Specifies the relationship between pclk and ic_clk Identical (0): clocks are identical; no meta-stability flops used for data passing between clock domains. Asynchronous (1): clocks may be completely asynchronous to each other, meta-stability flops are required for data passing between clock domains. Values: ■ Identical (0x0) ■ Asynchronous (0x1) Default Value: 0x1 Enabled: Always Parameter Name: IC_CLK_TYPE

Label	Description
Enable Asynchronous FIFO's for Data buffers?	This parameter controls whether DW_apb_i2c consist of Asynchronous or Synchronous FIFO's for the Transmit and Receive Data Buffers. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_CLK_TYPE==(0x1)) Parameter Name: IC_HAS_ASYNC_FIFO

3.5 Top Level Parameters / Bus Timing Parameters

Table 3-5 Top Level Parameters / Bus Timing Parameters

Label	Description
Standard Speed Mode Configuration	
Std speed SCL high count is?	Reset value of Standard Speed I2C Clock SCL High Count register (IC_SS_SCL_HCNT). The value must be calculated based on the I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see the IC_SS_SCL_HCNT register. Values: IC_HCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 3 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 1 : 6)) IC_CLOCK_PERIOD] Enabled: (IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE ==0) Parameter Name: IC_SS_SCL_HIGH_COUNT
Std speed SCL low count is?	Reset value of Standard Speed I2C Clock SCL Low Count register (IC_SS_SCL_LCNT). Value must be calculated based on I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see IC_SS_SCL_LCNT register. Values: IC_LCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 5 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 6 : 8)) IC_CLOCK_PERIOD] Enabled: (IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE ==0) Parameter Name: IC_SS_SCL_LOW_COUNT
Fast Mode or Fast Mode Plus Configuration	
Fast speed SCL high count is?	Reset value of Fast Mode or Fast Mode Plus I2C Clock SCL High Count register (IC_FS_SCL_HCNT). The value must be calculated based on I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see IC_FS_SCL_HCNT register. Values: IC_HCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_MAX_SPEED_MODE IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 3 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 1 : 6)) IC_CLOCK_PERIOD] Enabled: (IC_MAX_SPEED_MODE>=2 && IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==0) Parameter Name: IC_FS_SCL_HIGH_COUNT

Label	Description
Fast speed SCL low count is?	Reset value of Fast Mode or Fast Mode Plus I2C Clock SCL Low Count register (IC_FS_SCL_LCNT). The value must be calculated based on I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see the IC_FS_SCL_LCNT register Values: IC_LCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_MAX_SPEED_MODE IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 5 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 6 : 8)) IC_CLOCK_PERIOD] Enabled: (IC_MAX_SPEED_MODE>=2 && IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==0) Parameter Name: IC_FS_SCL_LOW_COUNT
High Speed Mode Configuration	
For high speed mode systems the I2C bus loading is? (pF)	For high speed mode, the bus loading affects the high and low pulse width of SCL. Values: ■ 100 (100) ■ 400 (400) Default Value: 100 Enabled: (IC_MAX_SPEED_MODE==3) && (IC_ULTRA_FAST_MODE ==0) Parameter Name: IC_CAP_LOADING
High speed SCL high count is?	Reset value of High Speed I2C Clock SCL High Count register (IC_HS_SCL_HCNT). The value must be calculated based on I2C data rate desired and high speed I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see IC_HS_SCL_HCNT register. Values: IC_HCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_MAX_SPEED_MODE IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 3 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 1 : 6)) IC_CLOCK_PERIOD IC_CAP_LOADING] Enabled: (IC_MAX_SPEED_MODE==3 && IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==0) Parameter Name: IC_HS_SCL_HIGH_COUNT
High speed SCL low count is?	Reset value of High Speed I2C Clock SCL Low Count register (IC_HS_SCL_LCNT). The value must be calculated based on I2C data rate and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. For more information, see IC_HS_SCL_LCNT register. Values: IC_LCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_MAX_SPEED_MODE IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 5 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 6 : 8)) IC_CLOCK_PERIOD IC_CAP_LOADING] Enabled: (IC_MAX_SPEED_MODE==3 && IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==0) Parameter Name: IC_HS_SCL_LOW_COUNT

Label	Description
Spike Suppression Configuration	
Maximum length (in ic_clk cycles) of suppressed spikes in Standard Mode, Fast Mode, and Fast Mode Plus	Reset value of maximum suppressed spike length register in Standard Mode, Fast Mode, and Fast Mode Plus modes (IC_FS_SPKLEN Register). Spike length is expressed in ic_clk cycles and this value is calculated based on the value of IC_CLOCK_PERIOD. Values: 0x1, ..., 0xff Default Value: [<functionof> IC_CLOCK_PERIOD (50)] Enabled: IC_ULTRA_FAST_MODE==0 Parameter Name: IC_DEFAULT_FS_SPKLEN
Maximum length (in ic_clk cycles) of suppressed spikes in HS mode	Reset value of maximum suppressed spike length register in HS modes (Register IC_HS_SPKLEN). Spike length is expressed in ic_clk cycles and this value is calculated based on the value of IC_CLOCK_PERIOD. Values: 0x1, ..., 0xff Default Value: [<functionof> IC_CLOCK_PERIOD (10)] Enabled: (IC_MAX_SPEED_MODE==3) && (IC_ULTRA_FAST_MODE ==0) Parameter Name: IC_DEFAULT_HS_SPKLEN

3.6 Top Level Parameters / Additional Features Parameters

Table 3-6 Top Level Parameters / Additional Features Parameters

Label	Description
Additional Features	
Allow dynamic updating of the TAR address?	When checked, allows the IC_TAR register to be updated dynamically. Setting this parameter affects the operation of DW_apb_i2c when it is in controller mode. For more details, see "Controller Mode Operation". Values: ■ false (0) ■ true (1) Default Value: false Enabled: Always Parameter Name: I2C_DYNAMIC_TAR_UPDATE
Enable register to generate NACKs for data received by Target?	Enables an additional register which controls whether the DW_apb_i2c generates a NACK after a data byte has been transferred to it. This NACK generation only occurs when the DW_apb_i2c is a Target-Receiver. If this register is set to a value of 1, it can only generate a NACK after a data byte is received; hence, the data transfer is terminated. Also, the data received is not pushed to the receive buffer. When the register is set to a value of 0, it generates NACK/ACK depending on normal criteria. If this option is selected, the default value of the register IC_SLV_DATA_NACK_ONLY is always 0. The register must be explicitly programmed to a value of 1 if NACKs are to be generated. The register can only be written to successfully if DW_apb_i2c is disabled (IC_ENABLE[0] = 0) or the target part is inactive (IC_STATUS[6] = 0). Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_ULTRA_FAST_MODE == 0 Parameter Name: IC_SLV_DATA_NACK_ONLY
When Receive FIFO is Physically full, Generate NACK for data received by target?	This parameter enables DW_apb_i2c in Target mode to generate NACK for a data byte received when Receive FIFO is physically full. The new data byte will not be pushed to the Receive FIFO, hence no overflow happens and rx_over interrupt will not be set. This works only when DW_apb_i2c is in Target/Receiver mode (data being written to the target) and is not applicable in Controller mode. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_ULTRA_FAST_MODE == 0) && (IC_MULTI_SAR_EN == 0) Parameter Name: IC_RX_FULL_GEN_NACK

Label	Description
Hold transfer when Tx FIFO is empty	If this parameter is set, the controller will only complete a transfer - that is issues a STOP - when it finds a Tx FIFO entry tagged with a Stop bit. If the Tx FIFO becomes empty and the last byte does not have the Stop bit set, the controller stalls the transfer by holding the SCL line low. If this parameter is not set, the controller completes a transfer when the Tx FIFO is empty. In SMBus Mode (IC_SMBUS=1), IC_EMPTYFIFO_HOLD_MASTER_EN should be always enabled. Values: ■ false (0) ■ true (1) Default Value: IC_SMBUS == 1 ? 1 : 0 Enabled: Always Parameter Name: IC_EMPTYFIFO_HOLD_MASTER_EN
When Receive FIFO is Physically Full, Hold the Bus Till Receive FIFO has Space Available?	When the Rx FIFO is physically full to its RX_BUFFER_DEPTH, this parameter provides a hardware method to hold the bus till Rx FIFO data is read out and there is a space available in the FIFO. This parameter can be used when DW_apb_i2c is either a target-receiver (that is, data is written to the device) or a controller-receiver (that is, the device reads data from a target). Note:If parameter "IC_RX_FULL_GEN_NACK" is enabled, then setting this parameter has no impact in target-receiver mode since, the controller NACK's the Data byte if Rx-FIFO has no empty space. Note:If this parameter is checked and bit 9 of the IC_CON register (RX_FIFO_FULL_HLD_CTRL) is programmed to HIGH, then the RX_OVER interrupt is never set to 1 as the criteria to set this interrupt is never met. The RX_OVER interrupt can be found in IC_INTR_STAT and IC_RAW_INTR_STAT registers. It is also an optional output signal, ic_rx_over_intr(_n). Values: ■ false (0x0) ■ true (0x1) Default Value: (IC_MULTI_SAR_EN == 1) ? 1 : 0 Enabled: (IC_ULTRA_FAST_MODE == 0) && (IC_MULTI_SAR_EN == 0) Parameter Name: IC_RX_FULL_HLD_BUS_EN
Enable restart detect interrupt in target mode?	When checked, allows the target to detect and issue the restart interrupt when target is addressed. Setting this parameter affects the operation of DW_apb_i2c only when it is in target mode. This controls the "RESTART_DET" bit in the IC_RAW_INTR_STAT, IC_INTR_MASK, IC_INTR_STAT, and IC_CLR_RESTART_DET registers. This also controls the ic_restart_det_intr(_n) and ic_intr(_n) signals. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_SLV_RESTART_DET_EN

Label	Description
Generate STOP_DET interrupt only if Controller is active?	Controls whether DW_apb_i2c generates STOP_DET interrupt when controller is active: ■ Checked (1): Allows the controller to detect and issue the stop interrupt when controller is active. ■ Unchecked (0): The controller always detects and issues the stop interrupt irrespective of whether it is active. This parameter affects the operation of DW_apb_i2c when it is in controller mode. This controls the STOP_DET bit of the IC_RAW_INTR_STAT, IC_INTR_MASK, IC_INTR_STAT and IC_CLR_STOP_DET registers. This also controls the ic_stop_det_intr(_n) and ic_intr(_n) signals. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_ULTRA_FAST_MODE ==0 Parameter Name: IC_STOP_DET_IF_MASTER_ACTIVE
Include Status bits to indicate the reason for clock stretching?	If this parameter is set, the DW_apb_i2c consists of status bits indicating the reason for clock stretching in the IC_STATUS Register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_ULTRA_FAST_MODE ==0 Parameter Name: IC_STAT_FOR_CLK_STRETCH
Include programmable bit for blocking Controller commands?	Controls whether DW_apb_i2c transmits data on I2C bus as soon as data is available in Tx FIFO. When checked, allows the controller to hold the transmission of data on I2C bus when Tx FIFO has data to transmit. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_TX_CMD_BLOCK
Enable blocking Controller commands after reset?	Controls whether DW_apb_i2c has its transmit command block enabled or disabled after reset. If checked, the DW_apb_i2c blocks the transmission of data on I2C bus. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_TX_CMD_BLOCK==1 Parameter Name: IC_TX_CMD_BLOCK_DEFAULT

Label	Description
Include First data byte indication in IC_DATA_CMD register?	Controls whether DW_apb_i2c generates FIRST_DATA_BYTE status bit in IC_DATA_CMD register. When checked, the controller/target receiver to set the FIRST_DATA_BYTE status bit in IC_DATA_CMD register to indicate whether the data present in IC_DATA_CMD register is first data byte after the address phase of a receive transfer. Note: In the case when APB_DATA_WIDTH is set to 8, you must perform two APB reads to the IC_DATA_CMD register to get status on bit 11. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: IC_FIRST_DATA_BYTE_STATUS
Avoid Rx FIFO Flush on Transmit Terminate?	This Parameter controls the Rx FIFO Flush during the Transmit Terminate. If this parameter is checked(1), only the Tx FIFO is flushed (not the Rx FIFO) Flush on the Transmit Terminate. If this parameter is unchecked(0), both Tx FIFO and Rx FIFO are flushed on Transmit Terminate. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_ULTRA_FAST_MODE ==0 Parameter Name: IC_AVOID_RX_FIFO_FLUSH_ON_TX_ABORT
Enable IC_CLK Frequency Reduction?	This parameter is used to reduce the system clock frequency (ic_clk) by reducing the internal latency required to generate the high period and low period of the SCL line. Values: ■ false (0x0) ■ true (0x1) Default Value: IC_ULTRA_FAST_MODE == 1 ? 1 : 0 Enabled: DWC-APB-Advanced-Source License Required and IC_ULTRA_FAST_MODE=0 Parameter Name: IC_CLK_FREQ_OPTIMIZATION

3.7 I2C Features Parameters

Table 3-7 I2C Features Parameters

Label	Description
I2C Features	
Include Bus Clear feature?	This parameter will enable the Bus clear feature for the DW_apb_i2c core. If this parameter is set: ■ If an SDA line is stuck at low for IC_SDA_STUCK_LOW_TIMEOUT period of ic_clk, DW_apb_i2c controller generates a controller transmit terminate (IC_TX_ABRT_SOURCE[17]: ABRT_SDA_STUCK_AT_LOW) to indicate SDA stuck at low. User can enable the SDA_STUCK_RECOVERY_EN (IC_ENABLE[3]) register bit to recover the SDA by sending at most 9 SCL clocks. If SDA line is recovered, then the controller generates a STOP and auto clear the 'SDA_STUCK_RECOVERY_EN' register bit and resume the normal I2C transfers. If an SDA line is not recovered, then the controller auto clears the SDA_STUCK_RECOVERY_EN register bit and asserts the SDA_STUCK_NOT_RECOVERED (IC_STATUS[11]) status bit to indicate the SDA is not recovered after sending 9 SCL clocks which intimate the user for system reset. ■ If SCL line is stuck at low for IC_SCL_STUCK_LOW_TIMEOUT period of ic_clk, DW_apb_i2c Controller will generate an SCL_STUCK_AT_LOW (IC_INTR_RAW_STATUS[14]) interrupt to intimate the user for system reset. Values: ■ false (0x0) ■ true (0x1) Default Value: IC_SMBUS==1 ? 1 : 0 Enabled: IC_ULTRA_FAST_MODE ==0 Parameter Name: IC_BUS_CLEAR_FEATURE
Has SCL Stuck Timeout value of?	Default value of the IC_SCL_STUCK_LOW_TIMEOUT Register. Values: 0x0, ..., 0xffffffff Default Value: 0xffffffff Enabled: IC_BUS_CLEAR_FEATURE==1 Parameter Name: IC_SCL_STUCK_TIMEOUT_DEFAULT
Has SDA Stuck Timeout value of?	Default value of the IC_SDA_STUCK_LOW_TIMEOUT Register. Values: 0x0, ..., 0xffffffff Default Value: 0xffffffff Enabled: IC_BUS_CLEAR_FEATURE==1 Parameter Name: IC_SDA_STUCK_TIMEOUT_DEFAULT

Label	Description
Enable Device ID feature?	If this Parameter is enabled, the DW_apb_i2c target includes a 24-bit IC_DEVICE_ID Register to store the value of Device ID and transmits whenever controller is requested. The Controller mode includes a DEVICE_ID bit 13 in IC_TAR register to initiate the Device ID read for a particular target address mentioned in IC_TAR[6:0] register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_ULTRA_FAST_MODE==0) & (IC_SMBUS==0) Parameter Name: IC_DEVICE_ID
Has I2C Target DEVICE ID value of?	Device ID Value of the I2C Target stored in the IC_DEVICE_ID Register (24 bit, MSB is transferred first on the Device ID read from the controller). Values: 0x0, ..., 0xfffff Default Value: 0x0 Enabled: IC_DEVICE_ID==1 Parameter Name: IC_DEVICE_ID_VALUE
Enable Ultra-Fast Mode?	This parameter is used to control whether DW_apb_i2c supports Ultra-Fast speed mode or not. If this Parameter is enabled, the Controller ■ Disables the Arbitration, clock synchronization features. ■ Support only write transfers. ■ Does not check the validity of ACK/NACK for each byte. The Target ■ Supports only write transfers. ■ Disables the logic to generate ACK/NACK after the end of each byte. ■ Disables the logic to stretch the clock if RX-FIFO is full. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: DWC-APB-Advanced-Source License Required. Parameter Name: IC_ULTRA_FAST_MODE
Ultra Fast speed SCL high count is?	Reset value of Ultra-Fast Speed I2C Clock SCL High Count register (IC_UFM_SCL_HCNT). The value must be calculated based on the I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. Values: IC_HCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 3 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 1 : 6)) IC_CLOCK_PERIOD IC_ULTRA_FAST_MODE] Enabled: (IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==1) Parameter Name: IC_UFM_SCL_HIGH_COUNT

Label	Description
Ultra Fast speed SCL low count is?	Reset value of Ultra-Fast Speed I2C Clock SCL Low Count register (IC_UFM_SCL_LCNT). The value must be calculated based on the I2C data rate desired and I2C clock frequency. When parameter IC_USE_COUNTS = 0, this parameter is automatically calculated using the IC_CLOCK_PERIOD parameter. Values: IC_LCNT_LO_LIMIT, ..., 0xffff Default Value: [<functionof> IC_USE_COUNTS ((IC_ULTRA_FAST_MODE==1)? 5 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 6 : 8)) IC_CLOCK_PERIOD IC_ULTRA_FAST_MODE] Enabled: (IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==1) Parameter Name: IC_UFM_SCL_LOW_COUNT
Maximum length (in ic_clk cycles) of suppressed spikes in Ultra Fast mode	Reset value of maximum suppressed spike length register in Ultra-Fast Mode (IC_UFM_SPKLEN Register). Spike length is expressed in ic_clk cycles and this value is calculated based on the value of IC_CLOCK_PERIOD. Values: 0x1, ..., 0xff Default Value: [<functionof> IC_CLOCK_PERIOD (10)] Enabled: IC_ULTRA_FAST_MODE ==1 Parameter Name: IC_DEFAULT_UFM_SPKLEN
Has Ultra Fast mode tBuf count Value of?	Default value of the IC_UFM_TBUF_CNT Register. This parameter is active when the IC_USE_COUNTS and IC_ULTRA_FAST_MODE parameters are checked (1); otherwise, this value is automatically calculated using the IC_CLOCK_PERIOD parameter. Values: 0x0, ..., 0xffff Default Value: [<functionof> IC_USE_COUNTS IC_CLOCK_PERIOD] Enabled: (IC_USE_COUNTS==1) && (IC_ULTRA_FAST_MODE==1) Parameter Name: IC_UFM_TBUF_CNT_DEFAULT

3.8 SMBus Features Parameters

Table 3-8 SMBus Features Parameters

Label	Description
I2C System Management Bus Features	
Enable SMBus Mode?	Controls whether DW_apb_i2c Controller/Target supports SMBus mode. If checked, the DW_apb_i2c includes the SMBus mode related registers, real-time checks, timeout interrupts, and SMBus optional signals. Note: If this parameter is selected (1), then the user can set the parameter IC_MAX_SPEED_MODE to Standard mode(1) or Fast Mode/Fast Mode Plus (2). Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: DWC-APB-Advanced-Source License Required and IC_ULTRA_FAST_MODE=0 Parameter Name: IC_SMBUS
Enable Multiple SAR feature?	This parameter allows user to include Multiple SAR feature in the DW_apb_i2c. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: DWC-APB-Advanced-Source License Required and SMBus feature is enabled (IC_SMBUS = 1). Parameter Name: IC_MULTI_SAR_EN
Number of SAR's Supported?	This parameter determines the number of SARs supported by DW_apb_i2c. Values: 2, 3, 4 Default Value: 2 Enabled: IC_MULTI_SAR_EN == 1 Parameter Name: IC_NUM_SARS
Has SMBus clock low Target extend default Timeout value of?	Default value of the IC_SMBUS_CLK_LOW_SEXT Register. Values: 0x0, ..., 0xffffffff Default Value: 0xffffffff Enabled: IC_SMBUS==1 Parameter Name: IC_SMBUS_CLK_LOW_SEXT_DEFAULT
Has SMBus clock low Controller extend default Timeout value of?	Default value of the IC_SMBUS_CLK_LOW_MEXT Register. Values: 0x0, ..., 0xffffffff Default Value: 0xffffffff Enabled: IC_SMBUS==1 Parameter Name: IC_SMBUS_CLK_LOW_MEXT_DEFAULT

Label	Description
Has SMBus Thigh:Max Idle count Value of?	Default value of the IC_SMBUS_THIGH_MAX_IDLE_COUNT Register. Values: 0x0, ..., 0xffff Default Value: 0xffff Enabled: IC_SMBUS==1 Parameter Name: IC_SMBUS_RST_IDLE_CNT_DEFAULT
Enable SMBus Optional Signals?	This parameter controls whether DW_apb_i2c includes Optional SMBus Suspend and Alert signals on the interface. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_SMBUS==1 Parameter Name: IC_SMBUS_SUSPEND_ALERT
Include Optional target address register?	This parameter controls whether to include optional Target Address Register in SMBus Mode. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_SMBUS == 1) && (IC_MULTI_SAR_EN==0) Parameter Name: IC_OPTIONAL_SAR
Has I2C default optional target address of?	Controls whether to include Optional Target Address Register in SMBus Mode. A user is not allowed to assign any reserved addresses. The reserved address are as follows: 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x78 0x79 0x7a 0x7b 0x7c 0x7d 0x7e 0x7f Values: 0x0, ..., 0x7f Default Value: 0x0 Enabled: IC_OPTIONAL_SAR==1 Parameter Name: IC_OPTIONAL_SAR_DEFAULT

3.9 SMBus Features / SMBus ARP Features Parameters

Table 3-9 SMBus Features / SMBus ARP Features Parameters

Label	Description
SMBus ARP Features	
Enable Address Resolution Protocol in SMBus Mode?	Controls whether DW_apb_i2c includes logic to detect and respond ARP commands in Target mode. It also includes logic to generate/validate the PEC byte at the end of the transfer in Target mode only. Values: 0x0, 0x1 Default Value: 0x0 Enabled: IC_SMBUS==1 Parameter Name: IC_SMBUS_ARP
Select the Multiple SAR UDID Identification Process?	When set to 0, the DW_apb_i2c uses UDID[LOG2_IC_NUM_SARS-1:0] bits to identify each target during the SMBus ARP sequence. When set to 1, the DW_apb_i2c incorporates separate UDID registers for each target. The DW_apb_i2c uses these values to identify each target during the SMBus ARP sequence. In Future this parameter will be removed and default value for this parameter will be set to 1. Values: ■ false (0x0) ■ true (0x1) Default Value: (IC_SMBUS_ARP==1 && IC_MULTI_SAR_EN==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1) ? 1 : 0 Enabled: IC_SMBUS_ARP==1 && IC_MULTI_SAR_EN==1 && (IC_SMBUS_FD_FND_SUPPORT_EN==0) Parameter Name: IC_MULTI_SAR_UDID_CTRL
Enable SMBus Fixed Discoverable and Fixed Non-Discoverable Device Support?	This parameter allows user to enable SMBus Fixed Discoverable (FD) and Fixed non-Discoverable (FnD) device support in DW_apb_i2c. The user can further program 'IC_SMBUS_ARP_CTL' register to select the desired device type for different IC_SAR target address among non-ARP, ARP capable, FD and FnD device types. Values: 0x0, 0x1 Default Value: 0x0 Enabled: IC_SMBUS_ARP==1 Parameter Name: IC_SMBUS_FD_FND_SUPPORT_EN

3.10 SMBus Features / IC_SAR SMBus ARP Features Parameters

Table 3-10 SMBus Features / IC_SAR SMBus ARP Features Parameters

Label	Description
IC_SAR SMBus ARP Features	
Default value of IC_SAR SMBus Device Type	This parameter determines the reset value for the SMBus device type of IC_SAR target address. This is the value after reset for 'IC_SMBUS_ARP_CTRL.IC_SAR_SMBUS_DEVICE_TYPE'. Values: ■ Non-ARP Device (0x0) ■ ARP Device (0x1) ■ Fixed and Discoverable Device (0x2) ■ Fixed and Not-Discoverable Device (0x3) Default Value: ARP Device Enabled: IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 Parameter Name: IC_SAR_DFLT_SMBUS_DEVICE_TYPE
SMBus Unique Device Identifier (UDID) Hardcode?	Controls whether Unique Device Identifier (UDID) used for Dynamic Address Resolution process in SMBus ARP Mode is Hardcoded (Upper 96-bits) or Complete UDID is Software Programmable. Values: 0x0, 0x1 Default Value: 0x1 Enabled: IC_SMBUS_ARP==1 Parameter Name: IC_SMBUS_UDID_HC
Has SMBUS Unique device identifier (MSB 96 bits) value of?	If the parameter IC_SMBUS_UDID_HC is 1, stores the Static Unique Device Identifier used for Dynamic Address Resolution process in SMBus ARP Mode (Upper 96bits of UDID). If the parameter IC_SMBUS_UDID_HC is 0, then this field is used as the default value of the upper 96bits of the UDID Registers {IC_SMBUS_UDID_WORD3, IC_SMBUS_UDID_WORD2, IC_SMBUS_UDID_WORD1} Values: 0x0, ..., 0xffffffffffffffffffff Default Value: 0x0 Enabled: IC_SMBUS_ARP==1 Parameter Name: IC_SMBUS_UDID_MSB
Has Default SMBus Unique device identifier (LSB 32 bits) value of?	If the parameter IC_SMBUS_UDID_HC is 1, specifies default value of the IC_SMBUS_UDID_LSB register used for Dynamic Address Resolution process in SMBus ARP mode (Lower 32bits of UDID). If the parameter IC_SMBUS_UDID_HC is 0, specifies default value of the IC_SMBUS_UDID_WORD0 register used for Dynamic Address Resolution process in SMBus ARP mode (Lower 32bits of UDID). Values: 0x0, ..., 0xffffffff Default Value: 0xffffffff Enabled: IC_SMBUS_ARP==1 Parameter Name: IC_SMBUS_UDID_LSB_DEFAULT

Label	Description
Has Default Persistent Target Address register bit Value of?	Default value of the Persistent Target Address register bit in IC_CON Register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: IC_SMBUS_ARP==1 Parameter Name: IC_PERSISTANT_SLV_ADDR_DEFAULT

3.11 SMBus Features / IC_SAR2 SMBus ARP Features Parameters

Table 3-11 SMBus Features / IC_SAR2 SMBus ARP Features Parameters

Label	Description
IC_SAR2 SMBus ARP Features	
Has Default Persistent Target Address register (IC_SAR2) bit Value of?	Default value of the Persistent Target Address register (IC_SAR2) bit in IC_CON Register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) && (IC_SMBUS_ARP==1) Parameter Name: IC_PERSISTANT_SLV_ADDR2_DEFAULT
Default value of IC_SAR2 SMBus Device Type	This parameter determines the reset value for the SMBus device type of IC_SAR2 target address. This is the value after reset for 'IC_SMBUS_ARP_CTRL.IC_SAR2_SMBUS_DEVICE_TYPE'. Values: ■ Non-ARP Device (0x0) ■ ARP Device (0x1) ■ Fixed and Discoverable Device (0x2) ■ Fixed and Not-Discoverable Device (0x3) Default Value: ARP Device Enabled: IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_MULTI_SAR_EN==1 && IC_NUM_SARS>=2 Parameter Name: IC_SAR2_DFLT_SMBUS_DEVICE_TYPE
Hardcode IC_SAR2 SMBus Unique Device Identifier (UDID)?	When set to 1, the UDID of the IC_SAR2 target address is hardcoded and registers IC_SAR2_SMBUS_UDID_WORD0/1/2/3 are removed. When set to 0, the UDID of IC_SAR2 is programmable through registers IC_SAR2_SMBUS_UDID_WORD0/1/2/3. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR2_SMBUS_UDID_HC

Label	Description
Default Value of Registers IC_SAR2_SMBUS_UDID_WORD0/1/2/3?	If the parameter IC_SAR2_SMBUS_UDID_HC is 1, stores the Static Unique Device Identifier used for Dynamic Address Resolution process in SMBus ARP Mode for IC_SAR2 target address. If the parameter IC_SAR2_SMBUS_UDID_HC is 0, then this field is used as the default value of the IC_SAR2 UDID Registers {IC_SAR2_SMBUS_UDID_WORD3, IC_SAR2_SMBUS_UDID_WORD2, IC_SAR2_SMBUS_UDID_WORD1, IC_SAR2_SMBUS_UDID_WORD0}. Values: 0x0, ..., 0xffffffffffffffffffffffff Default Value: 0xffff0000ffff0000ffff00003fff0000 Enabled: IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR2_SMBUS_UDID_DEFAULT

3.12 SMBus Features / IC_SAR3 SMBus ARP Features Parameters

Table 3-12 SMBus Features / IC_SAR3 SMBus ARP Features Parameters

Label	Description
IC_SAR3 SMBus ARP Features	
Has Default Persistent Target Address register (IC_SAR3) bit Value of?	Default value of the Persistent Target Address register (IC_SAR3) bit in IC_CON Register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) && (IC_SMBUS_ARP==1) Parameter Name: IC_PERSISTANT_SLV_ADDR3_DEFAULT
Default value of IC_SAR3 SMBus Device Type	This parameter determines the reset value for the SMBus device type of IC_SAR3 target address. This is the value after reset for 'IC_SMBUS_ARP_CTRL.IC_SAR3_SMBUS_DEVICE_TYPE'. Values: ■ Non-ARP Device (0x0) ■ ARP Device (0x1) ■ Fixed and Discoverable Device (0x2) ■ Fixed and Not-Discoverable Device (0x3) Default Value: ARP Device Enabled: IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_MULTI_SAR_EN==1 && IC_NUM_SARS>=3 Parameter Name: IC_SAR3_DFLT_SMBUS_DEVICE_TYPE
Hardcode IC_SAR3 SMBus Unique Device Identifier (UDID)?	When set to 1, the UDID of the IC_SAR3 target address is hardcoded and registers IC_SAR3_SMBUS_UDID_WORD0/1/2/3 are removed. When set to 0, the UDID of IC_SAR3 is programmable through registers IC_SAR3_SMBUS_UDID_WORD0/1/2/3. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: IC_NUM_SARS>=3 && IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR3_SMBUS_UDID_HC

Label	Description
Default Value of Registers IC_SAR3_SMBUS_UDID_WORD0/1/2/3?	If the parameter IC_SAR3_SMBUS_UDID_HC is 1, stores the Static Unique Device Identifier used for Dynamic Address Resolution process in SMBus ARP Mode for IC_SAR3 target address. If the parameter IC_SAR3_SMBUS_UDID_HC is 0, then this field is used as the default value of the IC_SAR3 UDID Registers {IC_SAR3_SMBUS_UDID_WORD3, IC_SAR3_SMBUS_UDID_WORD2, IC_SAR3_SMBUS_UDID_WORD1, IC_SAR3_SMBUS_UDID_WORD0}. Values: 0x0, ..., 0xffffffffffffffffffffffff Default Value: 0xffff0000ffff0000ffff00007fff0000 Enabled: IC_NUM_SARS>=3 && IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR3_SMBUS_UDID_DEFAULT

3.13 SMBus Features / IC_SAR4 SMBus ARP Features Parameters

Table 3-13 SMBus Features / IC_SAR4 SMBus ARP Features Parameters

Label	Description
IC_SAR4 SMBus ARP Features	
Has Default Persistent Target Address register (IC_SAR4) bit Value of?	Default value of the Persistent Target Address register (IC_SAR4) bit in IC_CON Register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) && (IC_SMBUS_ARP==1) Parameter Name: IC_PERSISTANT_SLV_ADDR4_DEFAULT
Default value of IC_SAR4 SMBus Device Type	This parameter determines the reset value for the SMBus device type of IC_SAR4 target address. This is the value after reset for 'IC_SMBUS_ARP_CTRL.IC_SAR4_SMBUS_DEVICE_TYPE'. Values: ■ Non-ARP Device (0x0) ■ ARP Device (0x1) ■ Fixed and Discoverable Device (0x2) ■ Fixed and Not-Discoverable Device (0x3) Default Value: ARP Device Enabled: IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_MULTI_SAR_EN==1 && IC_NUM_SARS>=4 Parameter Name: IC_SAR4_DFLT_SMBUS_DEVICE_TYPE
Hardcode IC_SAR4 SMBus Unique Device Identifier (UDID)?	When set to 1, the UDID of the IC_SAR4 target address is hardcoded and registers IC_SAR4_SMBUS_UDID_WORD0/1/2/3 are removed. When set to 0, the UDID of IC_SAR4 is programmable through registers IC_SAR4_SMBUS_UDID_WORD0/1/2/3. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: IC_NUM_SARS>=4 && IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR4_SMBUS_UDID_HC

Label	Description
Default Value of Registers IC_SAR4_SMBUS_UDID_WORD0/1/2/3?	If the parameter IC_SAR4_SMBUS_UDID_HC is 1, stores the Static Unique Device Identifier used for Dynamic Address Resolution process in SMBus ARP Mode for IC_SAR4 target address. If the parameter IC_SAR4_SMBUS_UDID_HC is 0, then this field is used as the default value of the IC_SAR4 UDID Registers {IC_SAR4_SMBUS_UDID_WORD3, IC_SAR4_SMBUS_UDID_WORD2, IC_SAR4_SMBUS_UDID_WORD1, IC_SAR4_SMBUS_UDID_WORD0}. Values: 0x0, ..., 0xffffffffffffffffffffffff Default Value: 0xffff0000ffff0000ffff0000bfff0000 Enabled: IC_NUM_SARS>=4 && IC_MULTI_SAR_UDID_CTRL==1 Parameter Name: IC_SAR4_SMBUS_UDID_DEFAULT

3.14 DesignWare Foundation Configuration Parameters

Table 3-14 DesignWare Foundation Configuration Parameters

Label	Description
DesignWare Foundation Configuration	
Use DesignWare Foundation Synthesis Library	Specifies the availability of a DesignWare license. The component code utilizes DesignWare Foundation parts for optimal Synthesis QoR. Customers with only a DesignWare license must use Foundation parts. Customers with only a Source license, cannot use Foundation parts. Customers with both Source and DesignWare licenses have the option of using Foundation parts. Values: ■ false (0) ■ true (1) Default Value: True if DesignWare License is available; False if DesignWare License is not available Enabled: When both Source and DesignWare licenses are available Parameter Name: USE_FOUNDATION

Signal Descriptions

This chapter details all possible I/O signals in the IP. For configurable IP titles, your actual configuration might not contain all of these signals.

Inputs are on the left of the signal diagrams; outputs are on the right.

Attention: For configurable IP titles, do not use this document to determine the exact I/O footprint of the controller. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the I/O signals for your actual configuration at workspace/report/IO.html or workspace/report/IO.xml after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the I/O signals that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the widths might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

In addition to describing the function of each signal, the signal descriptions in this chapter include the following information:

- **Active State:** Indicates whether the signal is active high or active low. When a signal is not intended to be used in a particular application, then this signal needs to be tied or driven to the inactive state (opposite of the active state).
- **Registered:** Indicates whether or not the signal is registered directly inside the IP boundary without intervening logic (excluding simple buffers). A value of *No* does not imply that the signal is not synchronous, only that there is some combinatorial logic between the signal's origin or destination register and the boundary of the controller. A value of *N/A* indicates that this information is not provided for this IP title.
- **Synchronous to:** Indicates which clocks in the IP sample this input (drive for an output). This clock might not be the same as the clock that your application logic should use to clock (sample/ drive) this pin. For more details, consult the clock section in the databook. The presence of the postfix SuperList indicates a list of all possible clocks over all possible configs. Consult coreConsultant report for which clock applies to your specific configuration.
- **Exists:** Name of configuration parameter that populates this signal in your configuration.

- **Power Domain:** Name of power/voltage domain that this signal is part of when power/voltage islands are used. The SINGLE_DOMAIN value indicates that there are no islands.

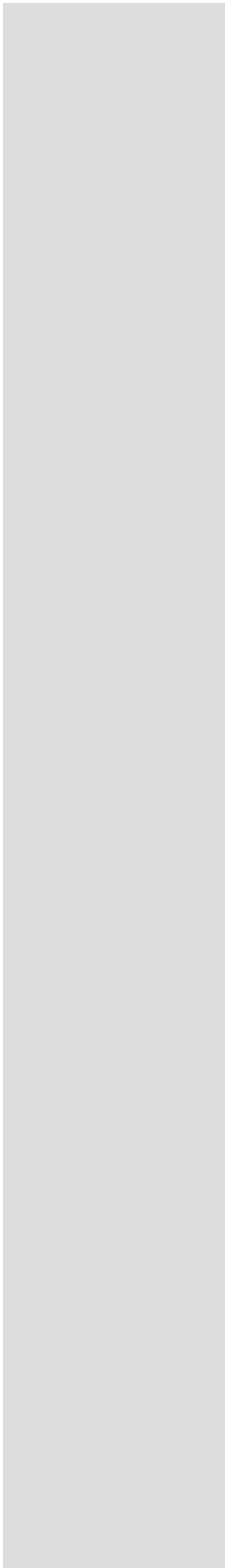
Attributes used with Synchronous To:

- Clock name - The name of the clock that samples an input or drives an output.
- None - This attribute may be used for clock inputs, hard-coded outputs, feed-through (direct or combinatorial), dangling inputs, unused inputs and asynchronous outputs.
- Asynchronous - This attribute is used for asynchronous inputs and asynchronous resets.

The I/O signals are grouped as follows:

- [“Interrupts Signals”](#) on page 168
- [“I2C Interface \(Controller/Target\) Signals”](#) on page 183
- [“APB Completer Interface Signals”](#) on page 186
- [“DMA Interface Signals”](#) on page 189
- [“SMBus Interface Signals”](#) on page 191
- [“I2C Debug Signals”](#) on page 192

4.1 Interrupts Signals



- ic_intr
- ic_mst_on_hold_intr
- ic_start_det_intr
- ic_stop_det_intr
- ic_restart_det_intr
- ic_scl_stuck_at_low_intr
- ic_smbus_clk_sext_intr
- ic_smbus_clk_mext_intr
- ic_smbus_quick_cmd_det_intr
- ic_smbus_arp_prepare_intr
- ic_smbus_arp_reset_intr
- ic_smbus_arp_get_udid_intr
- ic_smbus_arp_assign_address_intr
- ic_smbus_host_notify_intr
- ic_smbus_slv_rx_pec_nack_intr
- ic_smbalert_det_intr
- ic_smbus_det_intr
- ic_activity_intr
- ic_rx_done_intr
- ic_tx_abrt_intr
- ic_rd_req_intr
- ic_wr_req_intr
- ic_slv_addr_tag_intr
- ic_tx_empty_intr
- ic_tx_over_intr
- ic_rx_full_intr
- ic_rx_over_intr
- ic_rx_under_intr
- ic_gen_call_intr
- ic_intr_n
- ic_mst_on_hold_intr_n
- ic_start_det_intr_n
- ic_stop_det_intr_n
- ic_restart_det_intr_n
- ic_scl_stuck_at_low_intr_n
- ic_smbus_clk_sext_intr_n
- ic_smbus_clk_mext_intr_n
- ic_smbus_quick_cmd_det_intr_n
- ic_smbus_arp_prepare_intr_n
- ic_smbus_arp_reset_intr_n
- ic_smbus_arp_get_udid_intr_n
- ic_smbus_arp_assign_address_intr_n
- ic_smbus_host_notify_intr_n
- ic_smbus_slv_rx_pec_nack_intr_n
- ic_smbalert_det_intr_n
- ic_smbus_det_intr_n
- ic_activity_intr_n
- ic_rx_done_intr_n
- ic_tx_abrt_intr_n
- ic_rd_req_intr_n
- ic_wr_req_intr_n
- ic_slv_addr_tag_intr_n
- ic_tx_empty_intr_n
- ic_tx_over_intr_n
- ic_rx_full_intr_n
- ic_rx_over_intr_n
- ic_rx_under_intr_n
- ic_gen_call_intr_n

Table 4-1 Interrupts Signals

Port Name	I/O	Description
ic_intr	O	Combined active high interrupt. This signal is included on the interface when the configuration parameter IC_INTR_IO is checked (1) to indicate that only one interrupt line appears on the I/O (as opposed to individual interrupt signals). Polarity is set by the configuration parameter IC_INTR_POL (checked(1) = active high). Exists: (IC_INTR_IO==1) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_mst_on_hold_intr	O	Controller on hold I2C active high interrupt. This signal is included on the interface when the configuration parameters I2C_DYNAMIC_TAR_UPDATE and IC_EMPTYFIFO_HOLD_MASTER_EN are checked (1) and the configuration parameter IC_INTR_IO is unchecked (0), indicating that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (I2C_DYNAMIC_TAR_UPDATE==1 && IC_EMPTYFIFO_HOLD_MASTER_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_start_det_intr	O	Start condition detect on I2C active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_stop_det_intr	O	Stop condition detect on I2C active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_restart_det_intr	O	Restart condition detect on I2C active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SLV_RESTART_DET_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_scl_stuck_at_low_intr	O	SCL Stuck condition detect on I2C active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_BUS_CLEAR_FEATURE==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_clk_sext_intr	O	SMBUS Target clock extend timeout detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_clk_mext_intr	O	SMBUS Controller clock extend timeout detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_smbus_quick_cmd_det_intr	O	SMBUS Quick Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_arp_prepare_intr	O	SMBUS ARP Prepare Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_arp_reset_intr	O	SMBUS ARP Reset Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_arp_get_udid_intr	O	SMBUS ARP Get UDID Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_smbus_arp_assign_address_intr	O	SMBUS ARP Assign Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_host_notify_intr	O	SMBUS ARP Host Notify Command detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbus_slv_rx_pec_nack_intr	O	SMBUS ARP Target Received incorrect PEC Byte and generated Nack active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_smbalert_det_intr	O	SMBUS Alert detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_smbus_det_intr	O	SMBUS Suspend detect active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_activity_intr	O	I2C activity active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_rx_done_intr	O	Receive done active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_ULTRA_FAST_MODE==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_tx_abrt_intr	O	Transmit terminate active high interrupt. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_rd_req_intr	O	Target read request interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_ULTRA_FAST_MODE==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_wr_req_intr	O	Target write request interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_MULTI_SAR_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_slv_addr_tag_intr[(IC_NUM_SARS-1):0]	O	Target Address tag interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) && (IC_MULTI_SAR_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_tx_empty_intr	O	Transmit buffer empty active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When bit 0 of the IC_ENABLE register is 0, the TX FIFO is flushed and held in reset, where it looks like it has no data within it. The ic_tx_empty_intr bit is raised when bit 0 of the IC_ENABLE register is 0, provided there is activity in the controller or target state machines. When there is no longer activity, then this interrupt bit is masked with ic_en. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_tx_over_intr	O	Transmit buffer overflow active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_rx_full_intr	O	Receive buffer full active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When bit 0 of the IC_ENABLE register is 0, the RX FIFO is flushed and held in reset the RX FIFO is not full so this ic_rx_full_intr bit is cleared once the ic_enable bit is programmed with a 0, regardless of the activity that continues. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_rx_over_intr	O	Receive buffer overflow active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_rx_under_intr	O	Receive buffer underflow active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
ic_gen_call_intr	O	General Call received active high interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_intr_n	O	Combined active low interrupt. This signal is included on the interface when the configuration parameter IC_INTR_IO is unchecked (0) to indicate that only one interrupt line appears on the I/O (as opposed to individual interrupt signals). Exists: (IC_INTR_IO==1) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_mst_on_hold_intr_n	O	Controller on hold I2C active low interrupt. This signal is included on the interface when the configuration parameters I2C_DYNAMIC_TAR_UPDATE and IC_EMPTYFIFO_HOLD_MASTER_EN are checked (1) and the configuration parameter IC_INTR_IO is unchecked (0), indicating that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (I2C_DYNAMIC_TAR_UPDATE==1 && IC_EMPTYFIFO_HOLD_MASTER_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_start_det_intr_n	O	Start condition detect on I2C active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_stop_det_intr_n	O	Stop condition detect on I2C active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_restart_det_intr_n	O	Restart condition detect on I2C active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SLV_RESTART_DET_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_scl_stuck_at_low_intr_n	O	SCL Stuck condition detect on I2C active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_BUS_CLEAR_FEATURE==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_clk_sext_intr_n	O	SMBUS Target clock extend timeout detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_smbus_clk_mext_intr_n	O	SMBUS Controller clock extend timeout detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_quick_cmd_det_intr_n	O	SMBUS ARP Quick Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_arp_prepare_intr_n	O	SMBUS ARP Prepare Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_arp_reset_intr_n	O	SMBUS ARP Reset Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_smbus_arp_get_udid_intr_n	O	SMBUS ARP Get UDID Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_arp_assign_address_intr_n	O	SMBUS ARP Assign Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_host_notify_intr_n	O	SMBUS ARP Host Notify Command detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbus_slv_rx_pec_nack_intr_n	O	SMBUS ARP Target Received incorrect PEC Byte and generated Nack active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_ARP==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_smbalert_det_intr_n	O	SMBUS Alert detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbsus_det_intr_n	O	SMBUS Suspend detect active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_activity_intr_n	O	I2C activity active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_rx_done_intr_n	O	Receive done active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_ULTRA_FAST_MODE==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_tx_abrt_intr_n	O	Transmit terminate active low interrupt. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_rd_req_intr_n	O	Target read request active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_ULTRA_FAST_MODE==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_wr_req_intr_n	O	Target write request active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_MULTI_SAR_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_slv_addr_tag_intr_n[(IC_NUM_SARS-1):0]	O	Target Address tag active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) && (IC_MULTI_SAR_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_tx_empty_intr_n	O	Transmit buffer empty active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When bit 0 of the IC_ENABLE register is 0, the TX FIFO is flushed and held in reset, where it looks like it has no data within it. The ic_tx_empty_intr_n bit is raised when bit 0 of the IC_ENABLE register is 0, provided there is activity in the controller or target state machines. When there is no longer activity, then this interrupt bit is masked with ic_en. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_tx_over_intr_n	O	Transmit buffer overflow active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_rx_full_intr_n	O	Receive buffer full active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When bit 0 of the IC_ENABLE register is 0, the RX FIFO is flushed and held in reset the RX FIFO is not full so this ic_rx_full_intr_n bit is cleared once the ic_enable bit is programmed with a 0, regardless of the activity that continues. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_rx_over_intr_n	O	Receive buffer overflow active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_rx_under_intr_n	O	Receive buffer underflow active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. When the module is disabled, this interrupt keeps its level until the controller or target state machines go into idle and bit 0 of the IC_ENABLE register is 0. When ic_en goes to 0, this interrupt is cleared. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_gen_call_intr_n	O	General Call received active low interrupt. This signal is included on the interface when the configuration IC_INTR_IO parameter is unchecked (0), which indicates that individual interrupt lines appear on the I/O. Exists: (IC_INTR_IO==0) && (IC_INTR_POL==0) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

4.2 I2C Interface (Controller/Target) Signals

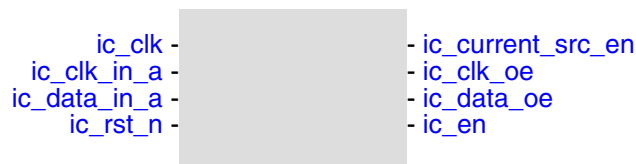


Table 4-2 I2C Interface (Controller/Target) Signals

Port Name	I/O	Description
ic_current_src_en	O	Current source pull-up. Controls the polarity of the current source pull-up on the SCLH. This pull-up is used to shorten the rise time on SCLH by activating an user-supplied external current source pull-up circuit. It is disabled after a RESTART condition and after each A/A bit when acting as the active controller. This signal enables other devices to delay the serial transfer by stretching the LOW period of the SCLH signal. The active controller re-enables its current source pull-up circuit again when all devices have released and the SCLH signal reaches high level, therefore, shortening the last part of the SCLH signal's rise time. Exists: (IC_MAX_SPEED_MODE==3) Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_clk	I	Peripheral clock. DW_apb_i2c runs on this clock and is used to clock transfers in standard, fast, and high-speed mode. Exists: Always Synchronous To: None Registered: N/A Power Domain: SINGLE_DOMAIN Active State: N/A

Port Name	I/O	Description
ic_clk_in_a	I	In (IC_ULTRA_FAST_MODE = 0) mode - Incoming I2C clock. This is the input SCL signal. Double-registered for metastability synchronization. Note:DW_apb_i2c provides filtering on the SDA (ic_data_in_a) and SCL (ic_clk_in_a) inputs, suppressing noise and signal spikes with durations less than one ic_clk period. In Ultra-Fast(IC_ULTRA_FAST_MODE = 1) mode - Incoming I2C clock. This is the input SCL signal. Double-registered for metastability synchronization. Note:DW_apb_i2c provides filtering on the SDA (ic_data_in_a) and SCL (ic_clk_in_a) inputs, suppressing noise and signal spikes with durations less than one ic_clk period. This signal is used as USCL input for target device. This signal is asynchronous to ic_clk. Exists: Always Synchronous To: Asynchronous Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_data_in_a	I	In (IC_ULTRA_FAST_MODE = 0) mode - Incoming I2C Data. It is the input SDA signal. Double-registered for metastability synchronization. Note:DW_apb_i2c provides filtering on the SDA (ic_data_in_a) and SCL (ic_clk_in_a) inputs, suppressing noise and signal spikes with durations less than one ic_clk period. In Ultra-Fast(IC_ULTRA_FAST_MODE = 1) mode - Incoming I2C Data. It is the input SDA signal. Double-registered for metastability synchronization. Note:DW_apb_i2c provides filtering on the SDA (ic_data_in_a) and SCL (ic_clk_in_a) inputs, suppressing noise and signal spikes with durations less than one ic_clk period. This signal is used as USDA input for target device. This signal is asynchronous to ic_clk. Exists: Always Synchronous To: Asynchronous Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
ic_rst_n	I	I2C reset. Used to reset flip-flops that are clocked by the ic_clk clock. Note:This signal does not reset DW_apb_i2c control, configuration, and status registers. The signal is asserted asynchronously, but is de-asserted synchronously after the rising edge of ic_clk. The synchronization must be provided external to this component. Exists: Always Synchronous To: Asynchronous Registered: N/A Power Domain: SINGLE_DOMAIN Active State: Low

Port Name	I/O	Description
ic_clk_oe	O	In (IC_ULTRA_FAST_MODE = 0) mode - Outgoing I2C clock. Open drain synchronous with ic_clk. In Ultra-Fast(IC_ULTRA_FAST_MODE = 1) mode - Outgoing I2C clock, inverted. This signal is used as USCL out from controller device. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_data_oe	O	In (IC_ULTRA_FAST_MODE = 0) mode - Outgoing I2C Data. Open Drain Synchronous to ic_clk. In Ultra-Fast(IC_ULTRA_FAST_MODE = 1) mode - Outgoing I2C Data, inverted. This signal is used as USDA out from controller device. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
ic_en	O	I2C/SMBus interface enable. Indicates whether DW_apb_i2c is enabled; this signal is set to 0 when register bit IC_ENABLE.ENABLE is set to 0 (disabled). Because DW_apb_i2c always finishes its current transfer before turning off ic_en, this signal may be used by a clock generator to control whether the DW_apb_i2c ic_clk is active or inactive. However, once the IP is re-enabled it is held into reset state. Clear TX terminate is required for the DW_apb_i2c to initiate the transfers. The DW_apb_i2c handles disable and TX terminate in similar fashion which requires a clear TX terminate after DW_apb_i2c disable as well. Exists: Always Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

4.3 APB Completer Interface Signals

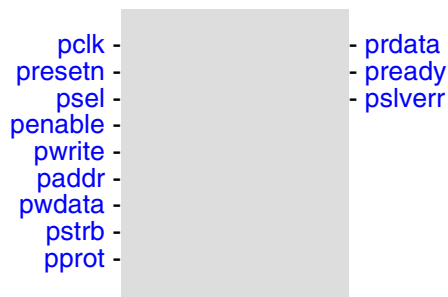


Table 4-3 APB Completer Interface Signals

Port Name	I/O	Description
pclk	I	APB clock for the bus interface unit. Exists: Always Synchronous To: None Registered: N/A Power Domain: SINGLE_DOMAIN Active State: N/A
presetn	I	An APB interface domain reset. The signal is asserted asynchronously, but is de-asserted synchronously after the rising edge of pclk. The synchronization must be provided external to this component. Exists: Always Synchronous To: Asynchronous Registered: N/A Power Domain: SINGLE_DOMAIN Active State: Low
psel	I	APB peripheral select that lasts for two pclk cycles. When asserted, indicates that the peripheral has been selected for a read/write operation. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
penable	I	APB enable control. Asserted for a single pclk cycle and used for timing read/write operations. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
pwrite	I	APB write control. When high, indicates a write access to the peripheral; when low, indicates a read access. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
paddr[IC_ADDR_SLICE_LHS:0]	I	APB address bus. Uses lower 7 bits of the address bus for register decode when IC_MULTI_SAR_EN = 0, otherwise 8 bits are used. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
pdata[(APB_DATA_WIDTH-1):0]	I	APB write data bus. Driven by the bus requester (DW_ahb to DW_apb bridge) during write cycles. Can be 8, 16, or 32 bits wide depending on APB_DATA_WIDTH parameter. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
prdata[(APB_DATA_WIDTH-1):0]	O	APB readback data. Driven by the selected peripheral during read cycles. Can be 8, 16, or 32 bits wide depending on APB_DATA_WIDTH parameter. Exists: Always Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
pready	O	The APB ready signal, used to extend the APB transfer and it is also used to indicate the end of a transaction when there is a high in the access phase of a transaction. Exists: (SLAVE_INTERFACE_TYPE>0) Synchronous To: pclk Registered: (SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1) ? Yes : No Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
pslverr	O	APB completer error response signal. The signal issues an error when some error condition occurs, as specified in databook. Exists: (SLAVE_INTERFACE_TYPE>0) Synchronous To: pclk Registered: (SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1) ? Yes : No Power Domain: SINGLE_DOMAIN Active State: High
pstrb[((APB_DATA_WIDTH/8)-1):0]	I	APB4 Write strobe bus. A high on individual bits in the pstrb bus indicate that the corresponding incoming write data byte on APB bus is to be updated in the addressed register. Exists: (SLAVE_INTERFACE_TYPE>1) Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
pprot[2:0]	I	APB4 Protection type. This signal is ignored internally in DW_apb_i2c. Exists: (SLAVE_INTERFACE_TYPE>1) Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A

4.4 DMA Interface Signals

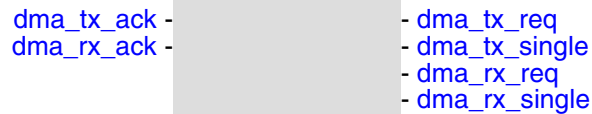


Table 4-4 DMA Interface Signals

Port Name	I/O	Description
dma_tx_ack	I	DMA Transmit Acknowledgement. Sent by the DMA controller to acknowledge the end of each APB transfer burst to the transmit FIFO. Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
dma_tx_req	O	Transmit FIFO DMA Request. Asserted when the transmit FIFO requires service from the DMA Controller; that is, the transmit FIFO is at or below the watermark level. 0: not requesting 1: requesting Software must set up the DMA controller with the number of words to be transferred when a request is made. When using the DW_ahb_dmac, this value is programmed in the DEST_MSIZE field of the CTLx register. Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_tx_single	O	DMA Transmit FIFO Single Signal. This DMA status output informs the DMA Controller that there is at least one free entry in the transmit FIFO. This output does not request a DMA transfer. 0: Transmit FIFO is full 1: Transmit FIFO is not full Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
dma_rx_ack	I	DMA Receive Acknowledgement. Sent by the DMA controller to acknowledge the end of each APB transfer burst from the receive FIFO. Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
dma_rx_req	O	Receive FIFO DMA Request. Asserted when the receive FIFO requires service from the DMA Controller; that is, the receive FIFO is at or above the watermark level. ■ 0: not requesting ■ 1: requesting Software must set up the DMA controller with the number of words to be transferred when a request is made. When using the DW_ahb_dmac, this value is programmed in the SRC_MSIZE field of the CTLx register. Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_rx_single	O	DMA Receive FIFO Single Signal. This DMA status output informs the DMA Controller that there is at least one valid data entry in the receive FIFO. This output does not request a DMA transfer. ■ 0: Receive FIFO is empty ■ 1: Receive FIFO is not empty Exists: (IC_HAS_DMA==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

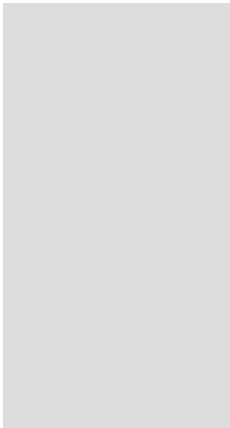
4.5 SMBus Interface Signals

ic_smbsus_in_n - ic_smbsus_out_n
ic_smbalert_in_n - ic_smbalert_oe

Table 4-5 SMBus Interface Signals

Port Name	I/O	Description
ic_smbsus_in_n	I	Incoming SMBus Suspend signal. This is the input SMBSUS signal. Double-registered for metastability synchronization. This signal is asynchronous to pclk. Exists: (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: Asynchronous Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbalert_in_n	I	Incoming SMBus Alert signal. This is the input SMBALERT signal. Double-registered for metastability synchronization. This signal is asynchronous to pclk. Exists: (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: Asynchronous Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbsus_out_n	O	Outgoing SMBus Suspend Signal. This signal is used to suspend the SMBus system, if DW_apb_i2c is used as SMBus Host. Exists: (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
ic_smbalert_oe[(IC_SMBALERT_OE_WIDTH-1):0]	O	Outgoing SMBus Alert Signal. This signal is used to intimate the Host that target wants to talk, if DW_apb_i2c is used as SMBus Target. Exists: (IC_SMBUS_SUSPEND_ALERT==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

4.6 I2C Debug Signals



- debug_s_gen
- debug_p_gen
- debug_data
- debug_addr
- debug_rd
- debug_wr
- debug_hs
- debug_master_act
- debug_slave_act
- debug_addr_10bit
- debug_mst_cstate
- debug_slv_cstate
- debug_smbalert_sarx
- debug_ic_operating_mode

Table 4-6 I2C Debug Signals

Port Name	I/O	Description
debug_s_gen	O	In the controller mode of operation, this signal is set to 1 when DW_apb_i2c is driving a START condition on the bus. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
debug_p_gen	O	In the controller mode of operation, this signal is set to 1 when DW_apb_i2c is driving a STOP condition on the bus. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
debug_data	O	In the controller or target mode of operation, this signal is set to 1 when a byte of data is actively being read or written by DW_apb_i2c. This bit remains 1 until the transaction has completed. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
debug_addr	O	In the controller or target mode of operation, this signal is set to 1 when the addressing phase is active on the I2C bus. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_rd	O	In the controller mode of operation, this signal is set to 1 whenever the controller is receiving data. This bit remains 1 until the transfer is complete or until the direction changes. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_wr	O	In the controller mode of operation, this signal is set to 1 whenever the controller is transmitting data. This bit remains 1 until the transfer is complete or the direction changes. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_hs	O	In the controller mode of operation, this signal is set to 1 when DW_apb_i2c is performing high-speed mode transfers. This bit is set after the high-speed controller code is transmitted and remains 1 until the controller leaves high-speed mode. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_master_act	O	This bit is set to 1 when the controller module is active. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Port Name	I/O	Description
debug_slave_act	O	This bit is set to 1 when the target module is active. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_addr_10bit	O	In the Target mode of operation, this signal is set if 10-bit addressing is enabled and if the target has received a matching 10-bit address with respect to IC_SAR register. This signal is not applicable in Controller Mode. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
debug_mst_cstate[4:0]	O	Controller FSM state vector. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
debug_slv_cstate[3:0]	O	Target FSM state vector. Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
debug_smbalert_sarx[6:0]	O	Target Address to be sent in response to SMBus Alert Response Address. This output is only valid when SMBus Alert is enabled in target mode. Exists: (IC_MSAR_SMBUS_ALERT_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A

Port Name	I/O	Description
debug_ic_operating_mode	O	Debug Signal. This signal indicates the operating mode of the DW_apb_i2c. ■ 0 : Controller Mode ■ 1 : Target Device Mode Exists: Always Synchronous To: ic_clk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Register Descriptions

This chapter details all possible registers in the controller. They are arranged hierarchically into maps and blocks (banks). For configurable IP titles, your actual configuration might not contain all of these registers.

Attention: For configurable IP titles, do not use this document to determine the exact attributes of your register map. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the register attributes for your actual configuration at `workspace/report/ComponentRegisters.html` or `workspace/report/ComponentRegisters.xml` after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the registers that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the Offset and Memory Access values might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Exists Expressions

These expressions indicate the combination of configuration parameters required for a register, field, or block to exist in the memory map. The expression is only valid in the local context and does not indicate the conditions for existence of the parent. For example, the expression for a bit field in a register assumes that the register exists and does not include the conditions for existence of the register.

Offset

The term *Offset* is synonymous with *Address*.

Memory Access Attributes

The Memory Access attribute is defined as `<ReadBehavior>/<WriteBehavior>` which are defined in the following table.

Table 5-1 Possible Read and Write Behaviors

Read (or Write) Behavior	Description
RC	A read clears this register field.
RS	A read sets this register field.
RM	A read modifies the contents of this register field in a manner not described by any of the above.
Wo	You can only write once to this register field.
W1C	A write of 1 clears this register field.
W1S	A write of 1 sets this register field.
W1T	A write of 1 toggles this register field.
W0C	A write of 0 clears this register field.
W0S	A write of 0 sets this register field.
W0T	A write of 0 toggles this register field.
WC	Any write clears this register field.
WS	Any write sets this register field.
WM	A write modifies this register field in a manner not described by any of the above.
no Read Behavior attribute	You cannot read this register. It is Write-Only.
no Write Behavior attribute	You cannot write to this register. It is Read-Only.

Table 5-2 Memory Access Examples

Memory Access	Description
R	Read-only register field.
W	Write-only register field.
R/W	Read/write register field.
R/W1C	You can read this register field. Writing 1 clears it.
RC/W1C	Reading this register field clears it. Writing 1 clears it.
R/Wo	You can read this register field. You can only write to it once.

Special Optional Attributes

Some register fields might use the following optional attributes.

Table 5-3 Optional Attributes

Attribute	Description
Volatile	As defined by the IP-XACT specification. If true, indicates in the case of a write followed by read, or in the case of two consecutive reads, there is no guarantee as to what is returned by the read on the second transaction or that this return value is consistent with the write or read of the first transaction. The element implies there is some additional mechanism by which this field can acquire new values other than by reads/writes/resets and other access methods known to IP-XACT. For example, when the controller updates the register field contents.
Testable	As defined by the IP-XACT specification. Possible values are unconstrained, untestable, readOnly, writeAsRead, restore. Untestable means that this field is untestable by a simple automated register test. For example, the read-write access of the register is controlled by a pin or another register. readOnly means that you should not write to this register; only read from it. This might apply for a register that modifies the contents of another register.
Reset Mask	As defined by the IP-XACT specification. Indicates that this register field has an unknown reset value. For example, the reset value is set by another register or an input pin; or the register is implemented using RAM.
* Varies	Indicates that the memory access (or reset) attribute (read, write behavior) is not fixed. For example, the read-write access of the register is controlled by a pin or another register. Or when the access depends on some configuration parameter; in this case the post-configuration report in coreConsultant gives the actual access value.

Register definitions for each component memory map.

Table 5-4 Registers for the DW_apb_i2c_mem_map Memory Map

Register	Offset	Description
DW_apb_i2c address block DW_apb_i2c_addr_block1 Exists: Always		
"IC_CON" on page 205	0x0	I2C Control Register
"IC_TAR" on page 214	0x4	I2C Target Address Register
"IC_SAR" on page 217	0x8	I2C Target Device Address Register
"IC_HS_MADDR" on page 218	0xc	I2C High Speed Controller Mode Code Address Register
"IC_DATA_CMD" on page 219	0x10	I2C Rx/Tx Data Buffer and Command Register
"IC_SS_SCL_HCNT" on page 223	0x14	Standard Speed I2C Clock SCL High Count Register
"IC_UFM_SCL_HCNT" on page 225	0x14	Ultra-Fast Speed I2C Clock SCL High Count Register
"IC_SS_SCL_LCNT" on page 227	0x18	Standard Speed I2C Clock SCL Low Count Register

Register	Offset	Description
"IC_UFM_SCL_LCNT" on page 229	0x18	Ultra-Fast Speed I2C Clock SCL Low Count Register
"IC_FS_SCL_HCNT" on page 231	0x1c	Fast Mode or Fast Mode Plus I2C Clock SCL High Count Register
"IC_UFM_TBUF_CNT" on page 233	0x1c	Ultra-Fast Speed mode TBuf Idle Count Register
"IC_FS_SCL_LCNT" on page 234	0x20	Fast Mode or Fast Mode Plus I2C Clock SCL Low Count Register
"IC_HS_SCL_HCNT" on page 236	0x24	High Speed I2C Clock SCL High Count Register
"IC_HS_SCL_LCNT" on page 238	0x28	High Speed I2C Clock SCL Low Count Register
"IC_INTR_STAT" on page 240	0x2c	I2C Interrupt Status Register
"IC_INTR_MASK" on page 245	0x30	I2C Interrupt Mask Register
"IC_RAW_INTR_STAT" on page 250	0x34	I2C Raw Interrupt Status Register
"IC_RX_TL" on page 259	0x38	I2C Receive FIFO Threshold Register
"IC_TX_TL" on page 260	0x3c	I2C Transmit FIFO Threshold Register
"IC_CLR_INTR" on page 261	0x40	Clear Combined and Individual Interrupt Register
"IC_CLR_RX_UNDER" on page 262	0x44	Clear RX_UNDER Interrupt Register
"IC_CLR_RX_OVER" on page 263	0x48	Clear RX_OVER Interrupt Register
"IC_CLR_TX_OVER" on page 264	0x4c	Clear TX_OVER Interrupt Register
"IC_CLR_RD_REQ" on page 265	0x50	Clear RD_REQ Interrupt Register
"IC_CLR_TX_ABRT" on page 266	0x54	Clear TX_ABRT Interrupt Register
"IC_CLR_RX_DONE" on page 267	0x58	Clear RX_DONE Interrupt Register
"IC_CLR_ACTIVITY" on page 268	0x5c	Clear ACTIVITY Interrupt Register
"IC_CLR_STOP_DET" on page 269	0x60	Clear STOP_DET Interrupt Register
"IC_CLR_START_DET" on page 270	0x64	Clear START_DET Interrupt Register
"IC_CLR_GEN_CALL" on page 271	0x68	Clear GEN_CALL Interrupt Register
"IC_ENABLE" on page 272	0x6c	I2C ENABLE Register
"IC_STATUS" on page 279	0x70	I2C STATUS Register
"IC_TXFLR" on page 287	0x74	I2C Transmit FIFO Level Register
"IC_RXFLR" on page 288	0x78	I2C Receive FIFO Level Register
"IC_SDA_HOLD" on page 289	0x7c	I2C SDA Hold Time Length Register
"IC_TX_ABRT_SOURCE" on page 291	0x80	I2C Transmit Terminate Source Register

Register	Offset	Description
“IC_SLV_DATA_NACK_ONLY” on page 299	0x84	Generate Target Data NACK Register
“IC_DMA_CR” on page 301	0x88	DMA Control Register
“IC_DMA_TDLR” on page 303	0x8c	DMA Transmit Data Level Register
“IC_DMA_RDLR” on page 304	0x90	DMA Receive Data Level Register
“IC_SDA_SETUP” on page 305	0x94	I2C SDA Setup Register
“IC_ACK_GENERAL_CALL” on page 306	0x98	I2C ACK General Call Register
“IC_ENABLE_STATUS” on page 307	0x9c	I2C Enable Status Register
“IC_FS_SPKLEN” on page 310	0xa0	I2C SS, FS or FM+ spike suppression limit
“IC_UFM_SPKLEN” on page 311	0xa0	I2C Ultra-Fast mode spike suppression limit
“IC_HS_SPKLEN” on page 312	0xa4	I2C HS spike suppression limit register
“IC_CLR_RESTART_DET” on page 313	0xa8	Clear RESTART_DET Interrupt Register
“IC_SCL_STUCK_AT_LOW_TIMEOUT” on page 314	0xac	I2C SCL Stuck at Low Timeout register
“IC_SDA_STUCK_AT_LOW_TIMEOUT” on page 315	0xb0	I2C SDA Stuck at Low Timeout register
“IC_CLR_SCL_STUCK_DET” on page 316	0xb4	Clear SCL Stuck at Low Detect interrupt Register
“IC_DEVICE_ID” on page 317	0xb8	I2C Device-Id register
“IC_SMBUS_CLK_LOW_SEXT” on page 318	0xbc	SMBus Target Clock Extend Timeout register
“IC_SMBUS_CLK_LOW_MEXT” on page 319	0xc0	SMBus Controller Clock Extend Timeout register
“IC_SMBUS_THIGH_MAX_IDLE_COUNT” on page 320	0xc4	SMBus Controller THigh MAX Bus-idle count Register
“IC_SMBUS_INTR_STAT” on page 322	0xc8	SMBus Interrupt Status Register
“IC_SMBUS_INTR_MASK” on page 326	0xcc	SMBus Interrupt Mask Register
“IC_SMBUS_RAW_INTR_STAT” on page 330	0xd0	SMBus Raw Interrupt Status Register
“IC_CLR_SMBUS_INTR” on page 334	0xd4	Clear SMBus Interrupt Register
“IC_OPTIONAL_SAR” on page 336	0xd8	I2C Optional Target Address Register
“IC_SMBUS_UDID_LSB” on page 337	0xdc	SMBUS ARP UDID LSB Register
“IC_SMBUS_UDID_WORD0” on page 338	0xdc	SMBUS ARP UDID WORD0 Register
“IC_SMBUS_UDID_WORD1” on page 339	0xe0	SMBUS ARP UDID WORD1 Register
“IC_SMBUS_UDID_WORD2” on page 340	0xe4	SMBUS ARP UDID WORD2 Register

Register	Offset	Description
“IC_SMBUS_UDID_WORD3” on page 341	0xe8	SMBUS ARP UDID WORD3 Register
“REG_TIMEOUT_RST” on page 342	0xf0	Register timeout counter reset value
“IC_COMP_PARAM_1” on page 343	0xf4	Component Parameter Register 1
“IC_COMP_VERSION” on page 346	0xf8	I2C Component Version Register
“IC_COMP_TYPE” on page 347	0xfc	I2C Component Type Register
“IC_SAR2” on page 348	0x100	I2C Target Device Address Register 2
“IC_SAR3” on page 349	0x104	I2C Target Device Address Register 3
“IC_SAR4” on page 350	0x108	I2C Target Device Address Register 4
“IC_CLR_WR_REQ” on page 351	0x11c	Clear WR_REQ Interrupt Register
“IC_CLR_SLV_ADDR_TAG” on page 352	0x120	Clear CLR_SLV_ADDR_TAG Interrupt Register
“IC_SAR2_SMBUS_UDID_WORD0” on page 353	0x124	IC_SAR2 SMBUS ARP UDID WORD0 Register
“IC_SAR2_SMBUS_UDID_WORD1” on page 354	0x128	IC_SAR2 SMBUS ARP UDID WORD1 Register
“IC_SAR2_SMBUS_UDID_WORD2” on page 355	0x12c	IC_SAR2 SMBUS ARP UDID WORD2 Register
“IC_SAR2_SMBUS_UDID_WORD3” on page 356	0x130	IC_SAR2 SMBUS ARP UDID WORD3 Register
“IC_SAR3_SMBUS_UDID_WORD0” on page 357	0x134	IC_SAR3 SMBUS ARP UDID WORD0 Register
“IC_SAR3_SMBUS_UDID_WORD1” on page 358	0x138	IC_SAR3 SMBUS ARP UDID WORD1 Register
“IC_SAR3_SMBUS_UDID_WORD2” on page 359	0x13c	IC_SAR3 SMBUS ARP UDID WORD2 Register
“IC_SAR3_SMBUS_UDID_WORD3” on page 360	0x140	IC_SAR3 SMBUS ARP UDID WORD3 Register
“IC_SAR4_SMBUS_UDID_WORD0” on page 361	0x144	IC_SAR4 SMBUS ARP UDID WORD0 Register
“IC_SAR4_SMBUS_UDID_WORD1” on page 362	0x148	IC_SAR4 SMBUS ARP UDID WORD1 Register
“IC_SAR4_SMBUS_UDID_WORD2” on page 363	0x14c	IC_SAR4 SMBUS ARP UDID WORD2 Register
“IC_SAR4_SMBUS_UDID_WORD3” on page 364	0x150	IC_SAR4 SMBUS ARP UDID WORD3 Register

Register	Offset	Description
“IC_SMBUS_ARP_CTRL” on page 365	0x154	I2C SMBus ARP Control Register

5.1 DW_apb_i2c_addr_block1 Registers

5.1.1 IC_CON

- **Name:** I2C Control Register

- **Description:** I2C Control Register.

This register can be written only when the DW_apb_i2c is disabled, which corresponds to the IC_EN-ABLE[0] register being set to 0. Writes at other times have no effect.

Read/Write Access:

- If configuration parameter I2C_DYNAMIC_TAR_UPDATE=1, bit 4 is read only.
- If configuration parameter IC_RX_FULL_HLD_BUS_EN =0, bit 9 is read only.
- If configuration parameter IC_STOP_DET_IF_MASTER_ACTIVE =0, bit 10 is read only.
- If configuration parameter IC_BUS_CLEAR_FEATURE=0, bit 11 is read only
- If configuration parameter IC_OPTIONAL_SAR=0, bit 16 is read only
- If configuration parameter IC_SMBUS=0, bit 17 is read only
- If configuration parameter IC_SMBUS_ARP=0, bits 18 and 19 are read only.
- **Size:** 32 bits
- **Offset:** 0x0
- **Exists:** Always

RSVD_IC_CON_2	31:26
IC_SAR4_SMBUS_ARP_EN	25
IC_SAR3_SMBUS_ARP_EN	24
IC_SAR2_SMBUS_ARP_EN	23
SMBUS_PERSISTENT_SLV_ADDR4_EN	22
SMBUS_PERSISTENT_SLV_ADDR3_EN	21
SMBUS_PERSISTENT_SLV_ADDR2_EN	20
SMBUS_PERSISTENT_SLV_ADDR_EN	19
SMBUS_ARP_EN	18
SMBUS_SLAVE_QUICK_EN	17
OPTIONAL_SAR_CTRL	16
RSVD_IC_CON_1	15:12
BUS_CLEAR_FEATURE_CTRL	11
STOP_DET_IF_MASTER_ACTIVE	10
RX_FIFO_FULL_HLD_CTRL	9
TX_EMPTY_CTRL	8
STOP_DET_IFADDRESSED	7
IC_SLAVE_DISABLE	6
IC_RESTART_EN	5
IC_10BITADDR_MASTER	4
IC_10BITADDR_SLAVE	3
SPEED	2:1
MASTER_MODE	0

Table 5-5 Fields for Register: IC_CON

Bits	Name	Memory Access	Description
31:26	RSVD_IC_CON_2	R	IC_CON_2 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
25	IC_SAR4_SMBUS_ARP_EN	R/W	This bit controls whether DW_apb_i2c should enable Address Resolution Logic in SMBus Mode for IC_SAR4 target device Address. The Target mode will decode the Address Resolution Protocol commands and respond to it. The DW_apb_i2c target also includes the generation/validity of PEC byte for Address Resolution Protocol commands. This bit is applicable only in Target mode. Values: ■ 0x1 (ENABLED): SMBus ARP control is enabled. ■ 0x0 (DISABLED): SMBus ARP control is disabled. Value After Reset: 0x0 Exists: IC_NUM_SARS>=4 && IC_MULTI_SAR_EN==1 && IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==0
24	IC_SAR3_SMBUS_ARP_EN	R/W	This bit controls whether DW_apb_i2c should enable Address Resolution Logic in SMBus Mode for IC_SAR3 target device Address. The Target mode will decode the Address Resolution Protocol commands and respond to it. The DW_apb_i2c target also includes the generation/validity of PEC byte for Address Resolution Protocol commands. This bit is applicable only in Target mode. Values: ■ 0x1 (ENABLED): SMBus ARP control is enabled. ■ 0x0 (DISABLED): SMBus ARP control is disabled. Value After Reset: 0x0 Exists: IC_NUM_SARS>=3 && IC_MULTI_SAR_EN==1 && IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==0
23	IC_SAR2_SMBUS_ARP_EN	R/W	This bit controls whether DW_apb_i2c should enable Address Resolution Logic in SMBus Mode for IC_SAR2 target device Address. The Target mode will decode the Address Resolution Protocol commands and respond to it. The DW_apb_i2c target also includes the generation/validity of PEC byte for Address Resolution Protocol commands. This bit is applicable only in Target mode. Values: ■ 0x1 (ENABLED): SMBus ARP control is enabled. ■ 0x0 (DISABLED): SMBus ARP control is disabled. Value After Reset: 0x0 Exists: IC_MULTI_SAR_EN==1 && IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==0

Bits	Name	Memory Access	Description
22	SMBUS_PERSISTENT_SLV_ADDR4_EN	R/W	The bit controls to enable DW_apb_i2c target with target device address 4 (IC_SAR4) as persistent or non persistent target. If the target is non-PSA then DW_apb_i2c target device clears the Address valid flag for both General and Directed Reset ARP command else the address valid flag will always set to 1. This bit is applicable only in Target mode. Values: ■ 0x1 (ENABLED): SMBus Persistent Target address (IC_SAR4) control is enabled. ■ 0x0 (DISABLED): SMBus Persistent Target address (IC_SAR4) control is disabled. Value After Reset: IC_PERSISTANT_SLV_ADDR4_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4)
21	SMBUS_PERSISTENT_SLV_ADDR3_EN	R/W	The bit controls to enable DW_apb_i2c target with target device address 3 (IC_SAR3) as persistent or non persistent target. If the target is non-PSA then DW_apb_i2c target device clears the Address valid flag for both General and Directed Reset ARP command else the address valid flag will always set to 1. This bit is applicable only in Target mode. Values: ■ 0x1 (ENABLED): SMBus Persistent Target address (IC_SAR3) control is enabled. ■ 0x0 (DISABLED): SMBus Persistent Target address (IC_SAR3) control is disabled. Value After Reset: IC_PERSISTANT_SLV_ADDR3_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3)

Bits	Name	Memory Access	Description
20	SMBUS_PERSISTENT_SLV_ADDR2_EN	R/W	The bit controls to enable DW_apb_i2c target with target device address 2 (IC_SAR2) as persistent or non persistent target. If the target is non-PSA then DW_apb_i2c target device clears the Address valid flag for both General and Directed Reset ARP command else the address valid flag will always set to 1. This bit is applicable only in Target mode. Values: 0x1 (ENABLED): SMBus Persistent Target address (IC_SAR2) control is enabled. 0x0 (DISABLED): SMBus Persistent Target address (IC_SAR2) control is disabled. Value After Reset: IC_PERSISTANT_SLV_ADDR2_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2)
19	SMBUS_PERSISTENT_SLV_ADDR_EN	R/W	The bit controls to enable DW_apb_i2c target as persistent or non persistent target. If the target is non-PSA then DW_apb_i2c target device clears the Address valid flag for both General and Directed Reset ARP command else the address valid flag will always set to 1. This bit is applicable only in Target mode. Values: 0x1 (ENABLED): SMBus Persistent Target address control is enabled. 0x0 (DISABLED): SMBus Persistent Target address control is disabled. Value After Reset: IC_PERSISTANT_SLV_ADDR_DEFAULT Exists: IC_SMBUS_ARP==1
18	SMBUS_ARP_EN	R/W	This bit controls whether DW_apb_i2c should enable Address Resolution Logic in SMBus Mode. The Target mode will decode the Address Resolution Protocol commands and respond to it. The DW_apb_i2c target also includes the generation/validity of PEC byte for Address Resolution Protocol commands. This bit is applicable only in Target mode. Values: 0x1 (ENABLED): SMBus ARP control is enabled. 0x0 (DISABLED): SMBus ARP control is disabled. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 && IC_SMBUS_FD_FND_SUPPORT_EN==0

Bits	Name	Memory Access	Description
17	SMBUS_SLAVE_QUICK_EN	R/W	If this bit is set to 1, DW_apb_i2c target only receives Quick commands in SMBus Mode. If this bit is set to 0, DW_apb_i2c target receives all bus protocols but not Quick commands. This bit is applicable only in target mode. Values: ■ 0x1 (ENABLED): SMBus Target is enabled to receive Quick command. ■ 0x0 (DISABLED): SMBus Target is disabled to receive Quick command. Value After Reset: 0x0 Exists: IC_SMBUS==1
16	OPTIONAL_SAR_CTRL	R/W	Enables the usage of IC_OPTIONAL_SAR register. If IC_OPTIONAL_SAR = 1, IC_OPTIONAL_SAR value is used as additional target address. User must program a valid address in IC_OPTIONAL_SAR before writing 1 to this field. If IC_OPTIONAL_SAR = 0, IC_OPTIONAL_SAR value is not used as additional target address. In this mode only one I2C target address is used. Values: ■ 0x1 (ENABLED): Optional SAR Address Register is enabled. ■ 0x0 (DISABLED): Optional SAR Address Register is disabled. Value After Reset: 0x0 Exists: IC_OPTIONAL_SAR==1
15:12	RSVD_IC_CON_1	R	IC_CON_1 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
11	BUS_CLEAR_FEATURE_CTRL	R/W	This bit controls whether the DW_apb_i2c enables the bus clear feature. For more information, refer to sections "Bus Clear Feature" and "SMBus/PMBus". Values: ■ 0x1 (ENABLED): Bus Clear Feature is enabled. ■ 0x0 (DISABLED): Bus Clear Feature is disabled. Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE==1

Bits	Name	Memory Access	Description
10	STOP_DET_IF_MASTER_ACTIVE	* Varies	In Controller mode: ■ 1'b1: issues the STOP_DET interrupt only when controller is active. ■ 1'b0: issues the STOP_DET irrespective of whether controller is active or not. Values: ■ 0x1 (ENABLED): Controller issues the STOP_DET interrupt only when controller is active ■ 0x0 (DISABLED): Controller issues the STOP_DET interrupt irrespective of whether controller is active or not Value After Reset: 0x0 Exists: Always Memory Access: "(IC_STOP_DET_IF_MASTER_ACTIVE==1) ? \"read-write\" : \"read-only\""
9	RX_FIFO_FULL_HLD_CTRL	* Varies	This bit controls whether DW_apb_i2c should hold the bus when the Rx FIFO is physically full to its RX_BUFFER_DEPTH, as described in the IC_RX_FULL_HLD_BUS_EN parameter. Values: ■ 0x1 (ENABLED): Hold bus when RX_FIFO is full ■ 0x0 (DISABLED): Overflow when RX_FIFO is full Value After Reset: 0x0 Exists: Always Memory Access: "(IC_RX_FULL_HLD_BUS_EN==1) ? \"read-write\" : \"read-only\""
8	TX_EMPTY_CTRL	R/W	This bit controls the generation of the TX_EMPTY interrupt, as described in the IC_RAW_INTR_STAT register. Values: ■ 0x1 (ENABLED): Controlled generation of TX_EMPTY interrupt ■ 0x0 (DISABLED): Default behaviour of TX_EMPTY interrupt Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
7	STOP_DET_IFADDRESSED	R/W	In target mode: ■ 1'b1: issues the STOP_DET interrupt only when it is addressed. ■ 1'b0: issues the STOP_DET irrespective of whether it's addressed or not. Note: During a general call address, this target does not issue the STOP_DET interrupt if STOP_DET_IF_ADDRESSED = 1'b1, even if the target responds to the general call address by generating ACK. The STOP_DET interrupt is generated only when the transmitted address matches the target address (SAR). Values: ■ 0x1 (ENABLED): target issues STOP_DET intr only if addressed ■ 0x0 (DISABLED): target issues STOP_DET intr always Value After Reset: 0x0 Exists: Always
6	IC_SLAVE_DISABLE	R/W	This bit controls whether I2C has its target disabled, which means once the presetrn signal is applied, then this bit takes on the value of the configuration parameter IC_SLAVE_DISABLE. You have the choice of having the target enabled or disabled after reset is applied, which means software does not have to configure the target. If this bit is set (target is disabled), DW_apb_i2c functions only as a controller and does not perform any action that requires a target. Note: Software should ensure that if this bit is written with 0, then bit 0 should also be written with a 0. Values: ■ 0x1 (SLAVE_DISABLED): Target mode is disabled ■ 0x0 (SLAVE_ENABLED): Target mode is enabled Value After Reset: IC_SLAVE_DISABLE Exists: Always

Bits	Name	Memory Access	Description
5	IC_RESTART_EN	R/W	Determines whether RESTART conditions may be sent when acting as a controller. Some older targets do not support handling RESTART conditions; however, RESTART conditions are used in several DW_apb_i2c operations. When RESTART is disabled, the controller is prohibited from performing the following functions: ■ Sending a START BYTE ■ Performing any high-speed mode operation ■ High-speed mode operation ■ Performing direction changes in combined format mode ■ Performing a read operation with a 10-bit address By replacing RESTART condition followed by a STOP and a subsequent START condition, split operations are broken down into multiple DW_apb_i2c transfers. If the above operations are performed, it will result in setting bit 6 (TX_ABRT) of the IC_RAW_INTR_STAT register. Values: ■ 0x1 (ENABLED): Controller restart enabled ■ 0x0 (DISABLED): Controller restart disabled Value After Reset: IC_RESTART_EN Exists: Always
4	IC_10BITADDR_MASTER	R/W	If the I2C_DYNAMIC_TAR_UPDATE configuration parameter is set to 'No' (0), this bit is named IC_10BITADDR_MASTER and controls whether the DW_apb_i2c starts its transfers in 7- or 10-bit addressing mode when acting as a controller. If I2C_DYNAMIC_TAR_UPDATE is set to 'Yes' (1), the function of this bit is handled by bit 12 of IC_TAR register, and becomes a read-only copy called IC_10BITADDR_MASTER_rd_only. In SMBus mode only 7-bit addressing mode is supported. Values: ■ 0x1 (ADDR_10BITS): Controller 10-bit addressing mode ■ 0x0 (ADDR_7BITS): Controller 7-bit addressing mode Value After Reset: IC_10BITADDR_MASTER Exists: I2C_DYNAMIC_TAR_UPDATE == 0

Bits	Name	Memory Access	Description
3	IC_10BITADDR_SLAVE	R/W	When acting as a target, this bit controls whether the DW_apb_i2c responds to 7- or 10-bit addresses. In SMBus mode only 7-bit addressing mode is supported. 0: 7-bit addressing. The DW_apb_i2c ignores transactions that involve 10-bit addressing; for 7-bit addressing, only the lower 7 bits of the IC_SAR register are compared. 1: 10-bit addressing. The DW_apb_i2c responds to only 10-bit addressing transfers that match the full 10 bits of the IC_SAR register. Values: 0x1 (ADDR_10BITS): Target 10Bit addressing 0x0 (ADDR_7BITS): Target 7Bit addressing Value After Reset: IC_10BITADDR_SLAVE Exists: Always
2:1	SPEED	R/W	These bits control at which speed the DW_apb_i2c operates; its setting is relevant only if one is operating the DW_apb_i2c in controller mode. Hardware protects against illegal values being programmed by software. These bits must be programmed appropriately for target mode also, as it is used to capture correct value of spike filter as per the speed mode. This register should be programmed only with a value in the range of 1 to IC_MAX_SPEED_MODE; otherwise, hardware updates this register with the value of IC_MAX_SPEED_MODE. 1: standard mode (100 kbit/s) 2: fast mode (<=400 kbit/s) or fast mode plus (<=1000Kbit/s) 3: high speed mode (3.4 Mbit/s) Note: This field is not applicable when IC_ULTRA_FAST_MODE=1 Values: 0x1 (STANDARD): Standard Speed mode of operation 0x2 (FAST): Fast or Fast Plus mode of operation 0x3 (HIGH): High Speed mode of operation Value After Reset: IC_MAX_SPEED_MODE Exists: Always
0	MASTER_MODE	R/W	This bit controls whether the DW_apb_i2c controller is enabled. Note: Software should ensure that if this bit is written with '1' then bit 6 should also be written with a '1'. Values: 0x1 (ENABLED): Controller mode is enabled 0x0 (DISABLED): Controller mode is disabled Value After Reset: IC_MASTER_MODE Exists: Always

5.1.2 IC_TAR

- **Name:** I2C Target Address Register

- **Description:** I2C Target Address Register

If the configuration parameter `I2C_DYNAMIC_TAR_UPDATE` is set to 'No' (0), this register is 12 bits wide, and bits 31:12 are reserved. This register can be written to only when `IC_ENABLE[0]` is set to 0. However, if `I2C_DYNAMIC_TAR_UPDATE` = 1, then the register becomes 13 bits wide. In this case, writes to `IC_TAR` succeed when one of the following conditions are true:

- `DW_apb_i2c` is NOT enabled (`IC_ENABLE[0]` is set to 0); or
- `DW_apb_i2c` is enabled (`IC_ENABLE[0]=1`); AND `DW_apb_i2c` is NOT engaged in any Controller (tx, rx) operation (`IC_STATUS[5]=0`); AND `DW_apb_i2c` is enabled to operate in Controller mode (`IC_CON[0]=1`); AND there are NO entries in the TX FIFO (`IC_STATUS[2]=1`)

You can change the TAR address dynamically without losing the bus, only if the following conditions are met.

- `DW_apb_i2c` is enabled (`IC_ENABLE[0]=1`); AND `IC_EMPTYFIFO_HOLD_MASTER_EN` configuration parameter is set to 1; AND `DW_apb_i2c` is enabled to operate in Controller mode (`IC_CON[0]=1`); AND there are NO entries in the Tx FIFO and the controller is in HOLD state (`IC_INTR_STAT[13]=1`).

Note: If the software or application is aware that the `DW_apb_i2c` is not using the TAR address for the pending commands in the Tx FIFO, then it is possible to update the TAR address even while the Tx FIFO has entries (`IC_STATUS[2] = 0`).

- It is not necessary to perform any write to this register if `DW_apb_i2c` is enabled as an I2C target only.

- **Size:** 32 bits

- **Offset:** 0x4

- **Exists:** Always

RSVD_IC_TAR_2	31:17
SMBUS_QUICK_CMD	16
RSVD_IC_TAR_1	15:14
DEVICE_ID	13
IC_10BITADDR_MASTER	12
SPECIAL	11
GC_OR_START	10
IC_TAR	9:0

Table 5-6 Fields for Register: IC_TAR

Bits	Name	Memory Access	Description
31:17	RSVD_IC_TAR_2	R	IC_TAR_2 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
16	SMBUS_QUICK_CMD	R/W	If bit 11 (SPECIAL) is set to 1, then this bit indicates whether a Quick command is to be performed by the DW_apb_i2c. Values: 0x1 (ENABLED): Enables programming of QUICK-CMD transmission 0x0 (DISABLED): Disables programming of QUICK-CMD transmission Value After Reset: 0x0 Exists: IC_SMBUS == 1
15:14	RSVD_IC_TAR_1	R	IC_TAR_1 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
13	DEVICE_ID	R/W	If bit 11 (SPECIAL) is set to 1, then this bit indicates whether a Device-ID of a particular target mentioned in IC_TAR[9:0] is to be performed by the DW_apb_i2c Controller. 0: Device-ID is not performed and checks ic_tar[10] to perform either general call or START byte command 1: Device-ID transfer is performed and bytes based on the number of read commands in the Tx-FIFO are received from the target and put in the Rx-FIFO. Values: 0x1 (ENABLED): Enables programming of DEVICE-ID transmission 0x0 (DISABLED): Disables programming of DEVICE-ID transmission Value After Reset: 0x0 Exists: IC_DEVICE_ID == 1
12	IC_10BITADDR_MASTER	R/W	This bit controls whether the DW_apb_i2c starts its transfers in 7- or 10-bit addressing mode when acting as a controller. In SMBus mode only 7-bit addressing mode is supported. Values: 0x1 (ADDR_10BITS): Address 10Bit transmission format 0x0 (ADDR_7BITS): Address 7Bit transmission format Value After Reset: IC_10BITADDR_MASTER Exists: I2C_DYNAMIC_TAR_UPDATE

Bits	Name	Memory Access	Description
11	SPECIAL	R/W	This bit indicates whether software performs a Device-ID or General Call or START BYTE command. 0: ignore bit 10 GC_OR_START and use IC_TAR normally 1: perform special I2C command as specified in Device_ID or GC_OR_START bit Values: 0x1 (ENABLED): Enables programming of GENERAL_CALL or START_BYTE transmission 0x0 (DISABLED): Disables programming of GENERAL_CALL or START_BYTE transmission Value After Reset: 0x0 Exists: Always
10	GC_OR_START	R/W	If bit 11 (SPECIAL) is set to 1 and bit 13(Device-ID) is set to 0, then this bit indicates whether a General Call or START byte command is to be performed by the DW_apb_i2c. 0: General Call Address - after issuing a General Call, only writes may be performed. Attempting to issue a read command results in setting bit 6 (TX_ABRT) of the IC_RAW_INTR_STAT register. The DW_apb_i2c remains in General Call mode until the SPECIAL bit value (bit 11) is cleared. 1: START BYTE Values: 0x1 (START_BYTE): START byte transmission 0x0 (GENERAL_CALL): GENERAL_CALL byte transmission Value After Reset: 0x0 Exists: Always
9:0	IC_TAR	R/W	This is the target address for any controller transaction. When transmitting a General Call, these bits are ignored. To generate a START BYTE, the CPU needs to write only once into these bits. If the IC_TAR and IC_SAR are the same, a controller cannot transmit to itself; it can transmit to only a target. Value After Reset: IC_DEFAULT_TAR_SLAVE_ADDR Exists: Always

5.1.3 IC_SAR

- **Name:** I2C Target Device Address Register
- **Description:** I2C Target Device Address Register
- **Size:** 32 bits
- **Offset:** 0x8
- **Exists:** Always

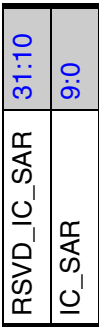


Table 5-7 Fields for Register: IC_SAR

Bits	Name	Memory Access	Description
31:10	RSVD_IC_SAR	R	IC_SAR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
9:0	IC_SAR	R/W	The IC_SAR holds the target device address when the I2C is operating as a target. For 7-bit addressing or SMBus (IC_SMBUS=1), only IC_SAR[6:0] is used. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Note: The default values cannot be any of the reserved address locations: that is, 0x00 to 0x07, or 0x78 to 0x7f. The correct operation of the device is not guaranteed if you program the IC_SAR or IC_TAR to a reserved value. Refer to Table "I2C/SMBus Definition of Bits in First Byte" for a complete list of these reserved values. Value After Reset: IC_DEFAULT_SLAVE_ADDR Exists: Always Volatile: true

5.1.4 IC_HS_MADDR

- **Name:** I2C High Speed Controller Mode Code Address Register
- **Description:** I2C High Speed Controller Mode Code Address Register
- **Size:** 32 bits
- **Offset:** 0xc
- **Exists:** IC_MAX_SPEED_MODE==3

31:3	RSVD_IC_HS_MAR
2:0	IC_HS_MAR

Table 5-8 Fields for Register: IC_HS_MADDR

Bits	Name	Memory Access	Description
31:3	RSVD_IC_HS_MAR	R	IC_HS_MAR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
2:0	IC_HS_MAR	R/W	This bit field holds the value of the I2C HS mode controller code. HS-mode controller codes are reserved 8-bit codes (00001xxx) that are not used for target addressing or other purposes. Each controller has its unique controller code; up to eight high-speed mode controllers can be present on the same I2C bus system. Valid values are from 0 to 7. This register goes away and becomes read-only returning 0's if the IC_MAX_SPEED_MODE configuration parameter is set to either Standard (1) or Fast (2). This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Value After Reset: IC_HS_MASTER_CODE Exists: Always

5.1.5 IC_DATA_CMD

- **Name:** I2C Rx/Tx Data Buffer and Command Register
- **Description:** I2C Rx/Tx Data Buffer and Command Register; this is the register the CPU writes to when filling the TX FIFO and the CPU reads from when retrieving bytes from RX FIFO.

Active bits of this registers are as follows:

Write:

- 10:0 bits when IC_EMPTYFIFO_HOLD_MASTER_EN=1
- 8:0 bits when IC_EMPTYFIFO_HOLD_MASTER_EN=0

Read:

- 11:0 bits when IC_FIRST_DATA_BYTE_STATUS = 1
- 7:0 bits when IC_FIRST_DATA_BYTE_STATUS = 0

Note: In order for the DW_apb_i2c to continue acknowledging reads, a read command should be written for every byte that is to be received; otherwise the DW_apb_i2c will stop acknowledging.

- **Size:** 32 bits
- **Offset:** 0x10
- **Exists:** Always

31:12	11	10	9	8	7:0
RSVD_IC_DATA_CMD	FIRST_DATA_BYTE	RESTART	STOP	CMD	DAT

Table 5-9 Fields for Register: IC_DATA_CMD

Bits	Name	Memory Access	Description
31:12	RSVD_IC_DATA_CMD	R	IC_DATA_CMD Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
11	FIRST_DATA_BYTE	R	Indicates the first data byte received after the address phase for receive transfer in Controller receiver or Target receiver mode. Note: In case of APB_DATA_WIDTH=8, ■ The user has to perform two APB Reads to IC_DATA_CMD in order to get status on 11 bit. ■ In order to read the 11 bit, the user has to perform the first data byte read [7:0] (offset 0x10) and then perform the second read[15:8](offset 0x11) in order to know the status of 11 bit (whether the data received in previous read is a first data byte or not). ■ The 11th bit is an optional read field, user can ignore 2nd byte read [15:8] (offset 0x11) if not interested in FIRST_DATA_BYTE status. Values: ■ 0x1 (ACTIVE): Non sequential data byte received ■ 0x0 (INACTIVE): Sequential data byte received Value After Reset: 0x0 Exists: IC_FIRST_DATA_BYTE_STATUS == 1 Volatile: true
10	RESTART	W	This bit controls whether a RESTART is issued before the byte is sent or received. This bit is available only if IC_EMPTYFIFO_HOLD_MASTER_EN is configured to 1. ■ 1: If IC_RESTART_EN is 1, a RESTART is issued before the data is sent/received (according to the value of CMD), regardless of whether or not the transfer direction is changing from the previous command; if IC_RESTART_EN is 0, a STOP followed by a START is issued instead. ■ 0: If IC_RESTART_EN is 1, a RESTART is issued only if the transfer direction is changing from the previous command; if IC_RESTART_EN is 0, a STOP followed by a START is issued instead. Values: ■ 0x1 (ENABLE): Issue RESTART before this command ■ 0x0 (DISABLE): Do not Issue RESTART before this command Value After Reset: 0x0 Exists: IC_EMPTYFIFO_HOLD_MASTER_EN Volatile: true

Bits	Name	Memory Access	Description
9	STOP	W	This bit controls whether a STOP is issued after the byte is sent or received. This bit is available only if IC_EMPTYFIFO_HOLD_MASTER_EN is configured to 1. 1 - STOP is issued after this byte, regardless of whether or not the Tx FIFO is empty. If the Tx FIFO is not empty, the controller immediately tries to start a new transfer by issuing a START and arbitrating for the bus. 0 - STOP is not issued after this byte, regardless of whether or not the Tx FIFO is empty. If the Tx FIFO is not empty, the controller continues the current transfer by sending/receiving data bytes according to the value of the CMD bit. If the Tx FIFO is empty, the controller holds the SCL line low and stalls the bus until a new command is available in the Tx FIFO. Values: 0x1 (ENABLE): Issue STOP after this command 0x0 (DISABLE): Do not Issue STOP after this command Value After Reset: 0x0 Exists: IC_EMPTYFIFO_HOLD_MASTER_EN Volatile: true
8	CMD	W	This bit controls whether a read or a write is performed. This bit does not control the direction when the DW_apb_i2c acts as a target. It controls only the direction when it acts as a controller. When a command is entered in the TX FIFO, this bit distinguishes the write and read commands. In target-receiver mode, this bit is a "don't care" because writes to this register are not required. In target-transmitter mode, a "0" indicates that the data in IC_DATA_CMD is to be transmitted. When programming this bit, you should remember the following: - Attempting to perform a read operation after a General Call command has been sent results in a TX_ABRT interrupt (bit 6 of the IC_RAW_INTR_STAT register), unless bit 11 (SPECIAL) in the IC_TAR register has been cleared. - If a "1" is written to this bit after receiving a RD_REQ interrupt, then a TX_ABRT interrupt occurs. Values: 0x1 (READ): Controller Read Command 0x0 (WRITE): Controller Write Command Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
7:0	DAT	R/W	This register contains the data to be transmitted or received on the I2C bus. If you are writing to this register and want to perform a read, bits 7:0 (DAT) are ignored by the DW_apb_i2c. However, when you read this register, these bits return the value of data received on the DW_apb_i2c interface. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.6 IC_SS_SCL_HCNT

- **Name:** Standard Speed I2C Clock SCL High Count Register
- **Description:** Standard Speed I2C Clock SCL High Count Register
- **Size:** 32 bits
- **Offset:** 0x14
- **Exists:** IC_ULTRA_FAST_MODE==0

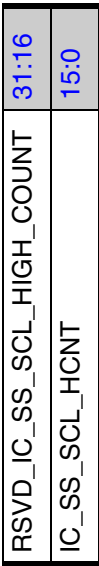


Table 5-10 Fields for Register: IC_SS_SCL_HCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_SS_SCL_HIGH_COUNT	R	IC_SS_SCL_HCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_SS_SCL_HCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock high-period count for standard speed. For more information, refer to "IC_CLK Frequency Configuration". This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE == 1) ? 3 : ((IC_CLK_FREQ_OPTIMIZATION == 1) ? 1 : 6))$ Hardware prevents values less than this being written, and if attempted results in minimum valid value being set. For designs with APB_DATA_WIDTH = 8, the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. When the configuration parameter IC_HC_COUNT_VALUES is set to 1, this register is read only. Note: This register must not be programmed to a value higher than 65525, because DW_apb_i2c uses a 16-bit counter to flag an I2C bus idle condition when this counter reaches a value of IC_SS_SCL_HCNT + 10. Value After Reset: IC_SS_SCL_HIGH_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES == 1) ? \"read-only\" : \"read-write\""

5.1.7 IC_UFM_SCL_HCNT

- **Name:** Ultra-Fast Speed I2C Clock SCL High Count Register
- **Description:** Ultra-Fast Speed I2C Clock SCL High Count Register
- **Size:** 32 bits
- **Offset:** 0x14
- **Exists:** IC_ULTRA_FAST_MODE==1

31:16	RSVD_IC_UFM_SCL_HCNT
15:0	IC_UFM_SCL_HCNT

Table 5-11 Fields for Register: IC_UFM_SCL_HCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_UFM_SCL_HCNT	R	IC_UFM_SCL_HCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_UFM_SCL_HCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock high-period count for Ultra-Fast speed. For more information, refer to "IC_CLK Frequency Configuration". This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is 3; hardware prevents values less than this being written, and if attempted results in 3 being set. For designs with APB_DATA_WIDTH = 8, the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. When the configuration parameter IC_HC_COUNT_VALUES is set to 1, this register is read only. Value After Reset: IC_UFM_SCL_HIGH_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES==1) ? \"read-only\" : \"read-write\""

5.1.8 IC_SS_SCL_LCNT

- **Name:** Standard Speed I2C Clock SCL Low Count Register
- **Description:** Standard Speed I2C Clock SCL Low Count Register
- **Size:** 32 bits
- **Offset:** 0x18
- **Exists:** IC_ULTRA_FAST_MODE==0

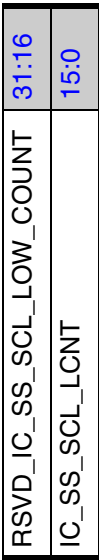


Table 5-12 Fields for Register: IC_SS_SCL_LCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_SS_SCL_LOW_COUNT	R	RSVD_IC_SS_SCL_LOW_COUNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_SS_SCL_LCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock low period count for standard speed. For more information, refer to "IC_CLK Frequency Configuration" This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE == 1) ? 5 : ((IC_CLK_FREQ_OPTIMIZATION == 1) ? 6 : 8))$ Hardware prevents values less than this being written, and if attempted, results in minimum valid value being set. For designs with APB_DATA_WIDTH = 8, the order of programming is important to ensure the correct operation of DW_apb_i2c. The lower byte must be programmed first, and then the upper byte is programmed. When the configuration parameter IC_HC_COUNT_VALUES is set to 1, this register is read only. Value After Reset: IC_SS_SCL_LOW_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES == 1) ? \"read-only\" : \"read-write\""

5.1.9 IC_UFM_SCL_LCNT

- **Name:** Ultra-Fast Speed I2C Clock SCL Low Count Register
- **Description:** Ultra-Fast Speed I2C Clock SCL Low Count Register
- **Size:** 32 bits
- **Offset:** 0x18
- **Exists:** IC_ULTRA_FAST_MODE==1

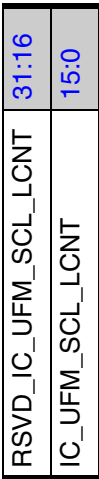


Table 5-13 Fields for Register: IC_UFM_SCL_LCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_UFM_SCL_LCNT	R	IC_UFM_SCL_LCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_UFM_SCL_LCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock low period count for Ultra-Fast speed. This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is 5; hardware prevents values less than this being written, and if attempted, results in 5 being set. For designs with APB_DATA_WIDTH = 8, the order of programming is important to ensure the correct operation of DW_apb_i2c. The lower byte must be programmed first, and then the upper byte is programmed. When the configuration parameter IC_HC_COUNT_VALUES is set to 1, this register is read only. Value After Reset: IC_UFM_SCL_LOW_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES==1) ? \"read-only\" : \"read-write\""

5.1.10 IC_FS_SCL_HCNT

- **Name:** Fast Mode or Fast Mode Plus I2C Clock SCL High Count Register
- **Description:** Fast Mode or Fast Mode Plus I2C Clock SCL High Count Register
- **Size:** 32 bits
- **Offset:** 0x1c
- **Exists:** IC_MAX_SPEED_MODE!=1

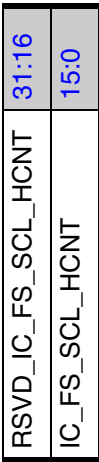


Table 5-14 Fields for Register: IC_FS_SCL_HCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_FS_SCL_HCNT	R	IC_FS_SCL_HCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_FS_SCL_HCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock high-period count for fast mode or fast mode plus. It is used in high-speed mode to send the Controller Code and START BYTE or General CALL. For more information, refer to "IC_CLK Frequency Configuration". This register goes away and becomes read-only returning 0s if IC_MAX_SPEED_MODE = standard. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE == 1) ? 3 : ((IC_CLK_FREQ_OPTIMIZATION == 1) ? 1 : 6))$ Hardware prevents values less than this being written, and if attempted results in minimum valid value being set. For designs with APB_DATA_WIDTH == 8 the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. Value After Reset: IC_FS_SCL_HIGH_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES==1) ? \"read-only\" : \"read-write\""

5.1.11 IC_UFM_TBUF_CNT

- **Name:** Ultra-Fast Speed mode TBuf Idle Count Register
- **Description:** Ultra-Fast Speed mode TBuf Idle Count Register
- **Size:** 32 bits
- **Offset:** 0x1c
- **Exists:** IC_ULTRA_FAST_MODE==1

31:16	RSVD_IC_UFM_TBUF_CNT
15:0	IC_UFM_TBUF_CNT

Table 5-15 Fields for Register: IC_UFM_TBUF_CNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_UFM_TBUF_CNT	R	IC_UFM_TBUF_CNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
15:0	IC_UFM_TBUF_CNT	R/W	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the Bus-Free time between a STOP and STOP condition count for Ultra-Fast speed. This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. For designs with APB_DATA_WIDTH = 8, the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first and then the upper byte is programmed. When the configuration parameter. Note: The DW_apb_i2c will add 9 ic_clks after tBuf time is expired to generate START on the Bus. Value After Reset: IC_UFM_TBUF_CNT_DEFAULT Exists: Always

5.1.12 IC_FS_SCL_LCNT

- **Name:** Fast Mode or Fast Mode Plus I2C Clock SCL Low Count Register
- **Description:** Fast Mode or Fast Mode Plus I2C Clock SCL Low Count Register
- **Size:** 32 bits
- **Offset:** 0x20
- **Exists:** IC_MAX_SPEED_MODE!=1

31:16	RSVD_IC_FS_SCL_LCNT
15:0	IC_FS_SCL_LCNT

Table 5-16 Fields for Register: IC_FS_SCL_LCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_FS_SCL_LCNT	R	IC_FS_SCL_LCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_FS_SCL_LCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock low period count for fast speed. It is used in high-speed mode to send the Controller Code and START BYTE or General CALL. For more information, refer to "IC_CLK Frequency Configuration". This register goes away and becomes read-only returning 0s if IC_MAX_SPEED_MODE = standard. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE==1) ? 5 : ((IC_CLK_FREQ_OPTIMIZATION==1) ? 6 : 8))$ Hardware prevents values less than this being written, and if attempted results in minimum valid value being set. For designs with APB_DATA_WIDTH = 8 the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. If the value is less than minimum valid value then the count value gets changed to minimum valid value. When the configuration parameter IC_HC_COUNT_VALUES is set to 1, this register is read only. Value After Reset: IC_FS_SCL_LOW_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES==1) ? \"read-only\" : \"read-write\""

5.1.13 IC_HS_SCL_HCNT

- **Name:** High Speed I2C Clock SCL High Count Register
- **Description:** High Speed I2C Clock SCL High Count Register
- **Size:** 32 bits
- **Offset:** 0x24
- **Exists:** IC_MAX_SPEED_MODE==3

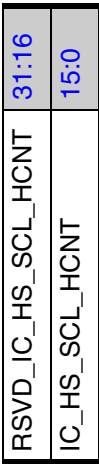


Table 5-17 Fields for Register: IC_HS_SCL_HCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_HS_SCL_HCNT	R	IC_HS_SCL_HCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_HS_SCL_HCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock high period count for high speed. refer to "IC_CLK Frequency Configuration". The SCL High time depends on the loading of the bus. For 100pF loading, the SCL High time is 60ns; for 400pF loading, the SCL High time is 120ns. This register goes away and becomes read-only returning 0s if IC_MAX_SPEED_MODE != high. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE == 1) ? 3 : ((IC_CLK_FREQ_OPTIMIZATION == 1) ? 1 : 6))$ Hardware prevents values less than this being written, and if attempted results in minimum valid value being set. For designs with APB_DATA_WIDTH = 8 the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. Value After Reset: IC_HS_SCL_HIGH_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES == 1) ? \"read-only\" : \"read-write\""

5.1.14 IC_HS_SCL_LCNT

- **Name:** High Speed I2C Clock SCL Low Count Register
- **Description:** High Speed I2C Clock SCL Low Count Register
- **Size:** 32 bits
- **Offset:** 0x28
- **Exists:** IC_MAX_SPEED_MODE==3



Table 5-18 Fields for Register: IC_HS_SCL_LCNT

Bits	Name	Memory Access	Description
31:16	RSVD_IC_HS_SCL_LOW_CNT	R	IC_HS_SCL_LCNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_HS_SCL_LCNT	* Varies	This register must be set before any I2C bus transaction can take place to ensure proper I/O timing. This register sets the SCL clock low period count for high speed. For more information, refer to "IC_CLK Frequency Configuration". The SCL low time depends on the loading of the bus. For 100pF loading, the SCL low time is 160ns; for 400pF loading, the SCL low time is 320ns. This register goes away and becomes read-only returning 0s if IC_MAX_SPEED_MODE != high. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is derived through below formula: $((IC_ULTRA_FAST_MODE == 1) ? 5 :$ $((IC_CLK_FREQ_OPTIMIZATION == 1) ? 6 : 8))$ Hardware prevents values less than this being written, and if attempted results in minimum valid value being set. For designs with APB_DATA_WIDTH == 8 the order of programming is important to ensure the correct operation of the DW_apb_i2c. The lower byte must be programmed first. Then the upper byte is programmed. If the value is less than minimum valid value then the count value gets changed to minimum valid value. Value After Reset: IC_HS_SCL_LOW_COUNT Exists: Always Memory Access: "(IC_HC_COUNT_VALUES == 1) ? \"read-only\" : \"read-write\""

5.1.15 IC_INTR_STAT

- **Name:** I2C Interrupt Status Register
- **Description:** I2C Interrupt Status Register.
Each bit in this register has a corresponding mask bit in the IC_INTR_MASK register. These bits are cleared by reading the matching interrupt clear register. The unmasked raw versions of these bits are available in the IC_RAW_INTR_STAT register.
- **Size:** 32 bits
- **Offset:** 0x2c
- **Exists:** Always

31:20	RSVD_IC_INTR_STAT
19	R_SLV_ADDR4_TAG
18	R_SLV_ADDR3_TAG
17	R_SLV_ADDR2_TAG
16	R_SLV_ADDR1_TAG
15	R_WR_REQ
14	R_SCL_STUCK_AT_LOW
13	R_MASTER_ON_HOLD
12	R_RESTART_DET
11	R_GEN_CALL
10	R_START_DET
9	R_STOP_DET
8	R_ACTIVITY
7	R_RX_DONE
6	R_TX_ABORT
5	R_RD_REQ
4	R_TX_EMPTY
3	R_TX_OVER
2	R_RX_FULL
1	R_RX_OVER
0	R_RX_UNDER

Table 5-19 Fields for Register: IC_INTR_STAT

Bits	Name	Memory Access	Description
31:20	RSVD_IC_INTR_STAT	R	IC_INTR_STAT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
19	R_SLV_ADDR4_TAG	R	See IC_RAW_INTR_STAT for a detailed description of R_SLV_ADDR4_TAG bit. Values: ■ 0x1 (DISABLED): SLV_ADDR4_TAG interrupt is active ■ 0x0 (ENABLED): SLV_ADDR4_TAG interrupt is inactive Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) Volatile: true

Bits	Name	Memory Access	Description
18	R_SLV_ADDR3_TAG	R	See IC_RAW_INTR_STAT for a detailed description of R_SLV_ADDR3_TAG bit. Values: 0x1 (DISABLED): SLV_ADDR3_TAG interrupt is active 0x0 (ENABLED): SLV_ADDR3_TAG interrupt is inactive Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) Volatile: true
17	R_SLV_ADDR2_TAG	R	See IC_RAW_INTR_STAT for a detailed description of R_SLV_ADDR2_TAG bit. Values: 0x1 (DISABLED): SLV_ADDR2_TAG interrupt is active 0x0 (ENABLED): SLV_ADDR2_TAG interrupt is inactive Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) Volatile: true
16	R_SLV_ADDR1_TAG	R	See IC_RAW_INTR_STAT for a detailed description of R_SLV_ADDR1_TAG bit. Values: 0x1 (DISABLED): SLV_ADDR1_TAG interrupt is active 0x0 (ENABLED): SLV_ADDR1_TAG interrupt is inactive Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 1) Volatile: true
15	R_WR_REQ	R	See IC_RAW_INTR_STAT for a detailed description of R_WR_REQ bit. Values: 0x1 (ACTIVE): WR_REQ interrupt is active 0x0 (INACTIVE): WR_REQ interrupt is inactive Value After Reset: 0x0 Exists: IC_MULTI_SAR_EN==1 Volatile: true
14	R_SCL_STUCK_AT_LOW	R	See IC_RAW_INTR_STAT for a detailed description of R_SCL_STUCK_AT_LOW bit. Values: 0x1 (ACTIVE): R_SCL_STUCK_AT_LOW interrupt is active 0x0 (INACTIVE): R_SCL_STUCK_AT_LOW interrupt is inactive Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE==1 Volatile: true

Bits	Name	Memory Access	Description
13	R_MASTER_ON_HOLD	R	See IC_RAW_INTR_STAT for a detailed description of R_MASTER_ON_HOLD bit. Values: ■ 0x1 (ACTIVE): R_MASTER_ON_HOLD interrupt is active ■ 0x0 (INACTIVE): R_MASTER_ON_HOLD interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
12	R_RESTART_DET	R	See IC_RAW_INTR_STAT for a detailed description of R_RESTART_DET bit. Values: ■ 0x1 (ACTIVE): R_RESTART_DET interrupt is active ■ 0x0 (INACTIVE): R_RESTART_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
11	R_GEN_CALL	R	See IC_RAW_INTR_STAT for a detailed description of R_GEN_CALL bit. Values: ■ 0x1 (ACTIVE): R_GEN_CALL interrupt is active ■ 0x0 (INACTIVE): R_GEN_CALL interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
10	R_START_DET	R	See IC_RAW_INTR_STAT for a detailed description of R_START_DET bit. Values: ■ 0x1 (ACTIVE): R_START_DET interrupt is active ■ 0x0 (INACTIVE): R_START_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
9	R_STOP_DET	R	See IC_RAW_INTR_STAT for a detailed description of R_STOP_DET bit. Values: ■ 0x1 (ACTIVE): R_STOP_DET interrupt is active ■ 0x0 (INACTIVE): R_STOP_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
8	R_ACTIVITY	R	See IC_RAW_INTR_STAT for a detailed description of R_ACTIVITY bit. Values: 0x1 (ACTIVE): R_ACTIVITY interrupt is active 0x0 (INACTIVE): R_ACTIVITY interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
7	R_RX_DONE	R	See IC_RAW_INTR_STAT for a detailed description of R_RX_DONE bit. Values: 0x1 (ACTIVE): R_RX_DONE interrupt is active 0x0 (INACTIVE): R_RX_DONE interrupt is inactive Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
6	R_TX_ABRT	R	See IC_RAW_INTR_STAT for a detailed description of R_TX_ABRT bit. Values: 0x1 (ACTIVE): R_TX_ABRT interrupt is active 0x0 (INACTIVE): R_TX_ABRT interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
5	R_RD_REQ	R	See IC_RAW_INTR_STAT for a detailed description of R_RD_REQ bit. Values: 0x1 (ACTIVE): R_RD_REQ interrupt is active 0x0 (INACTIVE): R_RD_REQ interrupt is inactive Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
4	R_TX_EMPTY	R	See IC_RAW_INTR_STAT for a detailed description of R_TX_EMPTY bit. Values: 0x1 (ACTIVE): R_TX_EMPTY interrupt is active 0x0 (INACTIVE): R_TX_EMPTY interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
3	R_TX_OVER	R	See IC_RAW_INTR_STAT for a detailed description of R_TX_OVER bit. Values: ■ 0x1 (ACTIVE): R_TX_OVER interrupt is active ■ 0x0 (INACTIVE): R_TX_OVER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
2	R_RX_FULL	R	See IC_RAW_INTR_STAT for a detailed description of R_RX_FULL bit. Values: ■ 0x1 (ACTIVE): R_RX_FULL interrupt is active ■ 0x0 (INACTIVE): R_RX_FULL interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
1	R_RX_OVER	R	See IC_RAW_INTR_STAT for a detailed description of R_RX_OVER bit. Values: ■ 0x1 (ACTIVE): R_RX_OVER interrupt is active ■ 0x0 (INACTIVE): R_RX_OVER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
0	R_RX_UNDER	R	See IC_RAW_INTR_STAT for a detailed description of R_RX_UNDER bit. Values: ■ 0x1 (ACTIVE): RX_UNDER interrupt is active ■ 0x0 (INACTIVE): RX_UNDER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

5.1.16 IC_INTR_MASK

- **Name:** I2C Interrupt Mask Register
- **Description:** I2C Interrupt Mask Register.
These bits mask their corresponding interrupt status bits. This register is active low; a value of 0 masks the interrupt, whereas a value of 1 unmask the interrupt.
- **Size:** 32 bits
- **Offset:** 0x30
- **Exists:** Always

31:20	RSVD_IC_INTR_STAT
19	M_SLV_ADDR4_TAG
18	M_SLV_ADDR3_TAG
17	M_SLV_ADDR2_TAG
16	M_SLV_ADDR1_TAG
15	M_WR_REQ
14	M_SCL_STUCK_AT_LOW
13	M_MASTER_ON_HOLD
12	M_RESTART_DET
11	M_GEN_CALL
10	M_START_DET
9	M_STOP_DET
8	M_ACTIVITY
7	M_RX_DONE
6	M_TX_ABRT
5	M_RD_REQ
4	M_TX_EMPTY
3	M_TX_OVER
2	M_RX_FULL
1	M_RX_OVER
0	M_RX_UNDER

Table 5-20 Fields for Register: IC_INTR_MASK

Bits	Name	Memory Access	Description
31:20	RSVD_IC_INTR_STAT	R	IC_INTR_STAT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
19	M_SLV_ADDR4_TAG	R/W	This bit masks the R_SLV_ADDR4_TAG interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): SLV_ADDR4_TAG interrupt is unmasked ■ 0x0 (ENABLED): SLV_ADDR4_TAG interrupt is masked Value After Reset: 0x1 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4)

Bits	Name	Memory Access	Description
18	M_SLV_ADDR3_TAG	R/W	This bit masks the R_SLV_ADDR3_TAG interrupt in IC_INTR_STAT register. Values: 0x1 (DISABLED): SLV_ADDR3_TAG interrupt is unmasked 0x0 (ENABLED): SLV_ADDR3_TAG interrupt is masked Value After Reset: 0x1 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3)
17	M_SLV_ADDR2_TAG	R/W	This bit masks the R_SLV_ADDR2_TAG interrupt in IC_INTR_STAT register. Values: 0x1 (DISABLED): SLV_ADDR2_TAG interrupt is unmasked 0x0 (ENABLED): SLV_ADDR2_TAG interrupt is masked Value After Reset: 0x1 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2)
16	M_SLV_ADDR1_TAG	R/W	This bit masks the R_SLV_ADDR1_TAG interrupt in IC_INTR_STAT register. Values: 0x1 (DISABLED): SLV_ADDR1_TAG interrupt is unmasked 0x0 (ENABLED): SLV_ADDR1_TAG interrupt is masked Value After Reset: 0x1 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 1)
15	M_WR_REQ	R/W	This bit masks the R_WR_REQ interrupt in IC_INTR_STAT register. Values: 0x1 (DISABLED): WR_REQ interrupt is unmasked 0x0 (ENABLED): WR_REQ interrupt is masked Value After Reset: 0x1 Exists: IC_MULTI_SAR_EN==1
14	M_SCL_STUCK_AT_LOW	R/W	This bit masks the R_SCL_STUCK_AT_LOW interrupt in IC_INTR_STAT register. Values: 0x1 (DISABLED): SCL_STUCK_AT_LOW interrupt is unmasked 0x0 (ENABLED): SCL_STUCK_AT_LOW interrupt is masked Value After Reset: "(IC_BUS_CLEAR_FEATURE_EN) ? \"0x1\" : \"0x0\"" Exists: IC_BUS_CLEAR_FEATURE==1

Bits	Name	Memory Access	Description
13	M_MASTER_ON_HOLD	R/W	This bit masks the R_MASTER_ON_HOLD interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): MASTER_ON_HOLD interrupt is unmasked ■ 0x0 (ENABLED): MASTER_ON_HOLD interrupt is masked Value After Reset: 0x0 Exists: I2C_DYNAMIC_TAR_UPDATE == 1 && IC_EMPTYFIFO_HOLD_MASTER_EN == 1
12	M_RESTART_DET	R/W	This bit masks the R_RESTART_DET interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RESTART_DET interrupt is unmasked ■ 0x0 (ENABLED): RESTART_DET interrupt is masked Value After Reset: 0x0 Exists: IC_SLV_RESTART_DET_EN == 1
11	M_GEN_CALL	R/W	This bit masks the R_GEN_CALL interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): GEN_CALL interrupt is unmasked ■ 0x0 (ENABLED): GEN_CALL interrupt is masked Value After Reset: 0x1 Exists: Always
10	M_START_DET	R/W	This bit masks the R_START_DET interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): START_DET interrupt is unmasked ■ 0x0 (ENABLED): START_DET interrupt is masked Value After Reset: 0x0 Exists: Always
9	M_STOP_DET	R/W	This bit masks the R_STOP_DET interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): STOP_DET interrupt is unmasked ■ 0x0 (ENABLED): STOP_DET interrupt is masked Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
8	M_ACTIVITY	R/W	This bit masks the R_ACTIVITY interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): ACTIVITY interrupt is unmasked ■ 0x0 (ENABLED): ACTIVITY interrupt is masked Value After Reset: 0x0 Exists: Always
7	M_RX_DONE	R/W	This bit masks the R_RX_DONE interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RX_DONE interrupt is unmasked ■ 0x0 (ENABLED): RX_DONE interrupt is masked Value After Reset: 0x1 Exists: IC_ULTRA_FAST_MODE == 0
6	M_TX_ABRT	R/W	This bit masks the R_TX_ABRT interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): TX_ABORT interrupt is unmasked ■ 0x0 (ENABLED): TX_ABORT interrupt is masked Value After Reset: 0x1 Exists: Always
5	M_RD_REQ	R/W	This bit masks the R_RD_REQ interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RD_REQ interrupt is unmasked ■ 0x0 (ENABLED): RD_REQ interrupt is masked Value After Reset: 0x1 Exists: IC_ULTRA_FAST_MODE == 0
4	M_TX_EMPTY	R/W	This bit masks the R_TX_EMPTY interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): TX_EMPTY interrupt is unmasked ■ 0x0 (ENABLED): TX_EMPTY interrupt is masked Value After Reset: 0x1 Exists: Always

Bits	Name	Memory Access	Description
3	M_TX_OVER	R/W	This bit masks the R_TX_OVER interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): TX_OVER interrupt is unmasked ■ 0x0 (ENABLED): TX_OVER interrupt is masked Value After Reset: 0x1 Exists: Always
2	M_RX_FULL	R/W	This bit masks the R_RX_FULL interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RX_FULL interrupt is unmasked ■ 0x0 (ENABLED): RX_FULL interrupt is masked Value After Reset: 0x1 Exists: Always
1	M_RX_OVER	R/W	This bit masks the R_RX_OVER interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RX_OVER interrupt is unmasked ■ 0x0 (ENABLED): RX_OVER interrupt is masked Value After Reset: 0x1 Exists: Always
0	M_RX_UNDER	R/W	This bit masks the R_RX_UNDER interrupt in IC_INTR_STAT register. Values: ■ 0x1 (DISABLED): RX_UNDER interrupt is unmasked ■ 0x0 (ENABLED): RX_UNDER interrupt is masked Value After Reset: 0x1 Exists: Always

5.1.17 IC_RAW_INTR_STAT

- **Name:** I2C Raw Interrupt Status Register
- **Description:** I2C Raw Interrupt Status Register.
Unlike the IC_INTR_STAT register, these bits are not masked so they always show the true status of the DW_apb_i2c.
- **Size:** 32 bits
- **Offset:** 0x34
- **Exists:** Always

31:20	RSVD_IC_RAW_INTR_STAT
19	SLV_ADDR4_TAG
18	SLV_ADDR3_TAG
17	SLV_ADDR2_TAG
16	SLV_ADDR1_TAG
15	WR_REQ
14	SCL_STUCK_AT_LOW
13	MASTER_ON_HOLD
12	RESTART_DET
11	GEN_CALL
10	START_DET
9	STOP_DET
8	ACTIVITY
7	RX_DONE
6	TX_ABORT
5	RD_REQ
4	TX_EMPTY
3	TX_OVER
2	RX_FULL
1	RX_OVER
0	RX_UNDER

Table 5-21 Fields for Register: IC_RAW_INTR_STAT

Bits	Name	Memory Access	Description
31:20	RSVD_IC_RAW_INTR_STAT	R	IC_RAW_INTR_STAT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
19	SLV_ADDR4_TAG	R	This bit is set to 1 when DW_apb_i2c is acting as target and another I2C controller is attempting to write/read I2C transfers with the target device address IC_SAR4. This can be used in conjunction with the WR_REQ/RD_REQ to determine which target address has been addressed. Values: ■ 0x1 (ACTIVE): SLV_ADDR4_TAG interrupt is active ■ 0x0 (INACTIVE): SLV_ADDR4_TAG interrupt is inactive. Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) Volatile: true

Bits	Name	Memory Access	Description
18	SLV_ADDR3_TAG	R	This bit is set to 1 when DW_apb_i2c is acting as target and another I2C controller is attempting to write/read I2C transfers with the target device address IC_SAR3. This can be used in conjunction with the WR_REQ/RD_REQ to determine which target address has been addressed. Values: ■ 0x1 (ACTIVE): SLV_ADDR3_TAG interrupt is active ■ 0x0 (INACTIVE): SLV_ADDR3_TAG interrupt is inactive. Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) Volatile: true
17	SLV_ADDR2_TAG	R	This bit is set to 1 when DW_apb_i2c is acting as target and another I2C controller is attempting to write/read I2C transfers with the target device address IC_SAR2. This can be used in conjunction with the WR_REQ/RD_REQ to determine which target address has been addressed. Values: ■ 0x1 (ACTIVE): SLV_ADDR2_TAG interrupt is active ■ 0x0 (INACTIVE): SLV_ADDR2_TAG interrupt is inactive. Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) Volatile: true
16	SLV_ADDR1_TAG	R	This bit is set to 1 when DW_apb_i2c is acting as target and another I2C controller is attempting to write/read I2C transfers with the target device address IC_SAR. This can be used in conjunction with the WR_REQ/RD_REQ to determine which target address has been addressed. Values: ■ 0x1 (ACTIVE): SLV_ADDR1_TAG interrupt is active ■ 0x0 (INACTIVE): SLV_ADDR1_TAG interrupt is inactive. Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 1) Volatile: true

Bits	Name	Memory Access	Description
15	WR_REQ	R	This bit is set to 1 when DW_apb_i2c is acting as target and another I2C controller is attempting to write data into DW_apb_i2c. This interrupt provides indication that, the write data from another I2C controller is going to follow and this interrupt can be used in conjunction of SLV_ADDRx_TAG (where x = 1 ... IC_NUM_SARS) to determine for which target device address the I2C transfer is received. Values: ■ 0x1 (ACTIVE): WR_REQ interrupt is active ■ 0x0 (INACTIVE): WR_REQ interrupt is inactive. Value After Reset: 0x0 Exists: IC_MULTI_SAR_EN==1 Volatile: true
14	SCL_STUCK_AT_LOW	R	Indicates whether the SCL Line is stuck at low for the IC_SCL_STUCK_LOW_TIMEOUT number of ic_clk periods. Enabled only when IC_BUS_CLEAR_FEATURE=1 and IC_ULTRA_FAST_MODE=0. Values: ■ 0x1 (ACTIVE): SCL_STUCK_AT_LOW interrupt is active ■ 0x0 (INACTIVE): SCL_STUCK_AT_LOW interrupt is inactive. Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE==1 Volatile: true
13	MASTER_ON_HOLD	R	Indicates whether controller is holding the bus and TX FIFO is empty. Enabled only when I2C_DYNAMIC_TAR_UPDATE=1 and IC_EMPTYFIFO_HOLD_MASTER_EN=1. Values: ■ 0x1 (ACTIVE): MASTER_ON_HOLD interrupt is active ■ 0x0 (INACTIVE): MASTER_ON_HOLD interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
12	RESTART_DET	R	Indicates whether a RESTART condition has occurred on the I2C interface when DW_apb_i2c is operating in Target mode and the target is being addressed. Enabled only when IC_SLV_RESTART_DET_EN=1. Note: However, in high-speed mode or during a START BYTE transfer, the RESTART comes before the address field as per the I2C protocol. In this case, the target is not the addressed target when the RESTART is issued, therefore DW_apb_i2c does not generate the RESTART_DET interrupt. Values: ■ 0x1 (ACTIVE): RESTART_DET interrupt is active ■ 0x0 (INACTIVE): RESTART_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
11	GEN_CALL	R	Set only when a General Call address is received and it is acknowledged. It stays set until it is cleared either by disabling DW_apb_i2c or when the CPU reads bit 0 of the IC_CLR_GEN_CALL register. DW_apb_i2c stores the received data in the Rx buffer. Values: ■ 0x1 (ACTIVE): GEN_CALL interrupt is active ■ 0x0 (INACTIVE): GEN_CALL interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
10	START_DET	R	Indicates whether a START or RESTART condition has occurred on the I2C interface regardless of whether DW_apb_i2c is operating in target or controller mode. Values: ■ 0x1 (ACTIVE): START_DET interrupt is active ■ 0x0 (INACTIVE): START_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
9	STOP_DET	R	Indicates whether a STOP condition has occurred on the I2C interface regardless of whether DW_apb_i2c is operating in target or controller mode. In Target Mode: ■ If IC_CON[7]=1'b1 (STOP_DET_IFADDRESSED), the STOP_DET interrupt will be issued only if target is addressed. Note: During a general call address, this target does not issue a STOP_DET interrupt if STOP_DET_IF_ADDRESSED=1'b1, even if the target responds to the general call address by generating ACK. The STOP_DET interrupt is generated only when the transmitted address matches the target device address (SAR). ■ If IC_CON[7]=1'b0 (STOP_DET_IFADDRESSED), the STOP_DET interrupt is issued irrespective of whether it is being addressed. In Controller Mode: ■ If IC_CON[10]=1'b1 (STOP_DET_IF_MASTER_ACTIVE), the STOP_DET interrupt will be issued only if Controller is active. ■ If IC_CON[10]=1'b0 (STOP_DET_IFADDRESSED), the STOP_DET interrupt will be issued irrespective of whether controller is active or not. Values: ■ 0x1 (ACTIVE): STOP_DET interrupt is active ■ 0x0 (INACTIVE): STOP_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
8	ACTIVITY	R	This bit captures DW_apb_i2c activity and stays set until it is cleared. There are four ways to clear it: ■ Disabling the DW_apb_i2c ■ Reading the IC_CLR_ACTIVITY register ■ Reading the IC_CLR_INTR register ■ System reset Once this bit is set, it stays set unless one of the four methods is used to clear it. Even if the DW_apb_i2c module is idle, this bit remains set until cleared, indicating that there was activity on the bus. Values: ■ 0x1 (ACTIVE): RAW_INTR_ACTIVITY interrupt is active ■ 0x0 (INACTIVE): RAW_INTR_ACTIVITY interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
7	RX_DONE	R	When the DW_apb_i2c is acting as a target-transmitter, this bit is set to 1 if the controller does not acknowledge a transmitted byte. This occurs on the last byte of the transmission, indicating that the transmission is done. Values: ■ 0x1 (ACTIVE): RX_DONE interrupt is active ■ 0x0 (INACTIVE): RX_DONE interrupt is inactive Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
6	TX_ABRT	R	This bit indicates if DW_apb_i2c, as an I2C transmitter, is unable to complete the intended actions on the contents of the transmit FIFO. This situation can occur both as an I2C controller or an I2C target, and is referred to as a 'transmit terminate'. When this bit is set to 1, the IC_TX_ABRT_SOURCE register indicates the reason why the transmit terminate takes places. Note: The DW_apb_i2c flushes/resets/empties only the TX_FIFO whenever there is a transmit terminate caused by any of the events tracked by the IC_TX_ABRT_SOURCE register. The Tx FIFO remains in this flushed state until the register IC_CLR_TX_ABRT is read. Once this read is performed, the Tx FIFO is then ready to accept more data bytes from the APB interface. RX FIFO flush because of TX_ABRT is controlled by the coreConsultant parameter IC_AVOID_RX_FIFO_FLUSH_ON_TX_ABRT. Values: ■ 0x1 (ACTIVE): TX_ABRT interrupt is active ■ 0x0 (INACTIVE): TX_ABRT interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
5	RD_REQ	R	This bit is set to 1 when DW_apb_i2c is acting as a target and another I2C controller is attempting to read data from DW_apb_i2c. The DW_apb_i2c holds the I2C bus in a wait state (SCL=0) until this interrupt is serviced, which means that the target has been addressed by a remote controller that is asking for data to be transferred. The processor must respond to this interrupt and then write the requested data to the IC_DATA_CMD register. This bit is set to 0 just after the processor reads the IC_CLR_RD_REQ register. Values: ■ 0x1 (ACTIVE): RD_REQ interrupt is active ■ 0x0 (INACTIVE): RD_REQ interrupt is inactive Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
4	TX_EMPTY	R	The behavior of the TX_EMPTY interrupt status differs based on the TX_EMPTY_CTRL selection in the IC_CON register. ■ When TX_EMPTY_CTRL = 0: This bit is set to 1 when the transmit buffer is at or below the threshold value set in the IC_TX_TL register. ■ When TX_EMPTY_CTRL = 1: This bit is set to 1 when the transmit buffer is at or below the threshold value set in the IC_TX_TL register and the transmission of the address/data from the internal shift register for the most recently popped command is completed. It is automatically cleared by hardware when the buffer level goes above the threshold. When IC_ENABLE[0] is set to 0, the TX FIFO is flushed and held in reset. There the TX FIFO looks like it has no data within it, so this bit is set to 1, provided there is activity in the controller or target state machines. When there is no longer any activity, then with ic_en=0, this bit is set to 0. Values: ■ 0x1 (ACTIVE): TX_EMPTY interrupt is active ■ 0x0 (INACTIVE): TX_EMPTY interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
3	TX_OVER	R	Set during transmit if the transmit buffer is filled to IC_TX_BUFFER_DEPTH and the processor attempts to issue another I2C command by writing to the IC_DATA_CMD register. When the module is disabled, this bit keeps its level until the controller or target state machines go into idle, and when ic_en goes to 0, this interrupt is cleared. Values: ■ 0x1 (ACTIVE): TX_OVER interrupt is active ■ 0x0 (INACTIVE): TX_OVER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
2	RX_FULL	R	Set when the receive buffer reaches or goes above the RX_TL threshold in the IC_RX_TL register. It is automatically cleared by hardware when buffer level goes below the threshold. If the module is disabled (IC_ENABLE[0]=0), the RX FIFO is flushed and held in reset; therefore the RX FIFO is not full. So this bit is cleared once the IC_ENABLE bit 0 is programmed with a 0, regardless of the activity that continues. Values: ■ 0x1 (ACTIVE): RX_FULL interrupt is active ■ 0x0 (INACTIVE): RX_FULL interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
1	RX_OVER	R	Set if the receive buffer is completely filled to IC_RX_BUFFER_DEPTH and an additional byte is received from an external I2C device. The DW_apb_i2c acknowledges this, but any data bytes received after the FIFO is full are lost. If the module is disabled (IC_ENABLE[0]=0), this bit keeps its level until the controller or target state machines go into idle, and when ic_en goes to 0, this interrupt is cleared. Note: If the configuration parameter IC_RX_FULL_HLD_BUS_EN is enabled and bit 9 of the IC_CON register (RX_FIFO_FULL_HLD_CTRL) is programmed to HIGH, then the RX_OVER interrupt never occurs, because the Rx FIFO never overflows. Values: ■ 0x1 (ACTIVE): RX_OVER interrupt is active ■ 0x0 (INACTIVE): RX_OVER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
0	RX_UNDER	R	Set if the processor attempts to read the receive buffer when it is empty by reading from the IC_DATA_CMD register. If the module is disabled (IC_ENABLE[0]=0), this bit keeps its level until the controller or target state machines go into idle, and when ic_en goes to 0, this interrupt is cleared. Values: ■ 0x1 (ACTIVE): RX_UNDER interrupt is active ■ 0x0 (INACTIVE): RX_UNDER interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

5.1.18 IC_RX_TL

- **Name:** I2C Receive FIFO Threshold Register
- **Description:** I2C Receive FIFO Threshold Register
- **Size:** 32 bits
- **Offset:** 0x38
- **Exists:** Always



Table 5-22 Fields for Register: IC_RX_TL

Bits	Name	Memory Access	Description
31:8	RSVD_IC_RX_TL	R	IC_RX_TL Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	RX_TL	R/W	Receive FIFO Threshold Level. Controls the level of entries (or above) that triggers the RX_FULL interrupt (bit 2 in IC_RAW_INTR_STAT register). The valid range is 0-255, with the additional restriction that hardware does not allow this value to be set to a value larger than the depth of the buffer. If an attempt is made to do that, the actual value set will be the maximum depth of the buffer. A value of 0 sets the threshold for 1 entry, and a value of 255 sets the threshold for 256 entries. Value After Reset: IC_RX_TL Exists: Always

5.1.19 IC_TX_TL

- **Name:** I2C Transmit FIFO Threshold Register
- **Description:** I2C Transmit FIFO Threshold Register
- **Size:** 32 bits
- **Offset:** 0x3c
- **Exists:** Always



Table 5-23 Fields for Register: IC_TX_TL

Bits	Name	Memory Access	Description
31:8	RSVD_IC_TX_TL	R	IC_TX_TL Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	TX_TL	R/W	Transmit FIFO Threshold Level. Controls the level of entries (or below) that trigger the TX_EMPTY interrupt (bit 4 in IC_RAW_INTR_STAT register). The valid range is 0-255, with the additional restriction that it may not be set to value larger than the depth of the buffer. If an attempt is made to do that, the actual value set will be the maximum depth of the buffer. A value of 0 sets the threshold for 0 entries, and a value of 255 sets the threshold for 255 entries. Value After Reset: IC_TX_TL Exists: Always

5.1.20 IC_CLR_INTR

- **Name:** Clear Combined and Individual Interrupt Register
- **Description:** Clear Combined and Individual Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x40
- **Exists:** Always

31:1	RSVD_IC_CLR_INTR
0	CLR_INTR

Table 5-24 Fields for Register: IC_CLR_INTR

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_INTR	R	CLR_INTR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_INTR	R	Read this register to clear the combined interrupt, all individual interrupts, and the IC_TX_ABRT_SOURCE register. This bit does not clear hardware clearable interrupts but software clearable interrupts. Refer to Bit 9 of the IC_TX_ABRT_SOURCE register for an exception to clearing IC_TX_ABRT_SOURCE. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.21 IC_CLR_RX_UNDER

- **Name:** Clear RX_UNDER Interrupt Register
- **Description:** Clear RX_UNDER Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x44
- **Exists:** Always

31:1	RSVD_IC_CLR_RX_UNDER
0	CLR_RX_UNDER

Table 5-25 Fields for Register: IC_CLR_RX_UNDER

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_RX_UNDER	R	IC_CLR_RX_UNDER Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_RX_UNDER	R	Read this register to clear the RX_UNDER interrupt (bit 0) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.22 IC_CLR_RX_OVER

- **Name:** Clear RX_OVER Interrupt Register
- **Description:** Clear RX_OVER Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x48
- **Exists:** Always

31:1	RSVD_IC_CLR_RX_OVER
0	CLR_RX_OVER

Table 5-26 Fields for Register: IC_CLR_RX_OVER

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_RX_OVER	R	IC_CLR_RX_OVER Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_RX_OVER	R	Read this register to clear the RX_OVER interrupt (bit 1) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.23 IC_CLR_TX_OVER

- **Name:** Clear TX_OVER Interrupt Register
- **Description:** Clear TX_OVER Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x4c
- **Exists:** Always

31:1	RSVD_IC_CLR_TX_OVER
0	CLR_TX_OVER

Table 5-27 Fields for Register: IC_CLR_TX_OVER

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_TX_OVER	R	IC_CLR_TX_OVER Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_TX_OVER	R	Read this register to clear the TX_OVER interrupt (bit 3) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.24 IC_CLR_RD_REQ

- **Name:** Clear RD_REQ Interrupt Register
- **Description:** Clear RD_REQ Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x50
- **Exists:** IC_ULTRA_FAST_MODE==0

31:1	RSVD_IC_CLR_RD_REQ
0	CLR_RD_REQ

Table 5-28 Fields for Register: IC_CLR_RD_REQ

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_RD_REQ	R	IC_CLR_RD_REQ Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_RD_REQ	R	Read this register to clear the RD_REQ interrupt (bit 5) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.25 IC_CLR_TX_ABRT

- **Name:** Clear TX_ABRT Interrupt Register
- **Description:** Clear TX_ABRT Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x54
- **Exists:** Always

31:1	RSVD_IC_CLR_TX_ABRT
0	CLR_TX_ABRT

Table 5-29 Fields for Register: IC_CLR_TX_ABRT

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_TX_ABRT	R	IC_CLR_TX_ABRT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_TX_ABRT	R	Read this register to clear the TX_ABRT interrupt (bit 6) of the IC_RAW_INTR_STAT register, and the IC_TX_ABRT_SOURCE register. This also releases the TX FIFO from the flushed/reset state, allowing more writes to the TX FIFO. Refer to Bit 9 of the IC_TX_ABRT_SOURCE register for an exception to clearing IC_TX_ABRT_SOURCE. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.26 IC_CLR_RX_DONE

- **Name:** Clear RX_DONE Interrupt Register
- **Description:** Clear RX_DONE Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x58
- **Exists:** IC_ULTRA_FAST_MODE==0

31:1	RSVD_IC_CLR_RX_DONE
0	CLR_RX_DONE

Table 5-30 Fields for Register: IC_CLR_RX_DONE

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_RX_DONE	R	IC_CLR_RX_DONE Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_RX_DONE	R	Read this register to clear the RX_DONE interrupt (bit 7) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.27 IC_CLR_ACTIVITY

- **Name:** Clear ACTIVITY Interrupt Register
- **Description:** Clear ACTIVITY Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x5c
- **Exists:** Always

31:1	RSVD_IC_CLR_ACTIVITY
0	CLR_ACTIVITY

Table 5-31 Fields for Register: IC_CLR_ACTIVITY

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_ACTIVITY	R	IC_CLR_ACTIVITY Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_ACTIVITY	R	Reading this register clears the ACTIVITY interrupt if the I2C is not active anymore. If the I2C module is still active on the bus, the ACTIVITY interrupt bit continues to be set. It is automatically cleared by hardware if the module is disabled and if there is no further activity on the bus. The value read from this register to get status of the ACTIVITY interrupt (bit 8) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.28 IC_CLR_STOP_DET

- **Name:** Clear STOP_DET Interrupt Register
- **Description:** Clear STOP_DET Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x60
- **Exists:** Always

31:1	RSVD_IC_CLR_STOP_DET
0	CLR_STOP_DET

Table 5-32 Fields for Register: IC_CLR_STOP_DET

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_STOP_DET	R	IC_CLR_STOP_DET Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_STOP_DET	R	Read this register to clear the STOP_DET interrupt (bit 9) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.29 IC_CLR_START_DET

- **Name:** Clear START_DET Interrupt Register
- **Description:** Clear START_DET Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x64
- **Exists:** Always



Table 5-33 Fields for Register: IC_CLR_START_DET

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_START_DET	R	IC_CLR_START_DET Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_START_DET	R	Read this register to clear the START_DET interrupt (bit 10) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.30 IC_CLR_GEN_CALL

- **Name:** Clear GEN_CALL Interrupt Register
- **Description:** Clear GEN_CALL Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x68
- **Exists:** Always



Table 5-34 Fields for Register: IC_CLR_GEN_CALL

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_GEN_CALL	R	IC_CLR_GEN_CALL Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_GEN_CALL	R	Read this register to clear the GEN_CALL interrupt (bit 11) of IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.31 IC_ENABLE

- **Name:** I2C ENABLE Register
- **Description:** I2C Enable Register
- **Size:** 32 bits
- **Offset:** 0x6c
- **Exists:** Always

RSVD_IC_ENABLE_2	31:26
IC_SAR4_SMBUS_ALERT_EN	25
IC_SAR3_SMBUS_ALERT_EN	24
IC_SAR2_SMBUS_ALERT_EN	23
IC_SAR4_EN	22
IC_SAR3_EN	21
IC_SAR2_EN	20
IC_SAR_EN	19
SMBUS_ALERT_EN	18
SMBUS_SUSPEND_EN	17
SMBUS_CLK_RESET	16
RSVD_IC_ENABLE_1	15:4
SDA_STUCK_RECOVERY_ENABLE	3
TX_CMD_BLOCK	2
ABORT	1
ENABLE	0

Table 5-35 Fields for Register: IC_ENABLE

Bits	Name	Memory Access	Description
31:26	RSVD_IC_ENABLE_2	R	IC_ENABLE Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
25	IC_SAR4_SMBUS_ALERT_EN	R/W	This register bit is used to control assertion of SMBALERT signal for IC_SAR4 Target Device Address. This register bit is auto-cleared after detection of Acknowledgment from controller for Alert Response address. Values: ■ 0x1 (ALERT_ENABLED): IC_SAR4 target initiates the Alert signal to indicate SMBus Host ■ 0x0 (SUSPEND_DISABLED): IC_SAR4 target will not initiate the Alert signal to indicate SMBus Host. Value After Reset: 0x0 Exists: (IC_MSAR_SMBUS_ALERT_EN == 1) && (IC_NUM_SARS >= 4)

Bits	Name	Memory Access	Description
24	IC_SAR3_SMBUS_ALERT_EN	R/W	This register bit is used to control assertion of SMBALERT signal for IC_SAR3 Target Device Address. This register bit is auto-cleared after detection of Acknowledgment from controller for Alert Response address. Values: ■ 0x1 (ALERT_ENABLED): IC_SAR3 target initiates the Alert signal to indicate SMBus Host ■ 0x0 (SUSPEND_DISABLED): IC_SAR3 target will not initiate the Alert signal to indicate SMBus Host. Value After Reset: 0x0 Exists: (IC_MSAR_SMBUS_ALERT_EN == 1) && (IC_NUM_SARS >= 3)
23	IC_SAR2_SMBUS_ALERT_EN	R/W	This register bit is used to control assertion of SMBALERT signal for IC_SAR2 Target Device Address. This register bit is auto-cleared after detection of Acknowledgment from controller for Alert Response address. Values: ■ 0x1 (ALERT_ENABLED): IC_SAR2 target initiates the Alert signal to indicate SMBus Host ■ 0x0 (SUSPEND_DISABLED): IC_SAR2 target will not initiate the Alert signal to indicate SMBus Host. Value After Reset: 0x0 Exists: (IC_MSAR_SMBUS_ALERT_EN == 1) && (IC_NUM_SARS >= 2)
22	IC_SAR4_EN	R/W	Controls whether the DW_apb_i2c SAR4 is enabled. If IC_SAR4_EN bit of the IC_ENABLE register is set to 1, then the DW_apb_i2c will acknowledge the address if it corresponds to IC_SAR4 and completes the I2C transfers as intended. Otherwise, the I2C transfer address will be Nacked. Note: This field needs to be updated only when IC_ENABLE[0] is 0. Values: ■ 0x1 (SLAVE_ENABLED): Target Device Address IC_SAR4 is enabled ■ 0x0 (SLAVE_DISABLED): Target Device Address IC_SAR4 is disabled Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4)

Bits	Name	Memory Access	Description
21	IC_SAR3_EN	R/W	Controls whether the DW_apb_i2c SAR3 is enabled. If IC_SAR3_EN bit of the IC_ENABLE register is set to 1, then the DW_apb_i2c will acknowledge the address if it corresponds to IC_SAR3 and completes the I2C transfers as intended. Otherwise, the I2C transfer address will be Nacked. Note: This field needs to be updated only when IC_ENABLE[0] is 0. Values: ■ 0x1 (SLAVE_ENABLED): Target Device Address IC_SAR3 is enabled ■ 0x0 (SLAVE_DISABLED): Target Device Address IC_SAR3 is disabled Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3)
20	IC_SAR2_EN	R/W	Controls whether the DW_apb_i2c SAR2 is enabled. If IC_SAR2_EN bit of the IC_ENABLE register is set to 1, then the DW_apb_i2c will acknowledge the address if it corresponds to IC_SAR2 and completes the I2C transfers as intended. Otherwise, the I2C transfer address will be Nacked. Note: This field needs to be updated only when IC_ENABLE[0] is 0. Values: ■ 0x1 (SLAVE_ENABLED): Target Device Address IC_SAR2 is enabled ■ 0x0 (SLAVE_DISABLED): Target Device Address IC_SAR2 is disabled Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2)
19	IC_SAR_EN	R/W	Controls whether the DW_apb_i2c SAR is enabled. If IC_SAR_EN bit of the IC_ENABLE register is set to 1, then the DW_apb_i2c will acknowledge the address if it corresponds to IC_SAR and completes the I2C transfers as intended. Otherwise, the I2C transfer address will be Nacked. Note: This field needs to be updated only when IC_ENABLE[0] is 0. Values: ■ 0x1 (SLAVE_ENABLED): Target Device Address IC_SAR is enabled ■ 0x0 (SLAVE_DISABLED): Target Device Address IC_SAR is disabled Value After Reset: 0x1 Exists: (IC_MULTI_SAR_EN == 1)

Bits	Name	Memory Access	Description
18	SMBUS_ALERT_EN	R/W	This register bit is used to control assertion of SMBALERT signal for IC_SAR target device address. This register bit is auto-cleared after detection of Acknowledgment from controller for Alert Response address. Values: ■ 0x1 (ALERT_ENABLED): IC_SAR target initiates the Alert signal to indicate SMBus Host ■ 0x0 (SUSPEND_DISABLED): IC_SAR target does not initiate the Alert signal to indicate SMBus Host. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1
17	SMBUS_SUSPEND_EN	R/W	The SMBUS_SUSPEND_EN register bit is used to control assertion and de-assertion of SMBSUS signal. ■ 0: De-assert SMBSUS signal ■ 1: Assert SMBSUS signal Values: ■ 0x1 (ENABLED): Host/Controller initiates the SMBUS system to enter Suspend Mode. ■ 0x0 (DISABLED): Host/Controller will not initiates the SMBUS system to enter Suspend Mode. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1
16	SMBUS_CLK_RESET	R/W	This bit is used in SMBus Host mode to initiate the SMBus Controller Clock Reset. This bit should be enabled only when Controller is in idle. Whenever this bit is enabled, the SMBCLK is held low for the IC_SCL_STUCK_TIMEOUT ic_clk cycles to reset the SMBus target devices. Values: ■ 0x1 (ENABLED): Controller initiates the SMBUS Clock Reset Mechanism. ■ 0x0 (DISABLED): Controller will not initiates SMBUS Clock Reset Mechanism. Value After Reset: 0x0 Exists: IC_SMBUS==1
15:4	RSVD_IC_ENABLE_1	R	RSVD_IC_ENABLE_1 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
3	SDA_STUCK_RECOVERY_ENABLE	R/W	If SDA is stuck at low indicated through the TX_ABORT interrupt (IC_TX_ABRT_SOURCE[17]), then this bit is used as a control knob to initiate the SDA Recovery Mechanism (that is, send at most 9 SCL clocks and STOP to release the SDA line) and then this bit gets auto clear Values: ■ 0x1 (SDA_STUCK_RECOVERY_ENABLED): Controller initiates the SDA stuck at low recovery mechanism. ■ 0x0 (SDA_STUCK_RECOVERY_DISABLED): Controller disabled the SDA stuck at low recovery mechanism. Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE==1
2	TX_CMD_BLOCK	R/W	In Controller mode: ■ 1'b1: Blocks the transmission of data on I2C bus even if Tx FIFO has data to transmit. ■ 1'b0: The transmission of data starts on I2C bus automatically, as soon as the first data is available in the Tx FIFO. Note: To block the execution of Controller commands, set the TX_CMD_BLOCK bit only when Tx FIFO is empty (IC_STATUS[2]==1) and Controller is in Idle state (IC_STATUS[5] == 0). Any further commands put in the Tx FIFO are not executed until TX_CMD_BLOCK bit is unset. Values: ■ 0x1 (BLOCKED): Tx Command execution blocked ■ 0x0 (NOT_BLOCKED): Tx Command execution not blocked Value After Reset: IC_TX_CMD_BLOCK_DEFAULT Exists: IC_TX_CMD_BLOCK==1

Bits	Name	Memory Access	Description
1	ABORT	R/W	When set, the controller initiates the transfer terminate. ■ 0: terminate not initiated or terminate done ■ 1: terminate operation in progress The software can terminate the I2C transfer in controller mode by setting this bit. The software can set this bit only when ENABLE is already set; otherwise, the controller ignores any write to ABORT bit. The software cannot clear the ABORT bit once set. In response to an terminate, the controller issues a STOP and flushes the Tx FIFO after completing the current transfer, then sets the TX_ABORT interrupt after the terminate operation. The ABORT bit is cleared automatically after the terminate operation. For a detailed description on how to terminate I2C transfers, refer to "Terminating I2C Transfers". Values: ■ 0x1 (ENABLED): Terminate operation in progress ■ 0x0 (DISABLE): Terminate operation not in progress Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
0	ENABLE	R/W	Controls whether the DW_apb_i2c is enabled. 0: Disables DW_apb_i2c (TX and RX FIFOs are held in an erased state) 1: Enables DW_apb_i2c Software can disable DW_apb_i2c while it is active. However, it is important that care be taken to ensure that DW_apb_i2c is disabled properly. A recommended procedure is described in "Disabling DW_apb_i2c". When DW_apb_i2c is disabled, the following occurs: - The TX FIFO and RX FIFO get flushed. - Status bits in the IC_INTR_STAT register are still active until DW_apb_i2c goes into IDLE state. If the module is transmitting, it stops as well as deletes the contents of the transmit buffer after the current transfer is complete. If the module is receiving, the DW_apb_i2c stops the current transfer at the end of the current transfer and does not acknowledge the transfer. In systems with asynchronous pclk and ic_clk when IC_CLK_TYPE parameter set to asynchronous (1), there is a two ic_clk delay when enabling or disabling the DW_apb_i2c. For a detailed description on how to disable DW_apb_i2c, refer to "Disabling DW_apb_i2c" Values: 0x1 (ENABLED): I2C is enabled 0x0 (DISABLED): I2C is disabled Value After Reset: 0x0 Exists: Always

5.1.32 IC_STATUS

- **Name:** I2C STATUS Register

- **Description:** I2C Status Register.

This is a read-only register used to indicate the current transfer status and FIFO status. The status register may be read at any time. None of the bits in this register request an interrupt.

When the I2C is disabled by writing 0 in bit 0 of the IC_ENABLE register:

- Bits 1 and 2 are set to 1
- Bits 3 and 10 are set to 0

When the controller or target state machines goes to idle and ic_en=0:

- Bits 5 and 6 are set to 0

- **Size:** 32 bits
- **Offset:** 0x70
- **Exists:** Always

31:27	RSVD_IC_STATUS_2
26	SMBUS_SLAVE_ADDR4_RESOLVED
25	SMBUS_SLAVE_ADDR4_VALID
24	SMBUS_SLAVE_ADDR3_RESOLVED
23	SMBUS_SLAVE_ADDR3_VALID
22	SMBUS_SLAVE_ADDR2_RESOLVED
21	SMBUS_SLAVE_ADDR2_VALID
20	SMBUS_ALERT_STATUS
19	SMBUS_SUSPEND_STATUS
18	SMBUS_SLAVE_ADDR_RESOLVED
17	SMBUS_SLAVE_ADDR_VALID
16	SMBUS_QUICK_CMD_BIT
15:13	RSVD_IC_STATUS_1
12	SLV_ISO_SAR_DATA_CLK_STRETCH
11	SDA_STUCK_NOT_RECOVERED
10	SLV_HOLD_RX_FIFO_FULL
9	SLV_HOLD_TX_FIFO_EMPTY
8	MST_HOLD_RX_FIFO_FULL
7	MST_HOLD_TX_FIFO_EMPTY
6	SLV_ACTIVITY
5	MST_ACTIVITY
4	RFF
3	RFNE
2	TFE
1	TFNF
0	ACTIVITY

Table 5-36 Fields for Register: IC_STATUS

Bits	Name	Memory Access	Description
31:27	RSVD_IC_STATUS_2	R	IC_STATUS Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
26	SMBUS_SLAVE_ADDR4_RESOLVED	R	This bit indicates whether the target device address 4 (IC_SAR4) is resolved by the ARP Controller. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address 4 is Resolved. ■ 0x0 (INACTIVE): SMBUS Target Device Address 4 is not Resolved. Value After Reset: 0x0 Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) Volatile: true
25	SMBUS_SLAVE_ADDR4_VALID	R	This bit indicates whether the target device address 4 (IC_SAR4) is valid or not. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address 4 is Valid. ■ 0x0 (INACTIVE): SMBUS Target Device Address 4 is not valid. Value After Reset: IC_PERSISTANT_SLV_ADDR4_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4) Volatile: true
24	SMBUS_SLAVE_ADDR3_RESOLVED	R	This bit indicates whether the target device address 3 (IC_SAR3) is resolved by the ARP Controller. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address 3 is Resolved. ■ 0x0 (INACTIVE): SMBUS Target Device Address 3 is not Resolved. Value After Reset: 0x0 Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) Volatile: true
23	SMBUS_SLAVE_ADDR3_VALID	R	This bit indicates whether the target device address 3 (IC_SAR3) is valid or not. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address 3 is Valid. ■ 0x0 (INACTIVE): SMBUS Target Device Address 3 is not valid. Value After Reset: IC_PERSISTANT_SLV_ADDR3_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3) Volatile: true

Bits	Name	Memory Access	Description
22	SMBUS_SLAVE_ADDR2_RESOLVED	R	This bit indicates whether the target device address 2 (IC_SAR2) is resolved by the ARP Controller. Values: ■ 0x1 (ACTIVE): SMBUS target device Address 2 is Resolved. ■ 0x0 (INACTIVE): SMBUS target device Address 2 is not Resolved. Value After Reset: 0x0 Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) Volatile: true
21	SMBUS_SLAVE_ADDR2_VALID	R	This bit indicates whether the target device address 2 (IC_SAR2) is valid or not. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address 2 is Valid. ■ 0x0 (INACTIVE): SMBUS Target Device Address 2 is not valid. Value After Reset: IC_PERSISTANT_SLV_ADDR2_DEFAULT Exists: (IC_SMBUS_ARP==1) && (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2) Volatile: true
20	SMBUS_ALERT_STATUS	R	This bit indicates the status of the SMBus Alert signal (ic_smbalert_in_n). This signal is asserted when the SMBus Alert signal is asserted by the SMBus Device. Values: ■ 0x1 (ACTIVE): SMBUS Alert is asserted. ■ 0x0 (INACTIVE): SMBUS Alert is not asserted. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true
19	SMBUS_SUSPEND_STATUS	R	This bit indicates the status of the SMBus Suspend signal (ic_smbsus_in_n). This signal is asserted when the SMBus Suspend signal is asserted by the SMBus Host. Values: ■ 0x1 (ACTIVE): SMBUS System is in Suspended mode. ■ 0x0 (INACTIVE): SMBUS System is not in Suspended mode. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true

Bits	Name	Memory Access	Description
18	SMBUS_SLAVE_ADDR_RESOLVED	R	This bit indicates whether the target device address (ic_sar) is resolved by the ARP Controller. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address is Resolved. ■ 0x0 (INACTIVE): SMBUS Target Device Address is not Resolved. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
17	SMBUS_SLAVE_ADDR_VALID	R	This bit indicates whether the target device address (ic_sar) is valid or not. Values: ■ 0x1 (ACTIVE): SMBUS Target Device Address is Valid. ■ 0x0 (INACTIVE): SMBUS Target Device Address is not valid. Value After Reset: IC_PERSISTANT_SLV_ADDR_DEFAULT Exists: IC_SMBUS_ARP==1 Volatile: true
16	SMBUS_QUICK_CMD_BIT	R	This bit indicates the R/W bit of the Quick command received. This bit will be cleared after the user has read this bit. Values: ■ 0x1 (ACTIVE): SMBUS QUICK CMD Read/write is set to 1. ■ 0x0 (INACTIVE): SMBUS QUICK CMD Read/write is set to 0. Value After Reset: 0x0 Exists: IC_SMBUS==1 Volatile: true
15:13	RSVD_IC_STATUS_1	R	RSVD_IC_STATUS_1 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
12	SLV_ISO_SAR_DATA_CLK_STRETCH	R	This bit indicates the Bus hold (clock stretching) in Target mode with Multiple SAR feature - for isolating the write transfer related data corresponding to one target address with another target address. Values: ■ 0x1 (ACTIVE): Target holds the bus to isolate current RX FIFO data from next transfer related to another target device address. ■ 0x0 (INACTIVE): Target is not holding the bus to isolate current RX FIFO data from next transfer related to another target device address. Value After Reset: 0x0 Exists: (IC_MULTI_SAR_EN == 1) && (IC_STAT_FOR_CLK_STRETCH == 1) Volatile: true
11	SDA_STUCK_NOT_RECOVERED	R	This bit indicates that SDA stuck at low is not recovered after the recovery mechanism. In Target mode, this register bit is not applicable. Values: ■ 0x1 (ACTIVE): SDA Stuck at low is not recovered after recovery mechanism. ■ 0x0 (INACTIVE): SDA Stuck at low is recovered after recovery mechanism. Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE==1 Volatile: true
10	SLV_HOLD_RX_FIFO_FULL	R	This bit indicates the BUS Hold in Target mode due to Rx FIFO is Full and an additional byte has been received (This kind of Bus hold is applicable if IC_RX_FULL_HLD_BUS_EN is set to 1 and bit 9 of the IC_CON register (RX_FIFO_FULL_HLD_CTRL) is programmed to HIGH). Values: ■ 0x1 (ACTIVE): Target holds the bus due to Rx FIFO is full ■ 0x0 (INACTIVE): Target is not holding the bus or Bus hold is not due to Rx FIFO is full Value After Reset: 0x0 Exists: IC_STAT_FOR_CLK_STRETCH == 1 Volatile: true

Bits	Name	Memory Access	Description
9	SLV_HOLD_TX_FIFO_EMPTY	R	This bit indicates the BUS Hold in Target mode for the Read request when the Tx FIFO is empty. The Bus is in hold until the Tx FIFO has data to Transmit for the read request. Values: ■ 0x1 (ACTIVE): Target holds the bus due to Tx FIFO is empty ■ 0x0 (INACTIVE): Target is not holding the bus or Bus hold is not due to Tx FIFO is empty Value After Reset: 0x0 Exists: IC_STAT_FOR_CLK_STRETCH == 1 Volatile: true
8	MST_HOLD_RX_FIFO_FULL	R	This bit indicates the BUS Hold in Controller mode due to Rx FIFO is Full and additional byte has been received (This kind of Bus hold is applicable if IC_RX_FULL_HLD_BUS_EN is set to 1 and bit 9 of the IC_CON register (RX_FIFO_FULL_HLD_CTRL) is programmed to HIGH). Values: ■ 0x1 (ACTIVE): Controller holds the bus due to Rx FIFO is full ■ 0x0 (INACTIVE): Controller is not holding the bus or Bus hold is not due to Rx FIFO is full Value After Reset: 0x0 Exists: IC_STAT_FOR_CLK_STRETCH == 1 Volatile: true
7	MST_HOLD_TX_FIFO_EMPTY	R	If the IC_EMPTYFIFO_HOLD_MASTER_EN parameter is set to 1, the DW_apb_i2c controller stalls the write transfer when Tx FIFO is empty, and the last byte does not have the Stop bit set. This bit indicates the BUS hold when the controller holds the bus because of the Tx FIFO being empty, and the previous transferred command does not have the Stop bit set. (This kind of Bus hold is applicable if IC_EMPTYFIFO_HOLD_MASTER_EN is set to 1). Values: ■ 0x1 (ACTIVE): Controller holds the bus due to Tx FIFO is empty ■ 0x0 (INACTIVE): Controller is not holding the bus or Bus hold is not due to Tx FIFO is empty Value After Reset: 0x0 Exists: IC_STAT_FOR_CLK_STRETCH == 1 Volatile: true

Bits	Name	Memory Access	Description
6	SLV_ACTIVITY	R	Target FSM Activity Status. When the Target Finite State Machine (FSM) is not in the IDLE state, this bit is set. Values: ■ 0x1 (ACTIVE): Target not idle ■ 0x0 (IDLE): Target is idle Value After Reset: 0x0 Exists: Always Volatile: true
5	MST_ACTIVITY	R	Controller FSM Activity Status. When the Controller Finite State Machine (FSM) is not in the IDLE state, this bit is set. Note: IC_STATUS[0]-that is, ACTIVITY bit-is the OR of SLV_ACTIVITY and MST_ACTIVITY bits. Values: ■ 0x1 (ACTIVE): Controller not idle ■ 0x0 (IDLE): Controller is idle Value After Reset: 0x0 Exists: Always Volatile: true
4	RFF	R	Receive FIFO Completely Full. When the receive FIFO is completely full, this bit is set. When the receive FIFO contains one or more empty location, this bit is cleared. Values: ■ 0x1 (FULL): Rx FIFO is full ■ 0x0 (NOT_FULL): Rx FIFO not full Value After Reset: 0x0 Exists: Always Volatile: true
3	RFNE	R	Receive FIFO Not Empty. This bit is set when the receive FIFO contains one or more entries; it is cleared when the receive FIFO is empty. Values: ■ 0x1 (NOT_EMPTY): Rx FIFO not empty ■ 0x0 (EMPTY): Rx FIFO is empty Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
2	TFE	R	Transmit FIFO Completely Empty. When the transmit FIFO is completely empty, this bit is set. When it contains one or more valid entries, this bit is cleared. This bit field does not request an interrupt. Values: ■ 0x1 (EMPTY): Tx FIFO is empty ■ 0x0 (NON_EMPTY): Tx FIFO not empty Value After Reset: 0x1 Exists: Always Volatile: true
1	TFNF	R	Transmit FIFO Not Full. Set when the transmit FIFO contains one or more empty locations, and is cleared when the FIFO is full. Values: ■ 0x1 (NOT_FULL): Tx FIFO not full ■ 0x0 (FULL): Tx FIFO is full Value After Reset: 0x1 Exists: Always Volatile: true
0	ACTIVITY	R	I2C Activity Status. Values: ■ 0x1 (ACTIVE): I2C is active ■ 0x0 (INACTIVE): I2C is idle Value After Reset: 0x0 Exists: Always Volatile: true

5.1.33 IC_TXFLR

- **Name:** I2C Transmit FIFO Level Register
- **Description:** I2C Transmit FIFO Level Register.
This register contains the number of valid data entries in the transmit FIFO buffer. It is cleared whenever:
 - The I2C is disabled
 - There is a transmit terminate - that is, TX_ABRT bit is set in the IC_RAW_INTR_STAT register
 - The target bulk transmit mode is terminated
 The register increments whenever data is placed into the transmit FIFO and decrements when data is taken from the transmit FIFO.
- **Size:** 32 bits
- **Offset:** 0x74
- **Exists:** Always

31:y	x:0
RSVD_TXFLR	TXFLR

Table 5-37 Fields for Register: IC_TXFLR

Bits	Name	Memory Access	Description
31:y	RSVD_TXFLR	R	TXFLR Register field Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[y]: TX_ABW_P1
x:0	TXFLR	R	Transmit FIFO Level. Contains the number of valid data entries in the transmit FIFO. Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[x]: TX_ABW_P1 - 1

5.1.34 IC_RXFLR

- **Name:** I2C Receive FIFO Level Register
- **Description:** I2C Receive FIFO Level Register.
This register contains the number of valid data entries in the receive FIFO buffer. It is cleared whenever:
 - The I2C is disabled
 - Whenever there is a transmit terminate caused by any of the events tracked in IC_TX_ABRT_SOURCE

The register increments whenever data is placed into the receive FIFO and decrements when data is taken from the receive FIFO.

- **Size:** 32 bits
- **Offset:** 0x78
- **Exists:** Always

31:y	RSVD_RXFLR
x:0	RXFLR

Table 5-38 Fields for Register: IC_RXFLR

Bits	Name	Memory Access	Description
31:y	RSVD_RXFLR	R	RXFLR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[y]: RX_ABW_P1
x:0	RXFLR	R	Receive FIFO Level. Contains the number of valid data entries in the receive FIFO. Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[x]: RX_ABW_P1 - 1

5.1.35 IC_SDA_HOLD

- **Name:** I2C SDA Hold Time Length Register
- **Description:** I2C SDA Hold Time Length Register.
 The bits [15:0] of this register are used to control the hold time of SDA during transmit in both target and controller mode (after SCL goes from HIGH to LOW).
 The bits [23:16] of this register are used to extend the SDA transition (if any) whenever SCL is HIGH in the receiver in either controller or target mode.
 Writes to this register succeed only when IC_ENABLE[0]=0.
 The values in this register are in units of ic_clk period. The value programmed in IC_SDA_TX_HOLD must be greater than the minimum hold time in each mode one cycle in controller mode, seven cycles in target mode for the value to be implemented.
 The programmed SDA hold time during transmit (IC_SDA_TX_HOLD) cannot exceed at any time the duration of the low part of scl. Therefore the programmed value cannot be larger than N_SCL_LOW-2, where N_SCL_LOW is the duration of the low part of the scl period measured in ic_clk cycles.
- **Size:** 32 bits
- **Offset:** 0x7c
- **Exists:** Always

31:24	RSVD_IC_SDA_HOLD
23:16	IC_SDA_RX_HOLD
15:0	IC_SDA_TX_HOLD

Table 5-39 Fields for Register: IC_SDA_HOLD

Bits	Name	Memory Access	Description
31:24	RSVD_IC_SDA_HOLD	R	IC_SDA_HOLD Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
23:16	IC_SDA_RX_HOLD	R/W	Sets the required SDA hold time in units of ic_clk period, when DW_apb_i2c acts as a receiver. Value After Reset: IC_DEFAULT_SDA_RX_HOLD Exists: Always

Bits	Name	Memory Access	Description
15:0	IC_SDA_TX_HOLD	R/W	Sets the required SDA hold time in units of ic_clk period, when DW_apb_i2c acts as a transmitter. Value After Reset: IC_DEFAULT_SDA_TX_HOLD Exists: Always

5.1.36 IC_TX_ABRT_SOURCE

- **Name:** I2C Transmit Terminate Source Register

- **Description:** I2C Transmit Terminate Source Register.

This register has 32 bits that indicate the source of the TX_ABRT bit. Except for Bit 9, this register is cleared whenever the IC_CLR_TX_ABRT register or the IC_CLR_INTR register is read. To clear bit 9, the source of the ABRT_SBYTE_NORSTRT must be fixed first; RESTART must be enabled (IC_CON[5]=1), the SPECIAL bit must be cleared (IC_TAR[11]), or the GC_OR_START bit must be cleared (IC_TAR[10]).

Once the source of the ABRT_SBYTE_NORSTRT is fixed, then this bit can be cleared in the same manner as other bits in this register. If the source of the ABRT_SBYTE_NORSTRT is not fixed before attempting to clear this bit, Bit 9 clears for one cycle and is then re-asserted.

- **Size:** 32 bits
- **Offset:** 0x80
- **Exists:** Always

31:23	TX_FLUSH_CNT
22:21	RSVD_IC_TX_ABRT_SOURCE
20	ABRT_DEVICE_WRITE
19	ABRT_DEVICE_SLVADDR_NOACK
18	ABRT_DEVICE_NOACK
17	ABRT_SDA_STUCK_AT_LOW
16	ABRT_USER_ABRT
15	ABRT_SLVRD_INTX
14	ABRT_SLV_ARBLOST
13	ABRT_SLVFLUSH_TXFIFO
12	ARB_LOST
11	ABRT_MASTER_DIS
10	ABRT_10B_RD_NORSTRT
9	ABRT_SBYTE_NORSTRT
8	ABRT_HS_NORSTRT
7	ABRT_SBYTE_ACKDET
6	ABRT_HS_ACKDET
5	ABRT_GCALL_READ
4	ABRT_GCALL_NOACK
3	ABRT_TXDATA_NOACK
2	ABRT_10ADDR2_NOACK
1	ABRT_10ADDR1_NOACK
0	ABRT_7B_ADDR_NOACK

Table 5-40 Fields for Register: IC_TX_ABRT_SOURCE

Bits	Name	Memory Access	Description
31:23	TX_FLUSH_CNT	R	This field indicates the number of Tx FIFO Data Commands which are flushed due to TX_ABRT interrupt. It is cleared whenever I2C is disabled. Role of DW_apb_i2c: Controller-Transmitter or Target-Transmitter Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
22:21	RSVD_IC_TX_ABRT_SOURCE	R	IC_TX_ABRT_SOURCE Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
20	ABRT_DEVICE_WRITE	R	This is a controller-mode-only bit. Controller is initiating the DEVICE_ID transfer and the Tx-FIFO consists of write commands. Role of DW_apb_i2c: Controller Values: ■ 0x1 (ACTIVE): This terminate is generated because of NOACK for target device address ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_DEVICE_ID == 1 Volatile: true
19	ABRT_DEVICE_SLVADDR_NOACK	R	This is a controller-mode-only bit. Controller is initiating the DEVICE_ID transfer and the target address sent was not acknowledged by any target. Role of DW_apb_i2c: Controller Values: ■ 0x1 (ACTIVE): This terminate is generated because of NOACK for target device address ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_DEVICE_ID == 1 Volatile: true
18	ABRT_DEVICE_NOACK	R	This is a controller-mode-only bit. Controller is initiating the DEVICE_ID transfer and the device id sent was not acknowledged by any target. Role of DW_apb_i2c: Controller Values: ■ 0x1 (ACTIVE): This terminate is generated because of NOACK for DEVICE-ID ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_DEVICE_ID == 1 Volatile: true

Bits	Name	Memory Access	Description
17	ABRT_SDA_STUCK_AT_LOW	R	This is a controller-mode-only bit. Controller detects the SDA Stuck at low for the IC_SDA_STUCK_AT_LOW_TIMEOUT value of ic_clks. Role of DW_apb_i2c:Controller Values: ■ 0x1 (ACTIVE): This terminate is generated because of Sda stuck at low for IC_SDA_STUCK_AT_LOW_TIMEOUT value of ic_clks ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_BUS_CLEAR_FEATURE == 1 Volatile: true
16	ABRT_USER_ABRT	R	This is a controller-mode-only bit. Controller has detected the transfer terminate (IC_ENABLE[1]) Role of DW_apb_i2c:Controller-Transmitter Values: ■ 0x1 (ABRT_USER_ABRT_GENERATED): Transfer terminate detected by controller ■ 0x0 (ABRT_USER_ABRT_VOID): Transfer terminate detected by controller- scenario not present Value After Reset: 0x0 Exists: Always Volatile: true
15	ABRT_SLVRD_INTX	R	When the processor side responds to a target mode request for data to be transmitted to a remote controller and user writes a 1 in CMD (bit 8) of IC_DATA_CMD register. Role of DW_apb_i2c:Target-Transmitter Values: ■ 0x1 (ABRT_SLVRD_INTX_GENERATED): Target trying to transmit to remote controller in read mode ■ 0x0 (ABRT_SLVRD_INTX_VOID): Target trying to transmit to remote controller in read mode- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
14	ABRT_SLV_ARBLOST	R	This field indicates that a Target has lost the bus while transmitting data to a remote controller. IC_TX_ABRT_SOURCE[12] is set at the same time. Note: Even though the target never 'owns' the bus, something could go wrong on the bus. This is a fail safe check. For instance, during a data transmission at the low-to-high transition of SCL, if what is on the data bus is not what is supposed to be transmitted, then DW_apb_i2c no longer own the bus. Role of DW_apb_i2c: Target-Transmitter Values: ■ 0x1 (ABRT_SLV_ARBLOST_GENERATED): Target lost arbitration to remote controller ■ 0x0 (ABRT_SLV_ARBLOST_VOID): Target lost arbitration to remote controller- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
13	ABRT_SLVFLUSH_TXFIFO	R	This field specifies that the Target has received a read command and some data exists in the TX FIFO, so the target issues a TX_ABRT interrupt to flush old data in TX FIFO. Role of DW_apb_i2c: Target-Transmitter Values: ■ 0x1 (ABRT_SLVFLUSH_TXFIFO_GENERATED): Target flushes existing data in TX-FIFO upon getting read command ■ 0x0 (ABRT_SLVFLUSH_TXFIFO_VOID): Target flushes existing data in TX-FIFO upon getting read command- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
12	ARB_LOST	R	This field specifies that the Controller has lost arbitration, or if IC_TX_ABRT_SOURCE[14] is also set, then the target transmitter has lost arbitration. Role of DW_apb_i2c: Controller-Transmitter or Target-Transmitter Values: ■ 0x1 (ARB_LOST_GENERATED): Controller or Target-Transmitter lost arbitration ■ 0x0 (ARB_LOST_VOID): Controller or Target-Transmitter lost arbitration- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
11	ABRT_MASTER_DIS	R	This field indicates that the User tries to initiate a Controller operation with the Controller mode disabled. Role of DW_apb_i2c: Controller-Transmitter or Controller-Receiver Values: ■ 0x1 (ABRT_MASTER_DIS_GENERATED): User initiating controller operation when CONTROLLER disabled ■ 0x0 (ABRT_MASTER_DIS_VOID): User initiating controller operation when CONTROLLER disabled- scenario not present Value After Reset: 0x0 Exists: Always Volatile: true
10	ABRT_10B_RD_NORSTRT	R	This field indicates that the restart is disabled (IC_RESTART_EN bit (IC_CON[5]) =0) and the controller sends a read command in 10-bit addressing mode. Role of DW_apb_i2c: Controller-Receiver Values: ■ 0x1 (ABRT_10B_RD_GENERATED): Controller trying to read in 10Bit addressing mode when RESTART disabled ■ 0x0 (ABRT_10B_RD_VOID): Controller not trying to read in 10Bit addressing mode when RESTART disabled Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
9	ABRT_SBYTE_NORSTRT	R	To clear Bit 9, the source of the ABRT_SBYTE_NORSTRT must be fixed first; restart must be enabled (IC_CON[5]=1), the SPECIAL bit must be cleared (IC_TAR[11]), or the GC_OR_START bit must be cleared (IC_TAR[10]). Once the source of the ABRT_SBYTE_NORSTRT is fixed, then this bit can be cleared in the same manner as other bits in this register. If the source of the ABRT_SBYTE_NORSTRT is not fixed before attempting to clear this bit, bit 9 clears for one cycle and then gets reasserted. When this field is set to 1, the restart is disabled (IC_RESTART_EN bit (IC_CON[5]) =0) and the user is trying to send a START Byte. Role of DW_apb_i2c: Controller Values: ■ 0x1 (ABRT_SBYTE_NORSTRT_GENERATED): User trying to send START byte when RESTART disabled ■ 0x0 (ABRT_SBYTE_NORSTRT_VOID): User trying to send START byte when RESTART disabled- scenario not present Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
8	ABRT_HS_NORSTRT	R	This field indicates that the restart is disabled (IC_RESTART_EN bit (IC_CON[5]) =0) and the user is trying to use the controller to transfer data in High Speed mode. Role of DW_apb_i2c:Controller-Transmitter or Controller-Receiver Values: ■ 0x1 (ABRT_HS_NORSTRT_GENERATED): User trying to switch Controller to HS mode when RESTART disabled ■ 0x0 (ABRT_HS_NORSTRT_VOID): User trying to switch Controller to HS mode when RESTART disabled- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
7	ABRT_SBYTE_ACKDET	R	This field indicates that the Controller has sent a START Byte and the START Byte was acknowledged (wrong behavior). Role of DW_apb_i2c:Controller Values: ■ 0x1 (ABRT_SBYTE_ACKDET_GENERATED): ACK detected for START byte ■ 0x0 (ABRT_SBYTE_ACKDET_VOID): ACK detected for START byte- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
6	ABRT_HS_ACKDET	R	This field indicates that the Controller is in High Speed mode and the High Speed Controller code was acknowledged (wrong behavior). Role of DW_apb_i2c:Controller Values: ■ 0x1 (ABRT_HS_ACK_GENERATED): HS Controller code ACKed in HS Mode ■ 0x0 (ABRT_HS_ACK_VOID): HS Controller code ACKed in HS Mode- scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
5	ABRT_GCALL_READ	R	This field indicates that DW_apb_i2c in the controller mode has sent a General Call but the user programmed the byte following the General Call to be a read from the bus (IC_DATA_CMD[9] is set to 1). Role of DW_apb_i2c: Controller-Transmitter Values: ■ 0x1 (ABRT_GCALL_READ_GENERATED): GCALL is followed by read from bus ■ 0x0 (ABRT_GCALL_READ_VOID): GCALL is followed by read from bus-scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
4	ABRT_GCALL_NOACK	R	This field indicates that DW_apb_i2c in controller mode has sent a General Call and no target on the bus acknowledged the General Call. Role of DW_apb_i2c: Controller-Transmitter Values: ■ 0x1 (ABRT_GCALL_NOACK_GENERATED): GCALL not ACKed by any target ■ 0x0 (ABRT_GCALL_NOACK_VOID): GCALL not ACKed by any target-scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
3	ABRT_TXDATA_NOACK	R	This field indicates the controller-mode only bit. When the controller receives an acknowledgment for the address, but when it sends data byte(s) following the address, it did not receive an acknowledge from the remote target(s). Role of DW_apb_i2c: Controller-Transmitter Values: ■ 0x1 (ABRT_TXDATA_NOACK_GENERATED): Transmitted data not ACKed by addressed target ■ 0x0 (ABRT_TXDATA_NOACK_VOID): Transmitted data non-ACKed by addressed target-scenario not present Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

Bits	Name	Memory Access	Description
2	ABRT_10ADDR2_NOACK	R	This field indicates that the Controller is in 10-bit address mode and that the second address byte of the 10-bit address was not acknowledged by any target. Role of DW_apb_i2c: Controller-Transmitter or Controller-Receiver Values: ■ 0x1 (ACTIVE): Byte 2 of 10Bit Address not ACKed by any target ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
1	ABRT_10ADDR1_NOACK	R	This field indicates that the Controller is in 10-bit address mode and the first 10-bit address byte was not acknowledged by any target. Role of DW_apb_i2c: Controller-Transmitter or Controller-Receiver Values: ■ 0x1 (ACTIVE): Byte 1 of 10Bit Address not ACKed by any target ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true
0	ABRT_7B_ADDR_NOACK	R	This field indicates that the Controller is in 7-bit addressing mode and the address sent was not acknowledged by any target. Role of DW_apb_i2c: Controller-Transmitter or Controller-Receiver Values: ■ 0x1 (ACTIVE): This terminate is generated because of NOACK for 7-bit address ■ 0x0 (INACTIVE): This terminate is not generated Value After Reset: 0x0 Exists: IC_ULTRA_FAST_MODE==0 Volatile: true

5.1.37 IC_SLV_DATA_NACK_ONLY

- **Name:** Generate Target Data NACK Register
- **Description:** Generate Target Data NACK Register
The register is used to generate a NACK for the data part of a transfer when DW_apb_i2c is acting as a target-receiver. This register only exists when the IC_SLV_DATA_NACK_ONLY parameter is set to 1. When this parameter disabled, this register does not exist and writing to the register's address has no effect.
A write can occur on this register if one of the following conditions are met:
 - DW_apb_i2c is disabled (IC_ENABLE[0] = 0)
 - Target part is inactive (IC_STATUS[6] = 0)**Note:** The IC_STATUS[6] is a register read-back location for the internal slv_activity signal; the user should poll this before writing the ic_slv_data_nack_only bit.
- **Size:** 32 bits
- **Offset:** 0x84
- **Exists:** [<functionof> "(IC_SLV_DATA_NACK_ONLY==0) ? 0 : 1"]

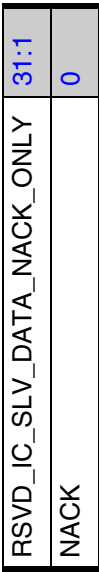


Table 5-41 Fields for Register: IC_SLV_DATA_NACK_ONLY

Bits	Name	Memory Access	Description
31:1	RSVD_IC_SLV_DATA_NACK_ONLY	R	IC_SLV_DATA_NACK_ONLY Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
0	NACK	R/W	Generate NACK. This NACK generation only occurs when DW_apb_i2c is a target-receiver. If this register is set to a value of 1, it can only generate a NACK after a data byte is received; hence, the data transfer is terminated and the data received is not pushed to the receive buffer. When the register is set to a value of 0, it generates NACK/ACK, depending on normal criteria. ■ 1: generate NACK after data byte received ■ 0: generate NACK/ACK normally Values: ■ 0x1 (ENABLED): Target receiver generates NACK upon data reception only ■ 0x0 (DISABLED): Target receiver generates NACK normally Value After Reset: 0x0 Exists: Always

5.1.38 IC_DMA_CR

- **Name:** DMA Control Register
- **Description:** DMA Control Register.
This register is only valid when DW_apb_i2c is configured with a set of DMA Controller interface signals (IC_HAS_DMA = 1). When DW_apb_i2c is not configured for DMA operation, this register does not exist and writing to the register's address has no effect and reading from this register address will return zero. The register is used to enable the DMA Controller interface operation. There is a separate bit for transmit and receive. This can be programmed regardless of the state of IC_ENABLE.
- **Size:** 32 bits
- **Offset:** 0x88
- **Exists:** [`<functionof> "(IC_HAS_DMA==1) ? 1 : 0"`]

31:2	1	0
RSVD_IC_DMA_CR_2_31	TDMAE	RDMAE

Table 5-42 Fields for Register: IC_DMA_CR

Bits	Name	Memory Access	Description
31:2	RSVD_IC_DMA_CR_2_31	R	RSVD_IC_DMA_CR_2_31 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
1	TDMAE	R/W	Transmit DMA Enable. This bit enables/disables the transmit FIFO DMA channel. Values: ■ 0x1 (ENABLED): Transmit FIFO DMA channel enabled ■ 0x0 (DISABLED): transmit FIFO DMA channel disabled Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
0	RDMAE	R/W	Receive DMA Enable. This bit enables/disables the receive FIFO DMA channel. Values: ■ 0x1 (ENABLED): Receive FIFO DMA channel enabled ■ 0x0 (DISABLED): Receive FIFO DMA channel disabled Value After Reset: 0x0 Exists: Always

5.1.39 IC_DMA_TDLR

- **Name:** DMA Transmit Data Level Register
- **Description:** DMA Transmit Data Level Register.
This register is only valid when the DW_apb_i2c is configured with a set of DMA interface signals (IC_HAS_DMA = 1). When DW_apb_i2c is not configured for DMA operation, this register does not exist; writing to its address has no effect; reading from its address returns zero.
- **Size:** 32 bits
- **Offset:** 0x8c
- **Exists:** IC_HAS_DMA==1

31:y	x:0
RSVD_DMA_TDLR	DMATDL

Table 5-43 Fields for Register: IC_DMA_TDLR

Bits	Name	Memory Access	Description
31:y	RSVD_DMA_TDLR	R	DMA_TDLR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Range Variable[y]: TX_ABW
x:0	DMATDL	R/W	Transmit Data Level. This bit field controls the level at which a DMA request is made by the transmit logic. It is equal to the watermark level; that is, the dma_tx_req signal is generated when the number of valid data entries in the transmit FIFO is equal to or below this field value, and TDMAE = 1. Value After Reset: 0x0 Exists: Always Range Variable[x]: TX_ABW - 1

5.1.40 IC_DMA_RDLR

- **Name:** DMA Receive Data Level Register
- **Description:** I2C Receive Data Level Register.
This register is only valid when DW_apb_i2c is configured with a set of DMA interface signals (IC_HAS_DMA = 1). When DW_apb_i2c is not configured for DMA operation, this register does not exist; writing to its address has no effect; reading from its address returns zero.
- **Size:** 32 bits
- **Offset:** 0x90
- **Exists:** [<functionof> "(IC_HAS_DMA==1) ? 1 : 0"]

31:y	x:0
RSVD_DMA_RDLR	DMARDL

Table 5-44 Fields for Register: IC_DMA_RDLR

Bits	Name	Memory Access	Description
31:y	RSVD_DMA_RDLR	R	DMA_RDLR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Range Variable[y]: RX_ABW
x:0	DMARDL	R/W	Receive Data Level. This bit field controls the level at which a DMA request is made by the receive logic. The watermark level = DMARDL+1; that is, dma_rx_req is generated when the number of valid data entries in the receive FIFO is equal to or more than this field value + 1, and RDMAE =1. For instance, when DMARDL is 0, then dma_rx_req is asserted when 1 or more data entries are present in the receive FIFO. Value After Reset: 0x0 Exists: Always Range Variable[x]: RX_ABW - 1

5.1.41 IC_SDA_SETUP

- **Name:** I2C SDA Setup Register
- **Description:** I2C SDA Setup Register.
This register controls the amount of time delay (in terms of number of ic_clk clock periods) introduced in the rising edge of SCL - relative to SDA changing - when DW_apb_i2c services a read request in a target-transmitter operation. The relevant I2C requirement is tSU:DAT as detailed in the I2C Bus Specification. This register must be programmed with a value greater than or equal to one ic_clock_period of time.
Writes to this register succeed only when IC_ENABLE[0] = 0.
Note: The length of setup time is calculated using below formula:
- If IC_CLK_FREQ_OPTIMIZATION = 0 : [(IC_SDA_SETUP - 1) * (ic_clock_period)]
- If IC_CLK_FREQ_OPTIMIZATION = 1 : [(IC_SDA_SETUP + 1) * (ic_clock_period)]
The IC_SDA_SETUP register is only used by the DW_apb_i2c when operating as a target transmitter.
- **Size:** 32 bits
- **Offset:** 0x94
- **Exists:** IC_ULTRA_FAST_MODE==0

31:8	RSVD_IC_SDA_SETUP
7:0	SDA_SETUP

Table 5-45 Fields for Register: IC_SDA_SETUP

Bits	Name	Memory Access	Description
31:8	RSVD_IC_SDA_SETUP	R	IC_SDA_SETUP Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	SDA_SETUP	R/W	SDA Setup (tSU:DAT). It is recommended that if the required delay is 1000ns, then for an ic_clk frequency of 10 MHz: - If IC_CLK_FREQ_OPTIMIZATION = 0 : IC_SDA_SETUP should be programmed to a value of 11. - If IC_CLK_FREQ_OPTIMIZATION = 1 : IC_SDA_SETUP should be programmed to a value of 9. Value After Reset: IC_DEFAULT_SDA_SETUP Exists: Always

5.1.42 IC_ACK_GENERAL_CALL

- **Name:** I2C ACK General Call Register
- **Description:** I2C ACK General Call Register.
The register controls whether DW_apb_i2c responds with a ACK or NACK when it receives an I2C General Call address.
This register is applicable only when the DW_apb_i2c is in target mode.
- **Size:** 32 bits
- **Offset:** 0x98
- **Exists:** IC_ULTRA_FAST_MODE==0

31:1	0
RSVD_IC_ACK_GEN_1_31	ACK_GEN_CALL

Table 5-46 Fields for Register: IC_ACK_GENERAL_CALL

Bits	Name	Memory Access	Description
31:1	RSVD_IC_ACK_GEN_1_31	R	RSVD_IC_ACK_GEN_1_31 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
0	ACK_GEN_CALL	R/W	ACK General Call. When set to 1, DW_apb_i2c responds with a ACK (by asserting ic_data_oe) when it receives a General Call. Otherwise, DW_apb_i2c responds with a NACK (by negating ic_data_oe). Values: ■ 0x1 (ENABLED): Generate ACK for a General Call ■ 0x0 (DISABLED): Generate NACK for General Call Value After Reset: IC_DEFAULT_ACK_GENERAL_CALL Exists: Always

5.1.43 IC_ENABLE_STATUS

- **Name:** I2C Enable Status Register
- **Description:** I2C Enable Status Register.
The register is used to report the DW_apb_i2c hardware status when the IC_ENABLE[0] register is set from 1 to 0; that is, when DW_apb_i2c is disabled.
If IC_ENABLE[0] has been set to 1, bits 2:1 are forced to 0, and bit 0 is forced to 1.
If IC_ENABLE[0] has been set to 0, bits 2:1 is only be valid as soon as bit 0 is read as '0'.
Note: When IC_ENABLE[0] has been set to 0, a delay occurs for bit 0 to be read as 0 because disabling the DW_apb_i2c depends on I2C bus activities.
- **Size:** 32 bits
- **Offset:** 0x9c
- **Exists:** Always

31:3	RSVD_IC_ENABLE_STATUS
2	SLV_RX_DATA_LOST
1	SLV_DISABLED_WHILE_BUSY
0	IC_EN

Table 5-47 Fields for Register: IC_ENABLE_STATUS

Bits	Name	Memory Access	Description
31:3	RSVD_IC_ENABLE_STATUS	R	IC_ENABLE_STATUS Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
2	SLV_RX_DATA_LOST	R	Target Received Data Lost. This bit indicates if a Target-Receiver operation has been terminated with at least one data byte received from an I2C transfer due to the setting bit 0 of IC_ENABLE from 1 to 0. When read as 1, DW_apb_i2c is deemed to have been actively engaged in an terminated I2C transfer (with matching address) and the data phase of the I2C transfer has been entered, even though a data byte has been responded with a NACK. Note: If the remote I2C controller terminates the transfer with a STOP condition before the DW_apb_i2c has a chance to NACK a transfer, and IC_ENABLE[0] has been set to 0, then this bit is also set to 1. When read as 0, DW_apb_i2c is deemed to have been disabled without being actively involved in the data phase of a Target-Receiver transfer. Note: The CPU can safely read this bit when IC_EN (bit 0) is read as 0. Values: ■ 0x1 (ACTIVE): Target RX Data is lost ■ 0x0 (INACTIVE): Target RX Data is not lost Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
1	SLV_DISABLED_WHILE_BUSY	R	Target Disabled While Busy (Transmit, Receive). This bit indicates if a potential or active Target operation has been terminated due to the setting bit 0 of the IC_ENABLE register from 1 to 0. This bit is set when the CPU writes a 0 to the IC_ENABLE register while: (a) DW_apb_i2c is receiving the address byte of the Target-Transmitter operation from a remote controller; OR, (b) address and data bytes of the Target-Receiver operation from a remote controller. When read as 1, DW_apb_i2c is deemed to have forced a NACK during any part of an I2C transfer, irrespective of whether the I2C address matches the target address set in DW_apb_i2c (IC_SAR register) OR if the transfer is completed before IC_ENABLE is set to 0 but has not taken effect. Note:If the remote I2C controller terminates the transfer with a STOP condition before the DW_apb_i2c has a chance to NACK a transfer, and IC_ENABLE[0] has been set to 0, then this bit will also be set to 1. When read as 0, DW_apb_i2c is deemed to have been disabled when there is controller activity, or when the I2C bus is idle. Note:The CPU can safely read this bit when IC_EN (bit 0) is read as 0. Values: ■ 0x1 (ACTIVE): Target is disabled when it is active ■ 0x0 (INACTIVE): Target is disabled when it is idle Value After Reset: 0x0 Exists: Always Volatile: true
0	IC_EN	R	IC Enable Status. This bit always reflects the value driven on the output port ic_en. ■ When read as 1, DW_apb_i2c is deemed to be in an enabled state. ■ When read as 0, DW_apb_i2c is deemed completely inactive. Note:The CPU can safely read this bit anytime. When this bit is read as 0, the CPU can safely read SLV_RX_DATA_LOST (bit 2) and SLV_DISABLED_WHILE_BUSY (bit 1). Values: ■ 0x1 (ENABLED): I2C enabled ■ 0x0 (DISABLED): I2C disabled Value After Reset: 0x0 Exists: Always Volatile: true

5.1.44 IC_FS_SPKLEN

- **Name:** I2C SS, FS or FM+ spike suppression limit
- **Description:** I2C SS, FS or FM+ Spike Suppression Limit.
This register is used to store the duration, measured in ic_clk cycles, of the longest spike that is filtered out by the spike suppression logic when the component is operating in SS, FS or FM+ modes. The relevant I2C requirement is tSP (table 4) as detailed in the I2C Bus Specification. This register must be programmed with a minimum value of 1.
- **Size:** 32 bits
- **Offset:** 0xa0
- **Exists:** IC_ULTRA_FAST_MODE==0

31:8	RSVD_IC_FS_SPKLEN
7:0	IC_FS_SPKLEN

Table 5-48 Fields for Register: IC_FS_SPKLEN

Bits	Name	Memory Access	Description
31:8	RSVD_IC_FS_SPKLEN	R	IC_FS_SPKLEN Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	IC_FS_SPKLEN	R/W	This register must be set before any I2C bus transaction can take place to ensure stable operation. This register sets the duration, measured in ic_clk cycles, of the longest spike in the SCL or SDA lines that will be filtered out by the spike suppression logic. This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is 1; hardware prevents values less than this being written, and if attempted results in 1 being set. or more information, refer to "Spike Suppression". Value After Reset: IC_DEFAULT_FS_SPKLEN Exists: Always

5.1.45 IC_UFM_SPKLEN

- **Name:** I2C Ultra-Fast mode spike suppression limit
- **Description:** I2C UFM spike suppression limit.
This register is used to store the duration, measured in ic_clk cycles, of the longest spike that is filtered out by the spike suppression logic when the component is operating in Ultra-Fast mode. The relevant I2C requirement is tSP (table 13) as detailed in the I2C Bus Specification. This register must be programmed with a minimum value of 1.
- **Size:** 32 bits
- **Offset:** 0xa0
- **Exists:** IC_ULTRA_FAST_MODE==1

31:8	RSVD_IC_UFM_SPKLEN
7:0	IC_UFM_SPKLEN

Table 5-49 Fields for Register: IC_UFM_SPKLEN

Bits	Name	Memory Access	Description
31:8	RSVD_IC_UFM_SPKLEN	R	IC_UFM_SPKLEN Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	IC_UFM_SPKLEN	R/W	This register must be set before any I2C bus transaction can take place to ensure stable operation. This register sets the duration, measured in ic_clk cycles, of the longest spike in the SCL or SDA lines that will be filtered out by the spike suppression logic. This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is 1; hardware prevents values less than this being written, and if attempted results in 1 being set. Value After Reset: IC_DEFAULT_UFM_SPKLEN Exists: Always

5.1.46 IC_HS_SPKLEN

- **Name:** I2C HS spike suppression limit register
- **Description:** I2C HS spike suppression limit register.
This register is used to store the duration, measured in ic_clk cycles, of the longest spike that is filtered out by the spike suppression logic when the component is operating in HS modes. The relevant I2C requirement is tSP (table 6) as detailed in the I2C Bus Specification. This register must be programmed with a minimum value of 1 and is implemented only if the component is configured to support HS mode; that is, if the IC_MAX_SPEED_MODE parameter is set to 3.
- **Size:** 32 bits
- **Offset:** 0xa4
- **Exists:** IC_HIGHSPEED_MODE_EN

31:8	RSVD_IC_HS_SPKLEN
7:0	IC_HS_SPKLEN

Table 5-50 Fields for Register: IC_HS_SPKLEN

Bits	Name	Memory Access	Description
31:8	RSVD_IC_HS_SPKLEN	R	IC_HS_SPKLEN Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
7:0	IC_HS_SPKLEN	R/W	This register must be set before any I2C bus transaction can take place to ensure stable operation. This register sets the duration, measured in ic_clk cycles, of the longest spike in the SCL or SDA lines that will be filtered out by the spike suppression logic; for more information, refer to "Spike Suppression". This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. The minimum valid value is 1; hardware prevents values less than this being written, and if attempted results in 1 being set. Value After Reset: IC_DEFAULT_HS_SPKLEN Exists: Always

5.1.47 IC_CLR_RESTART_DET

- **Name:** Clear RESTART_DET Interrupt Register
- **Description:** Clear RESTART_DET Interrupt Register
- **Size:** 32 bits
- **Offset:** 0xa8
- **Exists:** IC_SLV_RESTART_DET_EN == 1

31:1	RSVD_IC_CLR_RESTART_DET
0	CLR_RESTART_DET

Table 5-51 Fields for Register: IC_CLR_RESTART_DET

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_RESTART_DET	R	IC_CLR_RESTART_DET Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_RESTART_DET	R	Read this register to clear the RESTART_DET interrupt (bit 12) of IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.48 IC_SCL_STUCK_AT_LOW_TIMEOUT

- **Name:** I2C SCL Stuck at Low Timeout register
- **Description:** I2C SCL Stuck at Low Timeout.
This register is used to store the duration, measured in ic_clk cycles, used to generate an Interrupt (ic_scl_stuck_at_low_intr/ic_scl_stuck_at_low_intr_n) if SCL is held low for the IC_SCL_STUCK_LOW_TIMEOUT duration.
- **Size:** 32 bits
- **Offset:** 0xac
- **Exists:** IC_BUS_CLEAR_FEATURE==1



Table 5-52 Fields for Register: IC_SCL_STUCK_AT_LOW_TIMEOUT

Bits	Name	Memory Access	Description
31:0	IC_SCL_STUCK_LOW_TIMEOUT	R/W	DW_apb_i2c generate the interrupt to indicate SCL stuck at low (ic_scl_stuck_at_low_intr/ic_scl_stuck_at_low_intr_n) if it detects the SCL stuck at low for the IC_SCL_STUCK_LOW_TIMEOUT in units of ic_clk period. This register can be written only when the I2C interface is disabled which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Value After Reset: IC_SCL_STUCK_TIMEOUT_DEFAULT Exists: Always

5.1.49 IC_SDA_STUCK_AT_LOW_TIMEOUT

- **Name:** I2C SDA Stuck at Low Timeout register
- **Description:** I2C SDA Stuck at Low Timeout.
This register is used to store the duration, measured in ic_clk cycles, used to Recover the Data (SDA) line through sending SCL pulses if SDA is held low for the mentioned duration.
- **Size:** 32 bits
- **Offset:** 0xb0
- **Exists:** IC_BUS_CLEAR_FEATURE==1



Table 5-53 Fields for Register: IC_SDA_STUCK_AT_LOW_TIMEOUT

Bits	Name	Memory Access	Description
31:0	IC_SDA_STUCK_LOW_TIMEOUT	R/W	DW_apb_i2c initiates the recovery of SDA line through enabling the SDA_STUCK_RECOVERY_EN (IC_ENABLE[3]) register bit, if it detects the SDA stuck at low for the IC_SDA_STUCK_LOW_TIMEOUT in units of ic_clk period. Value After Reset: IC_SDA_STUCK_TIMEOUT_DEFAULT Exists: Always

5.1.50 IC_CLR_SCL_STUCK_DET

- **Name:** Clear SCL Stuck at Low Detect interrupt Register
- **Description:** Clear SCL Stuck at Low Detect Interrupt Register
- **Size:** 32 bits
- **Offset:** 0xb4
- **Exists:** IC_BUS_CLEAR_FEATURE==1

31:1	RSVD_CLR_SCL_STUCK_DET
0	CLR_SCL_STUCK_DET

Table 5-54 Fields for Register: IC_CLR_SCL_STUCK_DET

Bits	Name	Memory Access	Description
31:1	RSVD_CLR_SCL_STUCK_DET	R	CLR_SCL_STUCK_DET Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_SCL_STUCK_DET	R	Read this register to clear the SCL_STUCT_AT_LOW interrupt (bit 15) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.51 IC_DEVICE_ID

- **Name:** I2C Device-Id register
- **Description:** I2C Device-ID Register.
This Register contains the Device-ID of the component which includes 12-bits of Manufacturer name, and 9-bits of part identification and 3 bits of die-version.
- **Size:** 32 bits
- **Offset:** 0xb8
- **Exists:** IC_DEVICE_ID==1

31:24	RSVD_IC_DEVICE_ID
23:0	DEVICE-ID

Table 5-55 Fields for Register: IC_DEVICE_ID

Bits	Name	Memory Access	Description
31:24	RSVD_IC_DEVICE_ID	R	IC_DEVICE_ID Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
23:0	DEVICE-ID	R	Contains the Device-ID of the component assigned through the configuration parameter 'IC_DEVICE_ID_VALUE' Value After Reset: IC_DEVICE_ID_VALUE Exists: Always

5.1.52 IC_SMBUS_CLK_LOW_SEXT

- **Name:** SMBus Target Clock Extend Timeout register
- **Description:** SMBus Target Clock Extend Timeout Register.
This Register contains the Timeout value used to determine the Target Clock Extend Timeout in one transfer (from START to STOP).
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS is set to 1. This register is used to store the duration, measured in ic_clk cycles, used to detect the target clock extend timeout if target extends the clock (SCL) for the mentioned duration.
- **Size:** 32 bits
- **Offset:** 0xbc
- **Exists:** IC_SMBUS==1



Table 5-56 Fields for Register: IC_SMBUS_CLK_LOW_SEXT

Bits	Name	Memory Access	Description
31:0	SMBUS_CLK_LOW_SEXT_TIMEOUT	R/W	This field is used to detect the Target Clock Extend timeout (tLOW:SEXT) in controller mode extended by the target device in one message from the initial START to the STOP. The values in this register are in units of ic_clk period. Value After Reset: IC_SMBUS_CLK_LOW_SEXT_DEFAULT Exists: Always

5.1.53 IC_SMBUS_CLK_LOW_MEXT

- **Name:** SMBus Controller Clock Extend Timeout register
- **Description:** SMBus Controller Clock Extend Timeout Register.
This Register contains the Timeout value used to determine the Controller Clock Extend Timeout in one byte of transfer.
This register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS is set to 1. This register is used to store the duration, measured in ic_clk cycles, used to detect the Controller clock extend timeout if Controller extends the clock (SCL) for the mentioned duration.
- **Size:** 32 bits
- **Offset:** 0xc0
- **Exists:** IC_SMBUS==1



Table 5-57 Fields for Register: IC_SMBUS_CLK_LOW_MEXT

Bits	Name	Memory Access	Description
31:0	SMBUS_CLK_LOW_MEXT_TIME OUT	R/W	This field is used to detect the Controller extend SMBus clock (SCLK) timeout defined from START-to-ACK, ACK-to-ACK, or ACK-to-STOP in Controller mode. The values in this register are in units of ic_clk period. Value After Reset: IC_SMBUS_CLK_LOW_MEXT_DEFAULT Exists: Always

5.1.54 IC_SMBUS_THIGH_MAX_IDLE_COUNT

- **Name:** SMBus Controller THigh MAX Bus-idle count Register
- **Description:** SMBus Controller THigh MAX Bus-idle count Register.
This register programs the Bus-idle time period used when a controller has been dynamically added to the bus or when a controller has generated a clock reset on the bus. This register is used to store the duration, measured in ic_clk cycles, used to detect the Bus Idle condition if SCL and SDA are held high for the mentioned duration. This register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS is set to 1.
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** IC_SMBUS==1

31:16	RSVD_SMBUS_THIGH_MAX_BUS_IDLE_CNT
15:0	SMBUS_THIGH_MAX_BUS_IDLE_CNT

Table 5-58 Fields for Register: IC_SMBUS_THIGH_MAX_IDLE_COUNT

Bits	Name	Memory Access	Description
31:16	RSVD_SMBUS_THIGH_MAX_BUS_IDLE_CNT	R	SMBUS_THIGH_MAX_BUS_IDLE_CNT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
15:0	SMBUS_THIGH_MAX_BUS_IDLE_CNT	R/W	This field is used to set the required Bus-Idle time period used when a controller has been dynamically added to the bus and may not have detected a state transition on the SMBCLK or SMBDAT lines. In this case, the controller must wait long enough to ensure that a transfer is not currently in progress The values in this register are in units of ic_clk period. Value After Reset: IC_SMBUS_RST_IDLE_CNT_DEFAULT Exists: Always

5.1.55 IC_SMBUS_INTR_STAT

- **Name:** SMBus Interrupt Status Register
- **Description:** SMBUS Interrupt Status Register.
Each bit in this register has a corresponding mask bit in the IC_SMBUS_INTR_MASK register. These bits are cleared by writing the matching SMBus interrupt clear register (IC_CLR_SMBUS_INTR) bits. The unmasked raw versions of these bits are available in the IC_SMBUS_RAW_INTR_STAT register.
- **Size:** 32 bits
- **Offset:** 0xc8
- **Exists:** IC_SMBUS==1

31:11	RSVD_IC_SMBUS_INTR_STAT
10	R_SMBUS_ALERT_DET
9	R_SMBUS_SUSPEND_DET
8	R_SLV_RX_PEC_NACK
7	R_ARP_ASSGN_ADDR_CMD_DET
6	R_ARP_GET_UDID_CMD_DET
5	R_ARP_RST_CMD_DET
4	R_ARP_PREPARE_CMD_DET
3	R_HOST_NOTIFY_MST_DET
2	R_QUICK_CMD_DET
1	R_MST_CLOCK_EXTND_TIMEOUT
0	R_SLV_CLOCK_EXTND_TIMEOUT

Table 5-59 Fields for Register: IC_SMBUS_INTR_STAT

Bits	Name	Memory Access	Description
31:11	RSVD_IC_SMBUS_INTR_STAT	R	IC_SMBUS_INTR_STAT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
10	R_SMBUS_ALERT_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_SMBUS_ALERT_DET bit. Values: ■ 0x1 (ACTIVE): SMBUS_ALERT_DET interrupt is active ■ 0x0 (INACTIVE): SMBUS_ALERT_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true

Bits	Name	Memory Access	Description
9	R_SMBUS_SUSPEND_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_SMBUS_SUSPEND_DET bit. Values: ■ 0x1 (ACTIVE): SMBUS_SUSPEND_DET interrupt is active ■ 0x0 (INACTIVE): SMBUS_SUSPEND_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true
8	R_SLV_RX_PEC_NACK	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_SLV_RX_PEC_NACK bit. Values: ■ 0x1 (ACTIVE): SLV_RX_PEC_NACK interrupt is active ■ 0x0 (INACTIVE): SLV_RX_PEC_NACK interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
7	R_ARP_ASSGN_ADDR_CMD_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_ARP_ASSGN_ADDR_CMD_DET bit. Values: ■ 0x1 (ACTIVE): ARP_ASSGN_ADDR_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_ASSGN_ADDR_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
6	R_ARP_GET_UDID_CMD_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_ARP_GET_UDID_CMD_DET bit. Values: ■ 0x1 (ACTIVE): ARP_GET_UDID_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_GET_UDID_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true

Bits	Name	Memory Access	Description
5	R_ARP_RST_CMD_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_ARP_RST_CMD_DET bit. Values: ■ 0x1 (ACTIVE): ARP_RST_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_RST_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
4	R_ARP_PREPARE_CMD_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_ARP_PREPARE_CMD_DET bit. Values: ■ 0x1 (ACTIVE): ARP_PREPARE_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_PREPARE_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
3	R_HOST_NOTIFY_MST_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_HOST_NOTIFY_MST_DET bit. Values: ■ 0x1 (ACTIVE): HOST_NOTIFY_MST_DET interrupt is active ■ 0x0 (INACTIVE): HOST_NOTIFY_MST_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS==1 Volatile: true
2	R_QUICK_CMD_DET	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_QUICK_CMD_DET bit. Values: ■ 0x1 (ACTIVE): QUICK_CMD_DET interrupt is active ■ 0x0 (INACTIVE): QUICK_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
1	R_MST_CLOCK_EXTND_TIMEOUT	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_MST_CLOCK_EXTND_TIMEOUT bit. Values: ■ 0x1 (ACTIVE): MST_CLOCK_EXTND_TIMEOUT interrupt is active ■ 0x0 (INACTIVE): MST_CLOCK_EXTND_TIMEOUT interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
0	R_SLV_CLOCK_EXTND_TIMEOUT	R	See IC_SMBUS_RAW_INTR_STAT for a detailed description of R_SLV_CLOCK_EXTND_TIMEOUT bit. Values: ■ 0x1 (ACTIVE): SLV_CLOCK_EXTND_TIMEOUT interrupt is active ■ 0x0 (INACTIVE): SLV_CLOCK_EXTND_TIMEOUT interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

5.1.56 IC_SMBUS_INTR_MASK

- **Name:** SMBus Interrupt Mask Register
- **Description:** SMBus Interrupt Mask Register
- **Size:** 32 bits
- **Offset:** 0xcc
- **Exists:** IC_SMBUS==1

31:11	RSVD_IC_SMBUS_INTR_MASK
10	M_SMBUS_ALERT_DET
9	M_SMBUS_SUSPEND_DET
8	M_SLV_RX_PEC_NACK
7	M_ARP_ASSGN_ADDR_CMD_DET
6	M_ARP_GET_UDID_CMD_DET
5	M_ARP_RST_CMD_DET
4	M_ARP_PREPARE_CMD_DET
3	M_HOST_NOTIFY_MST_DET
2	M_QUICK_CMD_DET
1	M_MST_CLOCK_EXTND_TIMEOUT
0	M_SLV_CLOCK_EXTND_TIMEOUT

Table 5-60 Fields for Register: IC_SMBUS_INTR_MASK

Bits	Name	Memory Access	Description
31:11	RSVD_IC_SMBUS_INTR_MASK	R	IC_SMBUS_INTR_MASK Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
10	M_SMBUS_ALERT_DET	R/W	This bit masks the R_SMBUS_ALERT_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): SMBUS_ALERT_DET interrupt is unmasked ■ 0x0 (ENABLED): SMBUS_ALERT_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_SUSPEND_ALERT==1

Bits	Name	Memory Access	Description
9	M_SMBUS_SUSPEND_DET	R/W	This bit masks the M_SMBUS_SUSPEND_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): SMBUS_SUSPEND_DET interrupt is unmasked ■ 0x0 (ENABLED): SMBUS_SUSPEND_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_SUSPEND_ALERT==1
8	M_SLV_RX_PEC_NACK	R/W	This bit masks the R_SLV_RX_PEC_NACK interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): SLV_RX_PEC_NACK interrupt is unmasked ■ 0x0 (ENABLED): SLV_RX_PEC_NACK interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_ARP==1
7	M_ARP_ASSGN_ADDR_CMD_DET	R/W	This bit masks the R_ARP_ASSGN_ADDR_CMD_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): ARP_ASSGN_ADDR_CMD_DET interrupt is unmasked ■ 0x0 (ENABLED): ARP_ASSGN_ADDR_CMD_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_ARP==1
6	M_ARP_GET_UDID_CMD_DET	R/W	This bit masks the R_ARP_GET_UDID_CMD_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): ARP_GET_UDID_CMD_DET interrupt is unmasked ■ 0x0 (ENABLED): ARP_GET_UDID_CMD_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_ARP==1

Bits	Name	Memory Access	Description
5	M_ARP_RST_CMD_DET	R/W	This bit masks the R_ARP_RST_CMD_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): ARP_RST_CMD_DET interrupt is unmasked ■ 0x0 (ENABLED): ARP_RST_CMD_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_ARP==1
4	M_ARP_PREPARE_CMD_DET	R/W	This bit masks the R_ARP_PREPARE_CMD_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): ARP_PREPARE_CMD_DET interrupt is unmasked ■ 0x0 (ENABLED): ARP_PREPARE_CMD_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS_ARP==1
3	M_HOST_NOTIFY_MST_DET	R/W	This bit masks the R_HOST_NOTIFY_MST_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): HOST_NOTIFY_MST_DET interrupt is unmasked ■ 0x0 (ENABLED): HOST_NOTIFY_MST_DET interrupt is masked Value After Reset: 0x1 Exists: IC_SMBUS==1
2	M_QUICK_CMD_DET	R/W	This bit masks the R_QUICK_CMD_DET interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): QUICK_CMD_DET interrupt is unmasked ■ 0x0 (ENABLED): QUICK_CMD_DET interrupt is masked Value After Reset: 0x1 Exists: Always
1	M_MST_CLOCK_EXTND_TIMEOUT	R/W	This bit masks the R_MST_CLOCK_EXTND_TIMEOUT interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): MST_CLOCK_EXTND_TIMEOUT interrupt is unmasked ■ 0x0 (ENABLED): MST_CLOCK_EXTND_TIMEOUT interrupt is masked Value After Reset: 0x1 Exists: Always

Bits	Name	Memory Access	Description
0	M_SLV_CLOCK_EXTND_TIMEO T	R/W	This bit masks the R_SLV_CLOCK_EXTND_TIMEOUT interrupt in IC_SMBUS_INTR_STAT register. Values: ■ 0x1 (DISABLED): SLV_CLOCK_EXTND_TIMEOUT interrupt is unmasked ■ 0x0 (ENABLED): SLV_CLOCK_EXTND_TIMEOUT interrupt is masked Value After Reset: 0x1 Exists: Always

5.1.57 IC_SMBUS_RAW_INTR_STAT

- **Name:** SMBus Raw Interrupt Status Register
- **Description:** SMBus Raw Interrupt Status Register.
Unlike the IC_SMBUS_INTR_STAT register, these bits are not masked so they always show the true status of the DW_apb_i2c.
- **Size:** 32 bits
- **Offset:** 0xd0
- **Exists:** IC_SMBUS==1

31:11	RSVD_IC_SMBUS_RAW_INTR_STAT
10	SMBUS_ALERT_DET
9	SMBUS_SUSPEND_DET
8	SLV_RX_PEC_NACK
7	ARP_ASSGN_ADDR_CMD_DET
6	ARP_GET_UDID_CMD_DET
5	ARP_RST_CMD_DET
4	ARP_PREPARE_CMD_DET
3	HOST_NOTIFY_MST_DET
2	QUICK_CMD_DET
1	MST_CLOCK_EXTND_TIMEOUT
0	SLV_CLOCK_EXTND_TIMEOUT

Table 5-61 Fields for Register: IC_SMBUS_RAW_INTR_STAT

Bits	Name	Memory Access	Description
31:11	RSVD_IC_SMBUS_RAW_INTR_STAT	R	IC_SMBUS_RAW_INTR_STAT Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
10	SMBUS_ALERT_DET	R	Indicates whether a SMBALERT (ic_smbalert_in_n) signal is driven low by the target. Values: ■ 0x1 (ACTIVE): SMBUS Alert interrupt is active ■ 0x0 (INACTIVE): SMBUS Alert interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true

Bits	Name	Memory Access	Description
9	SMBUS_SUSPEND_DET	R	Indicates whether a SMBUS (ic_smbus_in_n) signal is driven low by the Host. Values: ■ 0x1 (ACTIVE): SMBUS System Suspended interrupt is active ■ 0x0 (INACTIVE): SMBUS System Suspended interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1 Volatile: true
8	SLV_RX_PEC_NACK	R	Indicates whether a NACK has been sent due to PEC mismatch while working as ARP target. Values: ■ 0x1 (ACTIVE): SLV_RX_PEC_NACK interrupt is active ■ 0x0 (INACTIVE): SLV_RX_PEC_NACK interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
7	ARP_ASSGN_ADDR_CMD_DET	R	Indicates whether an Assign Address ARP command has been received. Values: ■ 0x1 (ACTIVE): ARP_ASSGN_ADDR_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_ASSGN_ADDR_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
6	ARP_GET_UDID_CMD_DET	R	Indicates whether a Get UDID ARP command has been received. Values: ■ 0x1 (ACTIVE): ARP_GET_UDID_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_GET_UDID_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true

Bits	Name	Memory Access	Description
5	ARP_RST_CMD_DET	R	Indicates whether a General or Directed Reset ARP command has been received. Values: ■ 0x1 (ACTIVE): ARP_RST_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_RST_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
4	ARP_PREPARE_CMD_DET	R	Indicates whether a prepare to ARP command has been received. Values: ■ 0x1 (ACTIVE): ARP_PREPARE_CMD_DET interrupt is active ■ 0x0 (INACTIVE): ARP_PREPARE_CMD_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1 Volatile: true
3	HOST_NOTIFY_MST_DET	R	Indicates whether a Notify ARP Controller ARP command has been received. Values: ■ 0x1 (ACTIVE): HOST_NOTIFY_MST_DET interrupt is active ■ 0x0 (INACTIVE): HOST_NOTIFY_MST_DET interrupt is inactive Value After Reset: 0x0 Exists: IC_SMBUS==1 Volatile: true
2	QUICK_CMD_DET	R	Indicates whether a Quick command has been received on the SMBus interface regardless of whether DW_apb_i2c is operating in target mode. Enabled only when IC_SMBUS=1 is set to 1. Values: ■ 0x1 (ACTIVE): Quick Command interrupt is active ■ 0x0 (INACTIVE): Quick Command interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

Bits	Name	Memory Access	Description
1	MST_CLOCK_EXTND_TIMEOUT	R	Indicates whether the Controller device transaction (START-to-ACK, ACK-to-ACK, or ACK-to-STOP) from START to STOP exceeds IC_SMBUS_CLOCK_LOW_MEXT time with in each byte of message. This bit is enabled only when: ■ IC_SMBUS=1 ■ IC_CON[0]=1 ■ IC_EMPTYFIFO_HOLD_MASTER_EN=1 or ■ IC_RX_FULL_HLD_BUS_EN=1 Values: ■ 0x1 (ACTIVE): Controller Clock Extend Timeout interrupt is active ■ 0x0 (INACTIVE): Controller Clock Extend Timeout interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true
0	SLV_CLOCK_EXTND_TIMEOUT	R	Indicates whether the transaction from Target (i.e from START to STOP) exceeds IC_SMBUS_CLK_LOW_SEXT time. This bit is enabled only when: ■ IC_SMBUS=1 ■ IC_CON[0]=1 Values: ■ 0x1 (ACTIVE): Target Clock Extend Timeout interrupt is active ■ 0x0 (INACTIVE): Target Clock Extend Timeout interrupt is inactive Value After Reset: 0x0 Exists: Always Volatile: true

5.1.58 IC_CLR_SMBUS_INTR

- **Name:** Clear SMBus Interrupt Register
- **Description:** SMBus Clear Interrupt Register
- **Size:** 32 bits
- **Offset:** 0xd4
- **Exists:** IC_SMBUS==1

31:11	RSVD_IC_CLR_SMBUS_INTR
10	CLR_SMBUS_ALERT_DET
9	CLR_SMBUS_SUSPEND_DET
8	CLR_SLV_RX_PEC_NACK
7	CLR_ARP_ASSGN_ADDR_CMD_DET
6	CLR_ARP_GET_UDID_CMD_DET
5	CLR_ARP_RST_CMD_DET
4	CLR_ARP_PREPARE_CMD_DET
3	CLR_HOST_NOTIFY_MST_DET
2	CLR_QUICK_CMD_DET
1	CLR_MST_CLOCK_EXTND_TIMEOUT
0	CLR_SLV_CLOCK_EXTND_TIMEOUT

Table 5-62 Fields for Register: IC_CLR_SMBUS_INTR

Bits	Name	Memory Access	Description
31:11	RSVD_IC_CLR_SMBUS_INTR	W	IC_CLR_SMBUS_INTR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
10	CLR_SMBUS_ALERT_DET	W	Write this register bit to clear the SMBUS_ALERT_DET interrupt (bit 10) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1
9	CLR_SMBUS_SUSPEND_DET	W	Write this register bit to clear the SMBUS_SUSPEND_DET interrupt (bit 9) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_SUSPEND_ALERT==1

Bits	Name	Memory Access	Description
8	CLR_SLV_RX_PEC_NACK	W	Write this register bit to clear the SLV_RX_PEC_NACK interrupt (bit 8) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1
7	CLR_ARP_ASSGN_ADDR_CMD_DET	W	Write this register bit to clear the ARP_ASSGN_ADDR_CMD_DET interrupt (bit 7) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1
6	CLR_ARP_GET_UDID_CMD_DET	W	Write this register bit to clear the ARP_GET_UDID_CMD_DET interrupt (bit 6) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1
5	CLR_ARP_RST_CMD_DET	W	Write this register bit to clear the ARP_RST_CMD_DET interrupt (bit 5) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1
4	CLR_ARP_PREPARE_CMD_DET	W	Write this register bit to clear the ARP_PREPARE_CMD_DET interrupt (bit 4) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS_ARP==1
3	CLR_HOST_NOTIFY_MST_DET	W	Write this register bit to clear the HOST_NOTIFY_MST_DET interrupt (bit 3) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: IC_SMBUS==1
2	CLR_QUICK_CMD_DET	W	Write this register bit to clear the QUICK_CMD_DET interrupt (bit 2) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always
1	CLR_MST_CLOCK_EXTND_TIME_OUT	W	Write this register bit to clear the MST_CLOCK_EXTND_TIMEOUT interrupt (bit 1) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always
0	CLR_SLV_CLOCK_EXTND_TIME_OUT	W	Write this register bit to clear the SLV_CLOCK_EXTND_TIMEOUT interrupt (bit 0) of the IC_SMBUS_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always

5.1.59 IC_OPTIONAL_SAR

- **Name:** I2C Optional Target Address Register
- **Description:** I2C Optional Target Address Register.
Optional Target address for I2C in SMBus Mode. A same restriction as IC_SAR applies on IC_OPTIONAL_SAR.
- **Size:** 32 bits
- **Offset:** 0xd8
- **Exists:** IC_OPTIONAL_SAR==1



Table 5-63 Fields for Register: IC_OPTIONAL_SAR

Bits	Name	Memory Access	Description
31:7	RSVD_IC_OPTIONAL_SAR	R	IC_OPTIONAL_SAR Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
6:0	OPTIONAL_SAR	R/W	Optional Target address for DW_apb_i2c when operating as a target in SMBus Mode. Value After Reset: IC_OPTIONAL_SAR_DEFAULT Exists: Always

5.1.60 IC_SMBUS_UDID_LSB

- **Name:** SMBUS ARP UDID LSB Register
- **Description:** SMBUS ARP UDID LSB Register.
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS_UDID_HC is set to 1. This register is used to store the LSB 32 bit value of Target UDID register used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0xdc
- **Exists:** (IC_SMBUS_ARP == 1) && (IC_SMBUS_UDID_HC == 1)



Table 5-64 Fields for Register: IC_SMBUS_UDID_LSB

Bits	Name	Memory Access	Description
31:0	SMBUS_UDID_LSB	R/W	This field is used to store the LSB 32 bit value of target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SMBUS_UDID_LSB_DEFAULT Exists: Always

5.1.61 IC_SMBUS_UDID_WORD0

- **Name:** SMBUS ARP UDID WORD0 Register
- **Description:** SMBUS UDID WORD0 Register
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS_UDID_HC is set to 0. This register is used to store the Lower 32 bit value of Target UDID register i.e. UDID[31:0] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0xdc
- **Exists:** IC_SMBUS_UDID_HC==0



Table 5-65 Fields for Register: IC_SMBUS_UDID_WORD0

Bits	Name	Memory Access	Description
31:0	SMBUS_UDID_WORD0	R/W	This field is used to store the Lower 32 bit value of target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SMBUS_UDID_LSB_DEFAULT Exists: Always

5.1.62 IC_SMBUS_UDID_WORD1

- **Name:** SMBUS ARP UDID WORD1 Register
- **Description:** SMBUS UDID WORD1 Register
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Lower 32 bit value of Target UDID register i.e. UDID[63:32] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0xe0
- **Exists:** IC_SMBUS_UDID_HC==0



Table 5-66 Fields for Register: IC_SMBUS_UDID_WORD1

Bits	Name	Memory Access	Description
31:0	SMBUS_UDID_WORD1	R/W	This field is used to store the Middle-Lower 32 bit value of target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SMBUS_UDID_WORD1_DEFAULT Exists: Always

5.1.63 IC_SMBUS_UDID_WORD2

- **Name:** SMBUS ARP UDID WORD2 Register
- **Description:** SMBUS UDID WORD2 Register
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Upper 32 bit value of Target UDID register i.e. UDID[95:64] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0xe4
- **Exists:** IC_SMBUS_UDID_HC==0



Table 5-67 Fields for Register: IC_SMBUS_UDID_WORD2

Bits	Name	Memory Access	Description
31:0	SMBUS_UDID_WORD2	R/W	This field is used to store the Middle-Upper 32 bit value of target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SMBUS_UDID_WORD2_DEFAULT Exists: Always

5.1.64 IC_SMBUS_UDID_WORD3

- **Name:** SMBUS ARP UDID WORD3 Register
- **Description:** SMBUS UDID WORD3 Register
This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SMBUS_UDID_HC is set to 0. This register is used to store the Upper 32 bit value of Target UDID register i.e. UDID[127:96] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0xe8
- **Exists:** IC_SMBUS_UDID_HC==0



Table 5-68 Fields for Register: IC_SMBUS_UDID_WORD3

Bits	Name	Memory Access	Description
31:0	SMBUS_UDID_WORD3	R/W	This field is used to store the Upper 32 bit value of target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SMBUS_UDID_WORD3_DEFAULT Exists: Always

5.1.65 REG_TIMEOUT_RST

- **Name:** Register timeout counter reset value
- **Description:** Name: Register timeout counter reset register Size: REG_TIMEOUT_WIDTH bits
Address: 0xF0 Read/Write Access: Read/Write This register keeps the timeout value of register timer counter. The reset value of the register is REG_TIMEOUT_VALUE. The default reset value can be further modified if HC_REG_TIMEOUT_VALUE = 0. The final programmed value (or the default reset value if not programmed) determines from what value the register timeout counter starts counting down. A zero on this counter will break the waited transaction with PSLVERR as high. The minimum value written to this register must be greater than or equal to 4.
- **Size:** 32 bits
- **Offset:** 0xf0
- **Exists:** [`<functionof> "(( (SLAVE_INTERFACE_TYPE>0 && SLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0) ? 1 : 0)==1) ? 1 : 0"`]

31:y	RSVD_REG_TIMEOUT_RST
x:0	REG_TIMEOUT_RST_rw

Table 5-69 Fields for Register: REG_TIMEOUT_RST

Bits	Name	Memory Access	Description
31:y	RSVD_REG_TIMEOUT_RST	R	Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[y]: REG_TIMEOUT_WIDTH
x:0	REG_TIMEOUT_RST_rw	R/W	This field holds reset value of REG_TIMEOUT counter register. Value After Reset: REG_TIMEOUT_VALUE Exists: [<code><functionof> "(HC_REG_TIMEOUT_VALUE==0) ? 1 : 0"</code>] Volatile: true Range Variable[x]: REG_TIMEOUT_WIDTH - 1

5.1.66 IC_COMP_PARAM_1

- **Name:** Component Parameter Register 1
- **Description:** Component Parameter Register 1
Note: This is a constant read-only register that contains encoded information about the component's parameter settings. The reset value depends on coreConsultant parameter(s).
- **Size:** 32 bits
- **Offset:** 0xf4
- **Exists:** [`<functionof> "(IC_ADD_ENCODED_PARAMS==1) ? 1 : 0"`]

31:24	RSVD_IC_COMP_PARAM_1
23:16	TX_BUFFER_DEPTH
15:8	RX_BUFFER_DEPTH
7	ADD_ENCODED_PARAMS
6	HAS_DMA
5	INTR_IO
4	HC_COUNT_VALUES
3:2	MAX_SPEED_MODE
1:0	APB_DATA_WIDTH

Table 5-70 Fields for Register: IC_COMP_PARAM_1

Bits	Name	Memory Access	Description
31:24	RSVD_IC_COMP_PARAM_1	R	IC_COMP_PARAM_1 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always
23:16	TX_BUFFER_DEPTH	R	The value of this register is derived from the IC_TX_BUFFER_DEPTH coreConsultant parameter. ■ 0x00 = Reserved ■ 0x01 = 2 ■ 0x02 = 3 ■ ... ■ 0xFF = 256 Value After Reset: ENCODED_IC_TX_BUFFER_DEPTH Exists: Always

Bits	Name	Memory Access	Description
15:8	RX_BUFFER_DEPTH	R	The value of this register is derived from the IC_RX_BUFFER_DEPTH coreConsultant parameter. ■ 0x00: Reserved ■ 0x01: 2 ■ 0x02: 3 ■ ... ■ 0xFF: 256 Value After Reset: ENCODED_IC_RX_BUFFER_DEPTH Exists: Always
7	ADD_ENCODED_PARAMS	R	The value of this register is derived from the IC_ADD_ENCODED_PARAMS coreConsultant parameter. Reading 1 in this bit means that the capability of reading these encoded parameters via software has been included. Otherwise, the entire register is 0 regardless of the setting of any other parameters that are encoded in the bits. Values: ■ 0x1 (ENABLED): Enables capability of reading encoded parameters ■ 0x0 (DISABLED): Disables capability of reading encoded parameters Value After Reset: IC_ADD_ENCODED_PARAMS Exists: Always
6	HAS_DMA	R	The value of this register is derived from the IC_HAS_DMA coreConsultant parameter. Values: ■ 0x1 (ENABLED): DMA handshaking signals are enabled ■ 0x0 (DISABLED): DMA handshaking signals are disabled Value After Reset: IC_HAS_DMA Exists: Always
5	INTR_IO	R	The value of this register is derived from the IC_INTR_IO coreConsultant parameter. Values: ■ 0x1 (COMBINED): COMBINED Interrupt outputs ■ 0x0 (INDIVIDUAL): INDIVIDUAL Interrupt outputs Value After Reset: IC_INTR_IO Exists: Always

Bits	Name	Memory Access	Description
4	HC_COUNT_VALUES	R	The value of this register is derived from the IC_HC_COUNT_VALUES coreConsultant parameter. Values: ■ 0x1 (ENABLED): Hard code the count values for each mode. ■ 0x0 (DISABLED): Programmable count values for each mode. Value After Reset: IC_HC_COUNT_VALUES Exists: Always
3:2	MAX_SPEED_MODE	R	The value of this register is derived from the IC_MAX_SPEED_MODE coreConsultant parameter. ■ 0x0: Reserved ■ 0x1: Standard ■ 0x2: Fast ■ 0x3: High Values: ■ 0x1 (STANDARD): IC MAX SPEED is STANDARD MODE ■ 0x2 (FAST): IC MAX SPEED is FAST MODE ■ 0x3 (HIGH): IC MAX SPEED is HIGH MODE Value After Reset: "(IC_ULTRA_FAST_MODE_EN==0) ? (IC_MAX_SPEED_MODE) : \"0x00\"" Exists: IC_ULTRA_FAST_MODE==0
1:0	APB_DATA_WIDTH	R	The value of this register is derived from the APB_DATA_WIDTH coreConsultant parameter. Values: ■ 0x0 (APB_08BITS): APB data bus width is 08 bits ■ 0x1 (APB_16BITS): APB data bus width is 16 bits ■ 0x2 (APB_32BITS): APB data bus width is 32 bits ■ 0x3 (RESERVED): Reserved bits Value After Reset: ENCODED_APB_DATA_WIDTH Exists: Always

5.1.67 IC_COMP_VERSION

- **Name:** I2C Component Version Register
- **Description:** I2C Component Version Register
- **Size:** 32 bits
- **Offset:** 0xf8
- **Exists:** Always



Table 5-71 Fields for Register: IC_COMP_VERSION

Bits	Name	Memory Access	Description
31:0	IC_COMP_VERSION	R	Specific values for this register are described in the Releases Table in the DW_apb_i2c Release Notes Value After Reset: IC_VERSION_ID Exists: Always

5.1.68 IC_COMP_TYPE

- **Name:** I2C Component Type Register
- **Description:** I2C Component Type Register
- **Size:** 32 bits
- **Offset:** 0xfc
- **Exists:** Always



Table 5-72 Fields for Register: IC_COMP_TYPE

Bits	Name	Memory Access	Description
31:0	IC_COMP_TYPE	R	DesignWare Component Type number = 0x44_57_01_40. This assigned unique hex value is constant and is derived from the two ASCII letters 'DW' followed by a 16-bit unsigned number. Value After Reset: 0x44570140 Exists: Always

5.1.69 IC_SAR2

- **Name:** I2C Target Device Address Register 2
- **Description:** I2C Target Device Address Register 2
- **Size:** 32 bits
- **Offset:** 0x100
- **Exists:** (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 2)

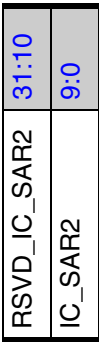


Table 5-73 Fields for Register: IC_SAR2

Bits	Name	Memory Access	Description
31:10	RSVD_IC_SAR2	R	IC_SAR2 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
9:0	IC_SAR2	R/W	The IC_SAR2 holds the target device address 2 when the I2C is operating as a target. For 7-bit addressing or SMBus (IC_SMBUS=1), only IC_SAR2[6:0] is used. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Note: The default values cannot be any of the reserved address locations: that is, 0x00 to 0x07, or 0x78 to 0x7f. The correct operation of the device is not guaranteed if you program the IC_SAR2 or IC_TAR to a reserved value. Refer to Table "I2C/SMBus Definition of Bits in First Byte" for a complete list of these reserved values. Value After Reset: IC_DEFAULT_SLAVE_ADDR2 Exists: Always Volatile: true

5.1.70 IC_SAR3

- **Name:** I2C Target Device Address Register 3
- **Description:** I2C Target Device Address Register 3
- **Size:** 32 bits
- **Offset:** 0x104
- **Exists:** (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 3)

31:10	RSVD_IC_SAR3
9:0	IC_SAR3

Table 5-74 Fields for Register: IC_SAR3

Bits	Name	Memory Access	Description
31:10	RSVD_IC_SAR3	R	IC_SAR3 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
9:0	IC_SAR3	R/W	The IC_SAR3 holds the target device address 3 when the I2C is operating as a target. For 7-bit addressing or SMBus (IC_SMBUS=1), only IC_SAR3[6:0] is used. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Note: The default values cannot be any of the reserved address locations: that is, 0x00 to 0x07, or 0x78 to 0x7f. The correct operation of the device is not guaranteed if you program the IC_SAR3 or IC_TAR to a reserved value. Refer to Table "I2C/SMBus Definition of Bits in First Byte" for a complete list of these reserved values. Value After Reset: IC_DEFAULT_SLAVE_ADDR3 Exists: Always Volatile: true

5.1.71 IC_SAR4

- **Name:** I2C Target Device Address Register 4
- **Description:** I2C Target Device Address Register 4
- **Size:** 32 bits
- **Offset:** 0x108
- **Exists:** (IC_MULTI_SAR_EN == 1) && (IC_NUM_SARS >= 4)

31:10	RSVD_IC_SAR4
9:0	IC_SAR4

Table 5-75 Fields for Register: IC_SAR4

Bits	Name	Memory Access	Description
31:10	RSVD_IC_SAR4	R	IC_SAR4 Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
9:0	IC_SAR4	R/W	The IC_SAR4 holds the target device address 4 when the I2C is operating as a target. For 7-bit addressing or SMBus mode (IC_SMBUS=1), only IC_SAR4[6:0] is used. This register can be written only when the I2C interface is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect. Note: The default values cannot be any of the reserved address locations: that is, 0x00 to 0x07, or 0x78 to 0x7f. The correct operation of the device is not guaranteed if you program the IC_SAR4 or IC_TAR to a reserved value. Refer to Table "I2C/SMBus Definition of Bits in First Byte" for a complete list of these reserved values. Value After Reset: IC_DEFAULT_SLAVE_ADDR4 Exists: Always Volatile: true

5.1.72 IC_CLR_WR_REQ

- **Name:** Clear WR_REQ Interrupt Register
- **Description:** Clear WR_REQ Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x11c
- **Exists:** IC_MULTI_SAR_EN==1

31:1	RSVD_IC_CLR_WR_REQ
0	CLR_WR_REQ

Table 5-76 Fields for Register: IC_CLR_WR_REQ

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_WR_REQ	R	IC_CLR_WR_REQ Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_WR_REQ	R	Read this register to clear the WR_REQ interrupt (bit 15) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.73 IC_CLR_SLV_ADDR_TAG

- **Name:** Clear CLR_SLV_ADDR_TAG Interrupt Register
- **Description:** Clear CLR_SLV_ADDR_TAG Interrupt Register
- **Size:** 32 bits
- **Offset:** 0x120
- **Exists:** IC_MULTI_SAR_EN==1

31:1	RSVD_IC_CLR_SLV_ADDR_TAG
0	CLR_SLV_ADDR_TAG

Table 5-77 Fields for Register: IC_CLR_SLV_ADDR_TAG

Bits	Name	Memory Access	Description
31:1	RSVD_IC_CLR_SLV_ADDR_TAG	R	IC_CLR_SLV_ADDR_TAG Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true
0	CLR_SLV_ADDR_TAG	R	Read this register to clear all SLV_ADDRx_TAG (Where x is 1 through IC_NUM_SARS) interrupts (bit 16 ... (16+IC_NUM_SARS-1)) of the IC_RAW_INTR_STAT register. Value After Reset: 0x0 Exists: Always Volatile: true

5.1.74 IC_SAR2_SMBUS_UDID_WORD0

- **Name:** IC_SAR2 SMBUS ARP UDID WORD0 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR2_SMBUS_UDID_HC is set to 0. This register is used to store the Lower 32 bit value of IC_SAR2 Target UDID register i.e. UDID[31:0] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x124
- **Exists:** IC_SAR2_SMBUS_UDID_HC==0



Table 5-78 Fields for Register: IC_SAR2_SMBUS_UDID_WORD0

Bits	Name	Memory Access	Description
31:0	SAR2_SMBUS_UDID_WORD0	R/W	This field is used to store the Lower 32 bit value of IC_SAR2 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR2_SMBUS_UDID_WORD0_DEFAULT Exists: Always

5.1.75 IC_SAR2_SMBUS_UDID_WORD1

- **Name:** IC_SAR2 SMBUS ARP UDID WORD1 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR2_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Lower 32 bit value of IC_SAR2 Target UDID register i.e. UDID[63:32] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x128
- **Exists:** IC_SAR2_SMBUS_UDID_HC==0



Table 5-79 Fields for Register: IC_SAR2_SMBUS_UDID_WORD1

Bits	Name	Memory Access	Description
31:0	SAR2_SMBUS_UDID_WORD1	R/W	This field is used to store the Middle-Lower 32 bit value of IC_SAR2 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR2_SMBUS_UDID_WORD1_DEFAULT Exists: Always

5.1.76 IC_SAR2_SMBUS_UDID_WORD2

- **Name:** IC_SAR2 SMBUS ARP UDID WORD2 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR2_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Upper 32 bit value of IC_SAR2 Target UDID register i.e. UDID[95:64] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x12c
- **Exists:** IC_SAR2_SMBUS_UDID_HC==0



Table 5-80 Fields for Register: IC_SAR2_SMBUS_UDID_WORD2

Bits	Name	Memory Access	Description
31:0	SAR2_SMBUS_UDID_WORD2	R/W	This field is used to store the Middle-Upper 32 bit value of IC_SAR2 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR2_SMBUS_UDID_WORD2_DEFAULT Exists: Always

5.1.77 IC_SAR2_SMBUS_UDID_WORD3

- **Name:** IC_SAR2 SMBUS ARP UDID WORD3 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR2_SMBUS_UDID_HC is set to 0. This register is used to store the Upper 32 bit value of IC_SAR2 Target UDID register i.e. UDID[127:96] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x130
- **Exists:** IC_SAR2_SMBUS_UDID_HC==0



Table 5-81 Fields for Register: IC_SAR2_SMBUS_UDID_WORD3

Bits	Name	Memory Access	Description
31:0	SAR2_SMBUS_UDID_WORD3	R/W	This field is used to store the Upper 32 bit value of IC_SAR2 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR2_SMBUS_UDID_WORD3_DEFAULT Exists: Always

5.1.78 IC_SAR3_SMBUS_UDID_WORD0

- **Name:** IC_SAR3 SMBUS ARP UDID WORD0 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR3_SMBUS_UDID_HC is set to 0. This register is used to store the Lower 32 bit value of IC_SAR3 Target UDID register i.e. UDID[31:0] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x134
- **Exists:** IC_SAR3_SMBUS_UDID_HC==0



Table 5-82 Fields for Register: IC_SAR3_SMBUS_UDID_WORD0

Bits	Name	Memory Access	Description
31:0	SAR3_SMBUS_UDID_WORD0	R/W	This field is used to store the Lower 32 bit value of IC_SAR3 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR3_SMBUS_UDID_WORD0_DEFAULT Exists: Always

5.1.79 IC_SAR3_SMBUS_UDID_WORD1

- **Name:** IC_SAR3 SMBUS ARP UDID WORD1 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR3_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Lower 32 bit value of IC_SAR3 Target UDID register i.e. UDID[63:32] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x138
- **Exists:** IC_SAR3_SMBUS_UDID_HC==0



Table 5-83 Fields for Register: IC_SAR3_SMBUS_UDID_WORD1

Bits	Name	Memory Access	Description
31:0	SAR3_SMBUS_UDID_WORD1	R/W	This field is used to store the Middle-Lower 32 bit value of IC_SAR3 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR3_SMBUS_UDID_WORD1_DEFAULT Exists: Always

5.1.80 IC_SAR3_SMBUS_UDID_WORD2

- **Name:** IC_SAR3 SMBUS ARP UDID WORD2 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR3_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Upper 32 bit value of IC_SAR3 Target UDID register i.e. UDID[95:64] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x13c
- **Exists:** IC_SAR3_SMBUS_UDID_HC==0



Table 5-84 Fields for Register: IC_SAR3_SMBUS_UDID_WORD2

Bits	Name	Memory Access	Description
31:0	SAR3_SMBUS_UDID_WORD2	R/W	This field is used to store the Middle-Upper 32 bit value of IC_SAR3 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR3_SMBUS_UDID_WORD2_DEFAULT Exists: Always

5.1.81 IC_SAR3_SMBUS_UDID_WORD3

- **Name:** IC_SAR3 SMBUS ARP UDID WORD3 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR3_SMBUS_UDID_HC is set to 0. This register is used to store the Upper 32 bit value of IC_SAR3 Target UDID register i.e. UDID[127:96] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x140
- **Exists:** IC_SAR3_SMBUS_UDID_HC==0



Table 5-85 Fields for Register: IC_SAR3_SMBUS_UDID_WORD3

Bits	Name	Memory Access	Description
31:0	SAR3_SMBUS_UDID_WORD3	R/W	This field is used to store the Upper 32 bit value of IC_SAR3 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR3_SMBUS_UDID_WORD3_DEFAULT Exists: Always

5.1.82 IC_SAR4_SMBUS_UDID_WORD0

- **Name:** IC_SAR4 SMBUS ARP UDID WORD0 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR4_SMBUS_UDID_HC is set to 0. This register is used to store the Lower 32 bit value of IC_SAR4 Target UDID register i.e. UDID[31:0] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x144
- **Exists:** IC_SAR4_SMBUS_UDID_HC==0



Table 5-86 Fields for Register: IC_SAR4_SMBUS_UDID_WORD0

Bits	Name	Memory Access	Description
31:0	SAR4_SMBUS_UDID_WORD0	R/W	This field is used to store the Lower 32 bit value of IC_SAR4 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR4_SMBUS_UDID_WORD0_DEFAULT Exists: Always

5.1.83 IC_SAR4_SMBUS_UDID_WORD1

- **Name:** IC_SAR4 SMBUS ARP UDID WORD1 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR4_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Lower 32 bit value of IC_SAR4 Target UDID register i.e. UDID[63:32] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x148
- **Exists:** IC_SAR4_SMBUS_UDID_HC==0



Table 5-87 Fields for Register: IC_SAR4_SMBUS_UDID_WORD1

Bits	Name	Memory Access	Description
31:0	SAR4_SMBUS_UDID_WORD1	R/W	This field is used to store the Middle-Lower 32 bit value of IC_SAR4 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR4_SMBUS_UDID_WORD1_DEFAULT Exists: Always

5.1.84 IC_SAR4_SMBUS_UDID_WORD2

- **Name:** IC_SAR4 SMBUS ARP UDID WORD2 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR4_SMBUS_UDID_HC is set to 0. This register is used to store the Middle-Upper 32 bit value of IC_SAR4 Target UDID register i.e. UDID[95:64] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x14c
- **Exists:** IC_SAR4_SMBUS_UDID_HC==0



Table 5-88 Fields for Register: IC_SAR4_SMBUS_UDID_WORD2

Bits	Name	Memory Access	Description
31:0	SAR4_SMBUS_UDID_WORD2	R/W	This field is used to store the Middle-Upper 32 bit value of IC_SAR4 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR4_SMBUS_UDID_WORD2_DEFAULT Exists: Always

5.1.85 IC_SAR4_SMBUS_UDID_WORD3

- **Name:** IC_SAR4 SMBUS ARP UDID WORD3 Register
- **Description:** This Register can be written only when the DW_apb_i2c is disabled, which corresponds to IC_ENABLE[0] being set to 0. This register is present only if configuration parameter IC_SAR4_SMBUS_UDID_HC is set to 0. This register is used to store the Upper 32 bit value of IC_SAR4 Target UDID register i.e. UDID[127:96] used in Address Resolution Protocol of SMBus.
- **Size:** 32 bits
- **Offset:** 0x150
- **Exists:** IC_SAR4_SMBUS_UDID_HC==0



Table 5-89 Fields for Register: IC_SAR4_SMBUS_UDID_WORD3

Bits	Name	Memory Access	Description
31:0	SAR4_SMBUS_UDID_WORD3	R/W	This field is used to store the Upper 32 bit value of IC_SAR4 target unique device identifier used in Address Resolution Protocol. Value After Reset: IC_SAR4_SMBUS_UDID_WORD3_DEFAULT Exists: Always

5.1.86 IC_SMBUS_ARP_CTRL

- **Name:** I2C SMBus ARP Control Register
- **Description:** I2C SMBus ARP Control Register.
This register is used to choose the different SMBus device types in DW_apb_i2c. This register can be written only when the DW_apb_i2c is disabled, which corresponds to the IC_ENABLE[0] register being set to 0. Writes at other times have no effect.
Note: When IC_SMBUS_FD_FND_SUPPORT_EN=1, 'SMBUS_ARP_EN' and 'IC_SARx_SMBUS_ARP_EN' bits of 'IC_CON' register cannot be programmed to enable the ARP capability.
- **Size:** 32 bits
- **Offset:** 0x154
- **Exists:** IC_SMBUS_FD_FND_SUPPORT_EN == 1

31:8	IC_SMBUS_ARP_CTRL_RSVD
7:6	IC_SAR4_SMBUS_DEVICE_TYPE
5:4	IC_SAR3_SMBUS_DEVICE_TYPE
3:2	IC_SAR2_SMBUS_DEVICE_TYPE
1:0	IC_SAR_SMBUS_DEVICE_TYPE

Table 5-90 Fields for Register: IC_SMBUS_ARP_CTRL

Bits	Name	Memory Access	Description
31:8	IC_SMBUS_ARP_CTRL_RSVD	R	IC_SMBUS_ARP_CTRL reserved bits - read only Value After Reset: 0x0 Exists: Always

Bits	Name	Memory Access	Description
7:6	IC_SAR4_SMBUS_DEVICE_TYPE	R/W	This bit can be programmed to select the desired SMBus device type for IC_SAR4 target address. Values: ■ 0x0 (NARP_DEVICE_TYPE): Non-ARP capable device ■ 0x1 (ARP_DEVICE_TYPE): ARP capable device ■ 0x2 (FD_DEVICE_TYPE): Fixed and Discoverable device ■ 0x3 (FND_DEVICE_TYPE): Fixed and non-Discoverable device Value After Reset: IC_SAR4_DFLT_SMBUS_DEVICE_TYPE Exists: IC_MULTI_SAR_EN==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_NUM_SARS>=4
5:4	IC_SAR3_SMBUS_DEVICE_TYPE	R/W	This bit can be programmed to select the desired SMBus device type for IC_SAR3 target address. Values: ■ 0x0 (NARP_DEVICE_TYPE): Non-ARP capable device ■ 0x1 (ARP_DEVICE_TYPE): ARP capable device ■ 0x2 (FD_DEVICE_TYPE): Fixed and Discoverable device ■ 0x3 (FND_DEVICE_TYPE): Fixed and non-Discoverable device Value After Reset: IC_SAR3_DFLT_SMBUS_DEVICE_TYPE Exists: IC_MULTI_SAR_EN==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_NUM_SARS>=3
3:2	IC_SAR2_SMBUS_DEVICE_TYPE	R/W	This bit can be programmed to select the desired SMBus device type for IC_SAR2 target address. Values: ■ 0x0 (NARP_DEVICE_TYPE): Non-ARP capable device ■ 0x1 (ARP_DEVICE_TYPE): ARP capable device ■ 0x2 (FD_DEVICE_TYPE): Fixed and Discoverable device ■ 0x3 (FND_DEVICE_TYPE): Fixed and non-Discoverable device Value After Reset: IC_SAR2_DFLT_SMBUS_DEVICE_TYPE Exists: IC_MULTI_SAR_EN==1 && IC_SMBUS_FD_FND_SUPPORT_EN==1 && IC_NUM_SARS>=2

Bits	Name	Memory Access	Description
1:0	IC_SAR_SMBUS_DEVICE_TYPE	R/W	This bit can be programmed to select the desired SMBus device type for IC_SAR target address. Values: ■ 0x0 (NARP_DEVICE_TYPE): Non-ARP capable device ■ 0x1 (ARP_DEVICE_TYPE): ARP capable device ■ 0x2 (FD_DEVICE_TYPE): Fixed and Discoverable device ■ 0x3 (FND_DEVICE_TYPE): Fixed and non-Discoverable device Value After Reset: IC_SAR_DFLT_SMBUS_DEVICE_TYPE Exists: IC_SMBUS_FD_FND_SUPPORT_EN==1

6

Programming the DW_apb_i2c

This chapter describes some programming flow and sequences in DW_apb_i2c.

Following are some programming flows:

- [“Programming Example” on page 371](#)
- [“Programming Flow for SCL and SDA Bus Recovery” on page 379](#)
- [“Programming Flow for Reading the Device ID” on page 380](#)
- [“Programming Flow of DW_apb_i2c in Ultra-Fast Mode” on page 381](#)

Following are some sequences:

- [“SMBus Timeout Sequence in Controller Mode” on page 383](#)
- [“SMBus Timeout Sequence in Target Mode” on page 384](#)
- [“ARP Controller Sequence” on page 385](#)
- [“ARP Target Sequence” on page 386](#)
- [“SMBus SUSPEND Sequence in Host Mode” on page 389](#)
- [“SMBus SUSPEND Sequence in Device Mode” on page 390](#)
- [“SMBus ALERT Sequence in Host Mode” on page 391](#)
- [“SMBus ALERT Sequence in Device Mode” on page 392](#)

6.1 Software Registers

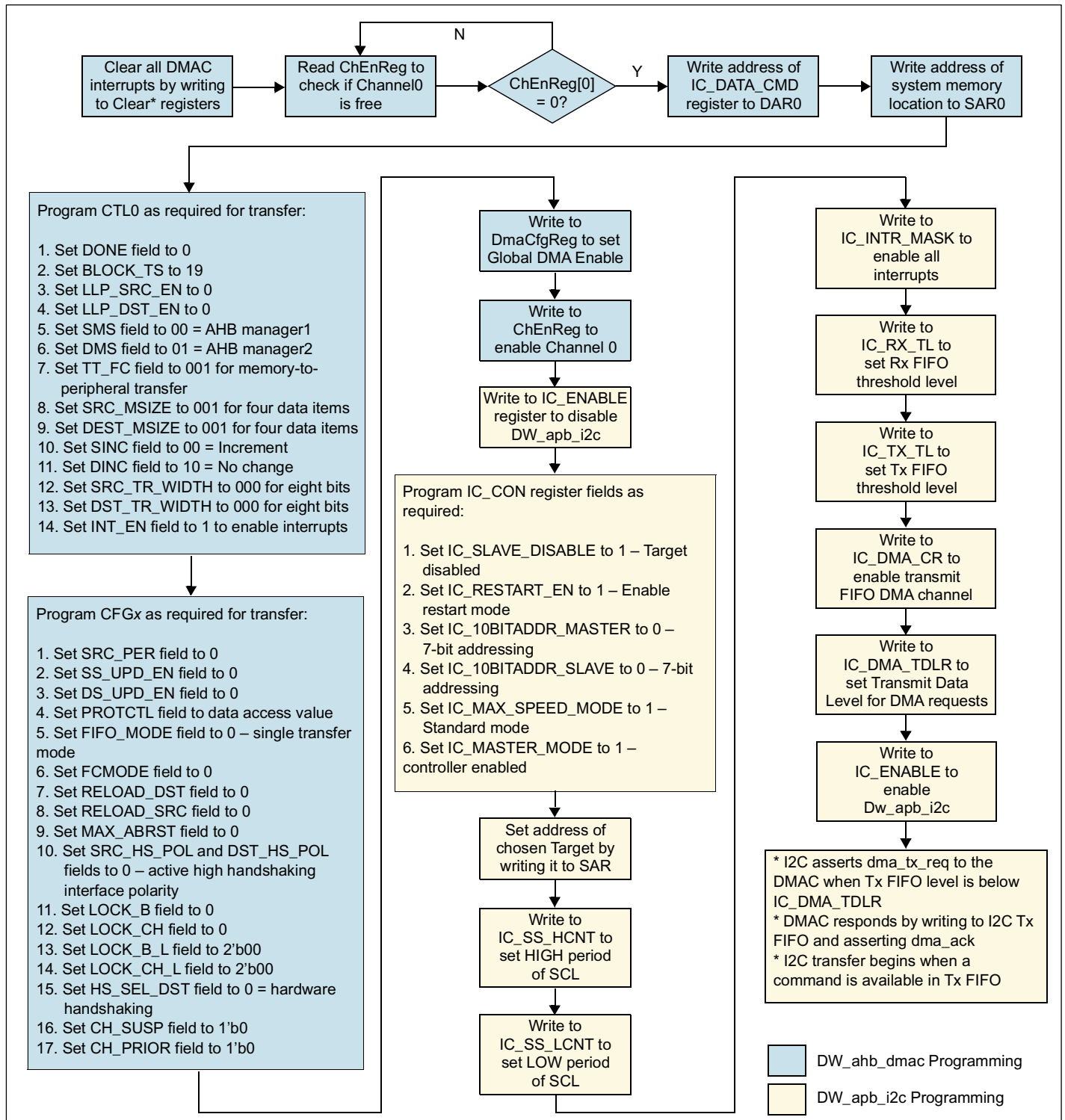
The DW_apb_i2c can be programmed through software registers.

For information about programming the software registers in terms of DW_apb_i2c operation, see [“Target Mode Operation”](#) on page 52 and [“Controller Mode Operation”](#) on page 56. The software registers are described in more detail in [Chapter 5, “Register Descriptions”](#).

6.2 Programming Example

The flow diagram in [Figure 6-1](#) shows an overview of programming the DW_apb_i2c.

Figure 6-1 Flowchart for DW_ahb_dmac and DW_apb_i2c Programming Example



**Note**

When there is at least one entry in the DW_apb_i2c Rx FIFO, the DW_apb_i2c asserts `dma_single` to the DMAC. When the number of entries in the DW_apb_i2c Rx FIFO reaches `IC_DMA_RDLR`, the DW_apb_i2c asserts `dma_rx_req` to the DMAC. In this example, in order to read nineteen data items from the DW_apb_i2c Rx FIFO, the DMAC samples `dma_req` for three BURST transfers of four beats of size 1 byte each, and it samples `dma_single` for three SINGLE transfers of size 1 byte each.

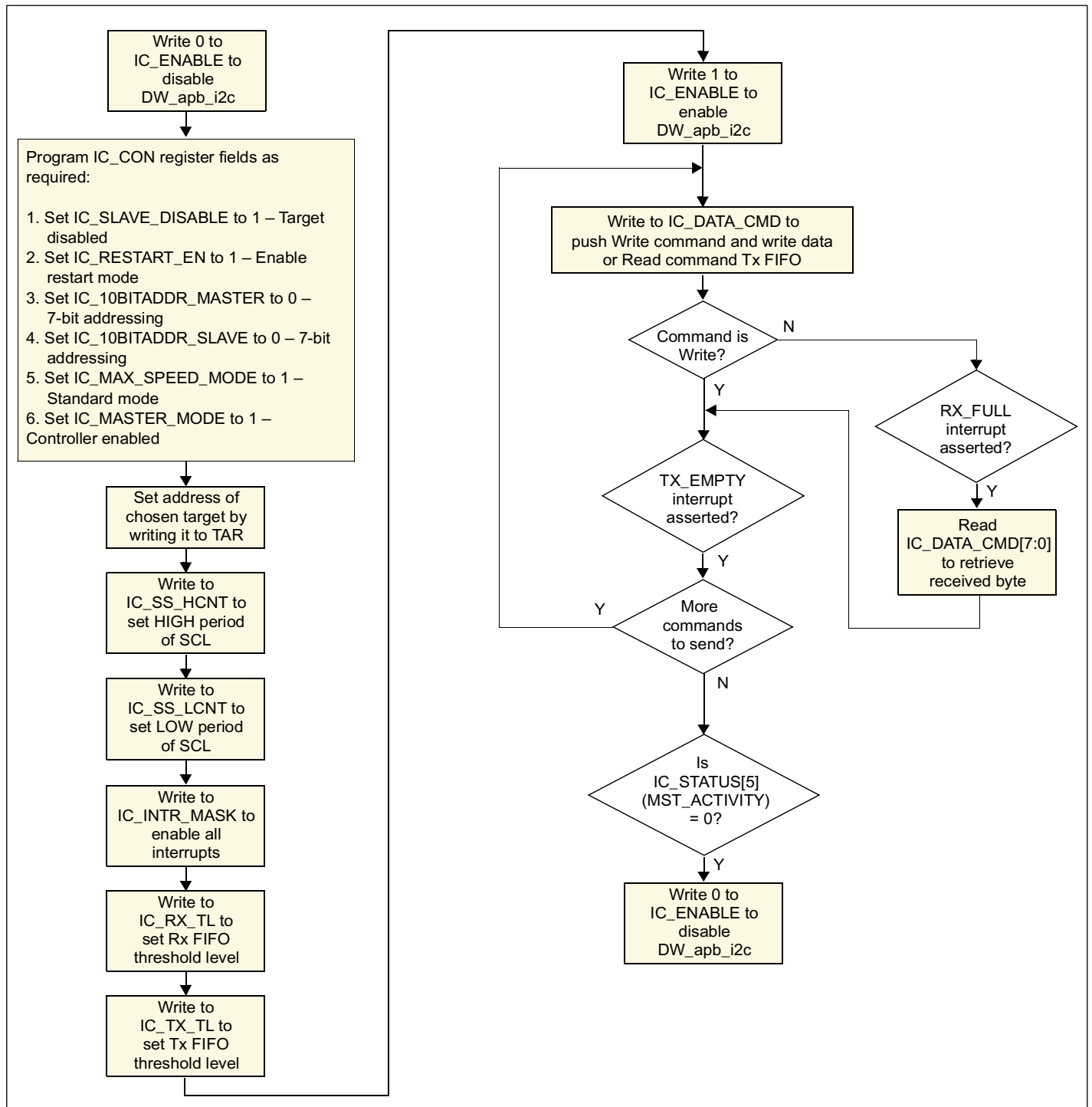
The following outlines details regarding reads and writes to/from DW_apb_i2c controllers/target devices and VIP controller/target devices:

- For DW_apb_i2c controller writes to a target VIP model, bear in mind when using the DMA that you are writing characters from the byte stream. However, for a write, the DW_apb_i2c needs a halfword. To use the DMA, you should write software similar to the following:

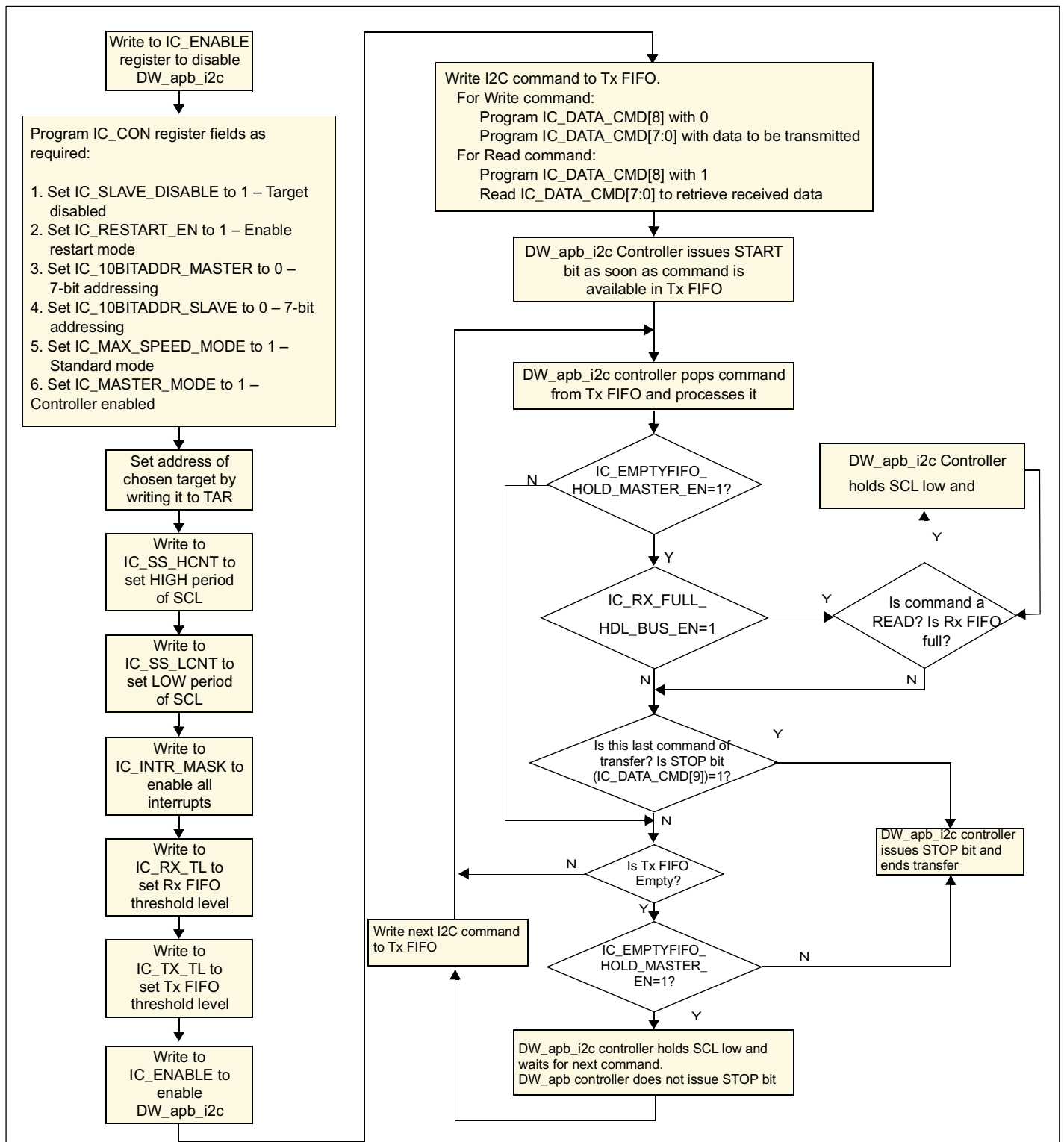

```
short int temp_array[];
char * ptr=(char *) temp_array;
foreach byte in bytes {
    store byte ptr++;
    store '0x01' write command ptr++
}
```

 - a. Program the DMA to read a stream of halfwords from memory and write them to the DW_apb_i2c using the hardware interface.
 - b. Program the DW_apb_i2c to do a write using the transmit DMA.
- For DW_apb_i2c controller reads from a target VIP model:
 - a. Create a read command halfword.
 - b. Program DMA channel 0 to do a fixed read of the read command halfword – that is, no address increment – to the DW_apb_i2c transmit buffer.
 - c. Program DMA channel 1 to read the data from the read buffer and store it in memory.
 - d. Program the DW_apb_i2c to do a controller read by setting both DMA channels.
- For DW_apb_i2c target writes from a controller VIP model:
 - a. Program the DW_apb_i2c to be a target with the RX buffer DMA enabled.
 - b. Program the DMA to read the buffer and store the bytes in memory.
- For DW_apb_i2c target reads from a controller VIP model:
 - a. Enable `IC_INTR_MASK.RD_REQ`; otherwise the DW_apb_i2c does not acknowledge the read.
 - b. When you get the `RD_REQ` interrupt, program the DMA to write the TX buffer with the read data.
 - c. Program the DW_apb_i2c to enable the TX DMA.

The flow diagram in [Figure 6-2](#) shows a programming example for the DW_apb_i2c controller.

Figure 6-2 Flowchart for DW_apb_i2c Controller

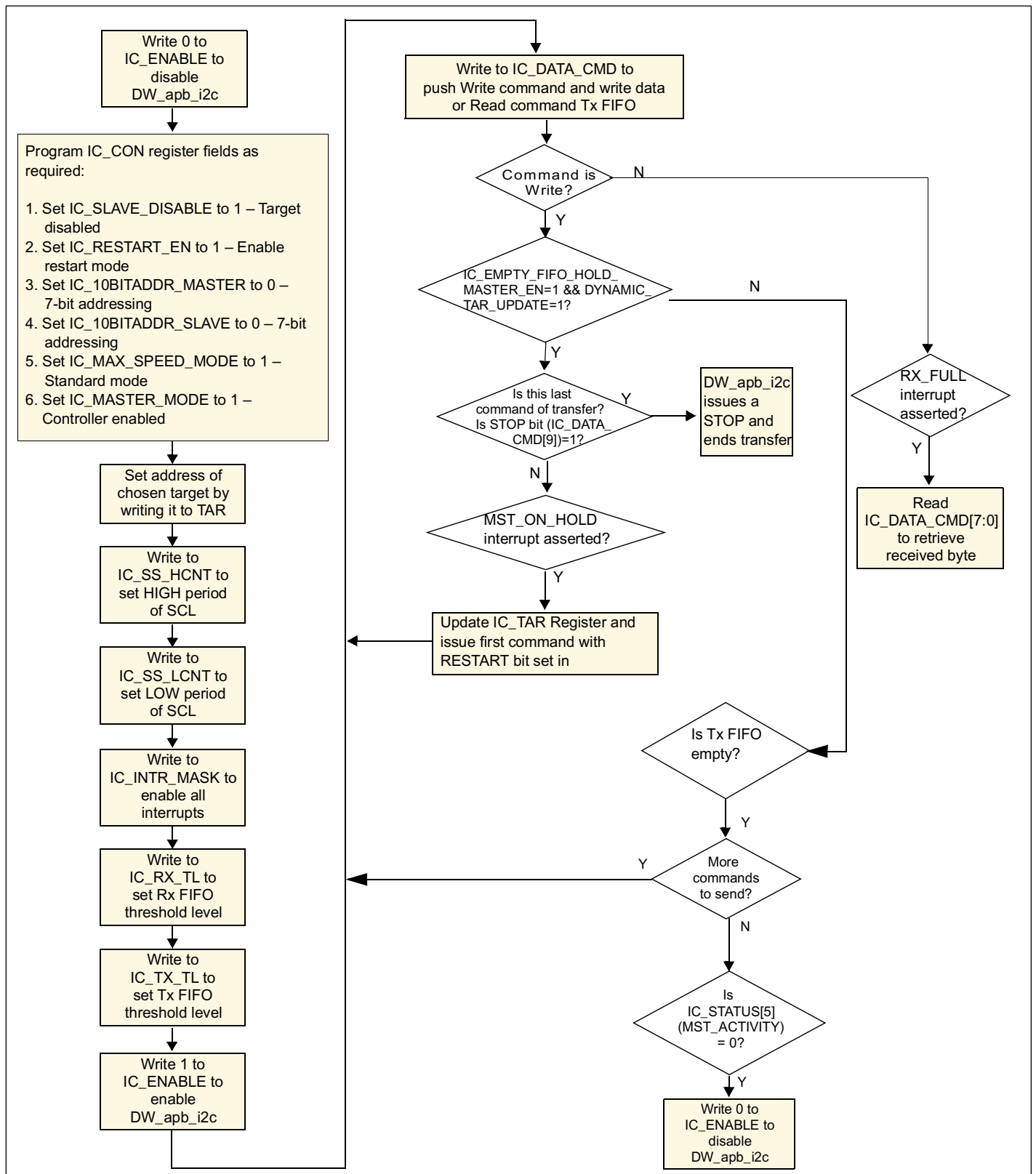
The flow diagram in [Figure 6-3](#) shows a programming example for the DW_apb_i2c controller in standard mode, fast mode, or fast mode plus with 7-bit addressing.

Figure 6-3 Flowchart for DW_apb_i2c Controller in Standard Mode, Fast Mode, or Fast Mode Plus

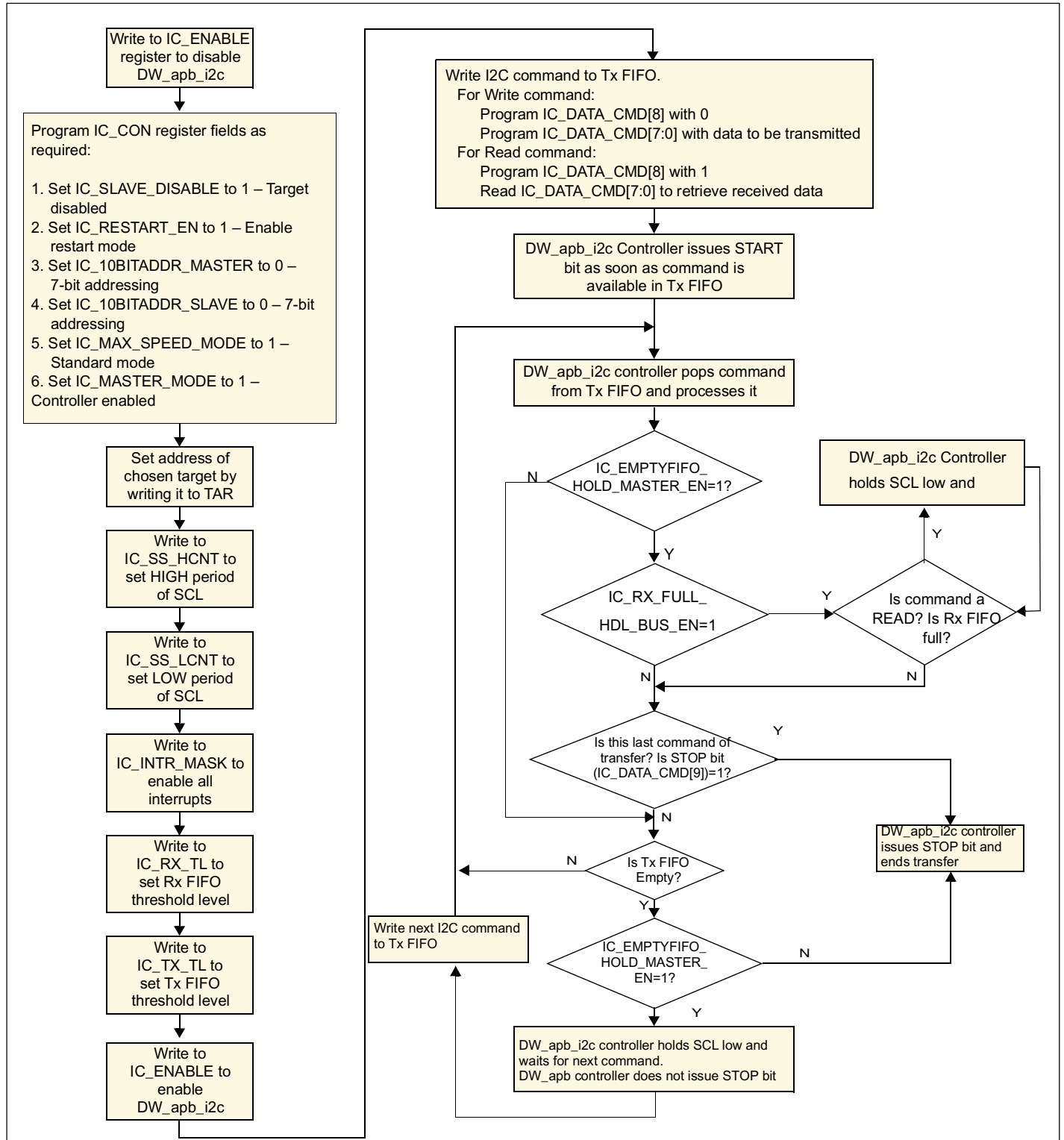
The flow diagram in [Figure 6-4](#) shows a programming example for DW_apb_i2c as controller with TAR address update. This flow shows how the MST_ON_HOLD interrupt is used when the software needs information from the hardware to safely update the TAR address.



When the software has full knowledge of when it is safe to update the TAR address without requiring information from hardware, the MST_ON_HOLD interrupt is not required to update the TAR address.

Figure 6-4 Flowchart for DW_apb_i2c Controller with TAR Address Update

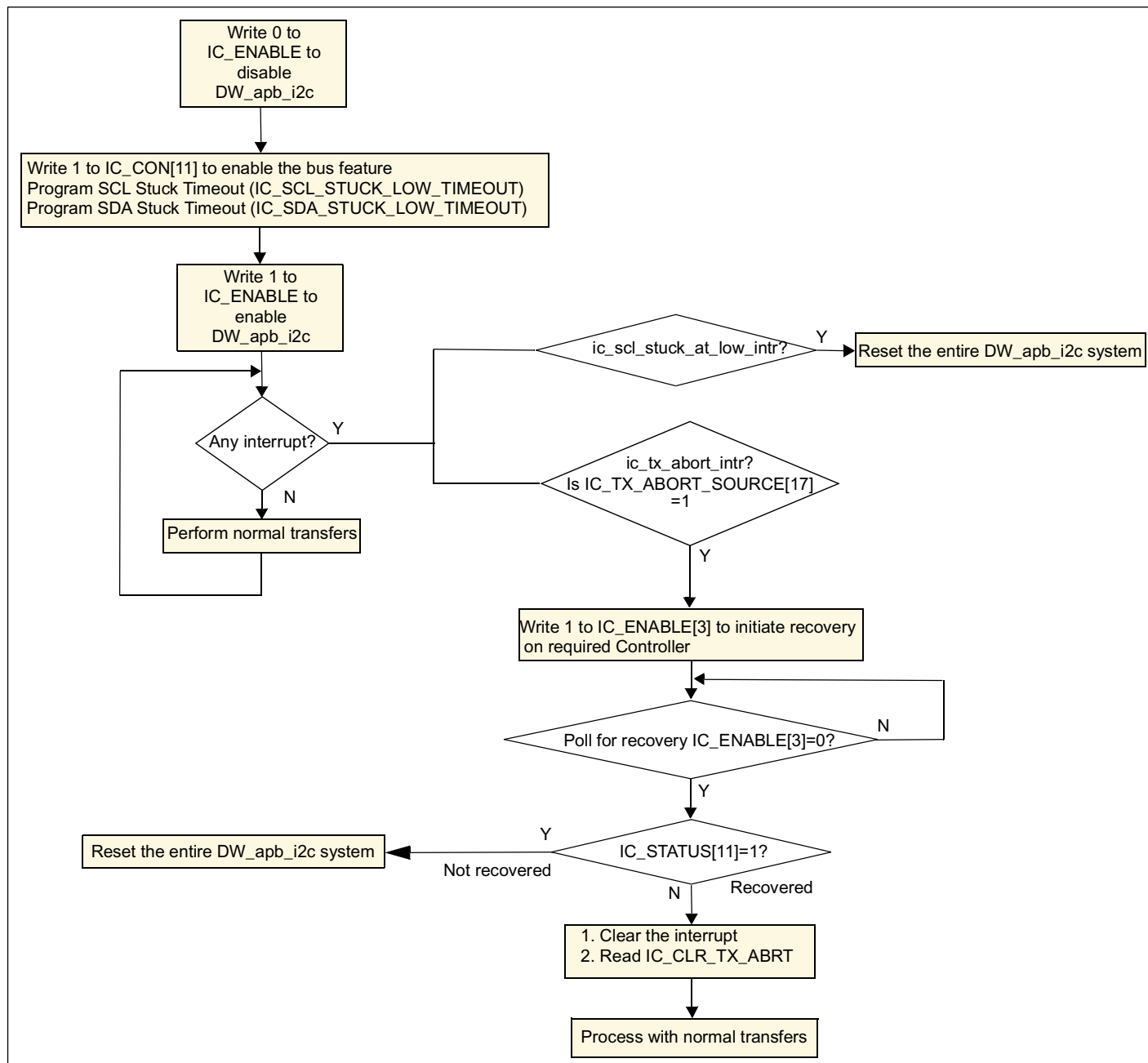
The flow diagram in [Figure 6-5](#) shows a programming example for the DW_apb_i2c target in standard mode, fast mode, or fast mode plus with 7-bit addressing.

Figure 6-5 Flowchart for DW_apb_i2c target in Standard Mode, Fast Mode, or Fast Mode Plus with 7-bit Addressing

6.3 Programming Flow for SCL and SDA Bus Recovery

The flow diagram in [Figure 6-6](#) shows a programming example for SCL and SDA bus recovery.

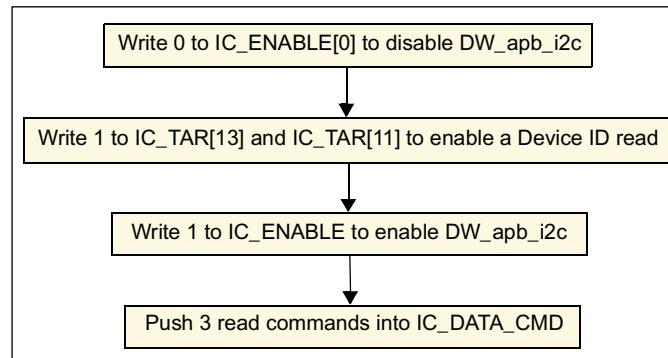
Figure 6-6 Flowchart for SCL and SDA Bus Recovery



6.4 Programming Flow for Reading the Device ID

Figure 6-7 shows a programming flow in the controller to initiate a Device ID read.

Figure 6-7 Flowchart for Reading a Device ID



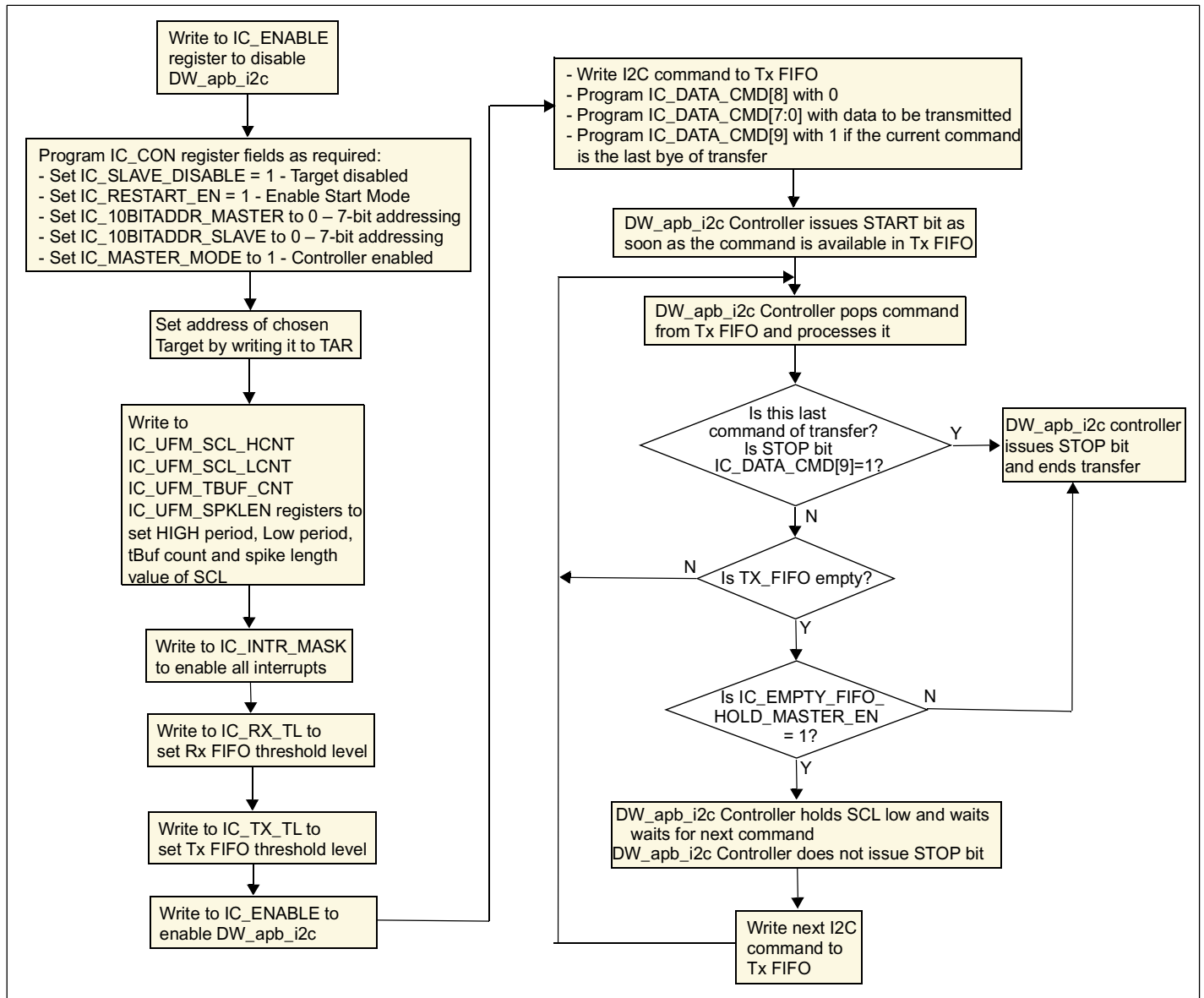
As the Device ID consists of 3 bytes, the user must issue 3 read commands in IC_DATA_CMD register. One read command populates one byte of Device ID in RX FIFO. If more than 3 commands are issued, the Device ID rolls back.

6.5 Programming Flow of DW_apb_i2c in Ultra-Fast Mode

This section describes the programming flow of DW_apb_i2c in Ultra-fast controller and target modes.

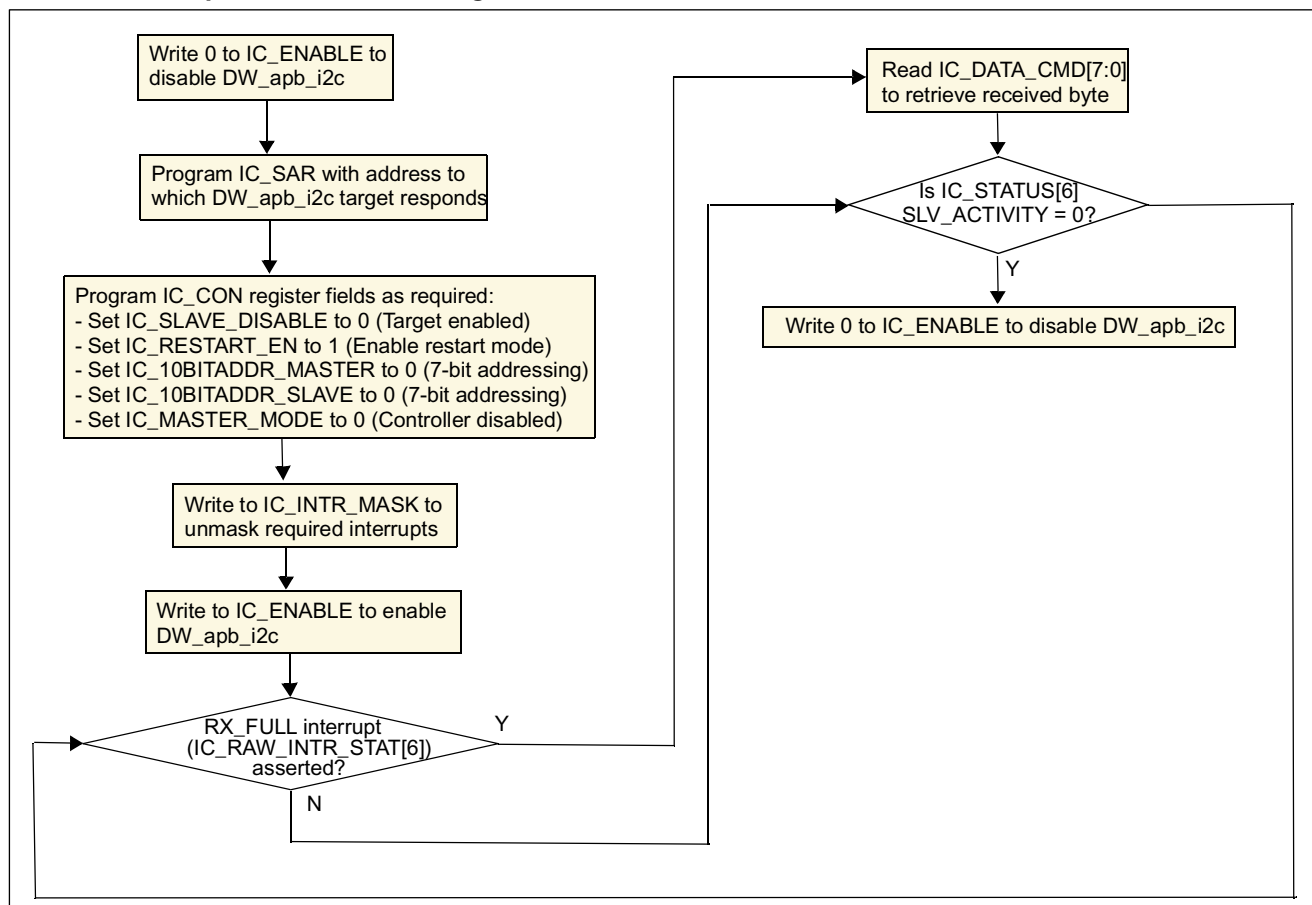
6.5.1 DW_apb_i2c Controller Mode

Figure 6-8 DW_apb_i2c Ultra-Fast Controller Mode



6.5.2 DW_apb_i2c Target Mode

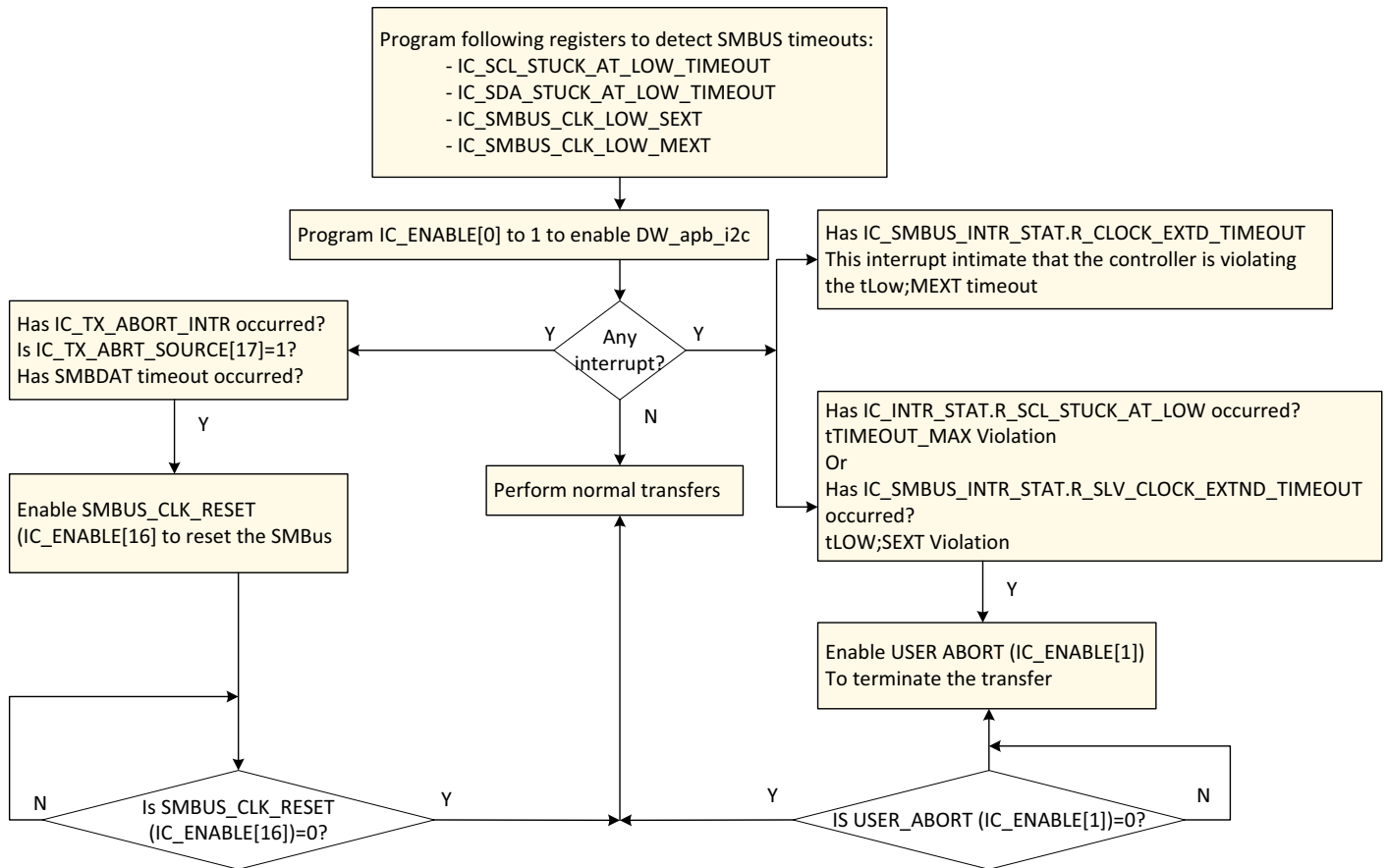
Figure 6-9 DW_apb_i2c Ultra-Fast Target Mode



6.6 SMBus Timeout Sequence in Controller Mode

Figure 6-10 shows the sequence for SMBus timeout in controller mode.

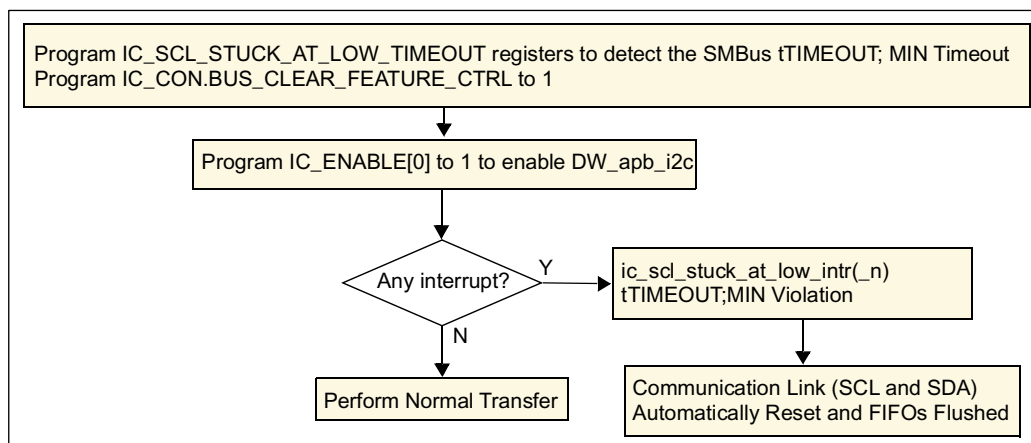
Figure 6-10 SMBus Timeout Sequence in Controller Mode



6.7 SMBus Timeout Sequence in Target Mode

Figure 6-11 shows a sequence for SMBus timeout in target mode.

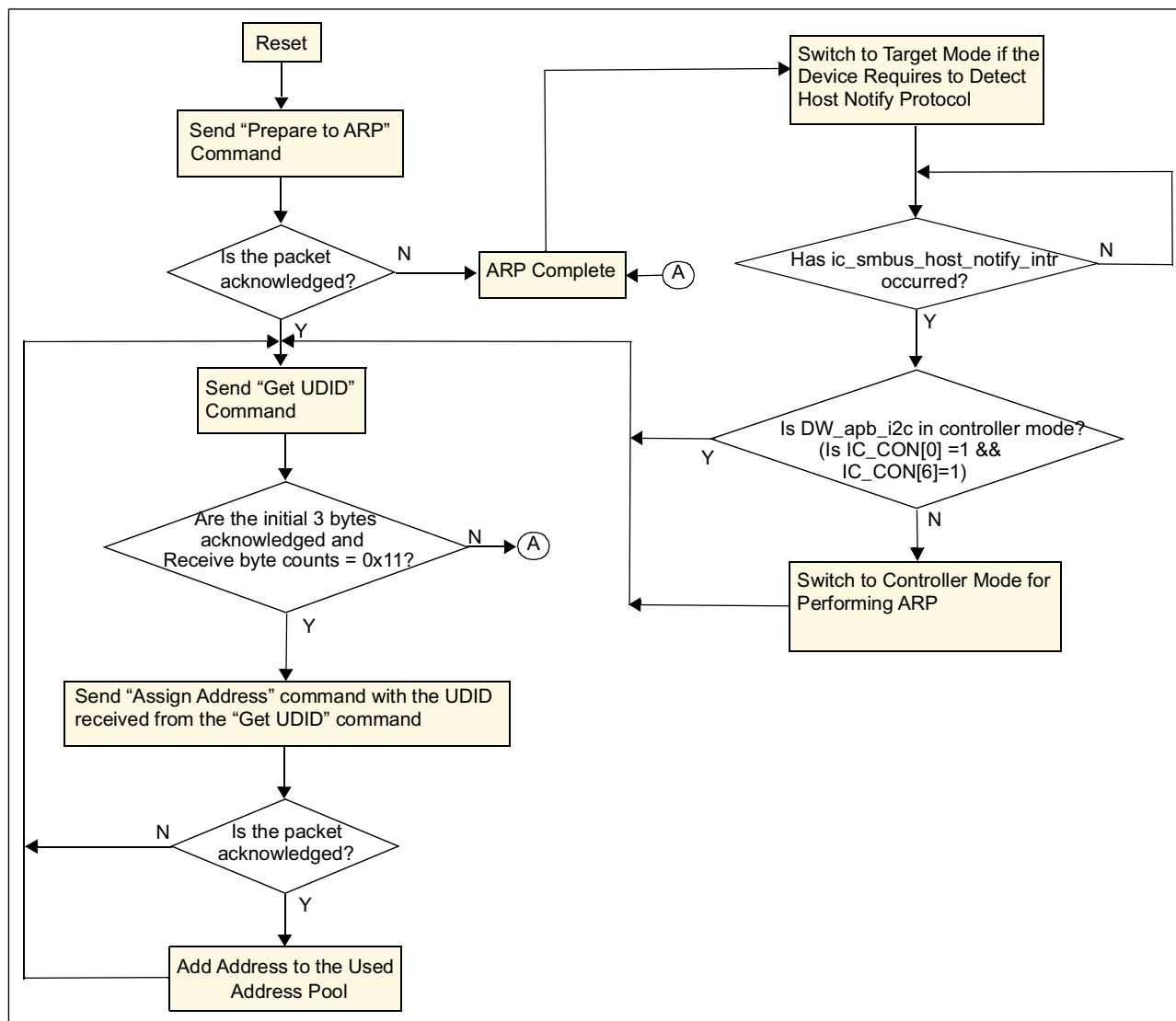
Figure 6-11 SMBus Timeout Sequence in Target Mode



6.8 ARP Controller Sequence

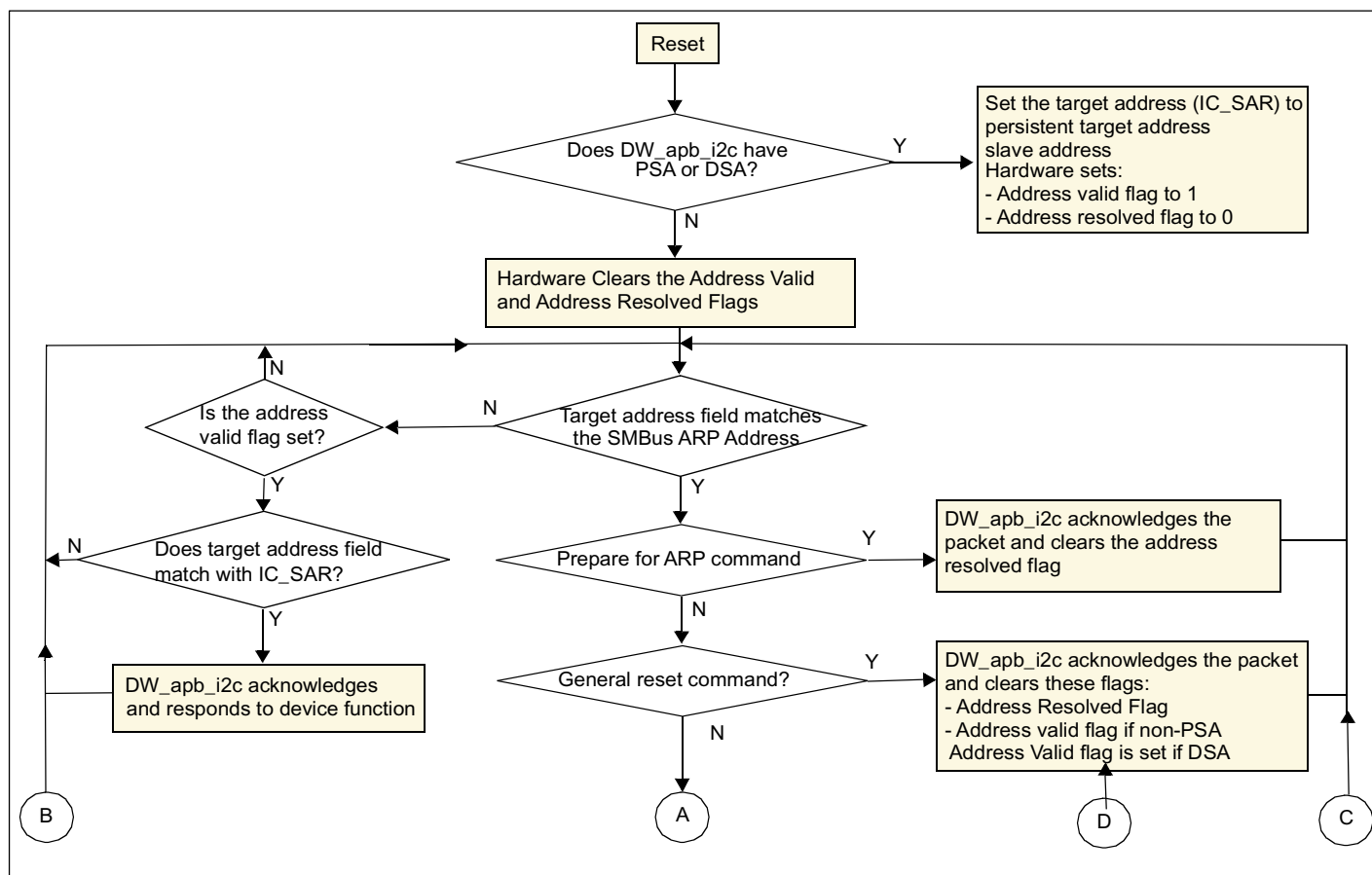
Figure 6-12 shows the sequence for an ARP controller.

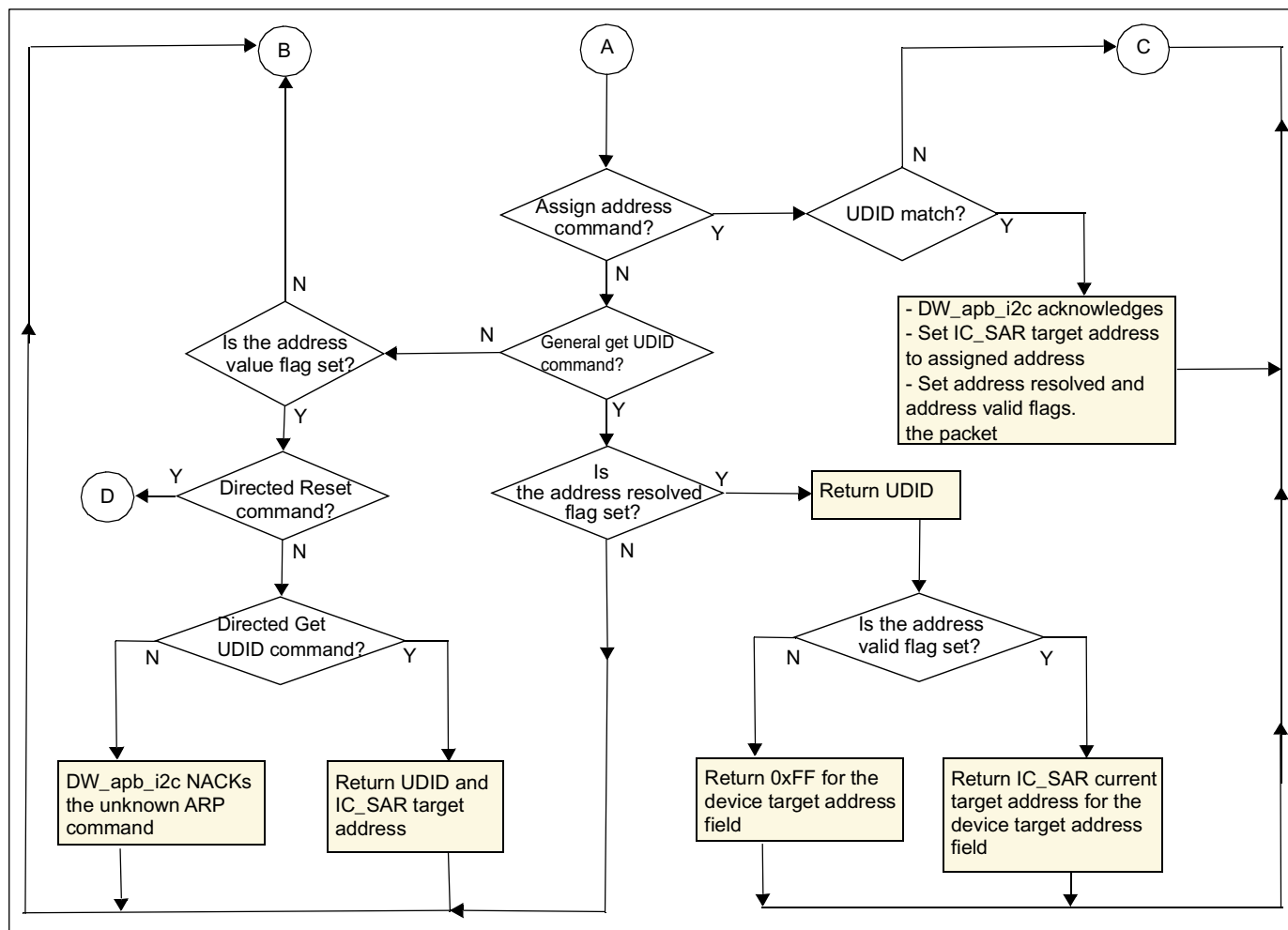
Figure 6-12 ARP Controller Sequence



6.9 ARP Target Sequence

Figure 6-13 shows the sequence for an ARP target.

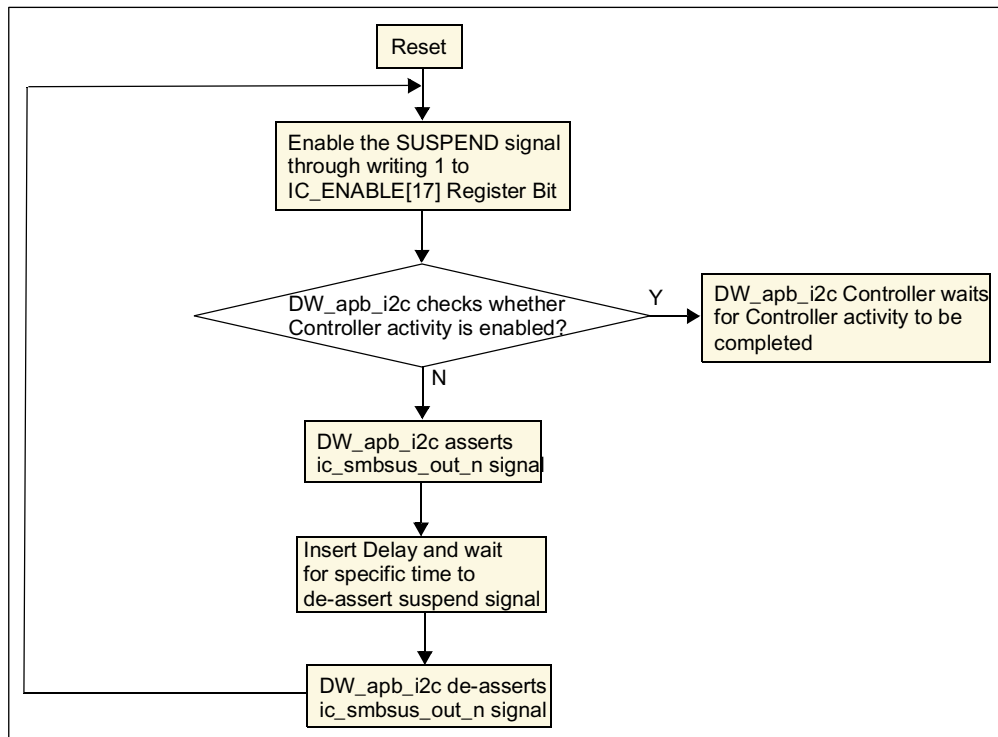
Figure 6-13 ARP Target Sequence



6.10 SMBus SUSPEND Sequence in Host Mode

Figure 6-14 shows the sequence for SMBus SUSPEND in the Host mode.

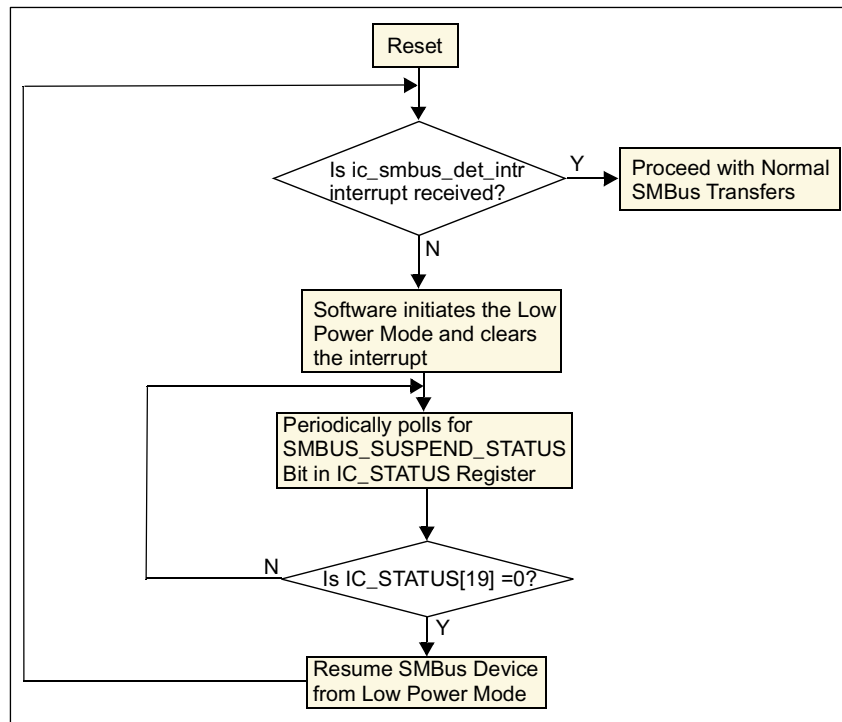
Figure 6-14 SMBus SUSPEND Sequence in Host Mode



6.11 SMBus SUSPEND Sequence in Device Mode

Figure 6-15 shows the SMBus SUSPEND sequence in the Device mode.

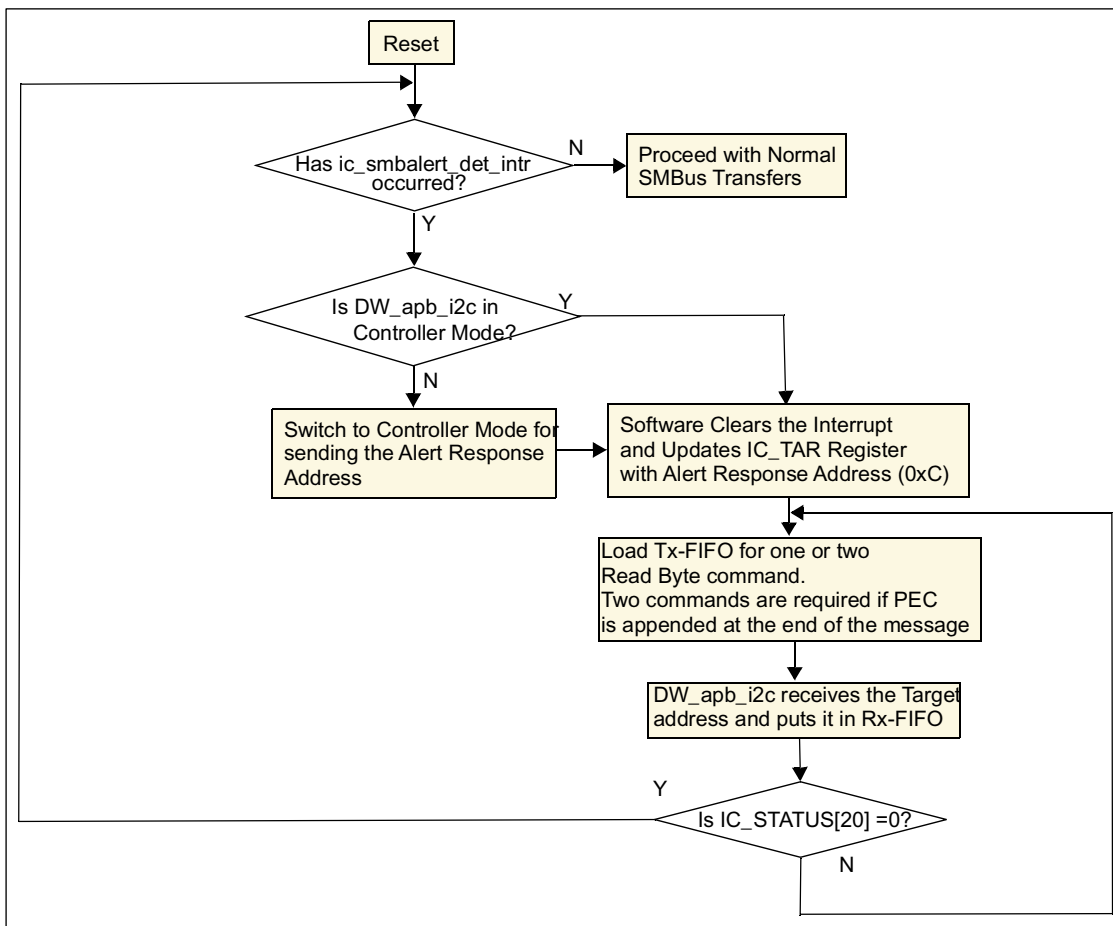
Figure 6-15 SMBus SUSPEND Sequence in Device Mode



6.12 SMBus ALERT Sequence in Host Mode

Figure 6-16 shows the SMBus ALERT sequence in the Host mode.

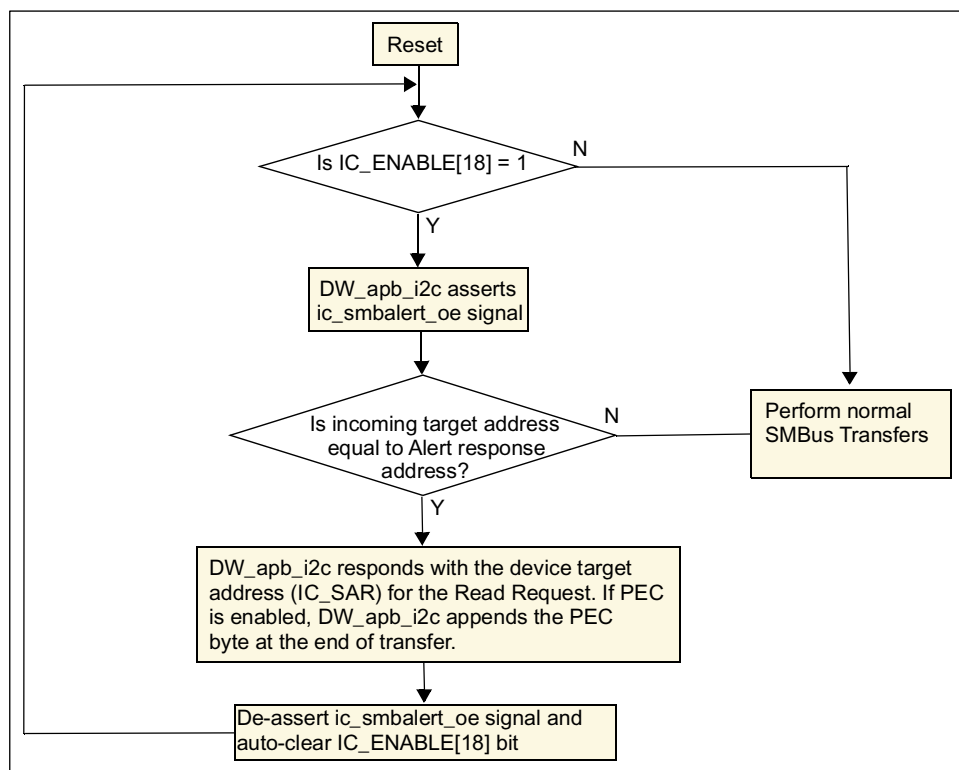
Figure 6-16 SMBus ALERT Sequence in Host Mode



6.13 SMBus ALERT Sequence in Device Mode

Figure 6-17 shows SMBus ALERT sequence in Device mode.

Figure 6-17 SMBus ALERT Sequence in Device Mode



7

Verification

This chapter provides an overview of the testbench available for the DW_apb_i2c verification. Once the DW_apb_i2c has been configured and the verification setup, simulations can be run automatically. For information on running simulations for DW_apb_i2c in coreAssembler or coreConsultant, refer to the “Simulating the AMBA Component” chapter in the *DesignWare Synthesizable Components for AMBA 2 User Guide*.

**Note**

The DW_apb_i2c verification testbench is built using Synopsys VC Verification IP (VIP). Ensure that you have the supported version of the VIP components for this release, otherwise, you might experience some tool compatibility problems. For more information about supported tools in this release, see the “Installing Tools and Verification IP Components” chapter in the *DesignWare Synthesizable Components for AMBA 2 Installation Guide*.

**Note**

The packaged test benches are only for validating the IP configuration in coreConsultant GUI. It is not for system level validation.

This chapter discusses the following sections:

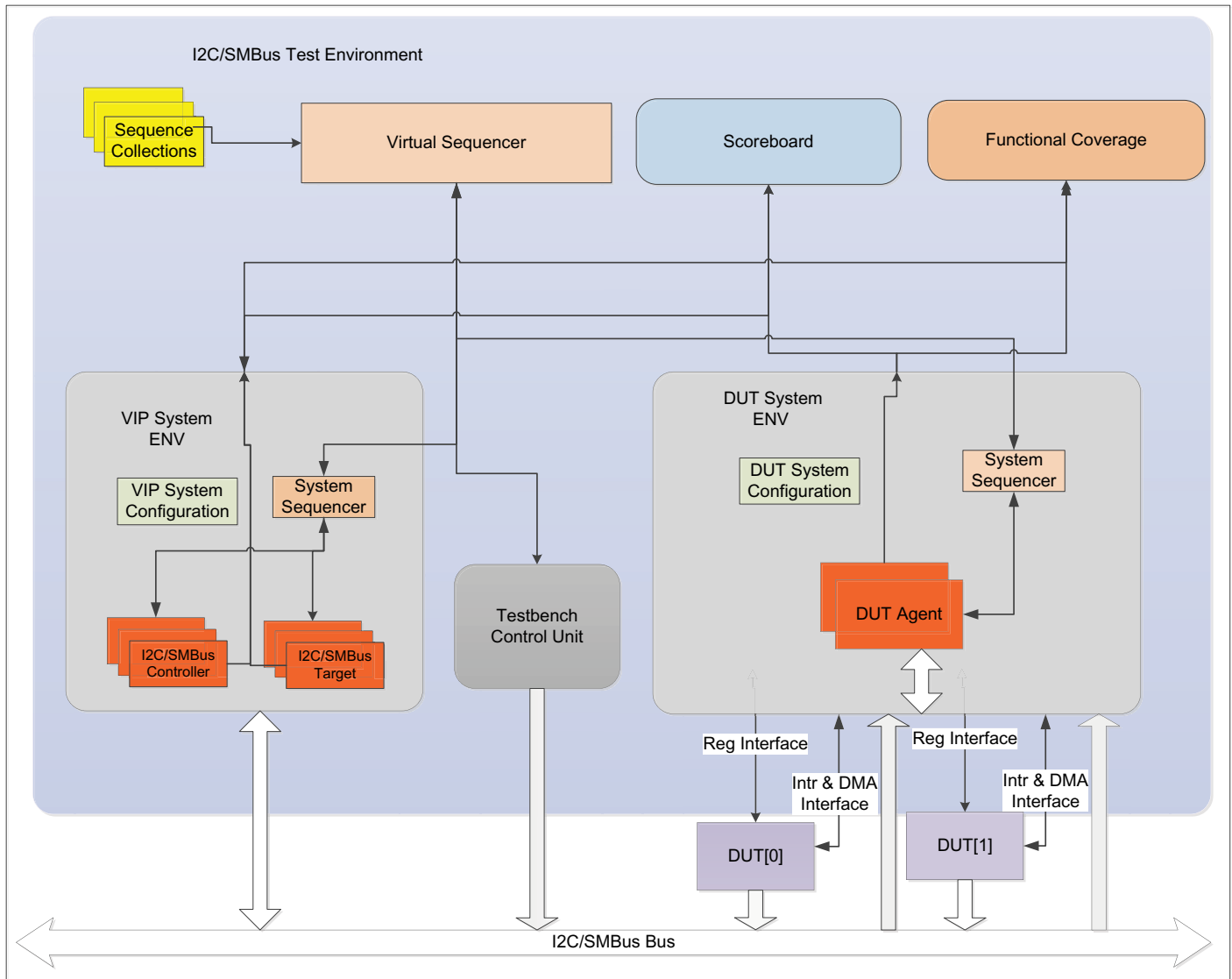
- “Verification Environment” on page 394
- “Testbench Directories and Files” on page 396
- “Packaged Testcases” on page 397

7.1 Verification Environment

DW_apb_i2c is verified using a UVM-methodology-based constrained random verification environment. The environment can generate random scenarios and the test case has hooks to control the scenarios to be generated.

Figure 7-1 shows the verification environment of the DW_apb_i2c testbench:

Figure 7-1 DW_apb_i2c UVM Verification Environment



The testbench consists of the following elements:

Testbench makes use of the standard SVT VIP for the protocol interfaces:

- AMBA SVT VIP
 - APB Requester VIP model is used to access register interface
- I²C SVT VIP

- I²C VIP for the interface with I²C Data transfer interface

7.1.1 VIP System Env – I²C Controller/Target VIP

This I²C VIP component is responsible for creating SMBus controller and target device agents and related transfers on the bus. Here one of the VIP agents is used as active Controller agent which initiates the necessary transfers. And a passive Target device agent is used to capture the initiated transfers and will be sent to the scoreboard for data integrity checking.

APB VIP component is responsible for register programming and issuing commands to FIFO and interrupt related checks on the primary side. Standard RAL based setup is being used for this.

7.1.2 DUT System Environment

On the DUT primary side, the transaction handling and monitoring for transactions and interrupt related things are being handled by DUT agent. In the top-level DUT system env, we can have multiple DUT agents.

DUT agent (dw_apb_i2c_agent): This represents an Agent which drives and monitors the DUT Interface. The DUT agent consists of the following component.

- DUT Driver (dw_apb_i2c_driver): This driver converts the dw_apb_i2c_transaction into the DUT commands and updates the same with interrupt and Status information.
- DUT Monitor (dw_apb_i2c_monitor): This Monitor the I²C bus with the DUT specific checking, such as timings.
- DUT common (dw_apb_i2c_common): This encapsulates the Shared Functionality between the driver and the monitor.
- DUT sequencer (dw_apb_i2c_sequencer): This Sequencer drives the dw_apb_i2c_transaction to the DUT driver (dw_apb_i2c_driver).
- RAL Model (ral_sys_DW_apb_i2c): This is the auto generated register model for the DUT, through this register model the DUT register access will be carried on
- APB Adapter (dw_apb_i2c_reg2apb_adapter): This adapter model is used convert the register model commands into Register interface APB VIP transaction and vice versa
- APB Requester Agent (svt_apb_requester_agent): This is an APB VIP Requester agent
- DUT configuration (dw_apb_i2c_configuration): This configuration file takes care of the programming attributes used in randomization of DUT programming flow and for some testbench control variables, status flags and so on.

7.2 Testbench Directories and Files

The DW_apb_i2c verification environment contains the following directories and associated files. [Table 7-1](#) shows the various directories and associated files:

Table 7-1 DW_apb_i2c Testbench Directory Structure

Directory	Description
<configured workspace>/sim/testbench	Top level testbench module (test_top.sv) and the DUT to the testbench wrapper (dut_sv_wrapper.sv) exist in this folder.
<configured workspace>/sim/testbench/env	Contains testbench files. For example, scoreboard, sequences, VIP, environment, sequencers, and agents.
<configured workspace>/sim/	Primarily contains the supporting files to compile and run the simulation. After the completion of the simulation, the log files are present here.
<configured workspace>/sim/test_*	Contains individual test cases. After the completion of the simulation, the test specific log files and if applicable the waveform files are stored here.

7.3 Packaged Testcases

The simulation environment that comes as a package files includes some demonstrative tests. Some or all the packaged demonstrative tests, depending upon their applicability to the chosen configuration, are displayed in **Setup and Run Simulations > Testcases** in the coreConsultant GUI.

The associated test cases shipped, and their description is explained in [Table 7-2](#).

Table 7-2 DW_apb_i2c Test Description

Test Name	
test_i2c_basic	This test consists of two instances of DW_apb_i2c (one is Controller and the other is Target) and is aimed at generating random I ² C read write traffic. The traffic is generated randomly based on the DW_apb_i2c (RAL based register) configuration and VIP is used to monitor the traffic and flag the errors in case of protocol violation. The I ² C transfer control attributes are randomized within the protocol limits and as per the DW_apb_i2c requirement.
test_smbus_basic	This test is aimed at generating random non-ARP SMBus traffic. I ² C Controller Sequence from the VIP will generate different possible non-ARP SMBus transfers and the DW_apb_i2c Target will respond accordingly. The SMBUS transfer control attributes are randomized within the protocol limits and as per the DW_apb_i2c requirement.

8

Integration Considerations

After you have configured, tested, and synthesized your component with the coreTools flow, you can integrate the component into your own design environment. The following sections discuss general integration considerations.

8.1 Accessing Top-level Constraints

To get SDC constraints out of coreConsultant, you need to first complete the synthesis activity and then use the “write_sdc” command to write out the results:

1. This cC command sets synthesis to write out scripts only, without running DC:

```
set_activity_parameter Synthesize ScriptsOnly 1
```

2. This cC command autocompletes the activity:

```
autocomplete_activity Synthesize
```

3. Finally, this cC command writes out SDC constraints:

```
write_sdc <filename>
```


8.2 Performance

This section discusses performance and the hardware configuration parameters that affect the performance of the DW_apb_i2c.

8.2.1 Power Consumption, Frequency, Area and DFT Coverage

[Table 8-1](#) provides information about the synthesis results (power consumption, frequency and area) and DFT coverage of the DW_apb_i2c using the industry standard 7nm technology library.

Table 8-1 Synthesis Results for DW_apb_i2c

Configuration	Operating Frequency	Gate Count	Power Consumption		TetraMax Coverage (%)		SpyGlass StuckAtCov (%)
			Static Power	Dynamic Power	StuckAtTest	Transition	
Default Configuration	100 MHz	12161	30 nW	0.068 mW	99.98	95.22	100
Typical Configuration - 1^a	100 MHz	15453	40 nW	0.088 mW	99.94	95.08	100
Typical Configuration - 2^b	100 MHz	26930	70 nW	0.131 mW	99.99	95.08	100

a. For the list of configuration parameters and their values for the Typical Configuration 1, see [Table 8-2](#)

b. For the list of configuration parameters and their values for the Typical Configuration 2, see [Table 8-3](#) on page 404

Table 8-2 Typical Configuration - 1

Parameter	Value
IC_ULTRA_FAST_MODE	0x0
IC_CLK_FREQ_OPTIMIZATION	0x0
IC_SMBUS	0x0
IC_ADD_ENCODED_PARAMS	0x1
APB_DATA_WIDTH	32
IC_MAX_SPEED_MODE	0x3
IC_10BITADDR_MASTER	0x1
IC_RESTART_EN	0x1
IC_10BITADDR_SLAVE	0x1
IC_MASTER_MODE	0x1
IC_SLAVE_DISABLE	0x1
IC_DEFAULT_SLAVE_ADDR	0x010
IC_DEFAULT_TAR_SLAVE_ADDR	0x011
IC_HS_MASTER_CODE	0x1
IC_TX_BUFFER_DEPTH	16

Parameter	Value
IC_RX_BUFFER_DEPTH	16
IC_INTR_POL	0x1
IC_INTR_IO	0x1
IC_HAS_DMA	0x1
IC_TX_TL	0x8
IC_RX_TL	0x8
IC_USE_COUNTS	0x1
IC_CLOCK_PERIOD	10
IC_SS_SCL_HIGH_COUNT	0x02f0
IC_SS_SCL_LOW_COUNT	0x0383
IC_FS_SCL_HIGH_COUNT	0x0075
IC_FS_SCL_LOW_COUNT	0x011c
IC_CAP_LOADING	100
IC_HS_SCL_HIGH_COUNT	0x0009
IC_HS_SCL_LOW_COUNT	0x0020
IC_HC_COUNT_VALUES	0x0
IC_CLK_TYPE	0x1
IC_HAS_ASYNC_FIFO	1
I2C_DYNAMIC_TAR_UPDATE	0x1
IC_SLV_DATA_NACK_ONLY	1
IC_EMPTYFIFO_HOLD_MASTER_EN	0x64
IC_DEFAULT_SDA_SETUP	0x000001
IC_DEFAULT_SDA_HOLD	0x1
IC_DEFAULT_ACK_GENERAL_CALL	0x1
IC_RX_FULL_HLD_BUS_EN	0x0
IC_SLV_RESTART_DET_EN	0x0
IC_STOP_DET_IF_MASTER_ACTIVE	0x0
IC_STAT_FOR_CLK_STRETCH	0x0
IC_TX_CMD_BLOCK	0x0

Parameter	Value
IC_TX_CMD_BLOCK_DEFAULT	0x0
IC_FIRST_DATA_BYTE_STATUS	0x0
IC_AVOID_RX_FIFO_FLUSH_ON_TX_ABRT	0x0
IC_BUS_CLEAR_FEATURE	0x0
IC_SCL_STUCK_TIMEOUT_DEFAULT	0xffffffff
IC_SDA_STUCK_TIMEOUT_DEFAULT	0xffffffff
IC_DEVICE_ID	0x0
IC_DEVICE_ID_VALUE	0x0
IC_SMBUS_CLK_LOW_SEXT_DEFAULT	0xffffffff
IC_SMBUS_CLK_LOW_MEXT_DEFAULT	0xffffffff
IC_SMBUS_RST_IDLE_CNT_DEFAULT	0xffff
IC_SMBUS_SUSPEND_ALERT	0x0
IC_OPTIONAL_SAR	0x0
IC_OPTIONAL_SAR_DEFAULT	0x0
IC_SMBUS_ARP	0x0
IC_SMBUS_UDID_MSB	0x0
IC_SMBUS_UDID_LSB_DEFAULT	0xffffffff
IC_PERSISTANT_SLV_ADDR_DEFAULT	0x0
IC_UFM_SCL_HIGH_COUNT	0x0006
IC_UFM_SCL_LOW_COUNT	0x0008
IC_UFM_TBUF_CNT_DEFAULT	0x8
IC_DEFAULT_FS_SPKLEN	0x8
IC_DEFAULT_HS_SPKLEN	0x1
IC_DEFAULT_UFM_SPKLEN	0x1
SLAVE_INTERFACE_TYPE	2
SLVERR_RESP_EN	1
REG_TIMEOUT_WIDTH	8
HC_REG_TIMEOUT_VALUE	0
REG_TIMEOUT_VALUE	8

Table 8-3 Typical Configuration - 2

Parameter	Value
SLAVE_INTERFACE_TYPE	0
SLVERR_RESP_EN	0
IC_ULTRA_FAST_MODE	0x0
IC_CLK_FREQ_OPTIMIZATION	0x1
IC_SMBUS	0x1
IC_MULTI_SAR_EN	0x1
IC_NUM_SARS	4
IC_ADD_ENCODED_PARAMS	0x1
APB_DATA_WIDTH	32
IC_MAX_SPEED_MODE	0x2
IC_10BITADDR_MASTER	0x0
IC_RESTART_EN	0x1
IC_10BITADDR_SLAVE	0x0
IC_MASTER_MODE	0x0
IC_SLAVE_DISABLE	0x1
IC_DEFAULT_SLAVE_ADDR	0x03a
IC_DEFAULT_SLAVE_ADDR2	0x056
IC_DEFAULT_SLAVE_ADDR3	0x057
IC_DEFAULT_SLAVE_ADDR4	0x058
IC_DEFAULT_TAR_SLAVE_ADDR	0x03a
IC_HS_MASTER_CODE	0x1
IC_TX_BUFFER_DEPTH	8
IC_RX_BUFFER_DEPTH	16
IC_INTR_POL	0x1
IC_INTR_IO	0x1
IC_HAS_DMA	0x0
IC_TX_TL	0x0
IC_RX_TL	0x0

Parameter	Value
IC_USE_COUNTS	0x0
IC_CLOCK_PERIOD	100
IC_SS_SCL_HIGH_COUNT	0x0028
IC_SS_SCL_LOW_COUNT	0x002f
IC_FS_SCL_HIGH_COUNT	0x0006
IC_FS_SCL_LOW_COUNT	0x000d
IC_CAP_LOADING	100
IC_HS_SCL_HIGH_COUNT	0x0001
IC_HS_SCL_LOW_COUNT	0x0006
IC_HC_COUNT_VALUES	0x0
IC_CLK_TYPE	0x0
IC_HAS_ASYNC_FIFO	0x0
I2C_DYNAMIC_TAR_UPDATE	1
IC_SLV_DATA_NACK_ONLY	0x1
IC_RX_FULL_GEN_NACK	0x0
IC_EMPTYFIFO_HOLD_MASTER_EN	1
IC_DEFAULT_SDA_SETUP	0x64
IC_DEFAULT_SDA_HOLD	0x000001
IC_DEFAULT_ACK_GENERAL_CALL	0x1
IC_RX_FULL_HLD_BUS_EN	0x1
IC_SLV_RESTART_DET_EN	0x1
IC_STOP_DET_IF_MASTER_ACTIVE	0x0
IC_STAT_FOR_CLK_STRETCH	0x1
IC_TX_CMD_BLOCK	0x1
IC_TX_CMD_BLOCK_DEFAULT	0x1
IC_FIRST_DATA_BYTE_STATUS	0x1
IC_AVOID_RX_FIFO_FLUSH_ON_TX_ABRT	0x1
IC_BUS_CLEAR_FEATURE	0x1
IC_SCL_STUCK_TIMEOUT_DEFAULT	0xffffffff

Parameter	Value
IC_SDA_STUCK_TIMEOUT_DEFAULT	0xffffffff
IC_DEVICE_ID	0x0
IC_DEVICE_ID_VALUE	0x0
IC_SMBUS_CLK_LOW_SEXT_DEFAULT	0xffffffff
IC_SMBUS_CLK_LOW_MEXT_DEFAULT	0xffffffff
IC_SMBUS_RST_IDLE_CNT_DEFAULT	0xffff
IC_SMBUS_SUSPEND_ALERT	0x1
IC_OPTIONAL_SAR	0x0
IC_OPTIONAL_SAR_DEFAULT	0x0
IC_SMBUS_ARP	0x1
IC_SMBUS_UDID_HC	0x0
IC_SMBUS_UDID_MSB	0x400815b70100000515b70100
IC_SMBUS_UDID_LSB_DEFAULT	0xffffffff
IC_PERSISTANT_SLV_ADDR_DEFAULT	0x1
IC_PERSISTANT_SLV_ADDR2_DEFAULT	0x1
IC_PERSISTANT_SLV_ADDR3_DEFAULT	0x1
IC_PERSISTANT_SLV_ADDR4_DEFAULT	0x1
IC_UFM_SCL_HIGH_COUNT	0x0001
IC_UFM_SCL_LOW_COUNT	0x0006
IC_UFM_TBUF_CNT_DEFAULT	0x8
IC_DEFAULT_FS_SPKLEN	0x1
IC_DEFAULT_HS_SPKLEN	0x1
IC_DEFAULT_UFM_SPKLEN	0x1
REG_TIMEOUT_WIDTH	4
HC_REG_TIMEOUT_VALUE	0
REG_TIMEOUT_VALUE	8
IC_MULTI_SAR_UDID_CTRL	0x1
IC_SAR2_SMBUS_UDID_HC	0x0
IC_SAR2_SMBUS_UDID_DEFAULT	0xffff0000ffff0000ffff00003fff0000

Parameter	Value
IC_SAR3_SMBUS_UDID_HC 0x0	0x0
IC_SAR3_SMBUS_UDID_DEFAULT	0xffff0000ffff0000ffff00007fff0000
IC_SAR4_SMBUS_UDID_HC	0x0
IC_SAR4_SMBUS_UDID_DEFAULT	0xffff0000ffff0000ffff0000bfff0000

8.3 Timing Exceptions

For details on quasi-static signals in the design, refer to manual.sgdc report generated by the SpyGlass tool.

A

Synchronizer Methods

The Basic Core Module (BCM) Library is a library of commonly used blocks for the Synopsys DesignWare IP development. These BCMs are configurable on an instance-by-instance basis and, for the majority of BCM designs, there is an equivalent (or nearly equivalent) DesignWare Building Block (DWBB) component.

This appendix contains the following sections:

- [“BCM Library Components”](#) on page [410](#)
- [“Synchronizer Methods”](#) on page [411](#)

A.1 BCM Library Components

Table A-1 describes the list of BCM library components used in DW_apb_i2c.

Table A-1 BCM Library Components

BCM Module Name	BCM Description	DWBB Equivalent
DW_ahb_eh2h_bcm21	Single clock data bus synchronizer	DW_sync
DW_apb_i2c_bcm05	FIFO controller interface; submodule of DWbb_bcm07	DW_fifoctrl_if
DW_apb_i2c_bcm06	Synchronous (Single Clock) FIFO Controller with Dynamic Flags	DW_fifoctrl_s1_df
DW_apb_i2c_bcm07	Synchronous (Dual-Clock) FIFO Controller with Static Flags	DW_fifoctrl_s2_sf
DW_apb_i2c_bcm41	Single Clock Data Bus Synchronizer with Configurable Polarity Reset Note: Same as DWbb_bcm21 but with additional RST_VAL parameter	--
DW_apb_i2c_bcm47	Universal Synchronous (Clocked) CRC Generator/Checker	DW_crc_s
DW_apb_i2c_bcm57	Synchronous Write-Port, Asynchronous Read-Port RAM (Flip-Flop-Based)	DW_ram_r_w_s_dff
DW_apb_i2c_bcm36_nhs	Bus Delay Component	--

A.2 Synchronizer Methods

This section describes the synchronizer methods (blocks of synchronizer functionality) that are used in the DW_apb_i2c to cross clock boundaries.

This section contains the following sections:

- “Synchronizers Used in DW_apb_i2c” on page 411
- “Synchronizer 1: Simple Double Register Synchronizer” on page 411
- “Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset” on page 412



Note

The DesignWare Building Blocks (DWBB) contains several synchronizer components with functionality similar to methods documented in this appendix. For more information about the DWBB synchronizer components go to:

<https://www.synopsys.com/dw/buildingblock.php>

A.2.1 Synchronizers Used in DW_apb_i2c

Each of the synchronizers and synchronizer sub-modules are comprised of verified DesignWare Basic Core (BCM) RTL designs. The BCM synchronizer designs are identified by the synchronizer type. The corresponding RTL files comprising the BCM synchronizers used in the DW_apb_i2c are listed and cross referenced to the synchronizer type in Table A-2. Note that certain BCM modules are contained in other BCM modules, as they are used in a building block fashion.

Table A-2 Synchronizers used in DW_apb_i2c

Synchronizer module file	Sub module file	Synchronizer Type and Number
DW_apb_i2c_bcm21.v	NA	Synchronizer 1: Simple Multiple Register Synchronizer
DW_apb_i2c_bcm41.v	DW_apb_i2c_bcm21.v	Synchronizer 2: Simple Multiple Register Synchronizer with Configurable Polarity Reset

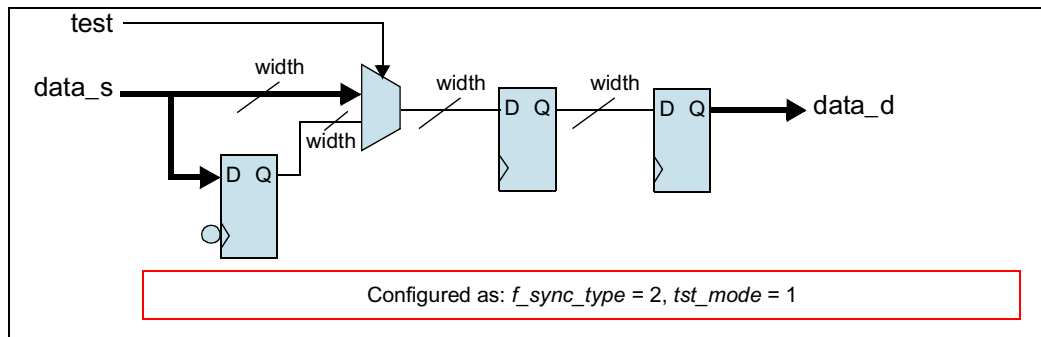


Note

The BCM21 is a basic multiple register based synchronizer module used in the design. It can be replaced with equivalent technology specific synchronizer cell.

A.2.2 Synchronizer 1: Simple Double Register Synchronizer

This is a single clock data bus synchronizer for synchronizing data that crosses asynchronous clock boundaries. The synchronization scheme depends on core configuration. If pclk and ic_clk are asynchronous (IC_CLK_TYPE =ASYNC) then DW_apb_i2c_bcm21 is instantiated inside the core for synchronization. This uses two stage synchronization process both using positive edge of clock.

Figure A-1 Block Diagram of Synchronizer 1 With Two Stage Synchronization (Both Positive Edges)

A.2.3 Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset

This is a single clock data bus synchronizer for synchronizing data that crosses asynchronous clock boundaries with configurable polarity reset. The synchronization scheme depends on core configuration. If `pclk` and `ic_clk` are asynchronous (`IC_CLK_TYPE = ASYNC`) then `DW_apb_i2c_bcm41` is instantiated inside the core for synchronization of `ic_clk_in_a` and `ic_data_in_a` input signals. This `DW_apb_i2c_bcm41` synchronizer is similar to the `DW_apb_i2c_bcm21` synchronizer and the polarity of the output of this synchronizer can be configured.

B

Internal Parameter Descriptions

Provides a description of the internal parameters that might be indirectly referenced in expressions in the Signals, Parameters, or Registers chapters. These parameters are not visible in the coreConsultant GUI and most of them are derived automatically from visible parameters. **You must not set any of these parameters directly.**

Some expressions might refer to TCL functions or procedures (sometimes identified as **function_of**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the core in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Table B-1 Internal Parameters

Parameter Name	Equals To
ENCODED_APB_DATA_WIDTH	[function_of: APB_DATA_WIDTH]
ENCODED_IC_RX_BUFFER_DEPTH	[function_of: IC_RX_BUFFER_DEPTH]
ENCODED_IC_TX_BUFFER_DEPTH	[function_of: IC_TX_BUFFER_DEPTH]
IC_ADDR_SLICE_LHS	((((IC_MULTI_SAR_EN==1) ((IC_SMBUS_FD_FND_SUPPORT_EN==1)))?0b1000:0b0111))
IC_BUS_CLEAR_FEATURE_EN	1
IC_DEFAULT_SDA_RX_HOLD	[function_of: IC_DEFAULT_SDA_HOLD]
IC_DEFAULT_SDA_TX_HOLD	[function_of: IC_DEFAULT_SDA_HOLD]
IC_HCNT_LO_LIMIT	((((IC_ULTRA_FAST_MODE==1)? 3 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 1 : 6))
IC_HIGHSPEED_MODE_EN	(IC_MAX_SPEED_MODE == 3 ? 1 : 0)
IC_LCNT_LO_LIMIT	((((IC_ULTRA_FAST_MODE==1)? 5 : ((IC_CLK_FREQ_OPTIMIZATION== 1) ? 6 : 8))
IC_MSAR_SMBUS_ALERT_EN	(IC_SMBUS_SUSPEND_ALERT==1 && IC_MULTI_SAR_EN==1) ? 1 : 0

Parameter Name	Equals To
IC_SAR2_SMBUS_UDID_WORD0_DEFAULT	(IC_SAR2_SMBUS_UDID_DEFAULT&0b11111111111111 11111111111111)
IC_SAR2_SMBUS_UDID_WORD1_DEFAULT	((IC_SAR2_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111110000000000000000000000000000 000)>>32)
IC_SAR2_SMBUS_UDID_WORD2_DEFAULT	((IC_SAR2_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000) >>64)
IC_SAR2_SMBUS_UDID_WORD3_DEFAULT	((IC_SAR2_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000 000) >>96)
IC_SAR3_SMBUS_UDID_WORD0_DEFAULT	(IC_SAR3_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111)
IC_SAR3_SMBUS_UDID_WORD1_DEFAULT	((IC_SAR3_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000)>>32)
IC_SAR3_SMBUS_UDID_WORD2_DEFAULT	((IC_SAR3_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000) >>64)
IC_SAR3_SMBUS_UDID_WORD3_DEFAULT	((IC_SAR3_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000 000) >>96)
IC_SAR4_SMBUS_UDID_WORD0_DEFAULT	(IC_SAR4_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111)
IC_SAR4_SMBUS_UDID_WORD1_DEFAULT	((IC_SAR4_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000)>>32)
IC_SAR4_SMBUS_UDID_WORD2_DEFAULT	((IC_SAR4_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000) >>64)
IC_SAR4_SMBUS_UDID_WORD3_DEFAULT	((IC_SAR4_SMBUS_UDID_DEFAULT&0b11111111111111 1111111111111111000000000000000000000000000 000 000) >>96)
IC_SMBALERT_OE_WIDTH	((IC_MSAR_SMBUS_ALERT_EN==1)?IC_NUM_SARS:1)
IC_SMBUS_UDID_WORD1_DEFAULT	(IC_SMBUS_UDID_MSB&0b1111111111111111111111 11111111)

[illegible]

C

Glossary

active command queue	Command queue from which a model is currently taking commands; see also command queue.
activity	A set of functions in coreConsultant that step you through configuration, verification, and synthesis of a selected core.
AHB	Advanced High-performance Bus — high-performance system backbone bus. AHB supports the efficient connection of processors, on-chip memories and off-chip external memory interfaces (Arm® Limited specification).
AMBA	Advanced Microcontroller Bus Architecture — a trademarked name by Arm® Limited that defines an on-chip communication standard for high speed micro-controller.
APB	Advanced Peripheral Bus — optimized for minimal power consumption and reduced interface complexity to support peripheral functions (Arm® Limited specification).
APB bridge	DW_apb submodule that converts protocol between the AHB bus and APB bus.
application design	Overall chip-level design into which a subsystem or subsystems are integrated.
arbiter	AMBA bus submodule that arbitrates bus activity between managers and subordinates.
BFM	Bus-Functional Model — A simulation model used for early hardware debug. A BFM simulates the bus cycles of a device and models device pins, as well as certain on-chip functions. See also Full-Functional Model.
big-endian	Data format in which most significant byte comes first; normal order of bytes in a word.
blocked command stream	A command stream that is blocked due to a blocking command issued to that stream; see also command stream, blocking command, and non-blocking command.

blocking command	A command that prevents a testbench from advancing to next testbench statement until this command executes in model. Blocking commands typically return data to the testbench from the model.
bus bridge	Logic that handles the interface and transactions between two bus standards, such as AHB and APB. See APB bridge.
command channel	Manages command streams. Models with multiple command channels execute command streams independently of each other to provide full-duplex mode function.
command stream	The communication channel between the testbench and the model.
component	A generic term that can refer to any synthesizable IP or verification IP in the DesignWare Library. In the context of synthesizable IP, this is a configurable block that can be instantiated as a single entity (VHDL) or module (Verilog) in a design.
configuration	The act of specifying parameters for a core prior to synthesis; can also be used in the context of VIP.
configuration intent	Range of values allowed for each parameter associated with a reusable core.
controller	Device or model that initiates and controls another device or peripheral.
core	Any configurable block of synthesizable IP that can be instantiated as a single entity (VHDL) or module (Verilog) in a design. Core is the preferred term for a big piece of IIP. Anything that requires coreConsultant for configuration, as well as anything in the DesignWare Cores library, is a core.
core developer	Person or company who creates or packages a reusable core. All the cores in the DesignWare Library are developed by Synopsys.
core integrator	Person who uses coreConsultant or coreAssembler to incorporate reusable cores into a system-level design.
coreAssembler	Synopsys product that enables automatic connection of a group of cores into a subsystem. Generates RTL and gate-level views of the entire subsystem.
coreConsultant	A Synopsys product that lets you configure a core and generate the design views and synthesis views you need to integrate the core into your design. Can also synthesize the core and run the unit-level testbench supplied with the core.
coreKit	An unconfigured core and associated files, including the core itself, a specified synthesis methodology, interfaces definitions, and optional items such as verification environment files and core-specific documentation.
cycle command	A command that executes and causes HDL simulation time to advance.
decoder	Software or hardware subsystem that translates from an “encoded” format back to standard format.
design context	Aspects of a component or subsystem target environment that affect the synthesis of the component or subsystem.

design creation	The process of capturing a design as parameterized RTL.
Design View	A simulation model for a core generated by coreConsultant.
DesignWare Synthesizable Components	The Synopsys name for the collection of AMBA-compliant coreKits and verification models delivered with DesignWare and used with coreConsultant or coreAssembler to quickly build DesignWare Synthesizable Component designs.
DesignWare cores	A specific collection of synthesizable cores that are licensed individually. For more information, refer to www.synopsys.com/designware .
DesignWare Library	A collection of synthesizable IP and verification IP components that is authorized by a single DesignWare license. Products include SmartModels, VMT model suites, DesignWare Memory Models, Building Block IP, and the DesignWare Synthesizable Components.
dual role device	Device having the capabilities of function and host (limited).
endian	Ordering of bytes in a multi-byte word; see also little-endian and big-endian.
Full-Functional Mode	A simulation model that describes the complete range of device behavior, including code execution. See also BFM.
GPIO	General Purpose Input Output.
GTECH	A generic technology view used for RTL simulation of encrypted source code by non-Synopsys simulators.
hard IP	Non-synthesizable implementation IP.
HDL	Hardware Description Language – examples include Verilog and VHDL.
IIP	Implementation Intellectual Property — A generic term for synthesizable HDL and non-synthesizable “hard” IP in all of its forms (coreKit, component, core, MacroCell, and so on).
implementation view	The RTL for a core. You can simulate, synthesize, and implement this view of a core in a real chip.
instantiate	The act of placing a core or model into a design.
interface	Set of ports and parameters that defines a connection point to a component.
IP	Intellectual property — A term that encompasses simulation models and synthesizable blocks of HDL code.
little-endian	Data format in which the least-significant byte comes first.
MacroCell	Bigger IP blocks (6811, 8051, memory controller) available in the DesignWare Library and delivered with coreConsultant.
model	A Verification IP component or a Design View of a core.
monitor	A device or model that gathers performance statistics of a system.
non-blocking command	A testbench command that advances to the next testbench statement without waiting for the command to complete.

peripheral	Generally refers to a small core that has a bus connection, specifically an APB interface.
RTL	Register Transfer Level. A higher level of abstraction that implies a certain gate-level structure. Synthesis of RTL code yields a gate-level design.
SDRAM	Synchronous Dynamic Random Access Memory; high-speed DRAM adds a separate clock signal to control signals.
SDRAM controller	A memory controller with specific connections for SDRAMs.
SoC	System on a chip.
soft IP	Any implementation IP that is configurable. Generally referred to as synthesizable IP.
static controller	Memory controller with specific connections for Static memories such as asynchronous SRAMs, Flash memory, and ROMs.
subsystem	In relation to coreAssembler, highest level of RTL that is automatically generated.
synthesis intent	Attributes that a core developer applies to a top-level design, ports, and core.
synthesizable IP	A type of Implementation IP that can be mapped to a target technology through synthesis. Sometimes referred to as Soft IP.
target	Device or model that is controlled by and responds to a controller.
technology-independent	Design that allows the technology (that is, the library that implements the gate and via widths for gates) to be specified later during synthesis.
Testsuite Regression Environment (TRE)	A collection of files for stand-alone verification of the configured component. The files, tests, and functionality vary from component to component.
VIP	Verification Intellectual Property — A generic term for a simulation model in any form, including a Design View.
workspace	A network location that contains a personal copy of a component or subsystem. After you configure the component or subsystem (using coreConsultant or coreAssembler), the workspace contains the configured component/subsystem and generated views needed for integration of the component/subsystem at the top level.
wrap, wrapper	Code, usually VHDL or Verilog, that surrounds a design or model, allowing easier interfacing. Usually requires an extra, sometimes automated, step to create the wrapper.
zero-cycle command	A command that executes without HDL simulation time advancing.